University of California
Santa Barbara

Efficient Algorithms for Graph Counting and Sampling from a Parameterized Perspective

A dissertation submitted in partial satisfaction
of the requirements for the degree

Doctor of Philosophy
in
Computer Science

by

Úrsula Hébert-Johnson

Committee in charge:

Professor Daniel Lokshtanov, Chair
Professor Subhash Suri
Professor Ambuj Singh
Professor Eric Vigoda

March 2025

The Dissertation of Úrsula Hébert-Johnson is approved.

Professor Subhash Suri

Professor Ambuj Singh

Professor Eric Vigoda

Professor Daniel Lokshtanov, Committee Chair

February 2025

Efficient Algorithms for Graph Counting and Sampling
from a Parameterized Perspective

by

Úrsula Hébert-Johnson

A mi mamá, mi papá, y Maicito

Acknowledgements

I would like to thank my advisor Daniel Lokshtanov for all that I have learned in our many meetings throughout my PhD. I have been very fortunate to have Daniel as my advisor. Thank you for finding just the right research problems for me to work on and for your guidance throughout the research process. And thank you for planning great ski trips for our lab. I would also like to thank the members of my committee, Subhash Suri, Ambuj Singh, and Eric Vigoda.

I am grateful to my parents in countless ways, including for passing on their enthusiasm for math and snow sports, which helped to keep my proofs airtight and my skis afloat during the past five years.

Thank you to my friends for all of the wonderful hikes and deep conversations. My original reason for going to graduate school was as much to meet great people and lifelong friends as it was for future career reasons. That definitely ended up coming true.

Finally and most importantly, thank you to Maicito for napping next to me while I wrote this thesis.

Curriculum Vitæ

Úrsula Hébert-Johnson

Education

2025	Ph.D. in Computer Science (Expected), University of California, Santa Barbara
2018	B.S. in Mathematics, Stanford University

Publications

1. Counting Yes-Instances of Vertex Deletion Problems in FPT Time. Úrsula Hébert-Johnson and Daniel Lokshtanov. *In preparation.*
2. Sampling Unlabeled Chordal Graphs in Expected Polynomial Time. Úrsula Hébert-Johnson and Daniel Lokshtanov. *Symposium on Theoretical Aspects of Computer Science (STACS)*, 2025.
3. Parameterized Complexity of Kidney Exchange Revisited. Úrsula Hébert-Johnson, Daniel Lokshtanov, Chinmay Sonar, and Vaishali Surianarayanan. *International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.
4. Min-Max Coverage Problems on Tree-Like Metrics. Eric Aaron, Úrsula Hébert-Johnson, Danny Krizanc, and Daniel Lokshtanov. *Latin-American Algorithms, Graphs and Optimization Symposium (LAGOS)*, 2023.
5. Counting and Sampling Labeled Chordal Graphs in Polynomial Time. Úrsula Hébert-Johnson, Daniel Lokshtanov, and Eric Vigoda. *European Symposium on Algorithms (ESA)*, 2023. Best ESA Paper Award.
6. Anonymity-Preserving Space Partitions. Úrsula Hébert-Johnson, Chinmay Sonar, Subhash Suri, and Vaishali Surianarayanan. *International Symposium on Algorithms and Computation (ISAAC)*, 2021.
7. Multicalibration: Calibration for the (Computationally-Identifiable) Masses. Úrsula Hébert-Johnson, Michael Kim, Omer Reingold, and Guy Rothblum. *International Conference on Machine Learning (ICML)*, 2018.
8. The Fibonacci Identities of Orthogonality. Kyle Hawkins, Úrsula Hébert-Johnson, and Benjamin Mathes. *Linear Algebra and Its Applications*, 475, 80-89, 2015.

Work Experience

January - September 2019	Software Engineer at Meta, Menlo Park, CA
October - December 2018	Software Engineer at Residential Energy Dynamics, Waterville, ME

Abstract

Efficient Algorithms for Graph Counting and Sampling
from a Parameterized Perspective

by

Úrsula Hébert-Johnson

Graph counting refers to counting how many n-vertex graphs belong to a particular graph class. This topic is closely intertwined with graph sampling, which refers to generating random elements of a graph class (uniformly or approximately uniformly at random). In this thesis, we develop algorithms for various graph counting and graph sampling problems.

For the class of chordal graphs, we present the first polynomial-time algorithm to exactly compute the number of labeled chordal graphs on n vertices. In fact, our algorithm solves a more general problem: given n and ω as input, it computes the number of ω-colorable labeled chordal graphs on n vertices, using $O(n^7)$ arithmetic operations. A standard sampling-to-counting reduction then yields a polynomial-time exact sampler that generates an ω-colorable labeled chordal graph on n vertices uniformly at random. Our counting algorithm improves upon the previous best result by Wormald (1985), which computes the number of labeled chordal graphs on n vertices in time exponential in n. We also design two approximation algorithms for this problem: (1) an approximate counting algorithm that computes a $(1 \pm \varepsilon)$-approximation of the number of n-vertex labeled chordal graphs, and (2) an approximate sampling algorithm that generates a random labeled chordal graph according to a distribution whose total variation distance from the uniform distribution is at most ε. The approximate counting algorithm runs in $O(n^3 \log n \log^7(1/\varepsilon))$ time, and the approximate sampling algorithm runs

in $O(n^3 \log n \log^7(1/\varepsilon))$ expected time.

Next, we turn our attention to *unlabeled* chordal graphs. We design an algorithm that generates an n-vertex unlabeled chordal graph uniformly at random in expected polynomial time. Along the way, we develop the following two results: (1) an FPT algorithm for counting and sampling labeled chordal graphs with a given automorphism π, parameterized by the number of moved points of π, and (2) a proof that the probability that a random n-vertex labeled chordal graph has a given automorphism $\pi \in S_n$ is at most $1/2^{c \max\{\mu^2, n\}}$, where μ is the number of moved points of π and c is a constant. Our algorithm for sampling unlabeled chordal graphs calls the aforementioned FPT algorithm as a black box with potentially large values of the parameter μ, but the probability of calling this algorithm with a large value of μ is exponentially small.

Our third and last topic is to design FPT algorithms for counting yes-instances of various vertex deletion problems. In particular, we describe FPT algorithms for counting yes-instances of Vertex Cover, Cluster Vertex Deletion, Feedback Vertex Set in Tournaments, and Split Vertex Deletion. We achieve this by designing a general framework that reduces each of these four counting problems to a simpler version of the problem with given deletion sets. For each of the problems in question, we then design an FPT algorithm for the version with given deletion sets. Overall, the algorithms for counting yes-instances of Vertex Cover, Cluster Vertex Deletion, and Feedback Vertex Set in Tournaments have running time at most $2^{3^{2k+o(k)}} \cdot n^{O(1)}$, and the running time of the algorithm for counting yes-instances of Split Vertex Deletion is a triple-exponential function of k. We also design a faster algorithm for counting yes-instances of Vertex Cover that runs in time $k^{O(k^2)}n^3$, using ideas inspired by the Buss kernel for that problem.

Contents

Chapter 1

Introduction

In graph algorithms, the topic of *counting* can be interpreted in one of two ways: we can count combinatorial objects within a given graph, or we can count the graphs themselves. For problems of the first type, we are given a graph G as input, and the goal is to count something in G. For instance, we could study the following problem: "Given a graph G and an integer k, how many vertex covers of G are of size at most k?" On the other hand, directly counting the graphs themselves leads to an equally fascinating collection of problems. In this case, the input is the number of vertices n (and possibly an integer k), and the goal is to count how many n-vertex graphs belong to the graph class in question. To continue with the previous example, rather than counting small vertex covers in a given graph, we could instead count how many graphs have a small vertex cover: "Given integers n and k, how many n-vertex graphs have a vertex cover of size at most k?" In this thesis, our main focus is to solve problems of the latter type, which we refer to as *graph counting*.

Sampling refers to generating random elements of a particular set or graph class. For instance, we could output a random vertex cover of size at most k for a given graph, or we could generate a random graph from the set of all n-vertex graphs with a vertex

cover of size at most k. As this example suggests, for every counting problem, there is a corresponding sampling problem. In general, if the counting problem is to compute $|S|$ for a set S, then the analogous sampling problem would be to output an element of S uniformly at random. The notions of counting and sampling are closely intertwined, and it is often possible to reduce from one to the other [1].

In this thesis, we develop algorithms for various graph counting and graph sampling problems. These new results are presented in Chapters 2 to 4. Each of these algorithms has connections to parameterized algorithms, in a different sense for each chapter. The algorithms in Chapter 2 are polynomial-time algorithms that deal with chordal graphs, which are closely connected to treewidth, a central concept in parameterized algorithms. The main result of Chapter 3 is an expected-polynomial-time algorithm (again having to do with chordal graphs) that heavily relies upon a fixed-parameter tractable (FPT) algorithm as a subroutine. In Chapter 4, all of our results are explicitly parameterized, since we design FPT algorithms for various parameterized graph classes. The first and simplest of these is an algorithm that counts how many n-vertex graphs have a vertex cover of size at most k.

Before describing these results, we begin with a survey of related work. At the moment, there is a much more extensive body of literature on counting and sampling combinatorial objects in a given graph [2] (e.g. vertex covers, independent sets, matchings, etc.) than there is on graph counting and sampling [3]. For this reason, to put our research in context, we begin with a brief survey of the former in Section 1.1. Next, in Section 1.2, we survey known algorithms for graph counting and sampling. Section 1.3 focuses on chordal graphs, since many of our new results are algorithms for counting and sampling chordal graphs. In Section 1.4, we discuss results related to parameterized algorithms that could be useful for future work along the lines of Chapter 4. The necessary notation and definitions for this thesis are provided in Section 1.5.

1.1 Counting and Sampling in a Given Graph

An enormous amount of research has been done on the topic of counting and sampling combinatorial objects in a given graph. In this section, we will give a brief overview of some of the most famous problems and types of algorithms. We also discuss the method of Markov chain Monte Carlo (MCMC), which is often used to design algorithms for these problems, since this technique could potentially also be transferable to future work on graph sampling. More information on the general topic of counting and sampling can be found in the book by Jerrum [2].

1.1.1 Counting algorithms

A natural place to begin is with #P-completeness, since many of the most famous problems in this area are #P-complete. Here are a few examples of #P-complete counting problems:

- counting independent sets in a given 3-regular graph [4]
- counting matchings in a given bipartite graph [5]
- counting perfect matchings in a given bipartite graph [6]
- counting 3-colorings of a given bipartite graph [7].

In the #SAT problem, we are given a Boolean formula as input, and the goal is to count the number of variable assignments that satisfy the formula. When a problem is #P-hard, this means that the problem is at least as hard as #SAT. (For a more formal definition, see [2].) We say a problem belongs to the complexity class #P if it can be formulated as counting the number of accepting paths in a polynomial-time nondeterministic Turning machine. A problem is #P-complete if it is #P-hard and belongs to #P.

We do not expect there to be polynomial-time algorithms to solve these problems exactly, since #P-complete problems are at least as difficult as NP-complete problems. Fortunately, there is a vast body of literature on approximate counting [2]. For some problems, it is possible to design a particularly efficient approximation algorithm known as a *fully polynomial randomized approximation scheme* (FPRAS). In the context of graph problems, an FPRAS is a randomized algorithm that takes as input a graph G and $\varepsilon, \delta \in (0, 1]$ and outputs a number $\hat{z}$ such that

$$\Pr(e^{-\varepsilon}\hat{z} \leq z \leq e^{\varepsilon}\hat{z}) \geq 1 - \delta,$$

where z is the exact solution for G, running in time $\text{poly}(n, \frac{1}{\varepsilon}, \log \frac{1}{\delta})$, where $n = |V(G)|$. Typically, $z = |S|$ for an implicitly defined set S whose size is exponential in n, even though the running time we are aiming for is polynomial in n.

There is an FPRAS for counting perfect matchings in a given bipartite graph that was presented by Jerrum, Sinclair, and Vigoda [8]. Around a decade earlier, Jerrum also showed that there is an FPRAS for counting k-colorings in graphs of low enough degree [9]. More precisely, suppose Δ and k are nonnegative integers such that $k > 2\Delta$. The algorithm in [9] is an FPRAS for counting k-colorings in graphs with maximum degree Δ. Both of these algorithms use MCMC techniques (see Section 1.1.3). More recently, the algorithm in [9] was also improved to a fully polynomal-time approximation scheme (FPTAS), which is a deterministic algorithm (using techniques other than Markov chains) [10].

For the problem of counting independent sets, there is a dichotomy between graphs with maximum degree $\Delta \leq 5$ and graphs with $\Delta > 5$. As was shown by Chen, Liu, and Vigoda in [11] using MCMC, there is an FPRAS for counting independent sets in graphs with maximum degree $\Delta \leq 5$ that runs in $O^*(n^2)$ time, where the O^* notation hides polylogarithmic factors and the dependence on ε and δ. On the other hand, there

is no FPRAS for counting independent sets in graphs with $\Delta = 6$ unless RP = NP [12].

Finally, it is worth mentioning two well-known counting problems that are not #P-complete and that have efficient exact algorithms. The number of spanning trees of a graph can be computed in polynomial time by computing the determinant of a submatrix of the graph's Laplacian matrix [13]. It is also possible to count perfect matchings in a planar graph in polynomial time by computing the square root of the determinant of a particular matrix [14].

1.1.2 Sampling algorithms

The sampling analogue of an FPRAS is known as a *fully polynomial almost uniform sampler* (FPAUS). Let S be the set of objects that we are sampling from, and let π be the uniform distribution over S. For example, if we are sampling independent sets in a graph G, then S would be the set of all independent sets in G. An FPAUS is a randomized algorithm that takes as input a graph G and $\delta \in (0, 1]$ and outputs $W \in S$ such that the total variation distance between π and the distribution of W is at most δ, running in time $\text{poly}(n, \log \frac{1}{\delta})$.

Counting and sampling are closely related for all problems that are "self-reducible," which is a condition that holds in many cases (including for independent sets and matchings). Assuming self-reducibility, a counting problem admits an FPRAS if and only if the corresponding sampling problem admits an FPAUS [1].

1.1.3 Markov chain Monte Carlo

Markov chain Monte Carlo refers to the technique of using a Markov chain to design a sampling algorithm. We give a brief summary of the concept of Markov chains (partly based on [2]). For more details, see [2].

A *discrete Markov chain* with *state space* S is a sequence $X_0, X_1, X_2, \ldots$ of random variables (each of which has domain S) that satisfies

$$\Pr(X_{t+1} = x \mid X_1 = x_1, X_2 = x_2, \ldots, X_t = x_t) = \Pr(X_{t+1} = x \mid X_t = x_t)$$

for all $t \geq 0$, $x, x_1, \ldots, x_t \in S$. The intuition is that the probability of moving to a particular state in S at time $t+1$ only depends on the current state X_t, not on what happened further back in the past. A discrete Markov chain is *time-homogenous* if

$$\Pr(X_{t+1} = x \mid X_t = y) = \Pr(X_t = x \mid X_{t-1} = y)$$

for all $t \geq 1$, $x, y \in S$. In other words, the probability of transitioning from one particular state to another does not depend on the time t. We assume S is finite. From now on, a discrete time-homogenous Markov chain will simply be called a *Markov chain*, since these are the ones that are useful for sampling algorithms.

A Markov chain can be described in a concise way by its *transition matrix*. Suppose P is a matrix whose rows and columns are indexed by the elements of the state space S. For $x, y \in S$, we write $P(x, y)$ to denote the entry of P at row x and column y. Furthermore, suppose P is *stochastic*, i.e, $P(x, y) \geq 0$ for all $x, y \in S$ and $\sum_{y \in S} P(x, y) = 1$ for all $x \in S$. The Markov chain with *transition matrix* P is defined as the Markov chain $X_0, X_1, X_2, \ldots$ such that

$$\Pr(X_{t+1} = x \mid X_t = y) = P(x, y)$$

for all $t \geq 0$, $x, y \in S$. Of course, for the Markov chain to be fully specified, we also need to specify the distribution of the initial state X_0. This is typically done by choosing a fixed value $x \in S$ for X_0.

Now suppose S is a set that we would like to sample from. If we can prove that the distribution of X_t converges to the uniform distribution on S as t approaches infinity, and

if we have a way of simulating the Markov chain up to a sufficiently large value of t, then this gives us a way of sampling from S approximately uniformly at random. To analyze such an algorithm, we need to (1) prove that the distribution of X_t indeed converges to the uniform distribution and (2) understand the running time.

Suppose $X_0, X_1, X_2, \ldots$ is a Markov chain with transition matrix P. A *stationary distribution* of this Markov chain is a probability distribution π on S that satisfies $\pi(y) = \sum_{x \in S} \pi(x) P(x, y)$. This means that if X_t is distributed according to π, then so is X_{t+1}. The Markov chain is *irreducible* if for every $x, y \in S$, there exists a nonnegative integer t such that $P^t(x, y) > 0$, meaning every state can be reached from every other state. The Markov chain is *aperiodic* if $\gcd(\{t : P^t(x, x) > 0\} = 1)$ for all $x \in S$. We say the Markov chain is *ergodic* if it is both irreducible and aperiodic. If it is ergodic, then there exists a unique stationary distribution π, and furthermore, $P^t(x, y) \to \pi(y)$ as $t \to \infty$ for all $x, y \in S$. In many cases, it is possible to show that π is in fact the uniform distribution, in which case step (1) is complete. For instance, a sufficient condition for this is that P is symmetric.

Now suppose we have been given an error bound $\varepsilon > 0$. For each $x \in S$, the *mixing time* (with initial state x) of our Markov chain is $\tau_x(\varepsilon) \coloneqq \min\{t : \|P^t(x, \cdot) - \pi\|_{\mathrm{TV}} \leq \varepsilon\}$. Here $\|P^t(x, \cdot) - \pi\|_{\mathrm{TV}}$ denotes the total variation distance between row x of P^t and the stationary distribution π. The mixing time can also be considered to be independent of the initial state, by defining $\tau(\varepsilon) \coloneqq \max_{x \in S} \tau_x(\varepsilon)$. For (2), the mixing time tells us how many time steps we need to simulate before we have a good approximation of the uniform distribution.

As an example, the FPRAS by Jerrum for counting k-colorings in graphs of low enough degree [9] uses a relatively simple Markov chain whose state space S is the set of all (proper) k-colorings of the input graph G. Let $C \coloneqq [k]$ be the set of k colors. The transition probabilities are given by the following procedure:

1. Choose a vertex $v \in V(G)$ and a color $c \in C$ uniformly at random

2. Recolor vertex v with color c; if the resulting coloring X' is proper, then let $X_{t+1} = X'$, otherwise, let $X_{t+1} = X_t$.

This Markov chain is ergodic and its stationary distribution is uniform, as long as $k \geq \Delta + 2$. Moreover, if we assume $k > 2\Delta$, then its mixing time is a polynomial in n. Note that since $k > \Delta$, it is easy to find an arbitrary k-coloring to use as the initial state.

Markov chains have the potential to be useful for graph sampling as well. In this case, each state in the Markov chain would be a graph rather than a subset of the vertices of a fixed graph. For example, in [15] Sun and Bezáková proposed a Markov chain for sampling chordal graphs. This Markov chain comes with few mixing time guarantees, but perhaps better guarantees could be proved for this Markov chain or a variation on it in the future.

1.2 Graph Counting and Sampling

When counting and sampling graphs, we first need to choose whether the graphs are labeled or unlabeled. A *graph* is a set $V(G)$ of *vertices* together with a set $E(G)$ of *edges*. The vertex set $V(G)$ is finite, and the edge set $E(G)$ is a set of two-element subsets of $V(G)$. We write (u, v) to denote the edge $\{u, v\}$, and in general, we use standard terminology for graphs (see Section 1.5). When a graph does not come with any particular labeling of its vertices, we call this an *unlabeled* graph.

A *labeled graph* is a graph in which the vertices have been assigned labels from $\mathbb{N}$, the set of natural numbers. Equivalently, we can define a labeled graph as a graph whose vertex set is a subset of $\mathbb{N}$. For simplicity, we will use this second formulation, since this allows the labels to also serve as the names of the vertices. For example, we will speak

of the vertex $5 \in V(G)$ rather than a vertex v with label 5. Also, will we often speak of "labeled graphs on n vertices" as a shorthand for "labeled graphs with vertex set $[n]$."

An *isomorphism* from a graph G to a graph H is a bijection $f\colon V(G) \to V(H)$ such that for all $u, v \in V(G)$, u and v are adjacent in G if and only if $f(u)$ and $f(v)$ are adjacent in H. We say G and H are *isomorphic* if there is an isomorphism from G to H. This is what determines equality of unlabeled graphs — two unlabeled graphs are considered equal whenever they are isomorphic. For labeled graphs, equality is more strict. Two labeled graphs are considered equal only when their vertex and edge sets are precisely the same. As an example, there are only two connected unlabeled graphs on 3 vertices (a path and a triangle), but there are four connected *labeled* graphs on 3 vertices (since the path can be labeled in three different ways).

Labeled graphs are easy to count and sample, if we do not restrict to a particular graph class, since there are exactly $2^{\binom{n}{2}}$ labeled graphs on n vertices. To sample a random one, just flip an unbiased coin for each of the $\binom{n}{2}$ potential edges. However, the first polynomial-time algorithm for generating an *unlabeled* graph uniformly at random was only given in 1987, by Wormald [16]. The algorithm of Wormald runs in polynomial time in expectation, and the existence of a worst-case polynomial-time sampler of random unlabeled graphs remains open to this day. Whether there is an expected-polynomial-time *counting* algorithm for unlabeled graphs remains open as well.

1.2.1 Algorithms for various graph classes

It often happens that we want to restrict to a particular graph class, in which case even labeled graph counting can be nontrivial. In the rest of this section, we will survey the known counting and sampling algorithms for various well-known graph classes. The definitions of these graph classes (if not specified here), can be found in the cited papers.

Bipartite graphs. A graph is *bipartite* if its vertex set can be partitioned into two (disjoint) independent sets. Counting labeled bipartite graphs is simple enough that the algorithm has not been explicitly written in the literature, so for completeness we write it out here. A *bipartition* of a bipartite graph G is a pair (L, R) such that $\{L, R\}$ is a partition of $V(G)$ and L and R are both independent sets. We define a *2-colored* labeled bipartite graph as a labeled bipartite graph that comes with a specified bipartition. For $k, c \in \mathbb{N}$ with $c \leq k$, let $f(k, c)$ be the number of 2-colored labeled bipartite graphs on k vertices with exactly c components, and let $g(k, c)$ be the number of 2-colored labeled bipartite graphs on k vertices with at least c components. For $n \in \mathbb{N}$, the number of labeled bipartite graphs on n vertices is equal to

$$\sum_{c=1}^{n} \frac{1}{2^c} f(n, c).$$

The $\frac{1}{2^c}$ term comes from the fact that for each of the c connected components, there are two possible colorings: the vertex with the smallest label in that component can either belong to L or R.

To compute the values of f, we observe that $f(k, k) = g(k, k)$ and $f(k, c) = g(k, c) - g(k, c + 1)$ for all $k \in [n]$, $1 \leq c < k$. The function g satisfies the recurrence

$$g(k, c) = \sum_{k'=1}^{k-1} \binom{k-1}{k'-1} f(k', 1) g(k - k', c - 1)$$

whenever $c > 1$. In this recurrence, k' is the number of vertices in the connected component that contains the label 1. There are $\binom{k-1}{k'-1}$ ways of choosing the labels for that component since it contains $k' - 1$ other labels besides the label 1. After setting aside that component, the remaining graph has $k - k'$ vertices and at least $c - 1$ components. When $c = 1$, we have

$$g(k, 1) = \sum_{i=0}^{n} \binom{n}{i} 2^{i(n-i)}$$

for all $k \in \mathbb{N}$. Here i is the number of vertices in L. There are $\binom{n}{i}$ possible label sets for L and $2^{i(n-i)}$ possibilities for the edges between L and R.

To compute all of these values, we begin with the base cases $f(1,1) = 1$ and $g(1,1) = 1$. Now suppose that for some $\hat{k} \in [n]$, we have computed $f(k,c)$ and $g(k,c)$ for all $1 \leq k < \hat{k}$, $c \in [k]$. We can use the above recurrences to compute $g(\hat{k},c)$ for all $c \in [\hat{k}]$ and then $f(\hat{k},c)$ for all $c \in [\hat{k}]$. Therefore, we can compute $f(n,c)$ for all $c \in [n]$ in $O(n^2)$ arithmetic operations. In [17], Wilf uses similar ideas to derive a generating function for labeled bipartite graphs. However, the focus of [17] is on generating functions, so this book does not give an explicit counting algorithm.

Bipartite permutation graphs. There is a uniform sampling algorithm for connected unlabeled bipartite permutation graphs by Saitoh et al. that runs in $O(n)$ time [18].

Chordal graphs. In Chapter 2, we design polynomial-time algorithms for counting and sampling labeled chordal graphs, as well as faster approximate counting and approximate sampling algorithms. In Chapter 3, we design an expected-polynomial-time algorithm for sampling *unlabeled* chordal graphs. See Section 1.3 for more information about chordal graphs.

Cluster graphs. A *cluster graph* is a graph in which every connected component is a clique. The number of labeled cluster graphs on n vertices is equal to B_n, the nth Bell number, which is defined as the number of partitions of $[n]$. The Bell numbers satisfy the recurrence

$$B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k$$

for all $n \geq 0$, with the base case $B_0 = 1$. Thus B_n can be computed in $O(n^2)$ time. It is also easy to derive from this an $O(n^2)$-time sampling algorithm for labeled cluster graphs on n vertices using the standard sampling-to-counting reduction of [1].

The number of *unlabeled* cluster graphs on n vertices is equal to the number of

partitions of the integer n. To compute this, let $p(m, k)$ be the number of partitions of m into at most k parts, for nonnegative integers m and k. Each partition of m into exactly k parts can be converted to a partition of $m - k$ into at most k parts by subtracting 1 from each part. This is in fact a bijection, so $p(m, k)$ satisfies the recurrence

$$p(m, k) = p(m, k - 1) + p(m - k, k)$$

for all $m, k \in \mathbb{N}$. For the base case, we have $p(0, k) = 1$ and $p(m, 0) = 0$ for all $m, k \in \mathbb{N}$, and $p(m, k) = p(m, m)$ if $k > m$. Thus we can compute $p(n, n)$, the number of n-vertex unlabeled cluster graphs, in $O(n^2)$ time. As before, we can also derive from this a sampling algorithm for unlabeled cluster graphs that runs in $O(n^2)$ time.

Cographs. A *cotree* is a rooted tree whose root is labeled with the number 0 or 1. Each cograph can be uniquely represented by a cotree whose leaves correspond to the vertices of the cograph [19]. Because of this correspondence, known algorithms for counting labeled (resp. unlabeled) rooted trees (with a given number of leaves) can be used to compute the number of labeled (resp. unlabeled) cographs on n vertices.

Interval graphs. Generating functions for labeled interval graphs and various subclasses of interval graphs are stated by Hanlon in [20], but the running time of computing the coefficients is not explicitly stated.

Outerplanar graphs. Bodirsky and Kang presented a polynomial-time exact counting and uniform sampling algorithms for labeled outerplanar graphs in 2006 [21]. These algorithms were also used to derive a uniform sampling algorithm for *unlabeled* outerplanar graphs that runs in expected polynomial time [21].

Permutation graphs. A given permutation graph has a unique representation if it is prime in the sense that all of its modules are trivial [22]. This could perhaps be useful for counting permutation graphs, but it is not clear whether this leads to a polynomial-time algorithm. Such an algorithm is not explicitly written in the literature.

Planar graphs. Bodirsky, Gröpl, and Kang presented a polynomial-time algorithm that uses dynamic programming to exactly compute the number of labeled planar graphs on n vertices and generate a planar graph uniformly at random in time $\widetilde{O}(n^7)$ [23]. This was improved to $O(n^2)$ expected time by Fusy, using a Boltzmann sampler [24]. For 2-connected *unlabeled* planar graphs, an expected-polynomial-time uniform sampling algorithm was presented by Bodirsky, Gröpl, and Kang in 2005 [25], followed by the same result for connected unlabeled cubic planar graphs in 2008 [26].

Proper interval graphs. Generating functions for labeled and unlabeled proper interval graphs are stated by Hanlon in [20], but the running time of computing the coefficients is not explicitly stated. However, there is an explicit formula for counting unlabeled connected proper interval graphs given in [27]. For $n \in \mathbb{N}$, the number of unlabeled connected proper interval graphs on $n+1$ vertices is

$$\frac{1}{2}\left(\frac{1}{n+1}\binom{2n}{n} + \binom{n}{\lfloor n/2 \rfloor}\right).$$

This formula can also be used to derive a uniform sampling algorithm for unlabeled connected proper interval graphs on n vertices. For the sampling algorithm, the running time to output a string representation of the generated graph is $O(n)$, and the running time to generate the graph itself is $O(n+m)$, where m is the number of edges in the output graph [27].

Split graphs. The exact number of labeled split graphs on n vertices can be computed in $O(n^2)$ time, using the explicit formula given in [28]. In Chapter 2, we show that a similar counting algorithm can be used to derive an approximate sampling algorithm for labeled split graphs. Given n and ε, this algorithm generates a random labeled split graph according to a distribution whose total variation distance from the uniform distribution is at most ε, running in $O(n^3 \log n)$ expected time for fixed ε.

Threshold graphs. There is an explicit formula for the number of labeled threshold

graphs using Eulerian numbers. For a permutation π of $[n]$, we say position i with $1 \leq i \leq n-1$ is an *ascent* of π if $\pi(i) < \pi(i+1)$. For nonnegative integers n and k, the *Eulerian number* $\left\langle {n \atop k} \right\rangle$ is defined as the number of permutations of $[n]$ that have exactly k ascents. The Eulerian numbers can be computed using the following formula from [29]:

$$\left\langle {n \atop k} \right\rangle = \sum_{j=0}^{k+1} (-1)^j \binom{n+1}{j} (k-j+1)^n.$$

For $n \geq 2$, the number of labeled threshold graphs on n vertices is

$$\sum_{k=1}^{n-1} (n-k) \left\langle {n-1 \atop k-1} \right\rangle 2^k.$$

A combinatorial proof of this formula can be found in [30], and it can also be derived from the generating functions in [31].

Counting *unlabeled* threshold graphs is very simple. For $n \in \mathbb{N}$, there are 2^{n-1} unlabeled threshold graphs on n vertices. This is because threshold graphs are defined as the graphs that can be constructed from the empty graph by repeatedly adding either an isolated vertex or a dominating vertex, i.e., a vertex that is adjacent to all other vertices. For a threshold graph G with $n \geq 2$ vertices, we know that G contains an isolated vertex if and only if the last operation used to construct G was "add an isolated vertex." We can then undo the last operation by deleting an isolated vertex of G if one exists, and otherwise by deleting the vertex of degree $n-1$. Therefore, given G, we can reconstruct (in reverse order) the sequence of operations used to construct G. For the first vertex that is added to G, an isolated vertex is the same as a dominating vertex. Therefore, there are 2^{n-1} distinct graphs that can be constructed by this process. To sample an n-vertex unlabeled threshold graph uniformly at random, simply generate a random sequence of "isolated vertex" and "dominating vertex" operations of length $n-1$.

Trees. Cayley's formula tells us that there are exactly n^{n-2} labeled trees on n vertices. A uniform sampling algorithm for labeled trees using Prüfer sequences [32] was

discovered in 1918. We can also count the exact number of *unlabeled* trees on n vertices in polynomial time [33, A000055], and there is a uniform sampling algorithm for unlabeled trees that runs in polynomial time [34].

Other graph classes. The problem of counting/sampling n-vertex graphs in polynomial time appears to be open for the following graph classes, even when the graphs are labeled: graphs of bounded degeneracy [35], graphs of bounded treewidth [36], chordal bipartite graphs, distance-hereditary graphs, perfect graphs, strongly chordal graphs, and weakly chordal graphs.

1.2.2 A word about hardness

As mentioned above, there are many graph classes for which a polynomial-time counting algorithm has not yet been discovered. Typically, as is often done for other types of graph problems, a natural response would be to try to prove hardness (e.g., NP-hardness or #P-hardness). However, since the input to each graph counting problem is simply a number n (or a pair of numbers n and k), the standard hardness frameworks do not seem amenable to this. If we were to reduce from a typical NP-complete problem such as Vertex Cover to a graph counting problem, this would require "compressing" a large graph of size n into a small input of size $\log n$. To prove hardness of such problems, it seems likely that an entirely different hardness framework would be needed, perhaps predicated on the intractability of a particular graph counting problem. In this thesis, all of our results are focused on designing algorithms rather than proving hardness.

1.3 Chordal Graphs

Considering that Chapters 2 and 3 deal entirely with chordal graphs, this graph class deserves particular attention. A graph is *chordal* if it does not have a cycle of length

four or more as an induced subgraph. As an example, the graph in Figure 1.1 is chordal, and the graph in Figure 1.2 is not. Discussions of chordal graphs in the literature go as far back as 1958 [37]. In the early days, chordal graphs were also known as *rigit circuit graphs*, *rigid cycle graphs*, and *triangulated graphs* [38, 39, 40].

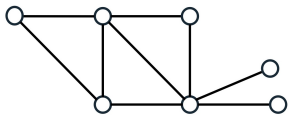

Figure 1.1: A chordal graph.

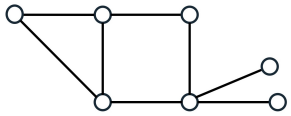

Figure 1.2: A graph that is not chordal.

The class of chordal graphs is a fundamental graph class that is a superclass or subclass of many other important graph classes. All interval graphs, split graphs, threshold graphs, strongly chordal graphs, and trivially perfect graphs are chordal. On the other hand, chordal graphs are perfect, weakly chordal, and odd-hole-free. Chordal graphs have close connections to treewidth, a central concept in parametrized algorithms. In particular, a graph is chordal if and only if it has a tree decomposition in which every bag is a clique [38, 41, 39]. An implication of this is that chordal graphs can be characterized as the intersections graphs of subtrees of a tree [38, 41, 39]. From this point of view, it is clear that interval graphs are chordal, since interval graphs are simply the intersection graphs of subpaths of a path. Like interval graphs, chordal graphs can be recognized in linear time [42, 43].

Many NP-hard optimization problems (such as *coloring* [40] and *maximum independent set* [44]), as well as #P-hard counting problems (such as *independent sets* [45, 46]), and many others [47], are polynomial-time solvable on chordal graphs. Chordal graphs have a wide variety of applications, including phylogeny in evolutionary biology [48, 49] and Bayesian networks in machine learning [50]. When doing Gaussian elimination on a symmetric matrix, the set of matrix entries that are nonzero for at least one time-point of

the elimination process corresponds to the edge set of a chordal graph. Thus the problem of finding an ordering in which to do Gaussian elimination that minimizes the number of nonzero matrix entries can be reduced to finding a chordal supergraph of a given graph with the minimum number of edges [51]. This problem, known as *minimum fill-in*, was shown to be NP-complete by Yannakakis [52].

An interesting and relevant result by Bender et al. [53] is that a random n-vertex labeled chordal graph is a split graph with probability $1 - o(1)$, i.e., the fraction of labeled chordal graphs that are not split is $o(1)$. This suggests the possibility of a simple *approximately uniform* sampler for labeled chordal graphs: simply sampling a random labeled split graph leads to an output distribution with total variation distance $o(1)$ from the uniform distribution on labeled chordal graphs. This is the main inspiration behind our approximate sampling algorithm for labeled chordal graphs in Chapter 2.

1.4 Parameterized Graph Counting Algorithms

For a given graph class $\mathcal{G}$, Vertex Deletion to $\mathcal{G}$ is the decision problem where the input is a graph G and a nonnegative integer k and the goal is to decide whether there exists a vertex subset $S \subseteq V(G)$ of size at most k such that $G \setminus S \in \mathcal{G}$. For example, when $\mathcal{G}$ is the class of bipartite graphs, Vertex Deletion to $\mathcal{G}$ is known as Odd Cycle Transversal. For forests and independent set graphs, the corresponding problems are Feedback Vertex Set and Vertex Cover, respectively.

In Chapter 4, we will design FPT algorithms for counting yes-instances of Vertex Deletion to $\mathcal{G}$ for various graph classes $\mathcal{G}$ (including independent set graphs, cluster graphs, and split graphs). These algorithms are inspired by the FPT algorithms and kernels for the corresponding vertex deletion problems (Vertex Cover, Cluster Vertex Deletion, and Split Vertex Deletion), parameterized by the number of

vertices deleted. For this reason, to facilitate future work on counting yes-instances of other vertex deletion problems, we survey what is known in terms of the kernel sizes and FPT algorithm running times for vertex deletion to various well-known graph classes.

Table 1.1: For each graph class $\mathcal{G}$, we state the size of the best-known kernel and the running time of the fastest-known FPT algorithm for Vertex Deletion to $\mathcal{G}$, if possible. For some graph classes, this problem is W[·]-hard. The parameter k is the number of vertices deleted.

Graph class	**Kernel size**	**FPT algorithm**
Bipartite[1]	$k^{O(1)}$ [54]	$2.3146^k n^{O(1)}$ [55]
Bipartite permutation	$k^{O(1)}$ [56]	$O(9^k n^9)$ [57]
Bounded degree (d)	$O(k)$ [58]	$(d+1)^k n^{O(1)}$ [59]
Bounded degeneracy ($r \geq 2$)	W[P]-hard [60]	W[P]-hard [60]
Bounded treewidth	$k^{O(1)}$ [61]	$2^{O(k)}n$ [61]
Chordal	$O(k^{12}\log^{10} k)$ [62]	$2^{O(k \log k)}n^{O(1)}$ [63]
Chordal bipartite	Open	Open
Cluster	$O(k^{5/3})$ [64]	$1.811^k n^{O(1)}$ [65]
Cograph[2]	$O(k^4)$	$3.08^k n^{O(1)}$ [66]
Distance-hereditary	$O(k^{20}\log^{15} k)$ [67]	$37^k n^{O(1)}$ [68]
Forest	$4k^2$ [69]	$3.83^k n^{O(1)}$ [70]
Independent set	$2k$ [71]	$1.25284^k n^{O(1)}$ [72]
Interval	$k^{O(1)}$ [73]	$8^k(n+m)$ [74]
Outerplanar	$O(k^4)$ [75]	$2^{O(k)}n$ [61]
Perfect	W[2]-hard [76]	W[2]-hard [76]

Continued on next page

[1]The polynomial kernel for bipartite graphs is randomized.

[2]The $O(k^4)$ kernel for 4-Hitting Set that uses the sunflower lemma [36] implies an $O(k^4)$ kernel for cographs, since cographs can be characterized as P_4-free graphs.

Table 1.1 – continued from previous page

Graph class	Kernel size	FPT algorithm
Permutation	Open [56]	Open [56]
Planar	Open [77]	$2^{O(k \log k)}n$ [78]
Proper interval	$k^{O(1)}$ [79]	$2^{O(k \log k)}(n+m)$ [80]
Split	$O(k^3)$ [81]	$1.25284^k k^{O(\log k)} n^{O(1)}$ [82]
Strongly chordal	Open	Open
Threshold[3]	$O(k^4)$	$3.08^k n^{O(1)}$ [66]
Trivially Perfect[4]	$O(k^4)$	$3.08^k n^{O(1)}$ [66]
Weakly chordal	W[2]-hard [76]	W[2]-hard [76]

In the "Kernel size" column, we have listed the bound on the number of vertices. For example, Vertex Cover admits a kernel with at most $2k$ vertices. In this column, "Open" means the existence of a polynomial kernel is open. In the "FPT algorithm" column, "Open" means it is not known whether this problem has an FPT algorithm or is W[1]-hard. For the bounded degeneracy graph class (i.e., graphs of degeneracy at most r), the W[P]-hardness result holds for $r \geq 2$.

1.5 Notation and Definitions

In Section 1.2, we defined graphs, and we discussed what it means for a graph to be labeled or unlabeled. The graphs in those definitions are simple (no multiple edges or self-loops) and undirected. In Chapter 4, we will also work with directed graphs. A

[3]The $O(k^4)$ kernel for 4-Hitting Set that uses the sunflower lemma [36] implies an $O(k^4)$ kernel for threshold graphs, since these graphs can be characterized as $(C_4, P_4, 2K_2)$-free graphs.

[4]The $O(k^4)$ kernel for 4-Hitting Set that uses the sunflower lemma [36] implies an $O(k^4)$ kernel for trivially perfect graphs, since these graphs can be characterized as (C_4, P_4)-free graphs.

directed graph is a set $V(G)$ of *vertices* together with a set $E(G)$ of ordered pairs of distinct vertices called *edges*. As before, the vertex set $V(G)$ is finite, and each edge is denoted as a pair (u, v). From now on until Chapter 4, all graphs will be undirected.

Sets and functions. Let $\mathbb{N}$ be the set of natural numbers $\{1, 2, 3, \ldots\}$. For nonnegative integers n, we use the notation $[n] \coloneqq \{1, 2, \ldots, n\}$. We also define $[a, b] \coloneqq \{a, a+1, \ldots, b\}$ for nonnegative integers a, b. Naturally, $[0] = \emptyset$ and $[a, b] = \emptyset$ if $b < a$.

Let $A = \{a_1, \ldots, a_r\}$ and $B = \{b_1, \ldots, b_r\}$ be finite subsets of $\mathbb{N}$ such that $|A| = |B|$, where the elements a_i and b_i are listed in increasing order. We define $\phi(A, B)\colon A \to B$ as the bijection that maps a_i to b_i for all $i \in [r]$.

Graphs. Suppose G is a graph. We say the vertices $u, v \in V(G)$ are *adjacent* if $(u, v) \in E(G)$, and we say the vertices u and v are *incident* to the edge (u, v). We also refer to u and v as the *endpoints* of the edge (u, v). The *neighborhood* of a vertex v is the set $N_G(v) \coloneqq \{u \in V(G) : (u, v) \in E(G)\}$ of vertices adjacent to v, and the *degree* of v is $\deg_G(v) \coloneqq |N_G(v)|$. For a vertex subset $S \subseteq V(G)$, the *neighborhood* of S is $N_G(S) \coloneqq (\bigcup_{v \in S} N_G(v)) \setminus S$. When the graph G is clear from the context, we omit the subscript. For $S, T \subseteq V(G)$, we say S *sees all of* T if $T \subseteq N(S)$.

A *subgraph* of G is a graph G' such that $V(G') \subseteq V(G)$ and $E(G') \subseteq E(G)$. For $S \subseteq V(G)$, we write $G \setminus S$ to denote the graph with vertex set $V(G) \setminus S$ and edge set $\{(u, v) \in E(G) : u, v \notin S\}$. An *induced* subgraph is a subgraph that can be written as $G' = G \setminus S$ for some $S \subseteq V(G)$. For a set $S \subseteq V(G)$, $G[S]$ denotes the induced subgraph with vertex set S and edge set $\{(u, v) \in E(G) : u, v \in S\}$. This called the *subgraph induced* by S. A *clique* is a set $S \subseteq V(G)$ such that $(u, v) \in E(G)$ for all $u, v \in S$, and an *independent set* is a set $S \subseteq V(G)$ such that $(u, v) \notin E(G)$ for all $u, v \in S$. A set $S \subseteq V(G)$ is called a *vertex cover* if $u \in S$ or $v \in S$ for every edge $(u, v) \in E(G)$.

A *walk* is a sequence of vertices $v_1, \ldots, v_\ell$ such that $(v_i, v_{i+1}) \in E(G)$ for all $1 \leq i < \ell$. A *path* is a walk in which the vertices are all distinct, and a *cycle* is a walk where the first

vertex is the same as the last but all others are distinct. The *length* of the path or cycle is the number of distinct vertices. The graph G is *connected* if there exists a path from u to v for every pair of vertices $u, v \in V(G)$. An induced subgraph $G[S]$ is a *connected component* of G if $G[S]$ is connected and there is no larger set $\hat{S} \supsetneq S$ such that $G[\hat{S}]$ is connected. A *non-edge* is a pair of distinct vertices $u, v \in V(G)$ such that $(u, v) \notin E(G)$.

Parameterized algorithms. For the definitions of parameterized algorithms terminology, including FPT algorithms, kernels, and W[1]-hardness, see [36].

Partitions and split graphs. In this thesis, a *partition* of a set is defined to allow empty parts. In particular, a partition of a set S is a collection of disjoint sets whose union is S. A *true partition* is a partition in which no part is empty.

A *split* graph is a graph whose vertex set can be partitioned into a clique and an independent set, with arbitrary edges between the two parts. A *split partition* of a graph G is a pair (C', I') such that C' is a clique, I' is an independent set, and $\{C', I'\}$ is a partition of $V(G)$. In other words, this is a partition (with labeled parts) that demonstrates that G is split. The split partition of a split graph is not unique, but it is possible to uniquely partition $V(G)$ into three parts in the following way.

Definition 1 *Let G be a split graph. The **always-clique set** of G is the subset of $V(G)$ consisting of vertices that belong to the clique in every split partition of G. The **always-independent set** of G is the set of vertices that belong to the independent set in every split partition of G. The **questioning set** of G is the set of vertices v such that there exists a split partition that places v in the clique as well as a split partition that places v in the independent set.*

These sets will typically be denoted by C, I, and Q, respectively. As an example, if G is a complete graph, then all of $V(G)$ belongs to Q.

1.6 Permissions and Attributions

Some of the research in this thesis has appeared in conference proceedings. We give the details of these conference articles below.

1. The contents of Chapter 2 are the result of a collaboration with Daniel Lokshtanov and Eric Vigoda. This work appeared in the *European Symposium on Algorithms (ESA 2023)* [83] and is available online at `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2023.58`.

2. The contents of Chapter 3 are the result of a collaboration with Daniel Lokshtanov. This work appeared in the *International Symposium on Theoretical Aspects of Computer Science (STACS 2025)* [84] and is available online at `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.STACS.2025.46`.

Chapter 2

Counting and Sampling Labeled Chordal Graphs in Polynomial Time

2.1 Introduction

In this chapter, we consider the problem of generating a labeled chordal graph on n vertices uniformly at random. Despite being one of the most fundamental and well-studied graph classes, prior to our work, the fastest uniform sampling algorithm for labeled chordal graphs was the exponential-time algorithm of Wormald from 1985 [85]. (To be precise, optimizing the running time of an algorithm for counting and sampling chordal graphs was not the main focus of Wormald; rather, the main goal was to determine the asymptotic number of chordal graphs with given connectivity, and the exponential-time algorithm is a corollary of these results.) Since then, various algorithmic approaches have been proposed for generating chordal graphs (e.g., [86, 87, 15, 88, 89]), but these algorithms do not come with any formal guarantees about their output distribution. In particular, [87] specifically asks for the existence of a polynomial-time algorithm to sample chordal graphs uniformly at random as an open problem. In a recent abstract, Sun

and Bezáková [15] proposed a Markov chain for sampling chordal graphs, but this Markov chain comes with few mixing time guarantees.

We obtain the first polynomial (in n) time algorithm to exactly count the number of labeled chordal graphs on n vertices, as well as the first polynomial-time uniform sampling algorithm for the class of labeled chordal graphs. Our algorithm also easily extends to counting and sampling ω-*colorable* labeled chordal graphs. A graph G is ω-colorable if there exists a function $c\colon V(G) \to \{1, \ldots, \omega\}$ such that every edge $(u, v) \in E(G)$ satisfies $c(u) \neq c(v)$.

Theorem 1 *There is a deterministic algorithm that given positive integers n and $\omega \leq n$, computes the number of ω-colorable labeled chordal graphs on n vertices using $O(n^7)$ arithmetic operations. Moreover, there is a randomized algorithm that given the same input, generates a graph uniformly at random from the set of all ω-colorable labeled chordal graphs on n vertices using $O(n^7)$ arithmetic operations.*

By the known equivalence between chromatic number, maximum clique size, and treewidth of chordal graphs [90], Theorem 1 can be reinterpreted as counting and sampling labeled chordal graphs of clique size at most ω, or treewidth at most $\omega - 1$. The running time bound of Theorem 1 is stated in terms of the number of arithmetic operations. Since there are at most 2^{n^2} labeled graphs on n vertices, the arithmetic operations need to deal with n^2-bit integers. Therefore, using the $O(n \log n)$-time algorithm for multiplying two n-bit integers [91] yields an $O(n^9 \log n)$-time upper bound for our algorithm in the RAM model.

A straightforward implementation of our counting algorithm gives the number of labeled chordal graphs on up to $n = 30$ vertices in less than three minutes on a standard desktop computer. Previously, the number of labeled chordal graphs was only known for graphs on up to 15 vertices [33]. In addition, we use our implementation to compute the

number of ω-colorable labeled chordal graphs for $n \leq 12$ and $\omega \leq 12$. We chose to stop at $n = 12$ to keep the table at a reasonable size, not because of the computation time. We present these computational results in Section 2.4.

In addition, we design two approximation algorithms: (1) an approximate counting algorithm that computes a $(1 \pm \varepsilon)$-approximation of the number of n-vertex labeled chordal graphs, and (2) an approximate sampling algorithm that generates a random labeled chordal graph according to a distribution whose total variation distance from the uniform distribution is at most ε. The approximate counting algorithm runs in $O(n^3 \log n \log^7(1/\varepsilon))$ time, and the approximate sampling algorithm runs in $O(n^3 \log n \log^7(1/\varepsilon))$ expected time.

2.1.1 Methods

Our exact counting algorithm is based on dynamic programming. While clique trees and tree decompositions are never explicitly mentioned in the description of the algorithm, the intuition behind the algorithm is based on these notions. Essentially, we generate a rooted clique tree where the dynamic-programming table encodes certain properties of the graph, including how it relates to the root node of the clique tree. A *clique tree* of a graph G is a tree T together with a function f that maps each vertex of G to a connected vertex subset of T, such that for every pair of vertices $u, v \in V(G)$, (u, v) is an edge in G if and only if $f(u) \cap f(v) \neq \emptyset$. It is well known that a graph G has a clique tree if and only if it is chordal [90].

The main difficulty with this approach is that different chordal graphs have different numbers of clique trees, so if we count the total number of clique trees, this will not give us an accurate count of the number of n-vertex chordal graphs. Therefore, the key idea behind our algorithm is to assign to each labeled chordal graph a unique "canonical"

clique tree and to only count these canonical clique trees. The information stored in the dynamic-programming table is sufficient to ensure that every clique tree that we do count is the canonical tree of some chordal graph, and that the canonical clique tree of every chordal graph is counted. As it turns out, the best way to phrase our algorithm is not in terms of clique trees at all, but rather in terms of an (essentially) equivalent notion that we call an "evaporation sequence." An evaporation sequence is closely related to the standard notion of a perfect elimination ordering (PEO) of a chordal graph. A *simplicial* vertex is a vertex whose neighborhood is a clique, and a PEO is an ordering of the vertices such that each is simplicial in the current induced subgraph, if we delete the vertices in that order (see Section 2.3.1 for more details). An evaporation sequence is a type of "canonical" PEO: at each step, we remove *all* simplicial vertices from the current subgraph, rather than making an arbitrary choice of a single simplicial vertex. We say that all of the simplicial vertices evaporate at time 1. Next, all of the vertices that become simplicial once the first set of simplicial vertices has been removed are said to evaporate at time 2, and so on. It is easy to see that every labeled chordal graph has a unique evaporation sequence, and that this sequence does not depend on the labeling of the vertices.

Therefore, the number of chordal graphs on n vertices is the sum over all evaporation sequences of the number of labeled chordal graphs with that evaporation sequence. While different evaporation sequences correspond to different numbers of chordal graphs, because this number is independent of the labels, we can simply guess the *number* x of vertices that evaporate at any given time, and then without loss of generality assign the labels $1, \ldots, x$ to those vertices.

In our dynamic-programming algorithm, the recursive subproblems deal with counting *rooted* clique trees. In this context, the root of a clique tree is the set of vertices that evaporate last. Just as we would like to "force" a set of nodes to be in the root of the

tree, we will sometimes want to force a set of nodes to evaporate last. This is done using what we call an *exception set*, i.e., a set of vertices that do not evaporate even if they are simplicial. The random sampling algorithm in Theorem 1 follows from our counting algorithm using the standard sampling-to-counting reduction of [1].

Our approximate counting algorithm is based on the result of Bender et al. [53] mentioned above, which states that almost every labeled chordal graph is a split graph. When n is large enough (compared to a function of $\frac{1}{\varepsilon}$), simply counting (even approximately) the number of labeled split graphs leads to a $(1 \pm \varepsilon)$-approximation of the number of labeled chordal graphs. Similarly, sampling a random labeled split graph gives an approximately uniform sampler of labeled chordal graphs.

2.1.2 Chapter overview

Our dynamic-programming algorithm, including the associated recurrences, is presented in Section 2.3. The proof of correctness of the counting algorithm can be found in Section 2.3.4 (here we reduce to counting connected chordal graphs) and Section 2.5 (here we establish the recurrences). In Section 2.4, one can find the tables of numbers produced by our implementation of the counting algorithm. In Section 2.6, we describe the details of how the sampling algorithm follows from the counting algorithm. Lastly, in Section 2.7, we describe our approximate counting and approximate sampling algorithms.

2.2 Preliminaries

In this chapter, we implicitly assume *all graphs that we consider are labeled graphs.*

Definition 2 *Let G_1, G_2 be two graphs, and suppose $C \coloneqq V(G_1) \cap V(G_2)$ is a clique in both G_1 and G_2. When we say we **glue** G_1 and G_2 together at C to obtain G, this means*

G is the union of G_1 and G_2: the vertex set is $V(G) = V(G_1) \cup V(G_2)$, and the edge set is $E(G) = E(G_1) \cup E(G_2)$.

2.3 Counting Labeled Chordal Graphs

2.3.1 The evaporation sequence

A vertex v in a graph G is *simplicial* if $N(v)$ is a clique. A *perfect elimination ordering* of a graph G is an ordering $v_1, \ldots, v_n$ of the vertices of G such that for all $i \in [n]$, v_i is simplicial in the subgraph induced by the vertices $v_i, \ldots, v_n$. It is well known that a graph is chordal if and only if it has a perfect elimination ordering [90]. For our counting algorithm, we define the notion of the evaporation sequence of a chordal graph, which can be viewed as a canonical version of the perfect elimination ordering. In the evaporation sequence, rather than making an arbitrary choice for which of the simplicial vertices in G will go first in the ordering, we place the set of all simplicial vertices as the first item in the sequence. As an example, if G is a tree, then the set of all simplicial vertices would be exactly the leaves of G.

To build the evaporation sequence, we use the fact that every chordal graph contains a simplicial vertex [90]. This means that given a chordal graph G, if we repeatedly remove all simplicial vertices, then eventually no vertices remain. By observing at which time step each vertex is deleted, we obtain a partition of $V(G)$, which allows us to classify and understand the structure of G. In our algorithm, it will also be useful to set aside a set of exceptional vertices which are never deleted, even if they are simplical.

To formalize this, suppose we are given a chordal graph G and a clique $X \subseteq V(G)$. We define the *evaporation sequence* of G with *exception set* X as follows: If $X = V(G)$, then the evaporation sequence of G is the empty sequence. If $X \subsetneq V(G)$, then let $\widetilde{L}_1$

be the set of all simplicial vertices in G, and let $L_1 = \widetilde{L}_1 \setminus X$. Suppose $L_2, \ldots, L_t$ is the evaporation sequence of $G \setminus L_1$ (with exception set X). Then $L_1, L_2, \ldots, L_t$ is the evaporation sequence of G.

To see that this is well-defined, we need to check that $\widetilde{L}_1 \setminus X$ is nonempty whenever $X \subsetneq V(G)$, to ensure that we eventually reach the base case $X = V(G)$. This follows from the fact that every chordal graph that is not a complete graph contains two non-adjacent simplicial vertices [90]. Suppose $\widetilde{L}_1 \setminus X = \emptyset$. This means $\widetilde{L}_1$ is a clique since X is a clique, so G does not contain any non-adjacent simplicial vertices. Thus G must be a complete graph, in which case $\widetilde{L}_1 \subseteq X$ implies $X = V(G)$. Therefore, we always reach the base case $X = V(G)$.

If the evaporation sequence $L_1, L_2, \ldots, L_t$ of G has length t, then we say G *evaporates* at time t with *exception set* X, and t is called the *evaporation time*. We define $L_G(X) \coloneqq L_t$ to be the last set in the evaporation sequence of G, and we let $L_G(X) = \emptyset$ if the sequence is empty. Similarly, we define the evaporation time of a vertex subset. Suppose G has evaporation sequence $L_1, L_2, \ldots, L_t$ with exception set X, and suppose $S \subseteq V(G) \setminus X$ is a nonempty vertex subset. Let t_S be the largest index i such that $L_i \cap S \neq \emptyset$. We say S *evaporates* at time t_S in G with exception set X.

2.3.2 Setup for the counting algorithm

Given positive integers n and ω, we wish to count the number of ω-colorable chordal graphs on n vertices. This can easily be reduced to the problem of counting *connected* ω-colorable chordal graphs on at most n vertices (see Lemma 10 in Section 2.3.4). Therefore, our main focus is to describe the following algorithm:

Theorem 2 *There is an algorithm that given $n \in \mathbb{N}$, computes the number of ω-colorable labeled connected chordal graphs with vertex set $[n]$ using $O(n^7)$ arithmetic operations.*

We first give an overview of this algorithm and the various dynamic-programming tables (Definition 3). Next, we describe the recurrences in detail in Section 2.3.3. In Section 2.3.4, we show that the counting portion of Theorem 1 (counting chordal graphs) follows from Theorem 2 (counting connected chordal graphs). In Section 2.5, we prove correctness of the recurrences and complete the proof of Theorem 2.

Algorithm overview. To count ω-colorable connected chordal graphs G, we classify these graphs based on the behavior of their evaporation sequence. We make use of several *counter functions* (these are our dynamic-programming tables), each of which keeps track of the number of chordal graphs in a particular subclass. The arguments of the counter functions tell us the number of vertices in the graph, the evaporation time, the size of the exception set X, the size of the last set of simplicial vertices $L_G(X)$, etc. Initially, we consider all possibilities for the evaporation time of G with exception set $X = \emptyset$. Then, using several of our recursive formulas, we reduce the number of vertices by dividing up the graph into smaller subgraphs and counting the number of possibilities for each subgraph. As we do so, the exception set X increases in size. When we consider these various subgraphs, we also make sure that in each subgraph, the maximum clique size is at most ω. In the end, the algorithm understands the possibilities for the entire graph, including the cliques that make up the very first set in its evaporation sequence.

The purpose of the exception set is to allow us to restrict to smaller subgraphs without distorting the evaporation behavior of the graph. For example, suppose we wish to count the number of connected chordal graphs on n vertices that evaporate at time t, such that the vertices $1, 2, \ldots, \ell$ make up the last set to evaporate, i.e., $L_G(\emptyset) = [\ell]$. Let $L = [\ell]$. One subproblem of interest would be to count the number of possibilities for the first connected component of $G \setminus L$. Formally, we count the number of possibilities for $G' \coloneqq G[L \cup C]$, where C is the connected component of $G \setminus L$ that contains the vertex

$\ell + 1$. For each possible number of vertices in G', we make a recursive call to count the number of possible subgraphs G' of that size. However, if we were to restrict to G' with a still-empty exception set, then the evaporation time of G' alone could be much less than the evaporation time of $V(G')$ in G. Indeed, there may be vertices in $G \setminus G'$ adjacent to L that prevent L from evaporating before time t, so when we restrict to the subgraph G', L would now evaporate too soon. This would cause a cascading effect, causing vertices near L to evaporate as well, and changing the entire evaporation sequence of G'. To resolve this, we add the vertices of L to the exception set to preserve the evaporation behavior of G'.

The list of counter functions is given in Definition 3; see Figure 2.1 for an illustration. In the figure, an arrow from one function to another, say from $\tilde{g}$ to g, indicates that the definition of $\tilde{g}$ is the same as that of g except where indicated otherwise. As shown below, the number of ω-colorable connected chordal graphs on n vertices is the sum of various calls to the fourth function $\tilde{g}_1$, since $\tilde{g}_1(t, 0, n)$ is the number of ω-colorable connected chordal graphs on n vertices that evaporate at time t with empty exception set.

To remember the names of these counter functions, one can think of them as follows. The g-functions keep track of the size of the exception set X, but these do not have information about the size of $L_G(X)$. The f-functions have an additional argument ℓ, which is the size of $L_G(X)$. As a mnemonic, one can say that g stands for "glued" and f stands for "free." In the g-functions, X is the "root," and all of the vertices of X are glued, in the sense that they cannot evaporate. In the f-functions, $X \cup L_G(X)$ is the "root," and some of the vertices in the root are free, since the vertices in $L_G(X)$ are allowed to evaporate. For functions with a tilde, the connected components of $G \setminus X$ (for g-functions) or $G \setminus (X \cup L(G, X))$ (for f-functions) all evaporate at the same time. Lastly, a subscript p means that the neighborhoods of the connected components of $G \setminus X$ (or $G \setminus (X \cup L(G, X))$), when intersected with X (or $X \cup L(G, X)$), are *proper* subsets of X

(or $X \cup L(G, X)$). For all of these functions, we only consider graphs that are ω-colorable.

Definition 3 *The following functions count various subclasses of chordal graphs. The arguments t, x, ℓ, k, z are nonnegative integers. They take on values up to at most n and satisfy the domain requirements listed below.*

1. *$g(t, x, k, z)$ is the number of ω-colorable connected chordal graphs G with vertex set $[x + k]$ that evaporate in time at most t with exception set $X \coloneqq [x]$, where X is a clique, with the following property: every connected component of $G \setminus X$ (if any) has at least one neighbor in $X \setminus [z]$.* ***Domain:*** *$t \geq 0$, $x \geq 1$, $z < x$.*

2. *$\tilde{g}(t, x, k, z)$ is the same as $g(t, x, k, z)$, except every connected component of $G \setminus X$ (if any) evaporates at time exactly t in G.* Note: A graph with $V(G) = X$ would be counted because in that case, $\tilde{g}$ is the same as g. ***Domain:*** *$t \geq 1$, $x \geq 1$, $z < x$.*

3. *$\tilde{g}_p(t, x, k, z)$ is the same as $\tilde{g}(t, x, k, z)$, except no connected component of $G \setminus X$ sees all of X.* ***Domain:*** *$t \geq 1$, $x \geq 1$, $z < x$.*

4. *$\tilde{g}_1(t, x, k)$ and $\tilde{g}_{\geq 2}(t, x, k)$ are the same as $\tilde{g}(t, x, k, z)$, except every connected component of $G \setminus X$ sees all of X (hence we no longer require every component of $G \setminus X$ to have a neighbor in $X \setminus [z]$), and furthermore, for $\tilde{g}_1$ we require that $G \setminus X$ has exactly one connected component, and for $\tilde{g}_{\geq 2}$ we require that $G \setminus X$ has at least two components.* ***Domain for $\tilde{g}_1$:*** *$t \geq 1$, $x \geq 0$.* ***Domain for $\tilde{g}_{\geq 2}$:*** *$t \geq 1$, $x \geq 1$.*

5. *$f(t, x, \ell, k)$ is the number of ω-colorable connected chordal graphs G with vertex set $[x + \ell + k]$ that evaporate at time exactly t with exception set $X \coloneqq [x]$, such that $G \setminus X$ is connected, $L_G(X) = [x + 1, x + \ell]$, and $X \cup L_G(X)$ is a clique.* ***Domain:*** *$t \geq 1$, $x \geq 0$, $\ell \geq 1$.*

6. *$\tilde{f}(t, x, \ell, k)$ is the same as $f(t, x, \ell, k)$, except every connected component of $G \setminus (X \cup L_G(X))$ evaporates at time exactly $t - 1$ in G, and there exists at least one such component, i.e., $X \cup L_G(X) \subsetneq V(G)$.* ***Domain:*** *$t \geq 2$, $x \geq 0$, $\ell \geq 1$.*

7. *$\tilde{f}_p(t, x, \ell, k)$ is the same as $\tilde{f}(t, x, \ell, k)$, except no connected component of $G \setminus (X \cup L_G(X))$ sees all of $X \cup L_G(X)$.* ***Domain:*** *$t \geq 2$, $x \geq 0$, $\ell \geq 1$.*

8. *$\tilde{f}_p(t, x, \ell, k, z)$ is the same as $\tilde{f}_p(t, x, \ell, k)$, except rather than requiring that $G \setminus X$ is connected, we require that $G \setminus [z]$ is connected.* ***Domain:*** *$t \geq 2$, $x \geq 0$, $\ell \geq 1$, $z \leq x$.*

2.3.3 Recurrences for the counting algorithm

We implicitly assume *all graphs in this section are connected and ω-colorable.* For $k \in \mathbb{N}$, let $c(k)$ denote the number of ω-colorable connected chordal graphs with vertex set $[k]$. To compute $c(n)$, we first consider all possibilities for the evaporation time. Initially, the exception set is empty. We observe that $\tilde{g}_1(t, 0, n)$ is the number of (connected, ω-colorable) chordal graphs with vertex set $[n]$ that evaporate at time exactly t with empty exception set. Therefore,

$$c(n) = \sum_{t=1}^{n} \tilde{g}_1(t, 0, n).$$

We compute the necessary values of $\tilde{g}_1$ by evaluating the following recurrences top-down using memoization. We take this approach rather than bottom-up dynamic programming to simplify the description slightly, since by memoizing we do not need to specify in what order the entries of the various dynamic-programming tables are computed. We simply compute each value of the counter functions as needed. For the following recurrences, let $X = [x]$ according to the current value of the argument x, and let $L = L_G(X)$.

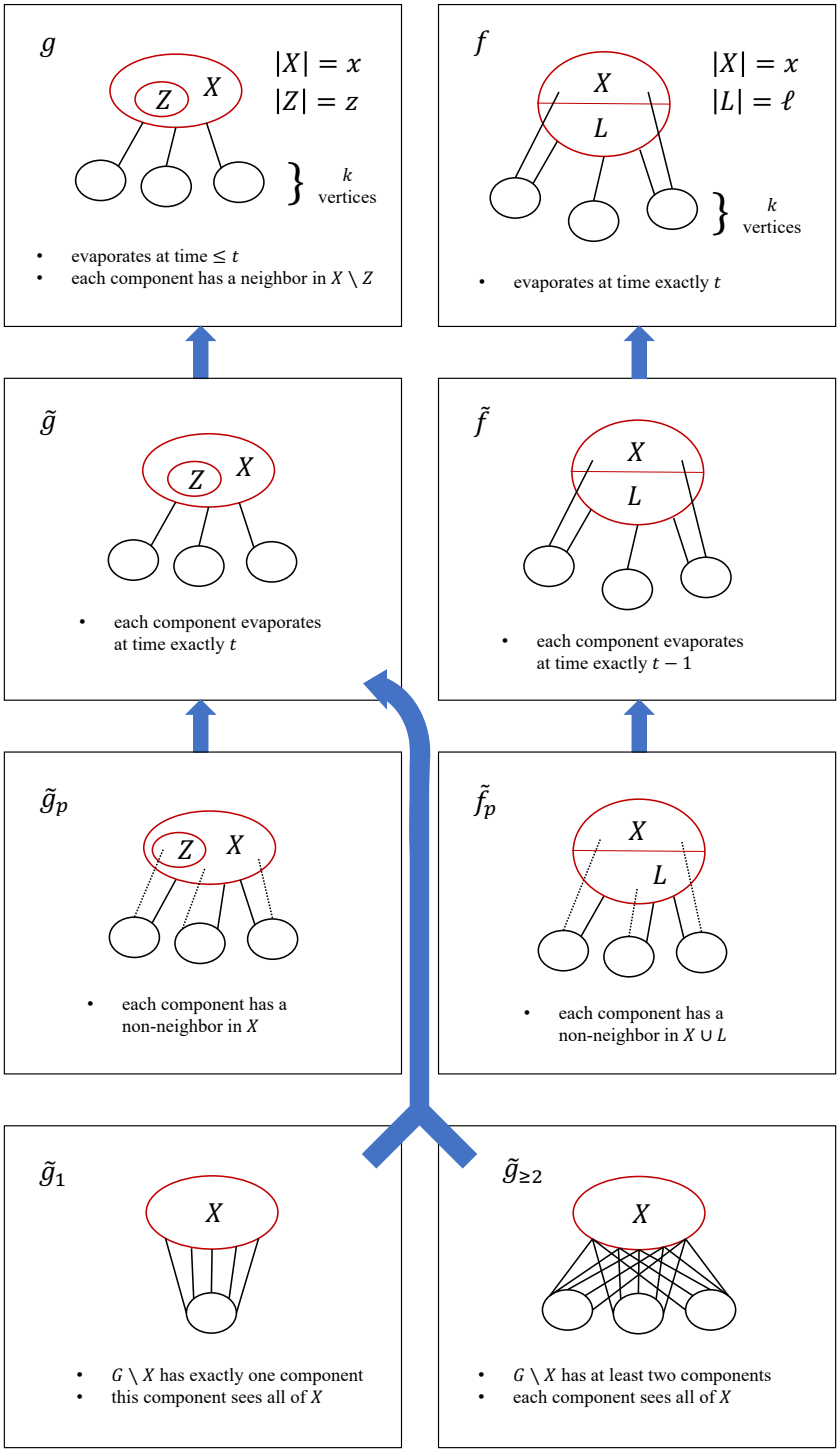

Figure 2.1: The counter functions. Here $L = L_G(X)$ and $Z = [z]$. The drawing of $\tilde{f}_p$ represents $\tilde{f}_p(t, x, \ell, k)$.

To compute $\tilde{g}_1(t, 0, n)$, the number of chordal graphs that evaporate at time exactly t, we consider all possibilities for the size ℓ of L. Once ℓ is given, there are $\binom{n}{\ell}$ possibilities for the label set of L. Recall that f counts the number of chordal graphs where L is fixed and evaporates at time t, and $G \setminus X$ is connected. Formally, we have the following recurrence — the first time this is used, the arguments are t, $x = 0$, and $k = n$.

Lemma 1 *For $\tilde{g}_1$, we have*

$$\tilde{g}_1(t, x, k) = \sum_{\ell=1}^{k} \binom{k}{\ell} f(t, x, \ell, k - \ell).$$

The proof of Lemma 1, along with the proofs of all of the following recurrences, can be found in Section 2.5.3. We proceed in this order, first presenting the recurrences and later presenting their proofs, to first give the reader the context of the entire algorithm. For now, to see the intuition behind Lemma 1, recall that in the definition of $f(t, x, \ell, k)$ we require a specific label set for $L_G(X)$, namely $[x + 1, x + \ell]$. If we were to replace that requirement with $L_G(X) = L'$ for any other subset L' of $[x + 1, x + \ell + k]$ of size ℓ, this would not change the value of $f(t, x, \ell, k)$. Therefore, in the recurrence for $\tilde{g}_1$, it is sufficient to compute $f(t, x, \ell, k - \ell)$ and multiply by $\binom{k}{\ell}$, rather than computing $\binom{k}{\ell}$ distinct counter functions.

For f, to count chordal graphs where L is fixed and evaporates at time t, and $G \setminus X$ is connected, we consider all possibilities for the set of labels that appear in connected components of $G \setminus (X \cup L)$ that evaporate at time exactly $t - 1$. Recall that $\tilde{f}$ is the same as f, except all components of $G \setminus (X \cup L)$ evaporate at time exactly $t - 1$. For each possible k' (the size of the chosen label set), $\tilde{f}$ allows us to count the number of possibilities for the subgraph G_1 consisting of $X \cup L$ and all components of $G \setminus (X \cup L)$ that evaporate at time exactly $t - 1$. The function g allows us to count the number of possibilities for the subgraph G_2 consisting of $X \cup L$ and all other components.

Lemma 2 *For f, we have*

$$f(t,x,\ell,k) = \sum_{k'=1}^{k} \binom{k}{k'} \tilde{f}(t,x,\ell,k') g(t-2, x+\ell, k-k', x).$$

When f is called for the first time in the initial steps of the algorithm, this is the first moment when X becomes nonempty (when we call the function g), since at this point we are restricting to a subgraph with fewer than n vertices. When we restrict to the subgraph G_2, we want to ensure that its vertices have the same evaporation behavior as they did in G. In particular, we need to ensure that the vertices of L do not evaporate too soon, since their presence may be essential for preventing other vertices from evaporating. For this reason, we let $X \cup L$ be the exception set for G_2. For G_1, the exception set is simply X because the components that evaporate at time exactly $t-1$ are still present in G_1, preventing the vertices of L from evaporating before time t.

For the subgraph G_2, now that all of L has been pushed into the exception set, we no longer have information about the argument ℓ since the last set of simplicial vertices in G_2 evaporates further back in time. This is why we call g rather than f to count the possibilities for G_2. In fact, $G_2 \setminus [x+\ell]$ might not even be connected, which is required by f. Finally, the fourth argument of g indicates that every connected component of $G_2 \setminus (X \cup L)$ has at least one neighbor in L. This ensures that $G \setminus X$ is connected.

For g, to count chordal graphs that evaporate in time at most t, we consider all possibilities for the set of labels that appear in connected components of $G \setminus X$ that evaporate at time exactly t. Recall that $\tilde{g}$ counts the number of chordal graphs where all connected components of $G \setminus X$ evaporate at time exactly t.

Lemma 3 *For g, we have*

$$g(t,x,k,z) = \sum_{k'=0}^{k} \binom{k}{k'} \tilde{g}(t,x,k',z) g(t-1, x, k-k', z).$$

For $\tilde{g}$, to count chordal graphs where all connected components of $G \setminus X$ evaporate at time exactly t, we consider all possibilities for the label set of the component C of $G \setminus X$ that contains the lowest label not in X (namely $x+1$). We also consider all possibilities for $N(C)$. In this recurrence, k' stands for $|C|$, and x' stands for $|N(C)|$.

Lemma 4 *For $\tilde{g}$, we have*

$$\tilde{g}(t,x,k,z) = \sum_{k'=1}^{k} \sum_{x'=1}^{x} \left(\binom{x}{x'} - \binom{z}{x'} \right) \binom{k-1}{k'-1} \tilde{g}_1(t,x',k') \tilde{g}(t,x,k-k',z).$$

The constraint $x' \geq 1$ ensures that G is connected. We subtract all ways of selecting x' elements from $[z]$ to ensure that $N(C)$ is not contained in $[z]$. We also subtract 1 in the binomial coefficient $\binom{k-1}{k'-1}$ because the label set for C always contains $x+1$, along with $k'-1$ other labels.

For $\tilde{f}$, we need to count chordal graphs where L is fixed and evaporates at time t, $G \setminus X$ is connected, and all connected components of $G \setminus (X \cup L)$ evaporate at time exactly $t-1$. The number of such graphs in which no component of $G \setminus (X \cup L)$ sees all of $X \cup L$ is $\tilde{f}_p(t,x,\ell,k)$, by the definition of $\tilde{f}_p(t,x,\ell,k)$. Now if there is at least one all-seeing component, then we break this down into two further cases: either exactly one component sees all of $X \cup L$, or at least two components see all of $X \cup L$. Recall that $\tilde{g}_1$ (resp. $\tilde{g}_{\geq 2}$) counts the number of chordal graphs where all components of $G \setminus X$ evaporate at time exactly t, every component sees all of X, and there is exactly one such component (resp. at least two such components). In the first (resp. second) case, $\tilde{g}_1$ (resp. $\tilde{g}_{\geq 2}$) corresponds to the all-seeing component(s), and $\tilde{f}_p$ (resp. $\tilde{g}_p$) corresponds to the remaining components.

Lemma 5 *For $\tilde{f}$, we have*

$$\tilde{f}(t,x,\ell,k) = \tilde{f}_p(t,x,\ell,k) + \sum_{k'=1}^{k} \binom{k}{k'} \tilde{g}_1(t-1,x+\ell,k')\tilde{f}_p(t,x,\ell,k-k') + \sum_{k'=1}^{k} \binom{k}{k'} \tilde{g}_{\geq 2}(t-1,x+\ell,k')\tilde{g}_p(t-1,x+\ell,k-k',x).$$

The above cases are relevant because if at least two components of $G \setminus (X \cup L)$ see all of $X \cup L$, then this prevents the vertices of L from evaporating before time t. Indeed, each vertex $u \in L$ has a neighbor in each of those two components, meaning u has two non-adjacent neighbors. Otherwise, if there is at most one such component, then the neighborhoods of the remaining components of $G \setminus (X \cup L)$ must together cover L to ensure that L does not evaporate until time t. In that case, for each vertex $u \in L$ that is covered by a proper-subset neighborhood $N(C)$ of a component C, u has a neighbor $v \in C$ as well as a neighbor $w \in (X \cup L) \setminus N(C)$, and v and w are non-adjacent.

The reason we require $G \setminus X$ to be connected in the definition of f (rather than just requiring G to be connected) can be seen from the recurrence for $\tilde{f}$. Since in the first sum over k' we only wish to consider graphs with exactly one all-seeing component, in the definition of $\tilde{g}_1$ we require $G \setminus X$ to be connected. The recurrence for $\tilde{g}_1$ depends on f, so this carries over into requiring $G \setminus X$ to be connected in the definition of f. This explains the need for the argument z (for example, in g): as was mentioned above when we discussed the recurrence for f, keeping track of z lets us ensure that $G \setminus X$ is connected in all graphs counted by f.

For $\tilde{g}_{\geq 2}$, to count chordal graphs where all connected components of $G \setminus X$ evaporate at time exactly t, every component sees all of X, and there are at least two such components, we consider all possibilities for the label set of the component that contains the lowest label not in X. For the remaining components, there is either exactly one of them or at least two.

Lemma 6 *For $\tilde{g}_{\geq 2}$, we have*

$$\tilde{g}_{\geq 2}(t,x,k) = \sum_{k'=1}^{k-1} \binom{k-1}{k'-1} \tilde{g}_1(t,x,k') \Big(\tilde{g}_1(t,x,k-k') + \tilde{g}_{\geq 2}(t,x,k-k') \Big).$$

For $\tilde{g}_p$, to count chordal graphs where all connected components of $G \setminus X$ evaporate at time exactly t and no component sees all of X, we proceed as we did for $\tilde{g}$, except we require $x' < x$ rather than $x' \leq x$.

Lemma 7 *For $\tilde{g}_p$, we have*

$$\tilde{g}_p(t,x,k,z) = \sum_{k'=1}^{k} \sum_{x'=1}^{x-1} \left(\binom{x}{x'} - \binom{z}{x'} \right) \binom{k-1}{k'-1} \tilde{g}_1(t,x',k') \tilde{g}_p(t,x,k-k',z).$$

For $\tilde{f}_p$, we need to count chordal graphs where L is fixed and evaporates at time t, all connected components of $G \setminus (X \cup L)$ evaporate at time exactly $t-1$, and no component sees all of $X \cup L$. We first observe that when $z = x$, requiring $G \setminus [z]$ to be connected is the same as requiring $G \setminus X$ to be connected.

Lemma 8 *We have $\tilde{f}_p(t,x,\ell,k) = \tilde{f}_p(t,x,\ell,k,x)$.*

The next recurrence for $\tilde{f}_p$ counts the number of such graphs in which $G \setminus [z]$ is connected. On the first reading, one can skip the two "otherwise" cases in Lemma 9. In this lemma, we consider all possibilities for the label set of the component C of $G \setminus (X \cup L)$ that contains the lowest label not in $X \cup L$. In the outer sum, k' stands for $|C|$. Additionally, we consider all possibilities for the size x' of $N(C) \cap X$ and the size ℓ' of $N(C) \cap L$, and we consider all possibilities for their respective label sets. If $\ell' > 0$, then $N(C)$ is automatically not contained in $[z]$ since $z \leq x$, so there are $\binom{x}{x'}$ possible label sets for $N(C) \cap X$.

The intuition behind the two "otherwise" cases is as follows. If $\ell' = 0$, then we must subtract $\binom{z}{x'}$ from the number of possible label sets for $N(C) \cap X$ to ensure that

$N(C) \not\subseteq [z]$. If $\ell' = \ell$, then all of the vertices of L have now been pushed into the exception set, so the evaporation time of the subgraph formed from the remaining components is $t-1$. In this case, we call $\tilde{g}_p$ since we no longer know the size of the last set of simplicial vertices.

The dot in front of each of the curly braces denotes multiplication. For example, if $\ell' > 0$, then we multiply by $\binom{x}{x'}$.

Lemma 9 *For $\tilde{f}_p(t, x, \ell, k, z)$, we have*

$$\tilde{f}_p(t,x,\ell,k,z) =$$

$$\sum_{k'=1}^{k} \sum_{\substack{0\le x'\le x \\ 0\le \ell'\le \ell \\ 0<x'+\ell'<x+\ell}} \binom{k-1}{k'-1}\binom{\ell}{\ell'}\tilde{g}_1(t-1,x'+\ell',k') \cdot \begin{cases} \binom{x}{x'} & \text{if } \ell' > 0 \\ \binom{x}{x'} - \binom{z}{x'} & \text{otherwise} \end{cases}$$

$$\cdot \begin{cases} \tilde{f}_p(t, x+\ell', \ell-\ell', k-k', z) & \text{if } \ell' < \ell \\ \tilde{g}_p(t-1, x+\ell, k-k', z) & \text{otherwise.} \end{cases}$$

The base cases are as follows. We reach the base case for g when $t = 0$:

$$g(0,x,k,z) = \begin{cases} 1 & \text{if } k = 0 \\ 0 & \text{if } k > 0. \end{cases}$$

For $\tilde{g}$ and $\tilde{g}_p$, we have $\tilde{g}(t,x,0,z) = 1$ and $\tilde{g}_p(t,x,0,z) = 1$ when $k = 0$. For, $\tilde{g}_1$ we observe that $\tilde{g}_1(t,x,k) = 0$ if $t = 0$ or $k = 0$. Similarly, for $\tilde{g}_{\ge 2}$ we have $\tilde{g}_{\ge 2}(t,x,k) = 0$ if $t = 0$ or $k = 0$. We reach the base case for f when $x + \ell > \omega$, $t = 1$, or $k = 0$. If $x + \ell > \omega$, then $f(t,x,\ell,k) = 0$. Remarkably, this is the only place where ω appears in the algorithm. If $x + \ell \le \omega$, then we have

$$f(1,x,\ell,k) = \begin{cases} 1 & \text{if } k = 0 \\ 0 & \text{otherwise.} \end{cases}$$

If $x + \ell \leq \omega$ and $t \geq 2$, then $f(t, x, \ell, 0) = 0$. For $\tilde{f}$, we have $\tilde{f}(t, x, \ell, k) = 0$ if $t = 1$ or $k = 0$. Similarly, for $\tilde{f}_p$ we have $\tilde{f}_p(t, x, \ell, k, z) = 0$ if $t = 1$ or $k = 0$. For the version of $\tilde{f}_p$ without the fifth argument z, we do not need a base case since we always immediately call $\tilde{f}_p$ with z.

The control flow formed by these recurrences is shown in Figure 2.2. The algorithm terminates because either the value of t or the number of vertices in the graph (i.e., $x+k$ or $x + \ell + k$) decreases each time we return to the same function. For the running time, note that we first compute all of the necessary binomial coefficients. In particular, we compute all values of $\binom{a}{b}$ such that $0 \leq a \leq n$ and $0 \leq b \leq a$, which can be done using $O(n^2)$ arithmetic operations. Next, when computing the counter functions, the running time is dominated by the arithmetic operations needed to compute $\tilde{f}_p$. The recurrence for $\tilde{f}_p$ involves a triple summation, and there are five arguments, so a naive implementation uses $O(n^8)$ arithmetic operations. However, in Section 2.5.4, we show that the running time can in fact be improved to $O(n^7)$ arithmetic operations.

2.3.4 Proof of Theorem 1 (counting)

In this section, we prove the counting portion of Theorem 1 using Theorem 2. (See Section 2.6 for the proof of the sampling portion of Theorem 1.) In other words, we describe an algorithm to count chordal graphs, assuming we have an algorithm to count connected chordal graphs. Theorem 2 — counting connected chordal graphs — is proved in Section 2.5.

For $k \in \mathbb{N}$, let $a(k)$ denote the number of ω-colorable chordal graphs with vertex set $[k]$. Recall that $c(k)$ is the number of ω-colorable connected chordal graphs with vertex set $[k]$.

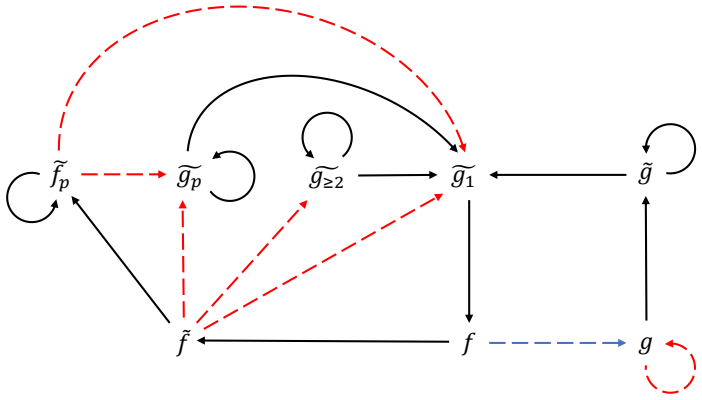

Figure 2.2: The control flow of the counting algorithm. An arrow from f to g indicates that the recursive formula for f depends on g. The arrow is red (and dashed) if t decreases by 1, blue (and dashed) if t decreases by 2, and black if t does not change. The black self-loops do not cause an infinite loop because in those recursive calls, the number of vertices decreases.

Lemma 10 *The number of ω-colorable chordal graphs with vertex set $[n]$ is given by*

$$a(n) = \sum_{k=1}^{n} \binom{n-1}{k-1} c(k)a(n-k)$$

for all $n \in \mathbb{N}$.

Proof: Suppose G is an ω-colorable chordal graph with vertex set $[n]$. Let G_1 be the graph formed by the connected component of G that contains the label 1, and let G_2 be the graph formed by all other connected components of G (which can potentially be empty). Let C be the set of labels that appear in G_1, let $k = |C|$, and let $D = [n] \setminus C$, i.e., D is the set of labels that appear in G_2. Now relabel G_1 by applying $\phi(C, [k])$ to the labels in C, and relabel G_2 by applying $\phi(D, [n-k])$ to the labels in D. (Recall that $\phi(A, B)$ is defined at the end of Section 1.5.) We can see that G_1 is now a connected ω-colorable chordal graph with vertex set $[k]$, and G_2 is an ω-colorable chordal graph with vertex set $[n-k]$. The map that takes any chordal graph G to the resulting pair (G_1, G_2) is injective since $\phi(C, [k])$ and $\phi(D, [n-k])$ are both bijections. Therefore, $a(n)$ is at most the number of possible triples (G_1, G_2, C), which is given by the summation above.

To see that $a(n)$ is bounded below by the same summation, suppose we are given $1 \leq k \leq n$, a connected ω-colorable chordal graph G_1 with vertex set $[k]$, an ω-colorable chordal graph G_2 with vertex set $[n-k]$, and a subset $C \subseteq [n]$ of size k that contains 1. Let $D = [n] \setminus C$. We construct an ω-colorable chordal graph G with vertex set $[n]$ as follows: Relabel G_1 by applying $\phi([k], C)$ to its label set, and relabel G_2 by applying $\phi([n-k], D)$ to its label set. Now let G be the union of G_1 and G_2 (by taking the union of the vertex sets and the edge sets). Clearly G is an ω-colorable chordal graph with vertex set $[n]$. The map that takes the triple (G_1, G_2, C) to the resulting graph G is injective, so $a(n)$ is at least the summation above, as desired. ■

Lemma 10 directly gives a dynamic-programming algorithm to compute the number of ω-colorable chordal graphs with vertex set $[n]$, given n as input. First, we compute all of the necessary binomial coefficients, i.e., all values of $\binom{a}{b}$ such that $0 \leq a \leq n$ and $0 \leq b \leq a$, which can be done using $O(n^2)$ arithmetic operations. Next, by Theorem 2, we can compute $c(k)$ for all $k \in [n]$ at a cost of $O(n^7)$ arithmetic operations. (Technically, the algorithm of Theorem 2 just computes $c(n)$, but by reusing the same dynamic-programming tables from that algorithm, we can also compute $c(k)$ for all $k \in [n]$.) To compute $a(n)$, we use the recurrence in Lemma 10 at a cost of $O(n^2)$ arithmetic operations. For the base case, we observe that $a(0) = 1$. Therefore, we have an algorithm to count ω-colorable chordal graphs on n vertices using $O(n^7)$ arithmetic operations.

2.4 Implementation of the Counting Algorithm

An implementation of the counting algorithm in C++ can be successfully run for inputs as large as $n = 30$ in about 2.5 minutes on a standard desktop computer.[1] Previously, the number of labeled chordal graphs was only known up to $n = 15$. Table 2.1 shows the number of connected chordal graphs on n vertices for $n \leq 30$, with the chromatic number unrestricted. Table 2.2 shows the number of ω-colorable connected chordal graphs on n vertices for various values of n and ω.[2]

2.5 Proof of Theorem 2 (Connected Chordal Graphs)

In this section, we prove correctness of the recurrences presented in Section 2.3.3, in order to prove Theorem 2. Sections 2.5.1 and 2.5.2 contain a number of lemmas on

[1] Our implementation is available on GitHub at https://github.com/uhebertj/chordal.

[2] These tables were made using the version of the algorithm that uses $O(n^8)$ arithmetic operations, without the optimization using the function h, since this version ran fastest in practice.

Table 2.1: Numbers of labeled connected chordal graphs on n vertices

$c(n)$	n
1	1
1	2
4	3
35	4
541	5
13302	6
489287	7
25864897	8
1910753782	9
193328835393	10
26404671468121	11
4818917841228328	12
1167442027829857677	13
374059462390709800421	14
158311620026439080777076	15
88561607724193506845709239	16
65629642803250494352023169033	17
64646285130595946195244365518454	18
84997214469704246545711429635276299	19
14988142356875294544461626191310904642 1	20
35626055123928426690872494367291110048855 8	21
11473744949464491944508258176053401236791504 61	22
50324868520402653224615508446959396780529673840 53	23
30210545039307528599583618386687349227933725131035504	24
24940038313065905058019326786145957925448982265006568596 1	25
28441345486995689815615546290431460703243324009448674823403 13	26
44993294034522185332489548856700572371349354518671249097245374660	27
99127725139236030144346028839700910906670827577808606147000987702773 9	28
30526157144572224953157514915475479605501638476250575941226904780179348933	29
13183638007395954271288355542312707702094261964027362487431622588244921589952 54	30

chordal graphs and their evaporation sequences. In Section 2.5.3, we use these lemmas to prove the recurrences. We then discuss the running time in Section 2.5.4.

2.5.1 Chordal graph properties

We make use of the following standard facts about chordal graphs. The proof of Lemma 11 can be found in [90].

Lemma 11 *Every chordal graph contains a simplicial vertex.*

Table 2.2: Numbers of ω-colorable labeled connected chordal graphs on n vertices. When $\omega = 2$, the algorithm counts labeled trees.

n									
2	3	4	5	6	7	8	9		
1	3	16	125	1296	16807	262144	4782969	2	
	4	34	480	9831	268093	9185436	379623492	3	
		35	540	13136	466683	22732032	1437072780	4	
			541	13301	488873	25736782	1873146621	5	ω
				13302	489286	25863916	1910084529	6	
					489287	25864896	1910751531	7	
						25864897	1910753781	8	
							1910753782	9	

n				
10	11	12		
100000000	2357947691	61917364224	2	
18376225525	1019282908941	63707908718994	3	
112588153700	10535042533301	1144261607209084	4	
181962472490	22726623077466	3513611793935959	5	
192919501307	26158547399061	4666697716137194	6	
193325509217	26400465973728	4813890013657154	7	ω
193328830337	26404655450778	4818876084111431	8	
193328835392	26404671456933	4818917765689886	9	
193328835393	26404671468120	4818917841203841	10	
	26404671468121	4818917841228327	11	
		4818917841228328	12	

Lemma 12 *Let G_1, G_2 be two chordal graphs, and suppose $Y \coloneqq V(G_1) \cap V(G_2)$ is a clique in both G_1 and G_2. If we glue G_1 and G_2 together at Y, then the resulting graph G is chordal.*

Proof: Suppose G contains an induced cycle C of length at least 4. Since G_1 and G_2 are chordal, C contains some vertex $u \in V(G_1) \setminus Y$ and some $v \in V(G_2) \setminus Y$. There are no edges connecting $G_1 \setminus Y$ to $G_2 \setminus Y$, so C contains two non-adjacent vertices that belong to the clique Y, which is a contradiction. ■

2.5.2 Properties of the evaporation sequence

Basic properties

In this section (Section 2.5.2), suppose G is a chordal graph that evaporates at time t with exception set X, where $X \subseteq V(G)$ is a clique, and let $L_1, \ldots, L_t$ be the evaporation sequence. For $i \in [t]$, let G_i be the state of the graph G just before time step i. In other words, $G_1 = G$ and $G_i = G \setminus (L_1 \cup \cdots \cup L_{i-1})$ for all $i \in [t]$.

Observation 1 *For all $i \in [t]$, every connected component of $G[L_i]$ is a clique.*

Proof: Let $i \in [t]$, and suppose u and v are two vertices that belong to the same connected component in $G[L_i]$. We have some path $u, w_1, w_2, \ldots, w_k, v$ from u to v in that connected component. Since w_1 is simplicial in G_i and thus in $G[L_i]$, u is adjacent to w_2. Similarly, by induction, u is adjacent to all of the vertices $w_1, \ldots, w_k, v$. Hence u and v are adjacent, as desired. ■

Observation 2 *If $G \setminus X$ is connected, then L_t is a clique.*

Proof: Let $u, v \in L_t$, and let P be a shortest path from u to v in $G \setminus X$. None of the vertices of P are simplicial in G_i for $i \in [t-1]$, since otherwise there would be a shorter path than P from u to v. Hence $P \subseteq L_t$, so u and v belong to the same connected component of $G[L_t]$, which is a clique by Observation 1. ■

Observation 3 *Let $i \in [t]$, let C be a connected component of $G[L_i]$, and let $u, v \in C$. Then $N_{G_i}(u) \setminus \{v\} = N_{G_i}(v) \setminus \{u\}$.*

Proof: If $u = v$ then we are done, so suppose $u \neq v$. Let $w \in N_{G_i}(u) \setminus \{v\}$. We know u and v are adjacent by Observation 1, and u is simplicial in G_i. Hence $w \in N_{G_i}(v) \setminus \{u\}$.

■

Observation 4 *If $G \setminus X$ is connected, X is a clique, and $X \subseteq N(L_t)$, then $X \cup L_t$ is a clique.*

Proof: By Observation 3 with $i = t$, we know $N(u) \cap X = N(v) \cap X$ for all $u, v \in L_t$. This implies that x and v are adjacent for all $x \in X$, $v \in L_t$. Since L_t is a clique by Observation 2, we conclude that $X \cup L_t$ is a clique. ■

Observation 5 *If $G \setminus X$ is connected, then $G_i \setminus X$ is connected for all $i \in [t]$.*

Proof: We proceed by induction by showing that if $G_i \setminus X$ is connected, then $G_{i+1} \setminus X$ is connected, for all $i \in [t-1]$. Suppose $G_i \setminus X$ is connected for some i, and let u, v be vertices in $G_{i+1} \setminus X$. Let P be a shortest path from u to v in $G_i \setminus X$. No intermediate vertices on P are simplicial in G_i, since otherwise there would be a shorter path from u to v in $G_i \setminus X$. Therefore, the same path P still exists in $G_{i+1} \setminus X$, and hence $G_{i+1} \setminus X$ is connected. ■

Lemma 13 *Let $C = V(G) \setminus X$. If $G \setminus X$ is connected, then $N(C) = N(L_t) \cap X$.*

Proof: It is clear that $N(L_t) \cap X \subseteq N(C)$. To see that $N(C) \subseteq N(L_t) \cap X$, it is sufficient to show $N(L_i) \cap X \subseteq N(L_{i+1} \cup \ldots \cup L_t) \cap X$ for all $i \in [t-1]$. Suppose $u \in N(L_i) \cap X$, so there is some vertex $v \in L_i$ that is adjacent to $u \in X$. We know $G_i \setminus X$ is connected by Observation 5, so there is a path $v, w_1, w_2, \ldots, w_k, w$ in $G_i \setminus X$ from v to some vertex $w \in L_{i+1} \cup \ldots \cup L_t$. We can assume $v, w_1, w_2, \ldots, w_k \in L_i$ by shortening the tail of the path if need be. Since v is simplicial in G_i, w_1 is adjacent to u. Similarly, by induction, all of the vertices $w_1, \ldots, w_k, w$ are adjacent to u. Hence $u \in N(L_{i+1} \cup \ldots \cup L_t) \cap X$. ■

Lemma 14 *If C is a connected component of $G \setminus X$, then $N(C) = N(L_t \cap C) \cap X$.*

Proof: By an argument similar to Observation 5, since C is connected, what remains of C in G_i is connected for all $i \in [t]$. Therefore, we can see that $N(L_i \cap C) \cap X \subseteq N((L_{i+1} \cup \ldots \cup L_t) \cap C) \cap X$ for all $i \in [t-1]$ by an argument similar to the proof of Lemma 13. ■

Evaporation time is preserved after gluing or taking induced subgraphs

The rest of the evaporation sequence lemmas that we need all have a similar flavor. For almost every recurrence, we need to show that the evaporation time of a chordal graph is preserved after gluing or taking induced subgraphs, as long as we choose the exception sets correctly and certain properties hold. For example, the simplest of these lemmas, Lemma 16, will be used to prove the recurrence for $g(t, x, k, z)$ (Lemma 3).

Lemma 15 *Suppose G is a connected chordal graph that contains a clique X, and suppose C is a connected component of $G \setminus X$. Then the evaporation time of C in G is equal to the evaporation time of $G[X \cup C]$, assuming both have exception set X.*

Proof: Let $G' = G[X \cup C]$. Let $L_1, \ldots, L_t$ be the evaporation sequence of G with exception set X, and let $G_i = G \setminus (L_1 \cup \cdots \cup L_{i-1})$ for $i \in [t]$. Similarly, let $L'_1, \ldots, L'_s$ be the evaporation sequence of G' with exception set X, and let $G'_i = G' \setminus (L'_1 \cup \cdots \cup L'_{i-1})$ for $i \in [s]$. For every $v \in C$, we know $N_{G'}(v) = N_G(v)$ since $N_G(v) \subseteq X \cup C$, so v is simplicial in G if and only if v is simplicial in G'. Hence $L'_1 = L_1 \cap C$. Continuing by induction, we can see that at each time step, the same set of vertices evaporates from C in both G_i and G'_i, and the vertices of X never evaporate. More precisely, for all $1 \leq i \leq s$, and for all $v \in C$ that have not yet evaporated, we have $N_{G'_i}(v) = N_{G_i}(v)$, and thus $L'_i = L_i \cap C$. Therefore, C has the same evaporation time in both G and G'. ■

Lemma 16 *Suppose G is a connected chordal graph that contains a clique X, and suppose $C_1, \ldots, C_r$ are connected components of $G \setminus X$. Then for every $j \in [r]$, the evapo-*

ration time of C_j in G is equal to the evaporation time of C_j in $G[X \cup C_1 \cup \cdots \cup C_r]$, assuming both have exception set X.

Proof: For each $j \in [r]$, the statement for C_j follows immediately from two applications of Lemma 15: first with the graph G and the component C_j, and then with the graph $G[X \cup C_1 \cup \cdots \cup C_r]$ and the component C_j. ■

Lemma 17 *Suppose G is a connected chordal graph that evaporates at time $t \geq 2$ with exception set X. Let $L = L_G(X)$, and suppose $X \cup L$ is a clique. Let A be the set of vertices in connected components C of $G \setminus (X \cup L)$ such that C evaporates at time exactly $t-1$ in G with exception set X, and let B be the set of vertices of all other components of $G \setminus (X \cup L)$. Let $G_1 = G[X \cup L \cup A]$ and $G_2 = G[X \cup L \cup B]$. Then the following statements hold:*

1. *G_1 evaporates at time t, every component of $G_1 \setminus (X \cup L)$ evaporates at time $t-1$ in G_1, and we have $L_{G_1}(X) = L$, when X is the exception set. Furthermore, A is nonempty.*

2. *G_2 evaporates in time at most $t-2$ with exception set $X \cup L$.*

Proof: To prove (1), we proceed using an argument similar to Lemma 15. In that lemma, the induction step relied on the fact that the vertices of X never evaporate. For a similar argument to hold here, we need to show that at time steps $1, \ldots, t-1$ in G_1, none of the vertices of L evaporate. Let $L_1, \ldots, L_t$ be the evaporation sequence of G with exception set X, and let $G_i = G \setminus (L_1 \cup \cdots \cup L_{i-1})$ for $i \in [t]$. Similarly, let $L_1^{(1)}, \ldots, L_s^{(1)}$ be the evaporation sequence of G_1 with exception set X, and let $G_i^{(1)} = G_1 \setminus (L_1^{(1)} \cup \cdots \cup L_{i-1}^{(1)})$ for $i \in [s]$. None of the vertices of L are simplicial in G_{t-1}. This means that every vertex $u \in L$ has two distinct non-adjacent neighbors $v_1, v_2 \in A$ that belong to G_{t-1}, since the

other components of $G \setminus (X \cup L)$ besides those in A have already evaporated before we reach G_{t-1}. The existence of v_1 and v_2 (since L is nonempty) implies that A is nonempty.

By an argument similar to Lemma 15, for every $u \in L$, its neighbors v_1 and v_2 do not evaporate in the first $t-2$ steps of the evaporation sequence of G_1, i.e., they still exist in $G_{t-1}^{(1)}$. Furthermore, the vertices that have evaporated from A in each time step to create G_t are the same as those for $G_t^{(1)}$. Therefore, every component of $G_1 \setminus (X \cup L)$ evaporates at time $t-1$ in G_1. In the end, $G_t^{(1)}$ is the clique $X \cup L$, so all of L is now simplicial, which yields $L_{G_1}(X) = L$.

To prove (2), we again use an argument similar to Lemma 15. In both G and G_2 (with the specified exception sets), none of the vertices of $X \cup L$ evaporate before time step t. Therefore, in the first $t-2$ time steps, the evaporation behavior of G_2 is exactly the same as it was for the corresponding vertices in G, which evaporate in time at most $t-2$. ■

Lemma 18 *Let G_1 and G_2 be connected chordal graphs such that $V(G_1) \cap V(G_2) = X \cup L$, where $L = L_{G_1}(X)$ and $X \cup L$ is a clique. Suppose G_1 evaporates at time $t \geq 2$ with exception set X, and suppose every connected component of $G_1 \setminus (X \cup L)$ evaporates at time exactly $t-1$ in G_1 with exception set X. Also, suppose every connected component of $G_2 \setminus (X \cup L)$ evaporates in time at most $t-2$ in G_2 with exception set $X \cup L$. If we glue G_1 and G_2 together at $X \cup L$, then the resulting graph G evaporates at time t with exception set X, and $L_G(X) = L$.*

Proof: Adding more vertices to a graph can only increase the evaporation time of a given vertex. Therefore, when we attach G_2 to G_1 to obtain G, the resulting evaporation time of each vertex in L is at least t (with exception set X). This implies that the evaporation behavior of the vertices of $G_2 \setminus (X \cup L)$ is the same in G as it was in G_2 by an argument similar to Lemma 15, since all of $X \cup L$ certainly stays alive until time t in

both G_2 (since it is the exception set) and G (by the argument above).

Now we just need to show that the evaporation behavior of the vertices of G_1 is the same in G as it was in G_1. When we attach G_2 to G_1, the only vertices of G_1 whose neighborhoods can change are those in $X \cup L$. The only way this could affect the evaporation behavior of G_1 would be by increasing the evaporation time of some vertices in L. However, we just showed that all of the vertices of $G_2 \setminus (X \cup L)$ evaporate in time at most $t-2$. Therefore, just before time step t (with exception set X), G_1 and G are identical since by then, all of $G_2 \setminus G_1$ has evaporated. Therefore, all of the vertices of L evaporate at time t in G, just as they did in G_1. ■

Lemma 19 *Suppose G is a connected chordal graph such that $X \cup L$ is a clique, where $L = L_G(X)$, and suppose every connected component of $G \setminus (X \cup L)$ evaporates at time exactly $t-1$ in G with exception set X, where $t \geq 2$. Suppose $C_1, \ldots, C_r$ are connected components of $G \setminus (X \cup L)$. Then for every $j \in [r]$, C_j evaporates at time exactly $t-1$ in $G[X \cup L \cup C_1 \cup \cdots \cup C_r]$ with exception set $X \cup L$.*

Proof: Let $G' = G[X \cup L \cup C_1 \cup \cdots \cup C_r]$. By Lemma 16, the evaporation time of C_j in G with exception set $X \cup L$ is equal to the evaporation time of C_j in G' with the same exception set. Furthermore, the evaporation time of C_j in G is the same with either exception set (X or $X \cup L$) since the evaporation time of C_j in G is less than t when X is the exception set. ■

Suppose G is a chordal graph with evaporation sequence $L_1, \ldots, L_t$ and exception set X, where $t \geq 2$. For a connected component C of $G[L_{t-1}]$, since by Observation 3 all vertices of C have the same neighborhood outside of C in G_{t-1}, we denote the neighborhood of C in $X \cup L_t$ by $\hat{N}(C) \coloneqq N_{G_{t-1}}(v) \cap (X \cup L_t) = N_G(v) \cap (X \cup L_t)$ for all $v \in C$.

Given a universe U and a subset $L \subseteq U$, we say a collection $\mathcal{S}$ of subsets of U is a *set*

cover for L if $\bigcup_{S\in\mathcal{S}} S \supseteq L$. For a chordal graph G as described above, let

$$\mathcal{S}_G = \{\hat{N}(C) : C \text{ is a connected component of } G[L_{t-1}]\}.$$

Each set in $\mathcal{S}_G$ is a subset of the universe $X \cup L_t$.

Lemma 20 *Suppose G is a chordal graph with evaporation sequence $L_1, \ldots, L_t$ and exception set X, where $t \geq 2$, and suppose $X \cup L_t$ is a clique. Then one of the following holds:*

1. *There is at most one connected component C of $G[L_{t-1}]$ such that $\hat{N}(C) = X \cup L_t$, and $\mathcal{S}_G \setminus \{X \cup L_t\}$ is a set cover for L_t.*

2. *There are at least two distinct connected components C_1, C_2 of $G[L_{t-1}]$ such that $\hat{N}(C_i) = X \cup L_t$ for $i \in \{1, 2\}$.*

Proof: We show that if there is at most one connected component of $G[L_{t-1}]$, say C, such that $\hat{N}(C) = X \cup L_t$, then $\mathcal{S}_G \setminus \{X \cup L_t\}$ is a set cover for L_t. If C does not exist, then we let $C = \emptyset$ to simplify notation. Suppose to the contrary that there is some vertex $v \in L_t$ that is not covered by any neighborhood $S \subsetneq X \cup L_t$ in $\mathcal{S}_G$. If there is a component C' of $G[L_{t-1}]$ such that $v \in \hat{N}(C')$, then we must have $\hat{N}(C') = X \cup L_t$. Therefore, there is at most one such component C', and if one exists we have $C' = C$. Hence $N_{G_{t-1}}(v) = (X \cup L_t \cup C) \setminus \{v\}$. Since C is a clique by Observation 1, $X \cup L_t \cup C$ is a clique. This means v is simplicial in G_{t-1}, which contradicts $v \in L_t$. ■

Lemma 21 *Suppose G is a connected chordal graph that evaporates at time $t \geq 2$ with exception set X. Let $L = L_G(X)$, and suppose $X \cup L$ is a clique. Furthermore, suppose every connected component of $G \setminus (X \cup L)$ evaporates at time exactly $t-1$ in G with exception set X. Suppose exactly one connected component C of $G \setminus (X \cup L)$ sees all of $X \cup L$, and let D be the set of vertices of all other components of $G \setminus (X \cup L)$. Let*

$G_2 = G[X \cup L \cup D]$. *Then when* X *is the exception set, we have the following:* G_2 *evaporates at time* t*, every component of* $G_2 \setminus (X \cup L)$ *evaporates at time* $t-1$ *in* G_2*, and* $L_{G_2}(X) = L$*. Furthermore,* D *is nonempty.*

Proof: Let $L_1, \ldots, L_t$ be the evaporation sequence of G with exception set X. By Lemma 14 (with exception set $X \cup L$ rather than X), we have $N(C) = N(L_{t-1} \cap C) \cap (X \cup L)$ for every connected component C' of $G \setminus (X \cup L)$. Furthermore, the connected components of $G \setminus (X \cup L)$ each intersected with L_{t-1} are the same as the connected components of $G[L_{t-1}]$ by an argument similar to Observation 5. Exactly one component of $G \setminus (X \cup L)$ sees all of $X \cup L$, so this means that exactly one component of $G[L_{t-1}]$ sees all of $X \cup L$. By Lemma 20, this implies that $\mathcal{S}_G \setminus \{X \cup L\}$ is a set cover for L. We observe that $\mathcal{S}_G \setminus \{X \cup L\} = \mathcal{S}_{G_2} \setminus \{X \cup L\}$ since we only removed the component C whose neighborhood is $\hat{N}(C) = X \cup L$ from G when defining G_2. Therefore, $\mathcal{S}_{G_2} \setminus \{X \cup L\}$ is also a set cover for L. This means D must be nonempty since L is nonempty.

As long as none of the vertices of L evaporate before time t in G_2, then we can see by an argument similar to Lemma 15 that the evaporation behavior of G_2 is the same as it was for the corresponding vertices in G (both with exception set X). Suppose for a contradiction that $u \in L$ is the first vertex in L to evaporate before time t in G_2. Suppose this happens at time s, where $s < t$. By an argument similar to Lemma 15, since none of L had evaporated before that point, all of L_{t-1} still exists in G_2 just before time s. The vertex u is contained in some neighborhood from $\mathcal{S}_{G_2} \setminus \{X \cup L\}$ since this is a set cover. Let v be a vertex in L_{t-1} from the component of $G_2 \setminus (X \cup L)$ corresponding to that neighborhood. Since that neighborhood is a proper subset of $X \cup L$, there is also some vertex $w \in X \cup L$ adjacent to u that does not belong to that neighborhood. Hence v and w are not adjacent, so u is not simplicial at time s, which is a contradiction.

Therefore, the evaporation behavior of G_2 is the same as it was for the corresponding

vertices in G, so G_2 evaporates at time t, every component of $G_2 \setminus (X \cup L)$ evaporates at time $t-1$, and $L_{G_2}(X) = L$. ■

Lemma 22 *Let G_1 and G_2 be connected chordal graphs such that $V(G_1) \cap V(G_2) = \hat{X}$, where $\hat{X}$ is a clique. Suppose $G_1 \setminus \hat{X}$ has at least two connected components, both of which see all of $\hat{X}$. Suppose every connected component of $G_1 \setminus \hat{X}$ (resp. $G_2 \setminus \hat{X}$) evaporates at time exactly $t-1$ in G_1 (resp. G_2) with exception set $\hat{X}$, where $t \geq 2$. Let $X \subsetneq \hat{X}$. If we glue G_1 and G_2 together at $\hat{X}$, then the resulting graph G evaporates at time t with exception set X, every connected component of $G \setminus \hat{X}$ evaporates at time $t-1$ in G, and $L_G(X) = \hat{X} \setminus X$.*

Proof: When we glue G_1 and G_2 together (and change the exception set from $\hat{X}$ to X), as long as it is still the case that none of the vertices of $\hat{X}$ evaporate before time t, then we can see by an argument similar to Lemma 15 that the evaporation behavior of these two graphs does not change. Suppose for a contradiction that $u \in \hat{X} \setminus X$ is the first vertex in $\hat{X}$ to evaporate before time t in G. Suppose this happens at time s, where $s < t$. Let $L_1, \ldots, L_t$ be the evaporation sequence of G_1 with exception set $\hat{X}$. Since u is the first such vertex to evaporate, all of L_{t-1} still exists in G just before time s. Let v and w be neighbors of u from L_{t-1} that belong to two distinct components of $G_1 \setminus \hat{X}$. Since v and w are not adjacent, this means u is not simplicial at time s, which is a contradiction. Therefore, the evaporation behavior of the vertices of $G_1 \setminus \hat{X}$ and $G_2 \setminus \hat{X}$ in G is the same as in G_1 and G_2. At time t, all of $\hat{X} \setminus X$ evaporates since $\hat{X}$ is a clique. ■

Lemma 23 *Suppose G is a connected chordal graph that evaporates at time $t \geq 2$ with exception set X. Let $L = L_G(X)$, and suppose $X \cup L$ is a clique. Furthermore, suppose every connected component of $G \setminus (X \cup L)$ evaporates at time exactly $t-1$ in G with exception set X. Let C be a component of $G \setminus (X \cup L)$, and let $L' = N(C) \cap L$. Let D be the*

set of vertices of all other components of $G \setminus (X \cup L)$*, and let* $G_2 = G[X \cup L \cup D]$*. If* $L' \subsetneq L$*, then when* $X \cup L'$ *is the exception set, we have the following:* G_2 *evaporates at time* t*, every component of* $G_2 \setminus (X \cup L)$ *evaporates at time* $t-1$ *in* G_2*, and* $L_{G_2}(X \cup L') = L \setminus L'$*; furthermore,* D *is nonempty.*

Proof: Let $L_1, \ldots, L_t$ be the evaporation sequence of G with exception set X. By an argument similar to the first paragraph of the proof of Lemma 21, we know $\hat{N}(C)$ is a proper subset of $X \cup L$ for every connected component C of $G[L_{t-1}]$. Therefore, by Lemma 20, $\mathcal{S}_G \setminus \{X \cup L\}$ is a set cover for L, so it follows that $\mathcal{S}_{G_2} \setminus \{X \cup L\}$ is a set cover for $L \setminus L'$. Since $L' \subsetneq L$, it is easy to see that D is nonempty.

By an argument similar to the second paragraph of the proof of Lemma 21, none of the vertices of $L \setminus L'$ evaporate before time t in G_2. Therefore, the evaporation behavior of G_2 (with exception set $X \cup L'$) is the same as it was for the corresponding vertices in G (with exception set X), so G_2 evaporates at time t, every component of $G_2 \setminus (X \cup L)$ evaporates at time $t-1$, and $L_{G_2}(X \cup L') = L \setminus L'$. ■

Lemma 24 *Let* G_1 *and* G_2 *be connected chordal graphs such that* $V(G_1) \cap V(G_2) = \widetilde{X}$*. Suppose* G_2 *contains a clique* $\hat{X} \supseteq \widetilde{X}$*. Furthermore, suppose* $G_1 \setminus \widetilde{X}$ *has exactly one connected component, which sees all of* $\widetilde{X}$ *and evaporates at time exactly* $t-1$ *in* G_1 *with exception set* $\widetilde{X}$*, where* $t \geq 2$*. Let* L' *be a subset of* $\widetilde{X}$*, and let* $X = \hat{X} \setminus L'$*. Let* G *be the graph obtained by gluing together* G_1 *and* G_2 *at* $\widetilde{X}$*. Then we have the following:*

1. *Suppose* G_2 *evaporates at time* t *with exception set* $\hat{X}$*, and let* $\hat{L} = L_{G_2}(\hat{X})$*. Suppose* $\hat{X} \cup \hat{L}$ *is a clique in* G_2*, and suppose every connected component of* $G_2 \setminus (\hat{X} \cup \hat{L})$ *evaporates at time exactly* $t-1$ *in* G_2*. Then* G *evaporates at time* t *with exception set* X*, every connected component of* $G \setminus (\hat{X} \cup \hat{L})$ *evaporates at time* $t-1$ *in* G*, and* $L_G(X) = L' \cup \hat{L}$*.*

2. *Suppose every connected component of $G_2 \setminus \hat{X}$ evaporates at time exactly $t-1$ in G_2 with exception set $\hat{X}$, and suppose $\widetilde{X} \subsetneq \hat{X}$. Then G evaporates at time t with exception set X, every connected component of $G \setminus \hat{X}$ evaporates at time $t-1$ in G, and $L_G(X) = L'$.*

Proof: First, we prove statement (1). When we glue G_1 and G_2 together (and change the exception set to X), as long as it is still the case that none of the vertices of $\hat{X}$ evaporate before time t, then we can see by an argument similar to Lemma 15 that the evaporation behavior of these two graphs does not change. Suppose for a contradiction that $u \in \hat{X} \setminus X$ is the first vertex in $\hat{X}$ to evaporate before time t in G. Suppose this happens at time s, where $s < t$. Let $L_1, \ldots, L_{t-1}$ be the evaporation sequence of G_1 with exception set $\widetilde{X}$. Let $v \in L_{t-1}$, and let $C = V(G_1) \setminus \widetilde{X}$. By Lemma 13, we know $\widetilde{X} = N(C) = N(L_{t-1}) \cap \widetilde{X}$, so u and v are adjacent since $u \in \widetilde{X}$. Since $\widetilde{X} \subsetneq \hat{X} \cup \hat{L}$, there is some vertex $w \in \hat{X} \cup \hat{L}$ that is adjacent to u but not v. Therefore, u is not simplicial at time s, which is a contradiction. Therefore, the evaporation behavior of the vertices of $G_1 \setminus \widetilde{X}$ and $G_2 \setminus \hat{X}$ in G is the same as in G_1 and G_2. At time t, all of $L' \cup \hat{L}$ evaporates since $\hat{X} \cup \hat{L}$ is a clique.

For statement (2), the proof is exactly the same except for the following differences. The sentence that begins "Since $\widetilde{X} \subsetneq \hat{X} \cup \hat{L} \ldots$" should instead say, "Since $\widetilde{X} \subsetneq \hat{X}$, there is some vertex $w \in \hat{X}$ that is adjacent to u but not v." And we change the concluding sentence to the following: "At time t, all of L' evaporates since $\hat{X}$ is a clique." ■

2.5.3 Proofs of recurrences

As in the counting algorithm, all graphs in this section are ω-colorable. We do not explicity mention this in each lemma, since ω-colorablitity does not change the recurrences (it only affects the base cases).

By Lemma 11, every induced subgraph of a chordal graph contains a simplicial vertex, so a chordal graph on n vertices evaporates in time at most n when the exception set is $X = \emptyset$. Therefore, the number of connected chordal graphs with vertex set $[n]$ is

$$c(n) = \sum_{t=1}^{n} \tilde{g}_1(t, 0, n).$$

Throughout this section, suppose t, x, ℓ, k, z are nonnegative integers in the domain of the relevant function, and suppose we are not yet at a base case. Let $G(t, x, k, z)$ be the set of graphs counted by $g(t, x, k, z)$, and similarly let $\widetilde{G}(t, x, k, z)$, $\widetilde{G}_p(t, x, k, z)$, $\widetilde{G}_1(t, x, k)$, $\widetilde{G}_{\geq 2}(t, x, k)$, $F(t, x, \ell, k)$, $\widetilde{F}(t, x, \ell, k)$, $\widetilde{F}_p(t, x, \ell, k)$, and $\widetilde{F}_p(t, x, \ell, k, z)$, be the sets of graphs counted by $\tilde{g}(t, x, k, z)$, $\tilde{g}_p(t, x, k, z)$, $\tilde{g}_1(t, x, k)$, $\tilde{g}_{\geq 2}(t, x, k)$, $f(t, x, \ell, k)$, $\tilde{f}(t, x, \ell, k)$, $\tilde{f}_p(t, x, \ell, k)$, and $\tilde{f}_p(t, x, \ell, k, z)$, respectively.

We are now ready to prove correctness of the recurrences. For the proofs of the following lemmas, let $X = [x]$ according to the current value of the argument x. We proceed in the order in which these functions were defined in Definition 3, so that similar functions and proofs appear consecutively. (This is somewhat different from the ordering of the lemma numbers.)

Lemma 3 *For g, we have*

$$g(t, x, k, z) = \sum_{k'=0}^{k} \binom{k}{k'} \tilde{g}(t, x, k', z) g(t-1, x, k-k', z).$$

Proof: To see that $g(t, x, k, z)$ is at most this summation, suppose $G \in G(t, x, k, z)$. Let A be the set of vertices in components C of $G \setminus X$ such that C evaporates at time exactly t in G with exception set X, and let B be the set of vertices of all other components of $G \setminus X$. Let $G_1 = G[X \cup A]$, $G_2 = G[X \cup B]$, and $k' = |A|$. Now relabel G_1 by applying $\phi(A, [x+1, x+k'])$ to the labels in A, and relabel G_2 by applying $\phi(B, [x+1, x+k-k'])$ to the labels in B. Note that G_1 and G_2 are connected since G is connected. Since every

component of $G \setminus X$ has at least one neighbor in $X \setminus [z]$, the same is true of $G_1 \setminus X$ and $G_2 \setminus X$. Therefore, by Lemma 16, we have $G_1 \in \widetilde{G}(t, x, k', z)$ and $G_2 \in G(t-1, x, k-k', z)$.

We claim that this process (including the relabeling step) is injective, i.e., distinct graphs G give rise to distinct triples (G_1, G_2, A). Indeed, suppose that starting from $G \in G(t, x, k, z)$, we obtain (G_1, G_2, A) along with the set B of remaining components, and starting from a distinct graph $G' \in G(t, x, k, z)$, we obtain (G'_1, G'_2, A') and the set B'. If $A \neq A'$, then we are done, so suppose $A = A'$. If $G[X \cup A] \neq G'[X \cup A']$, then after relabeling, $G[X \cup A]$ still differs from $G'[X \cup A']$ since $\phi(A, [x+1, x+k'])$ is a bijection. Otherwise, $G[X \cup B] \neq G'[X \cup B']$, in which case $G[X \cup B]$ still differs from $G'[X \cup B']$ even after relabeling. Therefore, $g(t, x, k, z)$ is at most the number of possible triples (G_1, G_2, A), which is given by the summation above.

We now show that $g(t, x, k, z)$ is bounded below by the same summation. Suppose we are given $0 \leq k' \leq k$, a graph $G_1 \in \widetilde{G}(t, x, k', z)$, a graph $G_2 \in G(t-1, x, k-k', z)$, and a subset $A \subseteq [x+1, x+k]$ of size k'. Let $B = [x+1, x+k] \setminus A$. We construct a graph $G \in G(t, x, k, z)$ as follows: Relabel G_1 by applying $\phi([x+1, x+k'], A)$ to the labels in $G_1 \setminus X$, and relabel G_2 by applying $\phi([x+1, x+k-k'], B)$ to the labels in $G_2 \setminus X$. By Lemma 12, since X is a clique in both G_1 and G_2, we can glue G_1 and G_2 together at X to obtain a chordal graph G. The resulting label set for G is $[x+k]$, and G is connected since G_1 and G_2 are connected. Furthermore, every component of $G \setminus X$ has at least one neighbor in $X \setminus [z]$ since the same is true of $G_1 \setminus X$ and $G_2 \setminus X$. By Lemma 16, G evaporates in time at most t, so we have $G \in G(t, x, k, z)$.

Suppose (G_1, G_2, A) and (G'_1, G'_2, A') are distinct triples of the above form that give rise to G and G', respectively. If $A \neq A'$, then by Lemma 16, $G \neq G'$. By the same lemma, if $G_1 \neq G'_1$ (resp. $G_2 \neq G'_2$), then the components of $G \setminus X$ and $G' \setminus X$ that evaporate at time exactly t (resp. less than t) will differ, which implies $G \neq G'$. Therefore, $g(t, x, k, z)$ is at least the summation above. ■

In the above lemma, we showed that the process that builds G_1 and G_2 from G (and vice versa) is injective, which ensures there is no double counting. In the lemmas that follow, we will build up similar injective maps in each direction. We only prove injectivity in detail in the proof of Lemma 3 since the arguments for the following lemmas are similar.

Lemma 4 *For $\tilde{g}$, we have*

$$\tilde{g}(t,x,k,z) = \sum_{k'=1}^{k}\sum_{x'=1}^{x}\left(\binom{x}{x'} - \binom{z}{x'}\right)\binom{k-1}{k'-1}\tilde{g}_1(t,x',k')\tilde{g}(t,x,k-k',z).$$

Proof: To see that $\tilde{g}(t,x,k,z)$ is at most this summation, suppose $G \in \widetilde{G}(t,x,k,z)$. Let C be the component of $G \setminus X$ that contains the label $x+1$, and let D be the set of vertices of all other components of $G \setminus X$. Let $X' = N(C)$, $G_1 = G[X' \cup C]$, $G_2 = G[X \cup D]$, $k' = |C|$, and $x' = |N(C)|$. Note that $X' \not\subseteq [z]$ since C has at least one neighbor in $X \setminus [z]$. Now relabel G_1 by applying $\phi(X', [x'])$ to the labels in X' and applying $\phi(C, [x'+1, x'+k'])$ to the labels in C. Relabel G_2 by applying $\phi(D, [x+1, x+k-k'])$ to the labels in D. Since every component of $G \setminus X$ has at least one neighbor in $X \setminus [z]$, the same is true of $G_2 \setminus X$. Therefore, by Lemma 16, we have $G_2 \in \widetilde{G}(t,x,k-k',z)$. We also have $G_1 \in \widetilde{G}_1(t,x',k')$ by an argument similar to Lemma 16; the only difference here is that the exception set is X' rather than X since C does not necessarily see all of X. (We chose C to be the component of $G \setminus X$ that contains the lowest label rather than choosing an arbitrary component of $G \setminus X$ so that this process is injective.) Hence $\tilde{g}(t,x,k,z)$ is at most the summation above.

We now show that $\tilde{g}(t,x,k,z)$ is bounded below by this summation. Suppose we are given $1 \le k' \le k$, $1 \le x' \le x$, a graph $G_1 \in \widetilde{G}_1(t,x',k')$, a graph $G_2 \in \widetilde{G}(t,x,k-k',z)$, a subset $C \subseteq [x+1, x+k]$ of size k' that contains $x+1$, and a subset $X' \subseteq X$ of size x' such that $X' \not\subseteq [z]$. Let $D = [x+1, x+k] \setminus C$. We construct a graph $G \in \widetilde{G}(t,x,k,z)$ as follows: Relabel G_1 by applying $\phi([x'], X')$ to the labels in $[x']$ and applying $\phi([x'+1, x'+k'], C)$

to the labels in $G_1 \setminus [x']$. Relabel G_2 by applying $\phi([x+1, x+k-k'], D)$ to the labels in $G_2 \setminus X$. By Lemma 12, we can glue G_1 and G_2 together at X' to obtain a chordal graph G. Every component of $G \setminus X$ has at least one neighbor in $X \setminus [z]$ since $G_2 \setminus X$ has that property, and because $X' \not\subseteq [z]$. By an argument similar to Lemma 16, G evaporates at time exactly t, so we have $G \in \widetilde{G}(t, x, k, z)$. Therefore, $\tilde{g}(t, x, k, z)$ is at least the number of possible choices for G_1, G_2, C, and X'. ■

Lemma 7 *For $\tilde{g}_p$, we have*

$$\tilde{g}_p(t, x, k, z) = \sum_{k'=1}^{k} \sum_{x'=1}^{x-1} \left(\binom{x}{x'} - \binom{z}{x'} \right) \binom{k-1}{k'-1} \tilde{g}_1(t, x', k') \tilde{g}_p(t, x, k-k', z).$$

Proof: The argument for $\tilde{g}_p$ is similar to the proof for $\tilde{g}$ (Lemma 4), except we do not allow $x' = x$ since no component of $G \setminus X$ sees all of X. ■

Lemma 1 *For $\tilde{g}_1$, we have*

$$\tilde{g}_1(t, x, k) = \sum_{\ell=1}^{k} \binom{k}{\ell} f(t, x, \ell, k-\ell).$$

Proof: To see that $\tilde{g}_1(t, x, k)$ is at most this summation, suppose $G \in \widetilde{G}_1(t, x, k)$. Let $C = V(G) \setminus X$, $L = L_G(X)$, $A = C \setminus L$, and $\ell = |L|$. We know $N(C) = X$ by the definition of $\widetilde{G}_1(t, x, k)$, and by Lemma 13, this implies $N(L) \cap X = X$. Therefore, $X \cup L$ is a clique by Observation 4. Relabel G by applying $\phi(L, [x+1, x+\ell])$ to L and applying $\phi(A, [x+\ell+1, x+k])$ to A. We now have $G \in F(t, x, \ell, k-\ell)$ since $L_G(X)$ uses the label set $[x+1, x+\ell]$, and relabeling vertices does not change the evaporation time of the graph. Hence $\tilde{g}_1(t, x, k)$ is at most the summation above.

We now show that $\tilde{g}_1(t, x, k)$ is bounded below by this summation. Suppose we are given $1 \le \ell \le k$, a graph $G \in F(t, x, \ell, k-\ell)$, and a subset $L \subseteq [x+1, x+k]$ of size ℓ. Let $A = [x+1, x+k] \setminus L$. Now relabel G by applying $\phi([x+1, x+\ell], L)$ to the labels in $[x+1, x+\ell]$ and applying $\phi([x+\ell+1, x+k], A)$ to the labels in $[x+\ell+1, x+k]$. Observe

that L sees all of X since $X \cup L$ is a clique, which implies $G \in \widetilde{G}_1(t, x, k)$. Therefore, $\tilde{g}_1(t, x, k)$ is at least the number of possible choices for G and L. ■

Lemma 6 *For $\tilde{g}_{\geq 2}$, we have*

$$\tilde{g}_{\geq 2}(t, x, k) = \sum_{k'=1}^{k-1} \binom{k-1}{k'-1} \tilde{g}_1(t, x, k') \Big(\tilde{g}_1(t, x, k-k') + \tilde{g}_{\geq 2}(t, x, k-k') \Big).$$

Proof: To see that $\tilde{g}_{\geq 2}(t, x, k)$ is at most this summation, suppose $G \in \widetilde{G}_{\geq 2}(t, x, k)$. Let C be the component of $G \setminus X$ that contains the lowest label not in X, and let D be the set of vertices of all other components of $G \setminus X$. Let $G_1 = G[X \cup C]$, $G_2 = G[X \cup D]$, and $k' = |C|$. Now relabel G_1 by applying $\phi(C, [x+1, x+k'])$ to the labels in C, and relabel G_2 by applying $\phi(D, [x+1, x+k-k'])$ to the labels in D. By Lemma 16, if $G \setminus X$ has exactly two connected components, then $G_2 \in \widetilde{G}_1(t, x, k-k')$, and otherwise, $G_2 \in \widetilde{G}_{\geq 2}(t, x, k-k')$. In either case, $G_1 \in \widetilde{G}_1(t, x, k')$. Hence $\tilde{g}_{\geq 2}(t, x, k)$ is at most the summation above.

We now show that $\tilde{g}_{\geq 2}(t, x, k)$ is bounded below by this summation. Suppose we are given $1 \leq k' \leq k$, a graph $G_1 \in \widetilde{G}_1(t, x, k')$, a graph G_2 that belongs to $\widetilde{G}_1(t, x, k-k')$ or $\widetilde{G}_{\geq 2}(t, x, k-k')$, and a subset $C \subseteq [x+1, x+k]$ of size k' that contains $x+1$. Let $D = [x+1, x+k] \setminus C$. We construct a graph $G \in \widetilde{G}(t, x, k)$ as follows: Relabel G_1 by applying $\phi([x+1, x+k'], C)$ to the labels in $[x+1, x+k']$, and relabel G_2 by applying $\phi([x+1, x+k-k'], D)$ to the labels in $[x+1, x+k-k']$. By Lemma 12, we can glue G_1 and G_2 together at X to obtain a chordal graph G. By Lemma 16, we have $G \in \widetilde{G}_{\geq 2}(t, x, k)$. Therefore, $\tilde{g}_{\geq 2}(t, x, k)$ is at least the summation above. ■

Lemma 2 *For f, we have*

$$f(t, x, \ell, k) = \sum_{k'=1}^{k} \binom{k}{k'} \tilde{f}(t, x, \ell, k') g(t-2, x+\ell, k-k', x).$$

Proof: To see that $f(t,x,\ell,k)$ is at most this summation, suppose $G \in F(t,x,\ell,k)$. Let $L = L_G(X)$. Let A be the set of vertices in components C of $G \setminus (X \cup L)$ such that C evaporates at time exactly $t-1$ in G with exception set X, and let B be the set of vertices of all other components of $G \setminus (X \cup L)$. Let $G_1 = G[X \cup L \cup A]$, $G_2 = G[X \cup L \cup B]$, and $k' = |A|$. Now relabel G_1 by applying $\phi(A, [x+\ell+1, x+\ell+k'])$ to the labels in A, and relabel G_2 by applying $\phi(B, [x+\ell+1, x+\ell+k-k'])$ to the labels in B. Note that G_1, $G_1 \setminus X$, and G_2 are connected since $G \setminus X$ is connected and $X \cup L$ is a clique. Hence we have $G_1 \in \widetilde{F}(t,x,\ell,k')$ by Lemma 17-(1). Since $G \setminus X$ is connected, every component of $G_2 \setminus (X \cup L)$ has a neighbor in L. Therefore, by Lemma 17-(2), we have $G_2 \in G(t-2, x+\ell, k-k', x)$, so $f(t,x,\ell,k)$ is at most the summation above.

We now show that $f(t,x,\ell,k)$ is bounded below by this summation. Suppose we are given $1 \le k' \le k$, a graph $G_1 \in \widetilde{F}(t,x,\ell,k')$, a graph $G_2 \in G(t-2, x+\ell, k-k', x)$, and a subset $A \subseteq [x+\ell+1, x+\ell+k]$ of size k'. Let $B = [x+\ell+1, x+\ell+k] \setminus A$. We construct a graph $G \in G(t,x,k)$ as follows: Relabel G_1 by applying $\phi([x+\ell+1, x+\ell+k'], A)$ to the labels in $[x+\ell+1, x+\ell+k']$, and relabel G_2 by applying $\phi([x+\ell+1, x+\ell+k-k'], B)$ to the labels in $[x+\ell+1, x+\ell+k-k']$. By Lemma 12, we can glue G_1 and G_2 together at $[x+\ell]$ to obtain a chordal graph G. We know $G \setminus X$ is connected since $G_1 \setminus X$ is connected and every component of $G_2 \setminus ([x+\ell])$ has a neighbor in $[x+1, x+\ell]$, so by Lemma 18, we have $G \in F(t,x,\ell,k)$. Therefore, $f(t,x,\ell,k)$ is at least the number of possible choices for G_1, G_2, and A. ■

Lemma 5 *For $\tilde{f}$, we have*

$$\tilde{f}(t,x,\ell,k) = \tilde{f}_p(t,x,\ell,k) + \sum_{k'=1}^{k} \binom{k}{k'} \tilde{g}_1(t-1, x+\ell, k') \tilde{f}_p(t,x,\ell,k-k') + \sum_{k'=1}^{k} \binom{k}{k'} \tilde{g}_{\ge 2}(t-1, x+\ell, k') \tilde{g}_p(t-1, x+\ell, k-k', x).$$

Proof: To see that $\tilde{f}(t,x,\ell,k)$ is at most this summation, suppose $G \in \widetilde{F}(t,x,\ell,k)$,

and let $L = L_G(X)$. The graph G falls into one of three cases: either zero, one, or at least two components of $G \setminus (X \cup L)$ see all of $X \cup L$. In the first case, we have $G \in \widetilde{F}_p(t, x, \ell, k)$. If G falls into the second or third case, then let A be the set of vertices in the components that see all of $X \cup L$, and let B be the set of vertices of all other components of $G \setminus (X \cup L)$. Let $G_1 = G[X \cup L \cup A]$, $G_2 = G[X \cup L \cup B]$, and $k' = |A|$. Now relabel G_1 by applying $\phi(A, [x+\ell+1, x+\ell+k'])$ to the labels in A, and relabel G_2 by applying $\phi(B, [x+\ell+1, x+\ell+k-k'])$ to the labels in B. If A consists of exactly one component of $G \setminus (X \cup L)$, then by Lemma 19, we have $G_1 \in \widetilde{G}_1(t-1, x+\ell, k')$, and by Lemma 21, we have $G_2 \in \widetilde{F}_p(t, x, \ell, k-k')$. Since $G \setminus X$ is connected, every component of $G_2 \setminus (X \cup L)$ has a neighbor in L. Therefore, if A consists of at least two components, then by Lemma 19, we have $G_1 \in \widetilde{G}_{\geq 2}(t-1, x+\ell, k')$ and $G_2 \in \widetilde{G}_p(t-1, x+\ell, k-k', x)$. Hence $\tilde{f}(t, x, \ell, k)$ is at most the summation above.

We now show that $\tilde{f}(t, x, \ell, k)$ is bounded below by this summation. First of all, every graph in $\widetilde{F}_p(t, x, \ell, k)$ belongs to $\widetilde{F}(t, x, \ell, k)$. For the second and third case, suppose we are given $1 \leq k' \leq k$, a subset $A \subseteq [x+\ell+1, x+\ell+k]$ of size k', and either a pair of graphs $G_1 \in \widetilde{G}_1(t-1, x+\ell, k')$ and $G_2 \in \widetilde{F}_p(t, x, \ell, k-k')$ or a pair of graphs $G_1 \in \widetilde{G}_{\geq 2}(t-1, x+\ell, k')$ and $G_2 \in \widetilde{G}_p(t-1, x+\ell, k-k', x)$. Let $B = [x+\ell+1, x+\ell+k] \setminus A$. We construct a graph $G \in \widetilde{F}(t, x, \ell, k)$ as follows: Relabel G_1 by applying $\phi([x+\ell+1, x+\ell+k'], A)$ to the labels in $[x+\ell+1, x+\ell+k']$, and relabel G_2 by applying $\phi([x+\ell+1, x+\ell+k-k'], B)$ to the labels in $[x+\ell+1, x+\ell+k-k']$. By Lemma 12, we can glue G_1 and G_2 together at $[x+\ell]$ to obtain a chordal graph G. If $G_1 \in \widetilde{G}_1(t-1, x+\ell, k')$ and $G_2 \in \widetilde{F}_p(t, x, \ell, k-k')$, then we know $G \setminus X$ is connected since $G_2 \setminus X$ is connected and the one component of $G_1 \setminus [x+\ell]$ sees all of $[x+\ell]$. In this case, by an argument similar to Lemma 18, gluing together G_1 and G_2 does not change the evaporation behavior of either of those graphs, so we have $G \in \widetilde{F}(t, x, \ell, k)$. For the case when $G_1 \in \widetilde{G}_{\geq 2}(t-1, x+\ell, k')$ and $G_2 \in \widetilde{G}_p(t-1, x+\ell, k-k', x)$, we know $G \setminus X$ is connected since every component of

$G_1 \setminus [x+\ell]$ sees all of $[x+\ell]$ and every component of $G_2 \setminus [x+\ell]$ has a neighbor in $[x+1, x+\ell]$. Hence we have $G \in \widetilde{F}(t,x,\ell,k)$ by Lemma 22. Therefore, $\tilde{f}(t,x,\ell,k)$ is at least the summation above. ■

Lemma 8 *We have* $\tilde{f}_p(t,x,\ell,k) = \tilde{f}_p(t,x,\ell,k,x)$.

Proof: When $z = x$ in $f_p(t,x,\ell,k,z)$, the definitions of $f_p(t,x,\ell,k)$ and $f_p(t,x,\ell,k,z)$ both require that $G \setminus X$ is connected, and they are otherwise identical. Therefore, $\tilde{f}_p(t,x,\ell,k) = \tilde{f}_p(t,x,\ell,k,x)$. ■

Lemma 9 *For* $\tilde{f}_p(t,x,\ell,k,z)$, *we have*

$$\tilde{f}_p(t,x,\ell,k,z) =$$

$$\sum_{k'=1}^{k} \sum_{\substack{0\le x'\le x \\ 0\le \ell'\le \ell \\ 0<x'+\ell'<x+\ell}} \binom{k-1}{k'-1}\binom{\ell}{\ell'} \tilde{g}_1(t-1, x'+\ell', k') \cdot \begin{cases} \binom{x}{x'} & \text{if } \ell' > 0 \\ \binom{x}{x'} - \binom{z}{x'} & \text{otherwise} \end{cases}$$

$$\cdot \begin{cases} \tilde{f}_p(t, x+\ell', \ell-\ell', k-k', z) & \text{if } \ell' < \ell \\ \tilde{g}_p(t-1, x+\ell, k-k', z) & \text{otherwise.} \end{cases}$$

Proof: To see that $\tilde{f}_p(t,x,\ell,k,z)$ is at most this summation, let $G \in \widetilde{F}_p(t,x,\ell,k,z)$, and let $L = L_G(X)$. Let C be the component of $G \setminus (X \cup L)$ that contains the lowest label not in $X \cup L$, and let D be the set of vertices of all other components of $G \setminus (X \cup L)$. Let $X' = N(C) \cap X$ and $L' = N(C) \cap L$. Let $G_1 = G[X' \cup L' \cup C]$, $G_2 = G[X \cup L \cup D]$, $k' = |C|$, $x' = |X'|$, and $\ell' = |L'|$. Note that $X' \not\subseteq [z]$ if $\ell' = 0$ since $G \setminus [z]$ is connected. Now relabel G_1 by applying $\phi(X', [x'])$ to the labels in X', applying $\phi(L', [x'+1, x'+\ell'])$ to the labels in L', and applying $\phi(C, [x'+\ell'+1, x'+\ell'+k'])$ to the labels in C. Relabel G_2 by applying $\phi(D, [x+\ell+1, x+\ell+k-k'])$ to the labels in D. If $\ell' < \ell$, then we also relabel G_2 further by applying $\phi(L', [x+1, x+\ell'])$ to the labels in L' and applying

$\phi(L \setminus L', [x+\ell'+1, x+\ell])$ to the labels in $L \setminus L'$. By an argument similar to Lemma 19, we have $G_1 \in \widetilde{G}_1(t-1, x'+\ell', k')$; the only difference here is that the exception set is $X' \cup L'$ rather than $X \cup L$ since C does not see all of $X \cup L$. We know $G_2 \setminus [z]$ is connected since $G \setminus [z]$ is connected. If $\ell' < \ell$, then by Lemma 23, we have $G_2 \in \widetilde{F}_p(t, x+\ell', \ell-\ell', k-k', z)$. If $\ell' = \ell$, then by Lemma 19, we have $G_2 \in \widetilde{G}_p(t-1, x+\ell, k-k', z)$. Hence $\tilde{f}_p(t, x, \ell, k, z)$ is at most the summation above.

We now show that $\tilde{f}_p(t, x, \ell, k)$ is bounded below by this summation. Suppose we are given $1 \leq k' \leq k$, $0 \leq x' \leq x$, $0 \leq \ell' \leq \ell$ such that $0 < x' + \ell' < x + \ell$, a graph $G_1 \in \widetilde{G}_1(t-1, x'+\ell', k')$, a graph G_2 such that $G_2 \in \widetilde{F}_p(t, x+\ell', \ell-\ell', k-k', z)$ if $\ell' < \ell$ and $G_2 \in \widetilde{G}_p(t-1, x+\ell, k-k', z)$ otherwise, a subset $C \subseteq [x+\ell+1, x+\ell+k]$ of size k' that contains $x+\ell+1$, a subset $X' \subseteq X$ of size x', with the added requirement that $X' \not\subseteq [z]$ if $\ell' = 0$, and a subset $L' \subseteq [x+1, x+\ell]$ of size ℓ'. Let $D = [x+\ell+1, x+\ell+k] \setminus C$. We construct a graph $G \in \widetilde{F}_p(t, x, \ell, k, z)$ as follows: Relabel G_1 by applying $\phi([x'], X')$ to the labels in $[x']$, applying $\phi([x'+1, x'+\ell'], L')$ to the labels in $[x'+1, x'+\ell']$, and applying $\phi([x'+\ell'+1, x'+\ell'+k'], C)$ to the labels in $[x'+\ell'+1, x'+\ell'+k']$. Relabel G_2 by applying $\phi([x+\ell+1, x+\ell+k-k'], D)$ to the labels in $[x+\ell+1, x+\ell+k-k']$. If $\ell' < \ell$, then we also relabel G_2 further by applying $\phi([x+1, x+\ell'], L')$ to the labels in $[x+1, x+\ell']$ and applying $\phi([x+\ell'+1, x+\ell], [x+1, \ldots, x+\ell] \setminus L')$ to the labels in $[x+\ell'+1, x+\ell]$. By Lemma 12, we can glue G_1 and G_2 together at $X' \cup L'$ to obtain a chordal graph G. We know $G \setminus [z]$ is connected since $G_2 \setminus [z]$ is connected and $X' \cup L' \not\subseteq [z]$. By Lemma 24, we have $G \in \widetilde{F}_p(t, x, \ell, k, z)$. Statement (1) of that lemma deals with the case when $\ell' < \ell$, and statement (2) deals with the case when $\ell' = \ell$. When applying that lemma, we let $\widetilde{X} = X' \cup L'$. For statement (1), we let $\hat{X} = X \cup L'$; for statement (2), we let $\hat{X} = [x+\ell]$. Therefore, $\tilde{f}_p(t, x, \ell, k, z)$ is at least the number of possible choices for G_1, G_2, C, X', and L'. ■

2.5.4 Wrapping up the proof of Theorem 2

Correctness of the counting algorithm for connected ω-colorable chordal graphs follows immediately from the proofs of the recurrences above. All that remains to show is that the algorithm terminates and uses at most $O(n^7)$ arithmetic operations.

To see that the algorithm terminates, we observe that every time a cycle of recursive calls returns to the same function where the cycle began, either the value of t decreases or the number of vertices in the graph decreases. We can see this from Figure 2.2: the only cycles where the value of t does not decrease are the black self-loops. For each these self-loops, the number of vertices in the graph decreases in each call to the function. For example, when $\tilde{g}$ calls itself in the recurrence for $\tilde{g}$, the number of vertices in the graph decreases from $x+k$ to $x+k-k'$, for some $k'>0$. Similarly, the number of vertices in the graph decreases in the self-loops corresponding to $\tilde{g}_{\geq 2}$, $\tilde{g}_p$, and $\tilde{f}_p$.

To speed up the computation of the recurrence for $\tilde{f}_p(t,x,\ell,k,z)$ (Lemma 9), we observe that when k' is fixed, $\tilde{g}_1(t-1,x'+\ell',k')$ is a common factor for all choices of x' and ℓ' that have the same sum $x'+\ell'$. Reordering the terms of the sum and letting $r=x'+\ell'$, we obtain

$$\tilde{f}_p(t,x,\ell,k,z) =$$

$$\sum_{k'=1}^{k}\sum_{r=1}^{x+\ell-1}\binom{k-1}{k'-1}\tilde{g}_1(t-1,r,k')\sum_{\ell'=\max\{0,r-x\}}^{\min\{r,\ell\}}\binom{\ell}{\ell'}\cdot\begin{cases}\binom{x}{r-\ell'} & \text{if } \ell'>0\\ \binom{x}{r-\ell'}-\binom{z}{r-\ell'} & \text{otherwise}\end{cases}$$

$$\cdot\begin{cases}\tilde{f}_p(t,x+\ell',\ell-\ell',k-k',z) & \text{if } \ell'<\ell\\ \tilde{g}_p(t-1,x+\ell,k-k',z) & \text{otherwise.}\end{cases}$$

Now the inner sum only depends on t, x, ℓ, z, r, and $k-k'$. Thus we define the helper

function

$$h(t,x,\ell,z,r,k) \coloneqq \sum_{\ell'=\max\{0,r-x\}}^{\min\{r,\ell\}} \binom{\ell}{\ell'} \cdot \begin{cases} \binom{x}{r-\ell'} & \text{if } \ell' > 0 \\ \binom{x}{r-\ell'} - \binom{z}{r-\ell'} & \text{otherwise} \end{cases} \cdot \begin{cases} \tilde{f}_p(t, x+\ell', \ell-\ell', k, z) & \text{if } \ell' < \ell \\ \tilde{g}_p(t-1, x+\ell, k, z) & \text{otherwise.} \end{cases}$$

The recurrence for $\tilde{f}_p$ can now be written as

$$\tilde{f}_p(t,x,\ell,k,z) = \sum_{k'=1}^{k} \sum_{r=1}^{x+\ell-1} \binom{k-1}{k'-1} \tilde{g}_1(t-1,r,k') h(t,x,\ell,z,r,k-k'). \tag{2.1}$$

Using Equation (2.1), it takes $O(n^7)$ arithmetic operations to compute all values of $\tilde{f}_p(t,x,\ell,k,z)$, since there are five arguments and now only a double summation. The cost of computing h is the same, i.e., $O(n^7)$ arithmetic operations, since h has six arguments and a single summation. Therefore, the overall running time of the counting algorithm is $O(n^7)$ arithmetic operations.

2.6 Sampling Labeled Chordal Graphs

Using the standard sampling-to-counting reduction of [1], we can use the information stored in the dynamic-programming tables from our counting algorithm to sample a uniformly random chordal graph. This sampling algorithm is all that remains to prove Theorem 1. Before sampling, assume we have already run the counting algorithm from Section 2.3, and thus we have constant-time access to the counter functions. In other words, given the name of a counter function and any particular values for its arguments, we can access the value of that function in constant time.

Theorem 3 *Suppose the counter functions have been precomputed and we have constant-time access to their values. We can sample uniformly at random from the following classes of chordal graphs, using $O(n^4)$ arithmetic operations for each sample:*

1. *ω-colorable chordal graphs with vertex set $[n]$*

2. *ω-colorable connected chordal graphs with vertex set $[n]$,*

where $n, \omega \in \mathbb{N}$ with $\omega \leq n$.

Proof: First, we show that we can sample uniformly at random from the following classes of chordal graphs: $G(t, x, k, z)$, $\widetilde{G}(t, x, k, z)$, $\widetilde{G}_p(t, x, k, z)$, $\widetilde{G}_1(t, x, k)$, $\widetilde{G}_{\geq 2}(t, x, k)$, $F(t, x, \ell, k)$, $\widetilde{F}(t, x, \ell, k)$, $\widetilde{F}_p(t, x, \ell, k)$, and $\widetilde{F}_p(t, x, \ell, k, z)$, where t, x, ℓ, k, z are any values in the domains of their respective functions. In the "greater than or equal" direction of the proof each recurrence in Section 2.5.3, we describe an injective map. For example, in Lemma 3 we are given $0 \leq k' \leq k$, a graph $G_1 \in \widetilde{G}(t, x, k', z)$, a graph $G_2 \in G(t - 1, x, k - k', z)$, and a subset $A \subseteq [x + 1, x + k]$ of size k', and we describe how one can map this to a graph $G \in G(t, x, k, z)$. In each lemma, we also describe an injective map in the reverse direction (to prove the "less than or equal" direction). Therefore, each of these injective maps is in fact a bijection.

Consider any one of the recurrence lemmas proved in Section 2.5.3. Let ψ be the injective map in the "greater than or equal" direction of the proof, let $\mathcal{C}_1$ be the domain of ψ and let $\mathcal{C}_2$ be the target space. Since $\psi \colon \mathcal{C}_1 \to \mathcal{C}_2$ is in fact a bijection, and our goal is to sample a uniformly random element of $\mathcal{C}_2$ (e.g. $G(t, x, k, z)$), it is sufficient to first sample a uniformly random element of $\mathcal{C}_1$, and then apply ψ.

The samplers for each of these graph classes call one another recursively/cyclically until reaching a base case. The base cases are the same as in the counting algorithm. Whenever we reach a base case, there is only one possible graph (since the counter

function equals 1), so we simply output that graph. We never choose a base case where the counter function equals 0, since the probability of taking such a path in the algorithm would have been 0.

To sample a *connected* chordal graph with vertex set $[n]$, we first choose a random value of the evaporation time, where each possible time $t \in [n]$ has probability proportional to the weight of the corresponding term in the summation for $c(n)$ (see section Section 2.3.3). Next, we sample a random graph with the chosen evaporation time t using the sampler for $\widetilde{G}_1(t, 0, n)$.

At the highest level, to sample a chordal graph with vertex set $[n]$, we use the standard sampling algorithm that is derived from the injective map in the "greater than or equal" direction of the proof Lemma 10.

One can also easily modify any of these samplers to only ouptut ω-colorable chordal graphs. This does not explicity change the pseudocode at all (see Section 2.8); we simply adjust the values of some of the base cases of the counting algorithm that are used by the sampling algorithm.

The correctness of each of these sampling procedures follows from the correctness of the counting algorithm. The sampling algorithm terminates by an argument similar to the proof of termination for the counting algorithm. For the sampling running time, the bottleneck is the time spent sampling from $\widetilde{F}_p(t, x, \ell, k, z)$. In this procedure, we compute the triple summation from the recurrence for $\widetilde{F}_p(t, x, \ell, k, z)$, and we call this procedure at most $O(n)$ times, so the overall cost is $O(n^4)$ arithmetic operations per sample. ■

This completes the proof of Theorem 1. The pseudocode for the sampling algorithm can be found in Section 2.8.

2.7 Approximate Counting and Sampling of Labeled Chordal Graphs

A *split* graph is a graph whose vertex set can be partitioned into a clique and an independent set, with arbitrary edges between the two parts. It is easy to see that every split graph is chordal. On the other hand, a result by Bender, Richmond, and Wormald [53] shows that a random n-vertex labeled chordal graph is a split graph with probability $1 - o(1)$. There is a relatively simple algorithm that counts the number of labeled split graphs on n vertices using $O(n^2)$ arithmetic operations [28]. In fact, we show that this algorithm can be modified to give an approximate answer using only $O(n \log^2(1/\varepsilon))$ arithmetic operations. By combining this with our (exact) counting and sampling algorithms for labeled chordal graphs, we obtain efficient algorithms for approximate counting and approximate sampling of labeled chordal graphs. In particular, if n is large enough (roughly $\log(1/\varepsilon)$ or larger), then we simply run the split graph counting or sampling algorithm. Otherwise, we run the exact counting or uniform sampling algorithm for chordal graphs.

For $\varepsilon > 0$, we say a number A is a $(1+\varepsilon)$-approximation of a number B if $B \leq A \leq B(1+\varepsilon)$, and we say A is a $(1-\varepsilon)$-approximation of B if $B(1-\varepsilon) \leq A \leq B$. Similarly, we say A is a $(1 \pm \varepsilon)$-approximation of B if $B(1-\varepsilon) \leq A \leq B(1+\varepsilon)$.

Theorem 4 *There is a deterministic algorithm that given a positive integer n and $0 < \varepsilon < 1$ as input, computes a $(1 \pm \varepsilon)$-approximation of the number of n-vertex labeled chordal graphs in $O(n^3 \log n \log^7(1/\varepsilon))$ time. Moreover, there is a randomized algorithm that given the same input, generates a random n-vertex labeled chordal graph according to a distribution whose total variation distance from the uniform distribution is at most ε. The expected running time of this randomized algorithm is $O(n^3 \log n \log^7(1/\varepsilon))$.*

In the $O(n^2)$ algorithm for counting split graphs that is described in [28], there is initially a certain amount of overcounting because of the fact that some split graphs have more than one valid way of being partitioned into a clique and an independent set. To account for this, the algorithm in [28] first computes the total number of split graphs *with overcounting*, and then it subtracts the value of the redundant terms to obtain the final answer. This gives an efficient counting algorithm, but this strategy does not lend itself as well to designing a sampling algorithm.

For Theorem 4, we wish to obtain not only an approximate *counting* algorithm, but also an approximate *sampling* algorithm. Therefore, we begin by designing a related, alternative counting algorithm for split graphs that partitions each split graph into three parts rather than two, using the unique three-part partition from Definition 1.

For our purposes, it is sufficient to *approximately* count split graphs, so the algorithm that we begin with is a $(1+\varepsilon)$-approximate counting algorithm that uses $O(n^2)$ arithmetic operations when ε is fixed (see Section 2.7.1). In Section 2.7.2, we then speed this up by computing only the largest terms of the summation given in Section 2.7.1, to obtain an approximate counting algorithm for split graphs that uses only $O(n \log^2(1/\varepsilon))$ arithmetic operations. In Section 2.7.3, we design an approximate sampling algorithm for split graphs. Finally, in Section 2.7.4, we describe the resulting approximate counting and sampling algorithms for *chordal* graphs, which proves Theorem 4.

2.7.1 Approximate counting of split graphs

Let C, I, and Q be the always-clique set, the always-independent set, and the questioning set, respectively, in a generic split graph (see Definition 1).

Observation 6 *For every split graph G, C is a clique and I is an independent set. Furthermore, every vertex in Q is adjacent to every vertex in C and is non-adjacent to*

every vertex in I.

Proof: Clearly, C is a clique and I is an independent set, since $C \subseteq C'$ and $I \subseteq I'$ for every split partition (C', I') of G. For the adjacencies of Q, let $v \in Q$. There exists some split partition (C', I') of G that places v in the clique C'. Since $C \subseteq C'$, this shows that v is adjacent to every vertex in C. Similarly, v is non-adjacent to every vertex in I. ■

In the proof of the next lemma, P_3 denotes a 3-vertex path, and $\overline{P_3}$ denotes the complement of a 3-vertex path. We say a graph is *P_3-free* (resp. *$\overline{P_3}$-free*) if it does not contain P_3 (resp. $\overline{P_3}$) as an induced subgraph.

Lemma 25 *The questioning set Q of a split graph G is either a clique or an independent set.*

Proof: We first observe that $G[Q]$ is P_3-free. To see this, suppose $G[Q]$ contains some induced path of u, v, w of length 3. Since $v \in Q$, there exists some split partition (C', I') of G such that $v \in I'$. This means $u \in C'$ and $w \in I'$ since $(u, v) \in E(G)$ and $(u, w) \notin E(G)$. Therefore, v and w are adjacent and both belong to I', which is a contradiction.

This shows that $G[Q]$ is P_3-free, so every connected component of $G[Q]$ is a clique. If Q is empty or $G[Q]$ has only one component, we are done. Now we claim that if $G[Q]$ has at least two components, then Q is an independent set. Suppose to the contrary that some component of $G[Q]$ contains an edge (u, v), and let w be a vertex in some other component. The vertices u, w, v form an induced $\overline{P_3}$ in $G[Q]$. However, we can see that $G[Q]$ is $\overline{P_3}$-free by an argument similar to the previous paragraph with the roles of C' and I' reversed, so this is a contradiction. ■

Lemma 26 *In a split graph, if Q is a clique, then every vertex in C has a neighbor in I. If Q is an independent set, then every vertex in I has a non-neighbor in C.*

Proof: For the case when Q is a clique, suppose there is some vertex $v \in C$ that is not adjacent to any vertex in I. Then $(C \cup Q \setminus \{v\}, I \cup \{v\})$ is a split partition that places v in the independent set, contradicting the fact that $v \in C$. Therefore, every vertex in C has a neighbor in I. The proof for the case when Q is an independent set is similar. ■

For the case when $|Q| \geq 2$, it is easy to exactly compute the number of split graphs. To do so, we will count the number of split graphs that satisfy the properties described in the following lemma.

Lemma 27 *Suppose G is a graph, and suppose $\{\hat{C}, \hat{I}, \hat{Q}\}$ is a partition of $V(G)$ with the following properties:*

1. *$\hat{C}$ is a clique and $\hat{I}$ is an independent set*
2. *$\hat{Q}$ is a clique or independent set, and $|\hat{Q}| \geq 2$*
3. *Every vertex in $\hat{Q}$ is adjacent to every vertex in $\hat{C}$ and is non-adjacent to every vertex in $\hat{I}$*
4. *If $\hat{Q}$ is a clique, then every vertex in $\hat{C}$ has a neighbor in $\hat{I}$*
5. *If $\hat{Q}$ is an independent set, then every vertex in $\hat{I}$ has a non-neighbor in $\hat{C}$.*

Then G is a split graph, and furthermore, $\hat{C}$ is the always-clique set, $\hat{I}$ is the always-independent set, and $\hat{Q}$ is the questioning set of G.

We have already established that the "converse" of this lemma is true: If G is a split graph with always-clique set, always-independent set, and questioning set equal to C, I, and Q, with $|Q| \geq 2$, then these three sets satisfy properties (1)-(5) of Lemma 27 when $\hat{C} = C$, $\hat{I} = I$, and $\hat{Q} = Q$. This was proved in Observation 6 and Lemmas 25 and 26. Combined with Lemma 27, this tells us that in order to count split graphs with $|Q| \geq 2$, we simply need to count the number of split graphs (for each given C, I, and Q) that satisfy properties (1)-(5).

Proof: [Proof of Lemma 27] Either $(\hat{C} \cup \hat{Q}, \hat{I})$ or $(\hat{C}, \hat{I} \cup \hat{Q})$ is a split partition of G, so G is a clearly a split graph. As usual, let C, I, and Q denote the always-clique set, always-independent set, and questioning set of G. From now on, we assume $\hat{Q}$ is a clique. For the case when $\hat{Q}$ is an independent set, the argument is similar.

We first show $\hat{Q} \subseteq Q$. Let $v \in \hat{Q}$. The split partition $(\hat{C} \cup \hat{Q}, \hat{I})$ places v in the clique, and the split partition $(\hat{C} \cup \hat{Q} \setminus \{v\}, \hat{I} \cup \{v\})$ places v in the independent set. Therefore, $v \in Q$.

Next, we show $\hat{C} \subseteq C$. Suppose there is some $v \in \hat{C} \setminus C$. This means there is some split partition (C', I') of G such that $v \in I'$. By property (4), v has some neighbor $u \in \hat{I}$, and we must have $u \in C'$. Now choose a vertex $q \in \hat{Q} \subseteq Q$. By property (3), we have $(v, q) \in E(G)$ and $(u, q) \notin E(G)$. This means q cannot belong to I' nor C', which is a contradiction.

To see that $\hat{I} \subseteq I$, let (C', I') be a split partition of G. Since $\hat{Q}$ is a clique and $|\hat{Q}| \geq 2$, there is at least one vertex $\hat{q} \in \hat{Q}$ that belongs to C'. For every vertex $v \in \hat{I}$, we know v is not adjacent to $\hat{q}$, so $v \in I'$. Therefore, $\hat{I} \subseteq I$. Since $\hat{C}$, $\hat{I}$, and $\hat{Q}$ together cover all of $V(G)$, this shows that $C = \hat{C}$, $I = \hat{I}$, and $Q = \hat{Q}$. ■

Putting this together, we have a formula that (exactly) counts split graphs with $|Q| \geq 2$.

Lemma 28 *Let $n \in \mathbb{N}$. The number of split graphs on n vertices with $|Q| \geq 2$ is given by the formula*[3]

$$2 \sum_{q=2}^{n} \sum_{c=0}^{n-q} \binom{n}{q} \binom{n-q}{c} (2^{n-c-q} - 1)^c.$$

Proof: By Lemma 25, Q is either a clique or an independent set. First consider the case when Q is a clique. Let q and c be integers such that $2 \leq q \leq n$ and $0 \leq q \leq n - q$. We wish to count the number of split graphs with $|Q| = q$, $|C| = c$, and $|I| = n - q - c$.

[3]We use the convention $0^0 = 1$, so the term where $q = n$ and $c = 0$ is equal to 1.

There are $\binom{n}{q}\binom{n-q}{c}$ ways of assigning the labels $1, \ldots, n$ to the sets C, I, and Q, since once we have chosen the labels for Q and C, the labels in I are known. For each possible choice of label sets, there are $(2^{n-c-q}-1)^c$ split graphs of this form, since we must choose a nonempty subset of I be to the neighborhood for each vertex in C. By Observation 6 and Lemmas 25 to 27, this counts exactly the correct set of graphs. Therefore, the number of split graphs on n vertices in which Q is a clique of size at least 2 is equal to

$$\sum_{q=2}^{n}\sum_{c=0}^{n-q}\binom{n}{q}\binom{n-q}{c}(2^{n-c-q}-1)^c.$$

By symmetry, this is exactly equal to the number of split graphs on n vertices in which Q is an independent set of size at least 2 (the bijection is given by taking the complement of each graph). Therefore, the number of split graphs with $|Q| \geq 2$ is twice the above summation. ■

We now move to the case when $|Q| \leq 1$.

Lemma 29 *Consider a split graph with $|Q| \leq 1$. In this case, every vertex in C has a neighbor in I, and every vertex in I has a non-neighbor in C.*

Proof: This follows immediately from Lemma 26. ■

We also need a lemma that is analogous to Lemma 27.

Lemma 30 *Suppose G is a graph, and suppose $\{\hat{C}, \hat{I}, \hat{Q}\}$ is a partition of $V(G)$ with the following properties:*

1. *$\hat{C}$ is a clique and $\hat{I}$ is an independent set*
2. *$|\hat{Q}| \leq 1$*
3. *Every vertex in $\hat{Q}$ is adjacent to every vertex in $\hat{C}$ and is non-adjacent to every vertex in $\hat{I}$*
4. *Every vertex in $\hat{C}$ has a neighbor in $\hat{I}$, and every vertex in $\hat{I}$ has a non-neighbor in $\hat{C}$.*

Then G is a split graph, and furthermore, $\hat{C}$ is the always-clique set, $\hat{I}$ is the always-independent set, and $\hat{Q}$ is the questioning set of G.

Proof: The graph G is clearly a split graph since $(\hat{C} \cup \hat{Q}, \hat{I})$ is a split partition of G. We also have the split partition $(\hat{C}, \hat{I} \cup \hat{Q})$, so $\hat{Q} \subseteq Q$.

To see that $\hat{C} \subseteq C$, suppose there is some $v \in \hat{C} \setminus C$. This means there is some split partition (C', I') of G such that $v \in I'$. By property (4), v has some neighbor $u \in \hat{I}$, and we must have $u \in C'$. And again by property (4), u has some non-neighbor $w \in \hat{C}$, and we must have $w \in I'$. Since $v, w \in \hat{C}$, this means v and w are two adjacent vertices in I', which is a contradiction. The proof that $\hat{I} \subseteq I$ is similar. Therefore, $C = \hat{C}$, $I = \hat{I}$, and $Q = \hat{Q}$. ■

For the proof of the following lemma, a *two-tone* graph is defined as a labeled graph in which, in addition to the vertex labels, each vertex is colored with one of two colors (cyan and indigo, for C and I). A *three-tone* graph is defined similarly, but with three colors (cyan, indigo, and white, for C, I, and Q).

Unlike the case when $|Q| \geq 2$, property (4) of Lemma 30 now has two neighbor/non-neighbor conditions that need to be true simultaneously, so it is no longer trivial to exactly count the number of possibilities. However, we can compute a very close approximation.

Lemma 31 *If n is sufficiently large, then the following formula gives a $\left(1 + \frac{1}{(3/2)^{n/3}}\right)$-approximation of the number of split graphs on n vertices with $Q = \emptyset$:*

$$\sum_{c=2}^{\lfloor \frac{n}{2} \rfloor} \binom{n}{c} (2^c - 1)^{n-c} + \sum_{c=\lfloor \frac{n}{2} \rfloor + 1}^{n-2} \binom{n}{c} (2^{n-c} - 1)^c.$$

Proof: We begin by working with the first summation. Fix an integer k such that $2 \leq k \leq \frac{n}{2}$. Let $\mathcal{T}$ be the set of all two-tone graphs with k cyan vertices and $n-k$ indigo vertices such that the cyan vertices form a clique, the indigo vertices form an independent

set, and every indigo vertex has a non-neighbor among the cyan vertices. The term with $c = k$ in the above summation counts the number of two-tone graphs in $\mathcal{T}$.

By Lemmas 29 and 30, the number of n-vertex split graphs with $|C| = k$ and $Q = \emptyset$ is equal to number of two-tone graphs in $\mathcal{T}$ where it happens that every cyan vertex has an indigo neighbor. Note that there are no split graphs with $|C| \in \{0, 1\}$ and $Q = \emptyset$ since if all of G is an independent set, then $Q = V(G)$. Now we just need to show that $|\mathcal{T}|$ is approximately the number of split graphs that we wish to count. Suppose we choose a graph G' from $\mathcal{T}$ uniformly at random. If $u \in V(G')$ is a cyan vertex and $v \in V(G')$ is an indigo vertex, then the probability that u and v are adjacent in G' is $\frac{2^{k-1}-1}{2^k-1}$. Therefore, the probability that a given cyan vertex u does not have an indigo neighbor is

$$\left(1 - \frac{2^{k-1}-1}{2^k-1}\right)^{n-k} \leq \left(\frac{2}{3}\right)^{n-k},$$

since $k \geq 2$. By a union bound, the probability that some cyan vertex u does not have an indigo neighbor is at most $n(2/3)^{n-k} \leq n(2/3)^{n/2}$ since $k \leq \frac{n}{2}$.

Therefore, the value of the summation from $c = 2$ to $\lfloor \frac{n}{2} \rfloor$ is at most α times the number of split graphs with $Q = \emptyset$ and $|C| \leq |I|$, where α is the following value:

$$\begin{aligned}
\alpha &= \frac{1}{1 - n\left(\frac{2}{3}\right)^{n/2}} \\
&= 1 + \frac{n}{\left(\frac{3}{2}\right)^{n/2} - n} \\
&\leq 1 + \frac{1}{\left(\frac{3}{2}\right)^{n/3}}.
\end{aligned}$$

The last inequality holds when n is sufficiently large.[4]

Similarly, the value of the summation from $c = \lfloor \frac{n}{2} \rfloor + 1$ to $n-2$ is at most α times the number of split graphs with $Q = \emptyset$ and $|C| > |I|$. The proof of this is similar, except $\mathcal{T}$ is defined as the set of all two-tone graphs with k cyan vertices and $n-k$ indigo vertices

[4] A loose bound shows that this lemma holds for $n \geq 61$.

such that the cyan vertices form a clique, the indigo vertices form an independent set, and *every cyan vertex has a neighbor among the indigo vertices.* Therefore, the overall sum is an α-approximation of the number of split graphs with $Q = \emptyset$. ■

Lemma 32 *If n is sufficiently large, then the following formula gives a $\left(1 + \frac{1}{(3/2)^{n/3}}\right)$-approximation of the number of split graphs on n vertices with $|Q| = 1$:*

$$\sum_{c=2}^{\lfloor \frac{n-1}{2} \rfloor} n\binom{n-1}{c}(2^c - 1)^{n-c-1} + \sum_{c=\lfloor \frac{n-1}{2} \rfloor + 1}^{n-2} n\binom{n-1}{c}(2^{n-c-1} - 1)^c.$$

Proof: For the first summation, fix an integer k such that $2 \leq k \leq \frac{n-1}{2}$. Let $\mathcal{T}$ be the set of all three-tone graphs with k cyan vertices, $n-k-1$ indigo vertices, and one white vertex, with the following properties: the cyan vertices form a clique, the indigo vertices form an independent set, the white vertex is adjacent to all of the cyan vertices and none of the indigo vertices, and every indigo vertex has a non-neighbor among the cyan vertices. The term with $c = k$ in the above summation counts the number of three-tone graphs in $\mathcal{T}$. There is now an additional factor of n (compared to the previous lemma) since there are n possible choices for the label of the white vertex.

By an argument similar to the previous lemma, the value of the summation from $c = 2$ to $\lfloor \frac{n-1}{2} \rfloor$ is at most α times the number of split graphs with $|Q| = 1$ and $|C| \leq |I|$, where α is the following value:

$$\alpha = \frac{1}{1 - (n-1)\left(\frac{2}{3}\right)^{(n-1)/2}}.$$

This is the same value as in the previous lemma, except with $n-1$ in place of n. As before, this is bounded above[5] by $1 + \frac{1}{(3/2)^{n/3}}$, and the case when $|C| > |I|$ is similar. ■

Suppose n is large enough that Lemmas 31 and 32 hold. Computing and adding together the summations from Lemmas 28, 31 and 32 gives a $\left(1 + \frac{1}{(3/2)^{n/3}}\right)$-approximation

[5] A loose bound shows that this lemma holds for $n \geq 65$.

for the number of split graphs on n vertices. If $n \geq 3\log_{3/2}\frac{1}{\varepsilon}$, then this is a $(1+\varepsilon)$-approximation.

The summation from Lemma 28 can be computed using $O(n^2)$ arithmetic operations by precomputing all binomial coefficients and precomputing all possible values of $(2^{n-c-q}-1)^c$. Similarly, the summations from Lemmas 31 and 32 can also be computed using $O(n^2)$ operations. Therefore, we have a $(1+\varepsilon)$-approximation algorithm for counting split graphs on n vertices that uses $O(n^2)$ arithmetic operations, for $n \geq \max\{N_1, 3\log_{3/2}\frac{1}{\varepsilon}\}$, where N_1 is a number large enough that Lemmas 31 and 32 hold for $n \geq N_1$.

This counting algorithm is already well-suited for designing a corresponding sampling algorithm. However, the running time still has some room for improvement. In the next section, we describe how to speed up this algorithm by only computing the terms that dominate the value of each summation, rather than computing all $\Theta(n^2)$ terms.

2.7.2 Faster approximate counting of split graphs

For the case when $|Q| \geq 2$, the term $(2^{n-c-q}-1)^c$ in Lemma 28 is maximized when $q=2$ and $c = \lfloor\frac{n-2}{2}\rfloor$. As it turns out, if we wish to compute a $(1-\varepsilon)$-approximation of the number of such split graphs, it is sufficient to compute only $O(\log^2(1/\varepsilon))$ terms of the double summation, near the values $q=2$ and $c=\lfloor\frac{n-2}{2}\rfloor$. In the following lemma, we show that we only need to compute $O(\log(1/\varepsilon))$ terms of the sum over q. Next, in Lemma 34, we show that we only need to compute $O(\log(1/\varepsilon))$ terms of the sum over c.

In the next few lemmas we assume $|Q| < n$, since this will be useful in the proof of Lemma 34. For the case when $|Q| = n$, we can easily count the number of split graphs. There are just two of them: G is either a complete graph or an independent set. (Lemma 33 also holds for the number of n-vertex split graphs with $2 \leq |Q| \leq n$, with the

same proof, if we take the sum from $q = 2$ to $\min\{s, n\}$, but requiring $|Q| < n$ is more convenient for the later lemmas.)

Lemma 33 *Let $0 < \varepsilon < 1$ be given. For sufficiently large n, the following formula gives a $(1-\varepsilon)$-approximation of the number of split graphs on n vertices with $2 \le |Q| < n$:*

$$2 \sum_{q=2}^{\min\{s,n-1\}} \sum_{c=0}^{n-q} \binom{n}{q} \binom{n-q}{c} (2^{n-c-q} - 1)^c,$$

where $s = \lceil 10 \log \frac{1}{\varepsilon} + 37 \rceil$.

Proof: The number of n-vertex split graphs with $2 \le |Q| < n$ is equal to the formula from Lemma 28, except with the sum over q going from 2 to $n-1$ rather than n. Now, in the current lemma, we have skipped the terms that go from $q = s+1$ to $n-1$. We need to show that their sum is at most ε times the total summation (from 2 to $n-1$).

For simplicity, in this proof we will not write the factor of 2 in front of the double summation, since including that factor does not change the multiplicative error. In the following inequality, we use the fact that $\lceil \frac{n-2}{2} \rceil \lfloor \frac{n-2}{2} \rfloor \ge \lceil \frac{n-2}{2} \rceil (\lfloor \frac{n-2}{2} \rfloor - 1) + 1$ since we can assume $\lceil \frac{n-2}{2} \rceil \ge 1$. The total value of the double summation is bounded below by the term with $q = 2$ and $c = \lfloor \frac{n-2}{2} \rfloor$:

$$\begin{aligned}
\binom{n}{2} \binom{n-2}{\lfloor \frac{n-2}{2} \rfloor} (2^{\lceil \frac{n-2}{2} \rceil} - 1)^{\lfloor \frac{n-2}{2} \rfloor} &\ge \binom{n-2}{\lfloor \frac{n-2}{2} \rfloor} \left(2^{\lceil \frac{n-2}{2} \rceil \lfloor \frac{n-2}{2} \rfloor} - 2^{\lceil \frac{n-2}{2} \rceil (\lfloor \frac{n-2}{2} \rfloor - 1)} \right) \\
&\ge \frac{1}{2} \binom{n-2}{\lfloor \frac{n-2}{2} \rfloor} 2^{\lceil \frac{n-2}{2} \rceil \lfloor \frac{n-2}{2} \rfloor} \\
&\ge \frac{1}{2} \binom{n-2}{\lfloor \frac{n-2}{2} \rfloor} 2^{(\frac{n-3}{2})(\frac{n-2}{2})}.
\end{aligned}$$

To prove a bound on the tail (the terms from $q = s+1$ to $n-1$), we first give a bound for the inner sum. Suppose $2 \le q < n$ is fixed. As c ranges from 0 to $\lfloor \frac{n-q}{2} \rfloor$, the product $c(n-q-c)$ increases by at least 1 whenever c increases by 1. Therefore,

$$\sum_{c=0}^{\lfloor \frac{n-q}{2} \rfloor} (2^{n-c-q} - 1)^c \le \sum_{c=0}^{\lfloor \frac{n-q}{2} \rfloor} (2^{n-c-q})^c \le \sum_{i=0}^{\lfloor \frac{n-q}{2} \rfloor \lceil \frac{n-q}{2} \rceil} 2^i \le 2^{(n-q)^2/4+1}.$$

Similarly, the same bound holds if we take the same sum from $c = \lceil \frac{n-q}{2} \rceil$ to $n-q$, so

$$\sum_{c=0}^{n-q} (2^{n-c-q} - 1)^c \leq 2^{(n-q)^2/4+2}.$$

Since $\binom{n-q}{c}$ is at most $\binom{n-2}{\lfloor \frac{n-2}{2} \rfloor}$ for all values of c and q, this implies

$$\sum_{c=0}^{n-q} \binom{n-q}{c} (2^{n-c-q} - 1)^c \leq \binom{n-2}{\lfloor \frac{n-2}{2} \rfloor} \cdot 2^{(n-q)^2/4+2}.$$

Therefore, we have the following bound on the tail:

$$\begin{aligned}\sum_{q=s+1}^{n-1} \sum_{c=0}^{n-q} \binom{n}{q} \binom{n-q}{c} (2^{n-c-q} - 1)^c &\leq \sum_{q=s+1}^{n-1} \binom{n}{q} \binom{n-2}{\lfloor \frac{n-2}{2} \rfloor} \cdot 2^{(n-q)^2/4+2} \\ &\leq \binom{n-2}{\lfloor \frac{n-2}{2} \rfloor} \cdot 2^{(\frac{n-3}{2})(\frac{n-2}{2})-1} \sum_{q=s+1}^{n-1} \binom{n}{q} 2^{-qn/4+5n/4+3/2}\end{aligned}$$

since $q^2 \leq qn$. We just need to show

$$\sum_{q=s+1}^{n-1} \binom{n}{q} 2^{-qn/4+5n/4+3/2} \leq \varepsilon$$

to conclude that we have a $(1-\varepsilon)$-approximation. Note that $\frac{n}{2^{n/4}} \leq \frac{1}{2^{n/10}}$ for $n \geq 34$. Therefore, we can bound this sum as follows:

$$\begin{aligned}\sum_{q=s+1}^{n-1} \binom{n}{q} 2^{-qn/4+5n/4+3/2} &\leq \sum_{q=s+1}^{n-1} n^q \cdot 2^{-qn/4+5n/4+3/2} \\ &= 2^{5n/4+3/2} \sum_{q=s+1}^{n-1} \left(\frac{n}{2^{n/4}} \right)^q \\ &\leq 2^{5n/4+3/2} \sum_{q=s+1}^{n-1} \left(\frac{1}{2^{n/10}} \right)^q \\ &\leq 2^{5n/4+3/2} \frac{2}{2^{(s+1)n/10}} \\ &= 2^{5n/4-(s+1)n/10+5/2} \\ &\leq \varepsilon,\end{aligned}$$

where the last step follows from the definition of s (see the next paragraph for more details). In the third-to-last step, we assumed $n \geq 10$ so that $2^{qn/10}$ increases by at least a factor of 2 each time q increases by 1.

The last step deserves more explanation since there are several steps contained in it. Starting from the definition of s, we have

$$\begin{aligned} s &= \lceil 10 \log \frac{1}{\varepsilon} + 37 \rceil \\ &\implies \log \frac{1}{\varepsilon} + \frac{5}{2} \leq -\frac{5}{4} + \frac{s}{10} + \frac{1}{10} \\ &\implies \log \frac{1}{\varepsilon} + \frac{5}{2} \leq \left(-\frac{5}{4} + \frac{s}{10} + \frac{1}{10} \right) n \\ &\implies \log \frac{1}{\varepsilon} \leq -\frac{5n}{4} + \frac{(s+1)n}{10} - \frac{5}{2}. \end{aligned}$$

In the second implication, where we use the fact that $1 \leq n$, we also need $-\frac{5}{4} + \frac{s}{10} + \frac{1}{10} \geq 0$ for this to work. This is true by the definition of s (since $\varepsilon \leq 5$).

■

Lemma 34 *Let $n, q \in \mathbb{N}$ with $2 \leq q < n$, and let $0 < \varepsilon < 1$ be given. We have the following approximation:*

$$\sum_{c=\max\{0,t_1\}}^{\min\{t_2, n-q\}} \binom{n-q}{c} (2^{n-c-q} - 1)^c \geq (1-\varepsilon) \sum_{c=0}^{n-q} \binom{n-q}{c} (2^{n-c-q} - 1)^c,$$

where $t_1 = \lfloor \frac{n-q}{2} \rfloor - \lceil \log \frac{1}{\varepsilon} \rceil - 2$ and $t_2 = \lceil \frac{n-q}{2} \rceil + \lceil \log \frac{1}{\varepsilon} \rceil + 2$.

Proof: As before, we show that the tail composed of terms we have skipped is at most ε times the total summation. In the following inequality, we use the fact that $\lceil \frac{n-q}{2} \rceil \lfloor \frac{n-q}{2} \rfloor \geq \lceil \frac{n-q}{2} \rceil (\lfloor \frac{n-q}{2} \rfloor - 1) + 1$ since $q < n$. The total value of the summation is bounded below by the term with $c = \lfloor \frac{n-q}{2} \rfloor$:

$$\begin{aligned} \binom{n-q}{\lfloor \frac{n-q}{2} \rfloor} (2^{\lceil \frac{n-q}{2} \rceil} - 1)^{\lfloor \frac{n-q}{2} \rfloor} &\geq \binom{n-q}{\lfloor \frac{n-q}{2} \rfloor} \left(2^{\lceil \frac{n-q}{2} \rceil \lfloor \frac{n-q}{2} \rfloor} - 2^{\lceil \frac{n-q}{2} \rceil (\lfloor \frac{n-q}{2} \rfloor - 1)} \right) \\ &\geq \frac{1}{2} \binom{n-q}{\lfloor \frac{n-q}{2} \rfloor} 2^{\lceil \frac{n-q}{2} \rceil \lfloor \frac{n-q}{2} \rfloor}. \end{aligned}$$

For the bound on the tail, we start with the upper end of the tail where $c > t_2$. We have

$$\sum_{c=t_2+1}^{n-q} \binom{n-q}{c} (2^{n-c-q}-1)^c = \sum_{\Delta=t_2+1-\lceil\frac{n-q}{2}\rceil}^{\lfloor\frac{n-q}{2}\rfloor} \binom{n-q}{\lceil\frac{n-q}{2}\rceil+\Delta} (2^{\lfloor\frac{n-q}{2}\rfloor-\Delta}-1)^{\lceil\frac{n-q}{2}\rceil+\Delta},$$

where $\Delta = c - \lceil\frac{n-q}{2}\rceil$. We have $(\lfloor\frac{n-q}{2}\rfloor - \Delta)(\lceil\frac{n-q}{2}\rceil + \Delta) \leq \lfloor\frac{n-q}{2}\rfloor\lceil\frac{n-q}{2}\rceil - \Delta^2$ since $\Delta \geq 0$. Therefore,

$$\begin{aligned}
\sum_{c=t_2+1}^{n-q} \binom{n-q}{c} (2^{n-c-q}-1)^c &\leq \binom{n-q}{\lceil\frac{n-q}{2}\rceil} \sum_{\Delta=t_2+1-\lceil\frac{n-q}{2}\rceil}^{\lfloor\frac{n-q}{2}\rfloor} 2^{(\lfloor\frac{n-q}{2}\rfloor-\Delta)(\lceil\frac{n-q}{2}\rceil+\Delta)} \\
&\leq \binom{n-q}{\lceil\frac{n-q}{2}\rceil} 2^{\lfloor\frac{n-q}{2}\rfloor\lceil\frac{n-q}{2}\rceil} \sum_{\Delta=t_2+1-\lceil\frac{n-q}{2}\rceil}^{\lfloor\frac{n-q}{2}\rfloor} 2^{-\Delta^2} \\
&= \frac{1}{2}\binom{n-q}{\lceil\frac{n-q}{2}\rceil} 2^{\lfloor\frac{n-q}{2}\rfloor\lceil\frac{n-q}{2}\rceil} \cdot 2 \sum_{\Delta=t_2+1-\lceil\frac{n-q}{2}\rceil}^{\lfloor\frac{n-q}{2}\rfloor} 2^{-\Delta^2}.
\end{aligned}$$

If $t \geq \sqrt{\log\frac{8}{\varepsilon}}$, then

$$2\sum_{\Delta=t}^{\infty} 2^{-\Delta^2} \leq \frac{\varepsilon}{2}.$$

When t_2 is defined as above, we have $t_2 + 1 - \lceil\frac{n-q}{2}\rceil \geq \sqrt{\log\frac{8}{\varepsilon}}$, so the value of the upper end of the tail is at most $\frac{\varepsilon}{2}$ times the total summation (since $\frac{\varepsilon}{8} \leq 1$).

Similarly, to bound the lower end of the tail, the same argument holds if we let $\Delta = c - \lfloor\frac{n-q}{2}\rfloor$. In this case, we use the fact that $(\lceil\frac{n-q}{2}\rceil - \Delta)(\lfloor\frac{n-q}{2}\rfloor + \Delta) \leq \lceil\frac{n-q}{2}\rceil\lfloor\frac{n-q}{2}\rfloor - \Delta^2$ since $\Delta \leq 0$. (This is the inequality that we need since we are now using a floor rather than a ceiling in the definition of Δ.) The lower end of the tail is at most $\frac{\varepsilon}{2}$ times the total summation as long as $|t_1 - 1 - \lfloor\frac{n-q}{2}\rfloor| \geq \sqrt{\log\frac{8}{\varepsilon}}$, which is true by the definition of t_1. Thus the sum of both tails is at most $\frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon$ times the total summation. ■

Combining Lemmas 33 and 34 with $\frac{\varepsilon}{2}$ in place of ε and using the fact that $(1-\frac{\varepsilon}{2})^2 \geq 1 - \varepsilon$, we immediately obtain the following:

Lemma 35 *Let $0 < \varepsilon < 1$ be given. For sufficiently large n, the following formula gives a $(1-\varepsilon)$-approximation of the number of split graphs on n vertices with $2 \le |Q| < n$:*

$$2 \sum_{q=2}^{\min\{s,n-1\}} \sum_{c=\max\{0,t_1\}}^{\min\{t_2,n-q\}} \binom{n}{q}\binom{n-q}{c}(2^{n-c-q}-1)^c,$$

where $s = \lceil 10 \log \frac{1}{\varepsilon} + 47 \rceil$, $t_1 = \lfloor \frac{n-q}{2} \rfloor - \lceil \log \frac{1}{\varepsilon} \rceil - 3$, and $t_2 = \lceil \frac{n-q}{2} \rceil + \lceil \log \frac{1}{\varepsilon} \rceil + 3$.

For the case when $|Q| \le 1$, we can prove similar statements. As above, let N_1 be large enough that Lemmas 31 and 32 hold for $n \ge N_1$.

Lemma 36 *Let $0 < \varepsilon < 1$ be given. If $n \ge \max\{N_1, 3\log_{3/2} \frac{1}{\varepsilon}\}$, then the following formula gives a $(1 \pm \varepsilon)$-approximation of the number of split graphs on n vertices with $Q = \emptyset$:*

$$\sum_{c=\max\{2,t_1\}}^{\lfloor \frac{n}{2} \rfloor} \binom{n}{c}(2^c-1)^{n-c} + \sum_{c=\lfloor \frac{n}{2} \rfloor + 1}^{\min\{t_2,n-2\}} \binom{n}{c}(2^{n-c}-1)^c,$$

where $t_1 = \lfloor \frac{n}{2} \rfloor - \lceil \log \frac{1}{\varepsilon} \rceil - 2$ and $t_2 = \lceil \frac{n}{2} \rceil + \lceil \log \frac{1}{\varepsilon} \rceil + 2$.

Proof: As we saw in the previous section, when $n \ge \max\{N_1, 3\log_{3/2} \frac{1}{\varepsilon}\}$, the summation in Lemma 31 is a $(1+\varepsilon)$-approximation of the number of n-vertex split graphs with $Q = \emptyset$. Therefore, we just need to show that the summation in this lemma (Lemma 36) is a $(1-\varepsilon)$-approximation of the summation from Lemma 31.

In the following inequality, we use the fact that $\lfloor \frac{n}{2} \rfloor \lceil \frac{n}{2} \rceil \ge \lfloor \frac{n}{2} \rfloor (\lceil \frac{n}{2} \rceil - 1) + 1$ since $n \ge 2$. The total value of the summation in this lemma is bounded below by the term with $c = \lfloor \frac{n}{2} \rfloor$:

$$\begin{aligned}\binom{n}{\lfloor \frac{n}{2} \rfloor}(2^{\lfloor \frac{n}{2} \rfloor}-1)^{\lceil \frac{n}{2} \rceil} &\ge \binom{n}{\lfloor \frac{n}{2} \rfloor}\left(2^{\lfloor \frac{n}{2} \rfloor \lceil \frac{n}{2} \rceil} - 2^{\lfloor \frac{n}{2} \rfloor (\lceil \frac{n}{2} \rceil - 1)}\right) \\ &\ge \frac{1}{2}\binom{n}{\lfloor \frac{n}{2} \rfloor} 2^{\lfloor \frac{n}{2} \rfloor \lceil \frac{n}{2} \rceil}.\end{aligned}$$

The upper end of the tail (where $c \ge \lfloor \frac{n}{2} \rfloor + 1$) and the lower end of the tail (where $c \le \lfloor \frac{n}{2} \rfloor$) are both at most $\frac{\varepsilon}{2}$ times the total summation by an argument similar to the

proof of Lemma 34 with $q = 0$. Indeed, it does not make a difference that the lower end of the tail contains the term $(2^c - 1)^{n-c}$ rather than $(2^{n-c} - 1)^c$ since both of these are bounded above by $2^{c(n-c)}$. Therefore, the sum of both tails is at most ε times the total summation. ■

Lemma 37 *Let $0 < \varepsilon < 1$ be given. If $n \geq \max\{N_1, 3\log_{3/2}\frac{1}{\varepsilon}\}$, then the following formula gives a $(1 \pm \varepsilon)$-approximation of the number of split graphs on n vertices with $|Q| = 1$:*

$$\sum_{c=\max\{2,t_1\}}^{\lfloor\frac{n-1}{2}\rfloor} n\binom{n-1}{c}(2^c-1)^{n-c-1} + \sum_{c=\lfloor\frac{n-1}{2}\rfloor+1}^{\min\{t_2,n-2\}} n\binom{n-1}{c}(2^{n-c-1}-1)^c,$$

where $t_1 = \lfloor\frac{n}{2}\rfloor - \lceil\log\frac{1}{\varepsilon}\rceil - 2$ and $t_2 = \lceil\frac{n}{2}\rceil + \lceil\log\frac{1}{\varepsilon}\rceil + 2$.

Proof: As in the previous lemma, we just need to show that the summation in this lemma is a $(1-\varepsilon)$-approximation of the summation from Lemma 32. For simplicity, we will not write the factor of n in front of each summation, since including that factor does not change the multiplicative error. We know $\lfloor\frac{n-1}{2}\rfloor\lceil\frac{n-1}{2}\rceil \geq \lfloor\frac{n-1}{2}\rfloor(\lceil\frac{n-1}{2}\rceil - 1) + 1$ since $n \geq 3$. The total value of the summation in this lemma is bounded below by the term with $c = \lfloor\frac{n-1}{2}\rfloor$, which is at most

$$\frac{1}{2}\binom{n-1}{\lfloor\frac{n-1}{2}\rfloor}2^{\lfloor\frac{n-1}{2}\rfloor\lceil\frac{n-1}{2}\rceil}$$

by an argument similar to the previous lemma with $n-1$ in place of n.

The upper end of the tail (where $c \geq \lfloor\frac{n-1}{2}\rfloor + 1$) and the lower end of the tail (where $c \leq \lfloor\frac{n-1}{2}\rfloor$) are both at most $\frac{\varepsilon}{2}$ times the total summation by an argument similar to the proof of Lemma 34 with $q = 1$. Therefore, the sum of both tails is at most ε times the total summation. ■

Let N_2 be large enough that Lemma 35 holds for $n \geq N_2$. We define the function f as follows: $f(\varepsilon) = \max\{N_1, N_2, 3\log_{3/2}(1/\varepsilon)\}$.

Lemma 38 *Given $0 < \varepsilon < 1$ and $n \geq f(\varepsilon)$, there is an algorithm that computes a $(1\pm\varepsilon)$-approximation of the number of n-vertex labeled split graphs in $O(n^3 \log n \log^2(1/\varepsilon))$ time.*

Proof: **Algorithm.** Let s be the total obtained from adding together the summations from Lemmas 35, 36, and 37. Return $s + 2$.

Correctness. There are exactly two split graphs with $|Q| = n$, so by Lemmas 35 to 37, $s + 2$ is a $(1 \pm \varepsilon)$-approximation of the number of n-vertex split graphs.

Running time. Let the intervals of values of q and c in the summation from Lemma 35 be called $I_q \coloneqq \{2, \ldots, \min\{s, n-1\}\}$ and $I_c \coloneqq \{\max\{0, t_1\}, \ldots, \min\{t_2, n - q\}\}$. For each $q \in I_q$, $c \in I_c$, we can compute $(2^{n-c-q}-1)^c$ using repeated squaring, which takes $O(\log n)$ arithmetic operations. Each of the binomial coefficients can be computed in $O(n)$ operations using the formula $\binom{a}{b} = \frac{a!}{b!(a-b)!}$. Therefore, we can compute the summation from Lemma 35 using $O(n|I_q||I_c|) = O(n \log^2(1/\varepsilon))$ operations. Similarly, the summations from Lemmas 36 and 37 can be computed using $O(n \log(1/\varepsilon))$ operations. Thus the overall running time is $O(n \log^2(1/\varepsilon))$ arithmetic operations. Since each number in this algorithm can be stored in $O(n^2)$ bits, this amounts to a running time of $O(n^3 \log n \log^2(1/\varepsilon))$ in the RAM model. ■

Interestingly, the factor of n in the running time of Lemma 38 only comes from computing the binomial coefficients. If one were to precompute the binomial coefficients, then the rest of the computation would only take $O(\log n \log^2(1/\varepsilon))$ arithmetic operations.

2.7.3 Approximate sampling of split graphs

The approximate counting algorithm from the previous section (Lemma 38) can naturally be extended to give an approximate sampling algorithm that works for $n \geq f(\frac{\varepsilon}{2})$. Recall that $I_q = \{2, \ldots, \min\{s, n-1\}\}$ and $I_c = \{\max\{0, t_1\}, \ldots, \min\{t_2, n-q\}\}$. For the sampling algorithm, we consider choosing a random number in $[0, 1)$ to be one operation.

Lemma 39 *Given $0 < \varepsilon < 1$ and $n \geq f(\frac{\varepsilon}{2})$, there is an algorithm that samples a random n-vertex labeled split graph according to a distribution whose total variation distance from the uniform distribution is at most ε. The expected running time of this algorithm is $O(n^3 \log n \log^2(1/\varepsilon))$.*

Proof: **Algorithm.** First, we run the counting algorithm from Lemma 38 with input (n, ε'), where $\varepsilon' = \min\{\frac{\varepsilon}{2}, \frac{1}{3}\}$. Let $s+2$ be the output of this algorithm.

We randomly choose one of the following four cases — $Q = \emptyset$, $|Q| = 1$, $2 \leq |Q| < n$, or $|Q| = n$ — with probabilities given by the computed summations. For example, the probability with which we choose $Q = \emptyset$ is the value of the summation from Lemma 36 divided by $s+2$, and the probability with which we choose $Q = n$ is $\frac{2}{s+2}$.

$\underline{|Q| = n}$: In this case, we return an n-vertex complete graph with probability $\frac{1}{2}$, and otherwise we return an n-vertex independent set.

$\underline{2 \leq |Q| < n}$: In this case, we choose a random pair $(q', c') \in I_q \times I_c$, where each pair (q, c) has probability proportional to the weight of the (q, c) term in the summation from Lemma 35. We then sample a uniformly random n-vertex split graph with $|Q| = q'$ and $|C| = c'$ (by following the proof of Lemma 28), and we return this graph.

$\underline{Q = \emptyset}$: In this case, we choose a random value of c, say c', from the values summed over in Lemma 36, where each value of c has probability proportional to the weight of the corresponding term in the summation. We sample a two-tone graph G uniformly at random from those with c' cyan vertices, $n - c'$ indigo vertices, and the following properties: (1) the cyan vertices form a clique; (2) the indigo vertices form an independent set; and (3) if $c' \leq \frac{n}{2}$, then every indigo vertex has a non-neighbor among the cyan vertices; otherwise, every cyan vertex has a neighbor among the indigo vertices. We then check whether every cyan vertex in G has an indigo neighbor if $c' \leq \frac{n}{2}$, and otherwise we check whether every indigo vertex has a cyan non-neighbor (call this property P).

If G satisfies property P, then we return G (without colors). Otherwise, we go back to the second paragraph of the algorithm (to again choose between the four possible cases for $|Q|$).

$|Q| = 1$: Similarly, in this case we choose a random value of c, say c', from the values summed over in Lemma 37, where each value of c has probability proportional to the weight of the corresponding term in the summation. We sample a three-tone graph G uniformly at random from those with c' cyan vertices, $n - c' - 1$ indigo vertices, one white vertex, and the following properties: (1) the cyan vertices form a clique; (2) the indigo vertices form an independent set; (3) the white vertex is adjacent to all of the cyan vertices and none of the indigo vertices; and (4) if $c' \leq \frac{n-1}{2}$, then every indigo vertex has a non-neighbor among the cyan vertices; otherwise, every cyan vertex has a neighbor among the indigo vertices. We then check whether every cyan vertex in G has an indigo neighbor if $c' \leq \frac{n-1}{2}$, and otherwise we check whether every indigo vertex has a cyan non-neighbor (call this property P). If G satisfies property P, then we return G (without colors). Otherwise, we go back to the second paragraph of the algorithm.

Correctness. Let s' be the number of split graphs computed by the approximate counting algorithm of Lemma 38 (with the given error bound being ε'), except we do not count the graphs that fail to satisfy property P (so s' is at most $s + 2$). By Lemma 38, $s + 2$ is a $(1 \pm \frac{\varepsilon}{2})$-approximation of the number of n-vertex split graphs. The number of graphs that are counted by this lemma but do not satisfy property P is at most $\frac{\varepsilon}{2}$ times the number of n-vertex split graphs, so s' is a $(1 - \varepsilon)$-approximation of the number of n-vertex split graphs.

The sampling algorithm never outputs a non-split graph or any graph that does not satisfy property P. Furthermore, the probabilities of all of the choices of $|Q|$ and $c = |C|$ are designed so that all output graphs are equally likely, since these probabilities are

based on the summations from the counting algorithm. Therefore, for every split graph G, if the probability that we output G is nonzero, then the probability that we output G is in fact $1/s'$. (We know $s' \neq 0$ since $1 - \varepsilon > 0$ and the number of n-vertex split graphs is nonzero for all n.) Since s' is a $(1-\varepsilon)$-approximation of the number of n-vertex split graphs, this implies that the total variation distance between the output distribution and the uniform distribution on split graphs is at most ε.

Running time. When we say one "iteration" of the algorithm, this refers to building a graph and then checking whether property P holds for that graph. First, we show that the expected number of iterations is $O(1)$. If property P does not hold for the current graph, then we call this a "failure." Each time we check property P, the probability of failure is $\frac{s+2-s'}{s+2}$. Let $\hat{s}$ be the exact number of n-vertex split graphs. We have $s+2-s' \leq \varepsilon' \cdot \hat{s} \leq \varepsilon'(\frac{s+2}{1-\varepsilon'})$ since $(1-\varepsilon')\hat{s} \leq s+2$, so the probability of failure is at most $\frac{\varepsilon'}{1-\varepsilon'}$. Therefore, the expected number of iterations is at most

$$\frac{1}{1-\frac{\varepsilon'}{1-\varepsilon'}},$$

which is at most 2, since $\varepsilon' \leq \frac{1}{3}$.

Initially, the counting algorithm takes $O(n^3 \log n \log^2(1/\varepsilon))$ time. Now we just need to bound the running time of each iteration of the algorithm after that. An iteration with $|Q| = n$ takes $O(n^2)$ time. For the case when $2 \leq |Q| < n$, let s_2 be the value of the summation from Lemma 35, and for each pair $(q,c) \in I_q \times I_c$, let $t_{(q,c)}$ denote the (q,c) term in that summation. We can choose a random pair (q',c') in the following way. First, we choose a random number $r \in [0,1)$. Next, we iterate over all pairs $(q,c) \in I_q \times I_c$. For each of these, we subtract $t_{(q,c)}/s_2$ from r and then check whether $r < 0$. When $r < 0$ for the first time, we let (q',c') be the current pair (q,c). This process takes $O(n \log^2(1/\varepsilon))$ arithmetic operations. At this point, we need to generate a random split graph with $|Q| = q'$ and $|C| = c'$. This can be done in $O(n^2)$ time in the following way. We assign

random labels to the sets C, I, and Q in $O(n)$ time. Next, for each vertex in C, we choose its neighborhood by repeatedly choosing a random subset of I until that subset happens to be nonempty. The expected number of attempts for this is at most 2, and each attempt takes $O(n)$ time for a given vertex in C. Therefore, for one iteration in the case when $2 \leq |Q| < n$, the expected running time is $O(n \log^2(1/\varepsilon))$ arithmetic operations plus $O(n^2)$ basic operations. Similarly, for an iteration with $Q = \emptyset$ or $|Q| = 1$, the expected running time is $O(n \log(1/\varepsilon))$ arithmetic operations plus $O(n^2)$ basic operations. This amounts to a running time of $O(n^3 \log n \log^2(1/\varepsilon) + n^2) = O(n^3 \log n \log^2(1/\varepsilon))$ in the RAM model. ■

2.7.4 Proof of Theorem 4: Approximate counting and sampling of chordal graphs

A random n-vertex chordal graph is a split graph with probability $1 - o(1)$. More precisely, we have the following result:

Proposition 1 (Bender et al. [53]) *If $\beta > \sqrt{3}/2$, n is sufficiently large, and G is a random n-vertex labeled chordal graph, then*

$$\Pr(G \text{ is a split graph}) > 1 - \beta^n.$$

Let N_3 be large enough that Proposition 1 holds for $n \geq N_3$. As above, let N_1 and N_2 be large enough that Lemmas 31 and 32 hold for $n \geq N_1$ and Lemma 35 holds for $n \geq N_2$. We define the function g as follows:

$$g(\varepsilon) = \max\{N_1, N_2, N_3, \log_{10/9}(2/\varepsilon), 3\log_{3/2}(2/\varepsilon)\}.$$

We are now ready to describe the approximate counting and sampling algorithms for Theorem 4.

Approximate counting algorithm. We are given n and ε as input. If $n < g(\varepsilon)$, then let z be the (exact) number of n-vertex chordal graphs computed by the counting algorithm from Theorem 1. Otherwise, we run the approximate split graph counting algorithm from Lemma 38 with input $(n, \frac{\varepsilon}{2})$ to obtain a $(1 \pm \frac{\varepsilon}{2})$-approximation of the number of n-vertex split graphs, and let z be this value. Return z.

Correctness and running time analysis (counting). To prove correctness, we need to check that we achieve the desired approximation ratio. This is clearly true if $n < g(\varepsilon)$ since in this case we output the exact answer. Now suppose $n \geq g(\varepsilon)$. By Proposition 1 with $\beta = \frac{9}{10}$, the (exact) number of n-vertex split graphs is a $(1 - \frac{\varepsilon}{2})$-approximation of the number n-vertex of chordal graphs since $n \geq \max\{N_3, \log_{10/9}(2/\varepsilon)\}$. The approximate counting algorithm from Lemma 38 computes a $(1 \pm \frac{\varepsilon}{2})$-approximation of the number of n-vertex split graphs since $n \geq \max\{N_1, N_2, 3\log_{3/2}(2/\varepsilon)\}$. Therefore, z is a $(1 \pm \varepsilon)$-approximation of the number of chordal graphs.

For the running time, if $n < g(\varepsilon)$, then the algorithm uses $O(\log^7(1/\varepsilon))$ arithmetic operations, so in this case the running time is $O(n^2 \log n \log^7(1/\varepsilon))$. Otherwise, the running time is $O(n^3 \log n \log^2(1/\varepsilon))$ by Lemma 38, so the overall running time is at most $O(n^3 \log n \log^7(1/\varepsilon))$.

Approximate sampling algorithm. We are given n and ε as input. If $n < g(\frac{\varepsilon}{2})$, then let G be a uniformly random chordal graph generated by the sampling algorithm from Theorem 1. Otherwise, let G be a random split graph generated by the sampling algorithm from Lemma 39 with input $(n, \frac{\varepsilon}{2})$. Return G.

Correctness and running time analysis (sampling). We need to show that the total variation distance between the output distribution and the uniform distribution is at most ε. This is clearly true if $n < g(\frac{\varepsilon}{2})$ since in this case the total variation distance is 0. For the case when $n \geq g(\frac{\varepsilon}{2})$, let us first compare the uniform distribution over n-vertex

split graphs to the uniform distribution over n-vertex chordal graphs. The total variation distance between these two distributions is equal to the number of non-split n-vertex chordal graphs divided by the total number of n-vertex chordal graphs. By Proposition 1, this is at most $\frac{\varepsilon}{2}$ since $n \geq \max\{N_3, \log_{10/9}(2/\varepsilon)\}$. Since $n \geq \max\{N_1, N_2, 3\log_{3/2}(4/\varepsilon)\}$, the approximate sampling algorithm from Lemma 39 samples a random n-vertex labeled split graph according to a distribution whose total variation distance from uniform (over split graphs) is at most $\frac{\varepsilon}{2}$. Therefore, by the triangle inequality, the total variation distance between the output distribution and the uniform distribution over n-vertex chordal graphs is at most ε.

For the running time, if $n < g(\frac{\varepsilon}{2})$, then the algorithm has a worst-cast running time bound of $O(\log^7(1/\varepsilon))$ arithmetic operations, i.e., $O(n^2 \log n \log^7(1/\varepsilon))$ time. Otherwise, the running time is $O(n^3 \log n \log^2(1/\varepsilon))$ by Lemma 39, so the overall running time is at most $O(n^3 \log n \log^7(1/\varepsilon))$.

This completes the proof of Theorem 4.

Remark 1 *We believe the running time of $O(n^3 \log n \log^7(1/\varepsilon))$ can be improved by storing only a truncated number digits of each number in the intermediate calculations.*

2.8 Pseudocode for the Sampling Algorithm

The procedure SAMPLE_CHORDAL(n) returns a uniformly random chordal graph with vertex set $[n]$, and SAMPLE_CONN_CHORDAL(n) returns a uniformly random *connected* chordal graph with vertex set $[n]$. To sample from $G(t, x, k, z)$, etc., one calls SAMPLE_$G(t, x, k, z)$, etc. Recall that $\phi(A, B)$ is the bijection defined in Section 1.5, which maps the smallest element of A to the smallest element of B, and so on.

Algorithm 1 Chordal graph sampler

procedure SAMPLE_CHORDAL(n)
  if $n = 0$ **then** ▷ Base case
    Let G be the empty graph with vertex set $\emptyset$ and edge set $\emptyset$
    return G
  end if

  Choose $k \in [n]$ at random such that $k = k'$ with probability $\binom{n-1}{k'-1} c(k') a(n-k')/a(n)$ for each $k' \in [n]$
  $G_1 \leftarrow$ SAMPLE_CONN_CHORDAL(k)
  $G_2 \leftarrow$ SAMPLE_CHORDAL($n-k$)
  Choose a uniformly random subset $C \subseteq [n]$ of size k containing 1
  $D \leftarrow [n] \setminus C$
  Relabel G_1 by applying $\phi([k], C)$ to its label set
  Relabel G_2 by applying $\phi([n-k], D)$ to its label set
  Let G be the union of G_1 and G_2
  return G
end procedure

procedure SAMPLE_CONN_CHORDAL(n)
  Choose $t \in [n]$ at random such that $t = t'$ with probability $\tilde{g}_1(t', 0, n)/c(n)$ for each $t' \in [n]$
  $G \leftarrow$ SAMPLE_$\widetilde{G}_1(t, 0, n)$
  return G
end procedure

procedure SAMPLE_$\widetilde{G}_1(t, x, k)$
  Choose $\ell \in [k]$ at random such that $\ell = \ell'$ with probability $\binom{k}{\ell'} f(t, x, \ell', k-\ell')/\tilde{g}_1(t, x, k)$ for each $\ell' \in [k]$
  $G \leftarrow$ SAMPLE_$F(t, x, \ell, k-\ell)$
  Choose a uniformly random subset $L \subseteq [x+1, x+k]$ of size ℓ
  $A \leftarrow [x+1, x+k] \setminus L$
  Relabel G by applying $\phi([x+1, x+\ell], L)$ to the labels in $[x+1, x+\ell]$ and applying $\phi([x+\ell+1, x+k], A)$ to the labels in $[x+\ell+1, x+k]$
  return G
end procedure

procedure GLUE(G_1, G_2, Y)
    Glue G_1 and G_2 together at Y, and let G be the resulting graph
    return G
end procedure

procedure SAMPLE_$F(t, x, \ell, k)$
    if $t = 1$ and $k = 0$ **then** ▷ Base case
        Let G be a complete graph with vertex set $[x + \ell]$
        return G
    end if

    Choose $k' \in [k]$ at random such that $k' = k''$ with probability
        $\binom{k}{k''} \tilde{f}(t, x, \ell, k'') g(t-2, x+\ell, k-k'', x) / f(t, x, \ell, k)$ for each $k'' \in [k]$
    $G_1 \leftarrow$ SAMPLE_$\widetilde{F}(t, x, \ell, k')$
    $G_2 \leftarrow$ SAMPLE_$G(t-2, x+\ell, k-k', x)$
    Choose a uniformly random subset $A \subseteq [x+\ell+1, x+\ell+k]$ of size k'
    $B \leftarrow [x+\ell+1, x+\ell+k] \setminus A$
    Relabel G_1 by applying $\phi([x+\ell+1, x+\ell+k'], A)$ to the labels in $[x+\ell+1, x+\ell+k']$
    Relabel G_2 by applying $\phi([x+\ell+1, x+\ell+k-k'], B)$ to the labels in $[x+\ell+1, x+\ell+k-k']$
    $G \leftarrow$ GLUE$(G_1, G_2, [x+\ell])$
    return G
end procedure

procedure SAMPLE_$G(t, x, k, z)$
  if $t = 0$ and $k = 0$ **then** ▷ Base case
    Let G be a complete graph with vertex set $[x]$
    return G
  end if

  Choose $k' \in \{0, 1, \ldots, k\}$ at random such that $k' = k''$ with probability $\binom{k}{k''}\tilde{g}(t, x, k'', z)g(t-1, x, k-k'', z)/g(t, x, k, z)$ for each $0 \le k'' \le k$
  $G_1 \leftarrow$ SAMPLE_$\widetilde{G}(t, x, k', z)$
  $G_2 \leftarrow$ SAMPLE_$G(t-1, x, k-k', z)$
  Choose a uniformly random subset $A \subseteq [x+1, x+k]$ of size k'
  $B \leftarrow [x+1, x+k] \setminus A$
  Relabel G_1 by applying $\phi([x+1, x+k'], A)$ to the labels in $[x+1, x+k']$
  Relabel G_2 by applying $\phi([x+1, x+k-k'], B)$ to the labels in $[x+1, x+k-k']$
  $G \leftarrow$ GLUE$(G_1, G_2, [x])$
  return G
end procedure

procedure SAMPLE_$\widetilde{G}(t, x, k, z)$
  if $k = 0$ **then** ▷ Base case
    Let G be a complete graph with vertex set $[x]$
    return G
  end if

  Choose $k' \in [k]$, $x' \in [x]$ at random such that $(k', x') = (k'', x'')$ with probability

$$\left(\binom{x}{x''} - \binom{z}{x''}\right)\binom{k-1}{k''-1}\tilde{g}_1(t, x'', k'')\tilde{g}(t, x, k-k'', z)/\tilde{g}(t, x, k, z)$$

    for each pair $(k'', x'') \in [k] \times [x]$
  $G_1 \leftarrow$ SAMPLE_$\widetilde{G}_1(t, x', k')$
  $G_2 \leftarrow$ SAMPLE_$\widetilde{G}(t, x, k-k', z)$
  Choose a uniformly random subset $X' \subseteq [x]$ of size x' such that $X' \not\subseteq [z]$
  Choose a uniformly random subset $C \subseteq [x+1, x+k]$ of size k' containing $x+1$
  $D \leftarrow [x+1, x+k] \setminus C$
  Relabel G_1 by applying $\phi([x'], X')$ to the labels in $[x']$ and applying
    $\phi([x'+1, x'+k'], C)$ to the labels in $[x'+1, x'+k']$
  Relabel G_2 by applying $\phi([x+1, x+k-k'], D)$ to the labels in $[x+1, x+k-k']$
  $G \leftarrow$ GLUE(G_1, G_2, X')
  return G
end procedure

procedure SAMPLE_$\widetilde{F}(t, x, \ell, k)$
  $S_1 \leftarrow \tilde{f}_p(t, x, \ell, k)$
  $S_2 \leftarrow \sum_{k'=1}^{k} \binom{k}{k'} \tilde{g}_1(t-1, x+\ell, k') \tilde{f}_p(t, x, \ell, k-k')$
  $S_3 \leftarrow \sum_{k'=1}^{k} \binom{k}{k'} \tilde{g}_{\geq 2}(t-1, x+\ell, k') \tilde{g}_p(t-1, x+\ell, k-k', x)$
  Choose $i \in [\tilde{f}(t, x, \ell, k)]$ uniformly at random
  if $i \leq S_1$ **then**
    $G \leftarrow$ SAMPLE_$\widetilde{F}_p(t, x, \ell, k)$
    return G
  else if $i \leq S_1 + S_2$ **then**
    Choose $k' \in [k]$ at random such that $k' = k''$ with probability
      $\binom{k}{k''} \tilde{g}_1(t-1, x+\ell, k'') \tilde{f}_p(t, x, \ell, k-k'')/S_2$ for each $k'' \in [k]$
    $G_1 \leftarrow$ SAMPLE_$\widetilde{G}_1(t-1, x+\ell, k')$
    $G_2 \leftarrow$ SAMPLE_$\widetilde{F}_p(t, x, \ell, k-k')$
  else
    Choose $k' \in [k]$ at random such that $k' = k''$ with probability
      $\binom{k}{k''} \tilde{g}_{\geq 2}(t-1, x+\ell, k'') \tilde{g}_p(t-1, x+\ell, k-k'', x)/S_3$ for each $k'' \in [k]$
    $G_1 \leftarrow$ SAMPLE_$\widetilde{G}_{\geq 2}(t-1, x+\ell, k')$
    $G_2 \leftarrow$ SAMPLE_$\widetilde{G}_p(t-1, x+\ell, k-k', x)$
  end if
  Choose a uniformly random subset $A \subseteq [x+\ell+1, x+\ell+k]$ of size k'
  $B \leftarrow [x+\ell+1, x+\ell+k] \setminus A$
  Relabel G_1 by applying $\phi([x+\ell+1, x+\ell+k'], A)$ to the labels in $[x+\ell+1, x+\ell+k']$
  Relabel G_2 by applying $\phi([x+\ell+1, x+\ell+k-k'], B)$ to the labels in $[x+\ell+1, x+\ell+k-k']$
  $G \leftarrow$ GLUE$(G_1, G_2, [x+\ell])$
  return G
end procedure

procedure SAMPLE_$\widetilde{G}_{\geq 2}(t, x, k)$
  $S_1 \leftarrow \sum_{k'=1}^{k-1} \binom{k-1}{k'-1} \tilde{g}_1(t, x, k') \tilde{g}_1(t, x, k - k')$
  $S_2 \leftarrow \sum_{k'=1}^{k-1} \binom{k-1}{k'-1} \tilde{g}_1(t, x, k') \tilde{g}_{\geq 2}(t, x, k - k')$
  Choose $i \in [\tilde{g}_{\geq 2}(t, x, k)]$ uniformly at random
  if $i \leq S_1$ **then**
    Choose $k' \in [k]$ at random such that $k = k''$ with probability
      $\binom{k-1}{k'-1} \tilde{g}_1(t, x, k'') \tilde{g}_1(t, x, k - k'')/S_1$ for each $k'' \in [k]$
    $G_1 \leftarrow$ SAMPLE_$\widetilde{G}_1(t, x, k')$
    $G_2 \leftarrow$ SAMPLE_$\widetilde{G}_1(t, x, k - k')$
  else
    Choose $k' \in [k]$ at random such that $k = k''$ with probability
      $\binom{k-1}{k'-1} \tilde{g}_1(t, x, k') \tilde{g}_{\geq 2}(t, x, k - k')/S_2$ for each $k'' \in [k]$
    $G_1 \leftarrow$ SAMPLE_$\widetilde{G}_1(t, x, k')$
    $G_2 \leftarrow$ SAMPLE_$\widetilde{G}_{\geq 2}(t, x, k - k')$
  end if
  Choose a uniformly random subset $C \subseteq [x + 1, x + k]$ of size k' containing $x + 1$
  $D \leftarrow [x + 1, x + k] \setminus C$
  Relabel G_1 by applying $\phi([x + 1, x + k'], C)$ to the labels in $[x + 1, x + k']$
  Relabel G_2 by applying $\phi([x + 1, x + k - k'], D)$ to the labels in $[x + 1, x + k - k']$
  $G \leftarrow$ GLUE$(G_1, G_2, [x])$
  return G
end procedure

procedure SAMPLE_$\widetilde{G}_p(t, x, k, z)$
  if $k = 0$ **then** ▷ Base case
    Let G be a complete graph with vertex set $[x]$
    return G
  end if

  Choose $k' \in [k]$, $x' \in [x-1]$ at random such that $(k', x') = (k'', x'')$ with probability

$$\left(\binom{x}{x''} - \binom{z}{x''}\right)\binom{k-1}{k''-1}\tilde{g}_1(t, x'', k'')\tilde{g}_p(t, x, k-k'', z)/\tilde{g}_p(t, x, k, z)$$

    for each pair $(k'', x'') \in [k] \times [x-1]$
  $G_1 \leftarrow$ SAMPLE_$\widetilde{G}_1(t, x', k')$
  $G_2 \leftarrow$ SAMPLE_$\widetilde{G}_p(t, x, k-k', z)$
  Choose a uniformly random subset $X' \subseteq [x]$ of size x' such that $X' \not\subseteq [z]$
  Choose a uniformly random subset $C \subseteq [x+1, x+k]$ of size k' containing $x+1$
  $D \leftarrow [x+1, x+k] \setminus C$
  Relabel G_1 by applying $\phi([x'], X')$ to the labels in $[x']$ and applying $\phi([x'+1, x'+k'], C)$
    to the labels in $[x'+1, x'+k']$
  Relabel G_2 by applying $\phi([x+1, x+k-k'], D)$ to the labels in $[x+1, x+k-k']$
  $G \leftarrow$ GLUE(G_1, G_2, X')
  return G
end procedure

procedure SAMPLE_$\widetilde{F}_p(t, x, \ell, k)$ ▷ Four arguments, without z
  return SAMPLE_$\widetilde{F}_p(t, x, \ell, k, x)$
end procedure

procedure SAMPLE_$\widetilde{F}_p$_$With_Z(t, x, \ell, k, z)$ ▷ Five arguments, including z

Choose $k' \in [k]$, $x' \in \{0, 1, \ldots, x\}$, $\ell' \in \{0, 1, \ldots, \ell\}$ at random such that $0 < x' + \ell' < x + \ell$ and such that $(k', x', \ell') = (k'', x'', \ell'')$ with probability

$$\binom{k-1}{k''-1}\binom{\ell}{\ell''}\tilde{g}_1(t-1, x''+\ell'', k'') \cdot \begin{cases} \binom{x}{x''} & \text{if } \ell'' > 0 \\ \binom{x}{x''} - \binom{z}{x''} & \text{otherwise} \end{cases} \cdot \begin{cases} \tilde{f}_p(t, x+\ell'', \ell-\ell'', k-k'', z) & \text{if } \ell'' < \ell \\ \tilde{g}_p(t-1, x+\ell'', k-k'', z) & \text{otherwise} \end{cases} \cdot 1/\tilde{f}_p(t, x, \ell, k, z)$$

for each possible triple (k'', x'', ℓ'')

$G_1 \leftarrow$ SAMPLE_$\widetilde{G}_1(t-1, x'+\ell', k')$

$G_2 \leftarrow \ell' < \ell$? SAMPLE_$\widetilde{F}_p(t, x+\ell', \ell-\ell', k-k', z)$: SAMPLE_$\widetilde{G}_p(t-1, x+\ell', k-k', z)$

if $\ell' > 0$ **then**

Choose a uniformly random subset $X' \subseteq X$ of size x' containing $x + \ell + 1$

else

Choose a uniformly random subset $X' \subseteq X$ of size x' containing $x + \ell + 1$ such that $X' \not\subseteq [z]$

end if

Choose a uniformly random subset $L' \subseteq [x+1, x+\ell]$ of size ℓ'

Choose a uniformly random subset $C \subseteq [x+\ell+1, x+\ell+k]$ of size k' containing $x + \ell + 1$

$D \leftarrow [x+\ell+1, x+\ell+k] \setminus C$

Relabel G_1 by applying $\phi([x'], X')$ to the labels in $[x']$, applying $\phi([x'+1, x'+\ell'], L')$ to the labels in $[x'+1, x'+\ell']$, and applying $\phi([x'+\ell'+1, x'+\ell'+k'], C)$ to the labels in $[x'+\ell'+1, x'+\ell'+k']$

Relabel G_2 by applying $\phi([x+\ell+1, x+\ell+k-k'], D)$ to the labels in $[x+\ell+1, x+\ell+k-k']$

if $\ell' < \ell$ **then**

Relabel G_2 by applying $\phi([x+1, x+\ell'], L')$ to the labels in $[x+1, x+\ell']$ and applying $\phi([x+\ell'+1, x+\ell], [x+1, \ldots, x+\ell] \setminus L')$ to the labels in $[x+\ell'+1, x+\ell]$

end if

$G \leftarrow$ GLUE$(G_1, G_2, X' \cup L')$

return G

end procedure

Chapter 3

Sampling Unlabeled Chordal Graphs in Expected Polynomial Time

3.1 Introduction

In Chapter 2, we designed an algorithm that generates n-vertex labeled chordal graphs uniformly at random. This algorithm runs in polynomial time, using at most $O(n^7)$ arithmetic operations for the first sample and $O(n^4)$ arithmetic operations for each subsequent sample. However, when discussing the performance of an algorithm that is being tested, the correct output typically does not depend on the labeling of the vertices. If we use a labeled-graph sampling algorithm to generate random test cases, then asymmetric graphs will be given too much weight/probability compared to those that happen to have many automorphisms. This naturally leads to the question of efficiently generating *unlabeled* chordal graphs uniformly at random. In this chapter, we present an algorithm that solves this problem and runs in expected polynomial time.

Theorem 5 *There is a randomized algorithm that given $n \in \mathbb{N}$, generates a graph uniformly at random from the set of all unlabeled chordal graphs on n vertices. This algorithm*

uses $O(n^7)$ arithmetic operations in expectation.

It is worth mentioning the difference between the running times for labeled vs. unlabeled chordal graph sampling. The sampling algorithm in Chapter 2 generates a random n-vertex labeled chordal graph in polynomial time, even in the worst case. However, obtaining a worst-case polynomial-time algorithm for sampling *unlabeled* chordal graphs — or an expected-polynomial-time counting algorithm for such graphs — appears to be very difficult since these questions remain open even for general graphs. This is relevant because the class of chordal graphs is known to be GI-complete. While there exist classes of graphs for which efficient unlabeled sampling algorithms are known (see Chapter 1), we are not aware of any GI-complete graph class with such an algorithm.

Our algorithm for sampling unlabeled chordal graphs builds upon an algorithm of Wormald that generates n-vertex unlabeled graphs uniformly at random in expected time $O(n^2)$ [16]. This in turn builds upon a related algorithm by Dixon and Wilf [92] that solves the same problem but assumes that the exact number of n-vertex unlabeled graphs has already been computed. In [16], the algorithm of Wormald follows a somewhat similar structure but removes that assumption. As mentioned above, the question of computing the exact number of n-vertex unlabeled graphs in expected polynomial time remains open to this day.

3.1.1 Methods

The sampling algorithm of Wormald [16] is based on the fact that unlabeled graphs correspond to orbits of the following group action: the symmetric group S_n acts on the set of labeled graphs by permuting the vertex labels. This correspondence follows from the Frobenius-Burnside lemma. Therefore, to sample a random unlabeled graph, it is enough to sample a random orbit of this group action.

To make this approach work for chordal graphs, we need two ingredients: (1) an algorithm for counting and sampling labeled chordal graphs with a given automorphism π, and (2) a proof that the probability that a random n-vertex labeled chordal graph has a given automorphism $\pi \in S_n$ is at most $1/2^{c \max\{\mu^2, n\}}$, where μ is the number of moved points of π (i.e., the non-fixed points) and c is a constant.

For (1), we design an FPT (fixed-parameter tractable) counting algorithm that is parameterized by μ, the number of moved points of π. This algorithm uses $O(2^{7\mu} n^9)$ arithmetic operations. Using the standard sampling-to-counting reduction of [1], we also obtain a corresponding sampling algorithm with the same running time. Our main algorithm (for sampling *unlabeled* chordal graphs) calls each of these FPT algorithms as a black box with potentially large values of the parameter μ. Nevertheless, using the bound from (2), we are able to show that the probability of using a large value of μ is exponentially small, so the expected running time is not significantly affected.

To design the counting algorithm for (1), we rewrite each of the recurrences from the algorithm in Chapter 2, now carrying around information about the automorphism π and its moved points. The original algorithm is a dynamic-programming algorithm in which there is a constant number of types of vertices (X, L, etc.) in each graph that we wish to count. In our updated version, we now keep track of the type of each vertex that is moved by π.

To prove the bound for (2), we distinguish between the cases when μ is small and μ is large (μ is either less than or greater than $\frac{n}{d \log n}$, where d is a constant). When μ is small, we use the fact that almost every chordal graph is a split graph [53], and we strengthen this by showing that in fact, almost every chordal graph is a balanced split graph. For the case of balanced split graphs, the argument is easy and is similar to the proof of the bound for general graphs [93]. When μ is large, the argument is more complicated. We observe that the vast majority of n-vertex labeled chordal graphs have maximum clique

size close to $n/2$. Along the way, we also use the fact that for every chordal graph G, there exists a PEO (perfect elimination ordering) of G such that some maximum clique appears at the tail end of that PEO.

3.2 Preliminaries

3.2.1 Permutations

For $n \in \mathbb{N}$, let S_n denote the group of all permutations of $[n]$. For a permutation $\pi \in S_n$, we define $M_\pi \coloneqq \{i \in [n] : \pi(i) \neq i\}$ to be the set of points moved by π. For $n \in \mathbb{N}$, $[n]_0 \coloneqq \{0, 2, 3, \ldots, n\}$ denotes the set of all possible values of $|M_\pi|$ for $\pi \in S_n$. We write $\mathrm{id} \in S_n$ for the identity permutation.

Suppose $\pi \in S_n$, $C \subseteq [n]$. We write $\pi(C) \coloneqq \{\pi(i) : i \in C\}$ to denote the image of C under π. We say C is *invariant* under π if $\pi(i) \in C$ for all $i \in C$. For a set C that is invariant under π, we write $\pi|_C$ to denote the permutation π restricted to the domain C. For a permutation $\pi \in S_n$ and a labeled graph G such that $V(G) \subseteq [n]$ is invariant under π, we say $\pi|_{V(G)}$ is an *automorphism* of G if for all $u, v \in V(G)$, u and v are adjacent if and only if $\pi(u)$ and $\pi(v)$ are adjacent. In other words, this is an isomorphism from $V(G)$ to itself.

3.2.2 Chordal graphs and evaporation sequences

Our algorithm for counting the number of labeled chordal graphs with a given automorphism uses the notion of evaporation sequences from Chapter 2. See Section 2.3.1 for the relevant definitions. Also, see Definition 2 for the concept of gluing together two chordal graphs. The following two lemmas are well-known facts about chordal graphs. The proof of the first can be found in [90].

Lemma 40 *Every chordal graph G contains a simplicial vertex. If G is not a complete graph, then G contains two non-adjacent simplicial vertices.*

Lemma 41 *A chordal graph G on n vertices has at most n maximal cliques.*

Proof: Let C be a maximal clique in G, and let v_i be the leftmost vertex of C in a given perfect elimination ordering $v_1, \ldots, v_n$ of G. We claim that C is equal to the closed neighborhood to the right of v_i, i.e., $C = N[v_i] \cap \{v_i, \ldots, v_n\}$. It is clear that C is contained in $N[v_i] \cap \{v_i, \ldots, v_n\}$ since v_i has no other neighbors to its right, so we indeed have $C = N[v_i] \cap \{v_i, \ldots, v_n\}$ by maximality of C. Therefore, there are at most n maximal cliques. ■

3.3 Sampling Unlabeled Chordal Graphs

In [16], Wormald presented an algorithm that generates an n-vertex unlabeled graph uniformly at random in expected time $O(n^2)$. In this chapter, we achieve a similar result for chordal graphs:

Theorem 5 *There is a randomized algorithm that given $n \in \mathbb{N}$, generates a graph uniformly at random from the set of all unlabeled chordal graphs on n vertices. This algorithm uses $O(n^7)$ arithmetic operations in expectation.*

The sampling algorithm of Wormald makes use of the fact that unlabeled graphs correspond to orbits of a particular group action. Since our algorithm will follow a similar outline, we begin by discussing some of the ideas behind the algorithm of Wormald.

Suppose we have been given $n \in \mathbb{N}$ as input, and let Ω be the set of all labeled graphs with vertex set $[n]$. The symmetric group S_n acts on Ω in the following way: For each $\pi \in S_n$ and $G \in \Omega$, $\pi \cdot G$ is the graph that results from permuting the vertex labels of

G according to π. The orbits of Ω under the action of S_n are the isomorphism classes of labeled graphs, each of which corresponds to an unlabeled graph. Let

$$\Gamma = \{(\pi, G) \in S_n \times \Omega : \pi \text{ is an automorphism of } G\}.$$

Suppose we fix an n-vertex unlabeled graph H, and let the corresponding isomorphism class of labeled graphs be $\mathcal{H}$. As is shown in [16], the number of pairs $(\pi, G) \in \Gamma$ such that $G \in \mathcal{H}$ is equal to $|S_n| = n!$. This follows from the Frobenius-Burnside lemma. Therefore, if we sample a random pair $(\pi, G) \in \Gamma$ uniformly at random, and then we forget the labels on the graph G, this amounts to sampling an n-vertex unlabeled graph uniformly at random.

In the case of chordal graphs, the same statements hold true. Let Ω_{chord} be the set of all labeled chordal graphs with vertex set $[n]$. The symmetric group S_n acts on Ω_{chord} in the same way as above, by permuting the vertex labels. The set of orbits of this group action corresponds to the set of unlabeled chordal graphs. Let

$$\Gamma_{\text{chord}} = \{(\pi, G) \in S_n \times \Omega_{\text{chord}} : \pi \text{ is an automorphism of } G\}.$$

Let H_c be an unlabeled chordal graph, and let $\mathcal{H}_c$ be the corresponding isomorphism class of labeled graphs. Applying the Frobenius-Burnside lemma to the orbit corresponding to $\mathcal{H}_c$ shows that the number of pairs $(\pi, G) \in \Gamma_{\text{chord}}$ such that $G \in \mathcal{H}_c$ is equal to $n!$.

In [16], Wormald describes an algorithm for sampling a random pair $(\pi, G) \in \Gamma$ in order to sample a random unlabeled graph. The same outline can be used to sample a random pair $(\pi, G) \in \Gamma_{\text{chord}}$. However, there are two key points where some difficulty arises. First of all, in one of the steps of the algorithm that samples from Γ, it is necessary to count the number of n-vertex labeled graphs with a given automorphism (and sample from the set of such graphs). This is easy to do for general graphs but becomes more complicated for chordal graphs (see Sections 3.4 and 3.5). Second, the

algorithm of Wormald uses the fact that the number of labeled graphs with a given automorphism π is at most $2^{\binom{n}{2}-\mu n/2+\mu(\mu+2)/4}$, where $\mu = |M_\pi|$ is the number of moved points of π. To transform this into an algorithm for sampling unlabeled chordal graphs, it is necessary to prove similar bounds on the number of labeled chordal graphs with a given automorphism. These will be the bounds B_μ in our algorithm.

3.3.1 Algorithm for sampling unlabeled chordal graphs

Let Count_Chordal_Lab(n) stand for the counting algorithm in Chapter 2 that computes the number of n-vertex labeled chordal graphs. In Sections 3.4 and 3.5, we will prove the following two theorems.

Theorem 6 *There is a deterministic algorithm that given $n \in \mathbb{N}$ and $\pi \in S_n$, computes the number of labeled chordal graphs with vertex set $[n]$ for which π is an automorphism. This algorithm uses $O(2^{7\mu}n^9)$ arithmetic operations, where $\mu = |M_\pi|$.*

Theorem 7 *There is a randomized algorithm that given $n \in \mathbb{N}$ and $\pi \in S_n$, generates a graph uniformly at random from the set of all labeled chordal graphs with vertex set $[n]$ for which π is an automorphism. This algorithm uses $O(2^{7\mu}n^9)$ arithmetic operations, where $\mu = |M_\pi|$.*

In our algorithm for sampling unlabeled chordal graphs, Count_Chordal_Lab(n, π) stands for the algorithm of Theorem 6 and Sample_Chordal_Lab(n, π) stands for the algorithm of Theorem 7.

Recall that $[n]_0 = \{0, 2, 3, \ldots, n\}$. For $\mu \in [n]_0$, let $R_\mu \coloneqq \{\pi \in S_n : |M_\pi| = \mu\}$ be the set of permutations with exactly μ moved points. For $\pi \in S_n$, let $\mathrm{Fix}(\pi)$ denote the set of n-vertex labeled chordal graphs G such that π is an automorphism of G. To follow the same approach as the algorithm of Wormald, for each $\mu \in [n]_0$, we need an upper bound

B_μ that satisfies $B_\mu \geq |R_\mu||\mathrm{Fix}(\pi)|$ for all $\pi \in R_\mu$. Let B_0 be the number of n-vertex labeled chordal graphs, which is exactly equal to $|R_0||\mathrm{Fix}(\mathrm{id})|$. For $2 \leq \mu \leq \frac{n}{200 \log n}$, let

$$B_\mu = \left(B_0 (9/10)^n + 2^n 2^{2n^2/9} + n \cdot 2^{n^2/4 + n/2} \right) n^\mu \mu!, \tag{3.1}$$

and for $\frac{n}{200 \log n} < \mu \leq n$, let

$$B_\mu = n^{2n+1} 2^{n^2/4 - f(\mu)} n^\mu \mu!, \tag{3.2}$$

where $f(\mu) = \frac{\mu^2}{900} - \frac{\mu}{10}$. In Section 3.3.2, we will prove that we indeed have $B_\mu \geq |R_\mu||\mathrm{Fix}(\pi)|$ for all $\pi \in R_\mu$, when n is sufficiently large. Let $B = \sum_{\mu \in [n]_0} B_\mu$.

Our algorithm for sampling a random n-vertex unlabeled chordal graph is given in Algorithm 2. The general idea is as follows. For $\mu \in [n]_0$, let $\Gamma_\mu = \{(\pi, G) \in \Gamma_{\mathrm{chord}} : |M_\pi| = \mu\}$. We choose μ such that the probability of each value of μ is B_μ / B, which is approximately equal to $\Gamma_\mu / \Gamma_{\mathrm{chord}}$. (We do not know how to efficiently compute the exact value of $\Gamma_\mu / \Gamma_{\mathrm{chord}}$ since we do not know the exact number of unlabeled chordal graphs.) Since B_μ / B is not exactly equal to $\Gamma_\mu / \Gamma_{\mathrm{chord}}$, we adjust for this by restarting with a certain probability in Step 11. We then proceed to select a random pair $(\pi, G) \in \Gamma_\mu$, and we output the graph G without labels.

Algorithm 2 Unlabeled chordal graph sampler

1: **procedure** SAMPLE_CHORDAL_UNLABELED(n)
2: ▷ Setup
3: $B_0 \leftarrow$ COUNT_CHORDAL_LAB(n)
4: Let B_μ be given by Equation (3.1) for $2 \le \mu \le \frac{n}{200 \log n}$
5: Let B_μ be given by Equation (3.2) for $\frac{n}{200 \log n} < \mu \le n$
6: $B \leftarrow \sum_{\mu \in [n]_0} B_\mu$
7:
8: ▷ Main algorithm
9: Choose $\mu \in [n]_0$ at random such that $\mu = i$ with probability B_i/B for each $i \in [n]_0$
10: Choose $\pi \in R_\mu$ uniformly at random
11: Restart (go back to Step 9) with probability
$1 - |R_\mu| \cdot$ COUNT_CHORDAL_LAB$(n, \pi)/B_\mu$
12: $G \leftarrow$ SAMPLE_CHORDAL_LAB(n, π)
13: Forget the labels on the vertices of G
14: **return** G
15: **end procedure**

Step 10 can easily be implemented in $O(n)$ time in the following way. If $\mu = 0$, let $\pi = \mathrm{id}$. For $\mu \ge 2$, we can repeatedly choose a random permutation of $[\mu]$ until we obtain a derangement. The expected number of trials for this is a constant (see [16]).

In Step 11, to compute $|R_\mu|$, we observe that $|R_0| = 1$ and $|R_\mu| = !\mu\binom{n}{\mu}$ for $\mu \ge 2$. Here $!\mu$ is the number of derangements of μ. We compute $!\mu$ using the formula $!m = m! \sum_{i=0}^{m} (-1)^i/i!$ for $m \in \mathbb{N}$, which can be derived using the inclusion-exclusion principle.

3.3.2 Correctness of Algorithm 2

See Section 3.5 for the proof of correctness of COUNT_CHORDAL_LAB(n, π) and SAMPLE_CHORDAL_LAB(n, π).

We need to show that the output graph is chosen uniformly at random. One "iteration" refers to one run of Steps 9 to 11 or 9 to 14. In a given iteration, we say (π, G) was "chosen" if π was chosen from R_μ and G was chosen by SAMPLE_CHORDAL_LAB(n, π). We claim that for all $(\pi, G) \in \Gamma_{\text{chord}}$, in any given iteration, the probability that (π, G)

is chosen is $1/B$. Indeed, this probability is equal to

$$\frac{B_\mu}{B}\frac{1}{|R_\mu|}\frac{|R_\mu||\mathrm{Fix}(\pi)|}{B_\mu}\frac{1}{|\mathrm{Fix}(\pi)|} = \frac{1}{B},$$

where $\mu = |M_\pi|$, since the probability of choosing G in Step 12 is $1/|\mathrm{Fix}(\pi)|$. Therefore, we output all n-vertex unlabeled chordal graphs with equal probability.

Next, we need to show that $B_\mu \geq |R_\mu||\mathrm{Fix}(\pi)|$ for all $\pi \in R_\mu$ to verify that the probability $|R_\mu||\mathrm{Fix}(\pi)|/B_\mu$ in Step 11 is at most 1. When $\mu = 0$ this is an equality, so suppose $\mu \geq 2$. Clearly $|R_\mu| \leq n^\mu \mu!$, so we just need to prove that the number of n-vertex labeled chordal graphs with automorphism π is at most $B_\mu/(n^\mu\mu!)$ for all $\pi \in S_n$ with μ moved points.

In the case when B_μ is defined according to Equation (3.2), this follows from Theorem 8. The proof of this theorem can be found in Section 3.6. (The bound in Theorem 8 is in fact true for all values of μ — the reason why we define B_μ differently for smaller values of μ will become clear when we discuss the running time in Section 3.3.3.)

Theorem 8 *Let $n \in \mathbb{N}$, $\pi \in S_n$, and let $\mu = |M_\pi|$. The number of labeled chordal graphs with vertex set $[n]$ for which π is an automorphism is at most*

$$n^{2n+1}2^{n^2/4-f(\mu)},$$

where $f(\mu) = \frac{\mu^2}{900} - \frac{\mu}{10}$.

For the other case, suppose $2 \leq \mu \leq \frac{n}{200\log n}$, and suppose $\pi \in S_n$ is a permutation with μ moved points. We need to show that the number of n-vertex labeled chordal graphs with automorphism π is at most $B_\mu/(n^\mu\mu!)$. We begin by reducing to the case of split graphs. A *split* graph is a graph whose vertex set can be partitioned into a clique and an independent set, with arbitrary edges between the two parts. It is easy to see that every split graph is chordal. Furthermore, the following result by Bender et al. [53]

shows that a random n-vertex labeled chordal graph is a split graph with probability $1 - o(1)$.

Proposition 2 (Bender et al. [53]) *If $\alpha > \sqrt{3}/2$, n is sufficiently large, and G is a random n-vertex labeled chordal graph, then*

$$\Pr(G \text{ is a split graph}) > 1 - \alpha^n.$$

Applying this proposition with $\alpha = \frac{9}{10}$ tells us that the number of n-vertex labeled chordal graphs that are *not split* is at most $B_0\left(\frac{9}{10}\right)^n$. To bound the number of n-vertex labeled split graphs with automorphism π, we consider two cases: balanced split graphs and unbalanced split graphs. We say an n-vertex split graph is *balanced* if $|C| \geq \frac{n}{3}$ and $|I| \geq \frac{n}{3}$ for every split partition (C, I) of G. It is easy to bound the number of unbalanced split graph as follows.

Lemma 42 *The number of labeled split graphs on n vertices that are not balanced is at most $2^n 2^{2n^2/9}$.*

Proof: Suppose (C, I) is a partition of $[n]$ into two parts such that $|C| < \frac{n}{3}$ or $|I| < \frac{n}{3}$. The number of labeled split graphs with this particular split partition is at most $2^{|I||C|} \leq 2^{\frac{n}{3}\cdot\frac{2n}{3}} = 2^{2n^2/9}$. Therefore, the number of unbalanced labeled split graphs on n vertices is at most $2^n 2^{2n^2/9}$ since there are at most 2^n possible partitions. ■

The following lemma will be useful for bounding the number of balanced split graphs.

Lemma 43 *Let $\pi \in S_n$, and let G be an n-vertex labeled split graph. If π is an automorphism of G, then there exists a split partition (C, I) of G such that C and I are invariant under π.*

Proof: Let $\hat{C}$ be the set of vertices that belong to the clique in every split partition of G, let $\hat{I}$ be the set of vertices that belong to the independent set in every split partition

of G, and let $\hat{Q} = V(G) \setminus (\hat{C} \cup \hat{I})$. By Observation 6 in Chapter 2, every vertex in $\hat{Q}$ is adjacent to every vertex in $\hat{C}$ and is non-adjacent to every vertex in $\hat{I}$. By Lemma 25, $\hat{Q}$ is either a clique or an independent set. Suppose π is an automorphism of G. If $\hat{Q}$ is a clique, then the split partition $(\hat{C} \cup \hat{Q}, \hat{I})$ has the desired property. If $\hat{Q}$ is an independent set, then the split partition $(\hat{C}, \hat{I} \cup \hat{Q})$ has the desired property. ■

Lemma 44 *Let $\pi \in S_n$, and suppose $|M_\pi| \geq 2$. The number of balanced labeled split graphs G on n vertices such that π is an automorphism of G is at most*

$$n \cdot 2^{n^2/4+n/2}.$$

Proof: Suppose (C, I) is a partition of $[n]$ into two parts. Let $i = |I|$ and $c = |C|$. The number of labeled split graphs with this particular split partition is 2^{ic}. Therefore, the number of labeled split graphs with automorphism π for which (C, I) has the property from Lemma 43 is at most 2^{ic}. Let $Z_{(C,I)}$ be the number of such graphs. Whenever two vertices $u, v \in I$ (resp. C) belong to the same cycle in the cycle decomposition of π, u and v must have the same relationship as each other to each of the vertices in C (resp. I). Therefore, we in fact have an upper bound of $\max\{2^{(i-1)c}, 2^{i(c-1)}\}$ on $Z_{(C,I)}$, since π has at least two moved points. If $i \geq \frac{n}{3}$ and $c \geq \frac{n}{3}$, then we have $\max\{2^{(i-1)c}, 2^{i(c-1)}\} \leq 2^{n^2/4-n/2}$.

By Lemma 43, every balanced labeled split graph on n vertices with automorphism π has a split partition (C, I) such that C and I are invariant under π. Furthermore, this split partition is balanced (i.e., both parts have size at least $\frac{n}{3}$). Therefore, the number of balanced labeled split graphs on n vertices with automorphism π is at most

$$\sum_{\lceil \frac{n}{3} \rceil \leq i \leq \lfloor \frac{2n}{3} \rfloor} 2^n \cdot 2^{n^2/4-n/2} \leq n \cdot 2^{n^2/4+n/2}$$

since the number of partitions (C, I) with $|I| = i$ is certainly at most 2^n. ■

Putting together Proposition 2 and Lemmas 42 and 44, we can see that

$$\left(B_0(9/10)^n + 2^n 2^{2n^2/9} + n \cdot 2^{n^2/4+n/2}\right) n^\mu \mu!$$

is an upper bound on $|R_\mu||\mathrm{Fix}(\pi)|$ when n is sufficiently large.

Let N_0 be the cutoff such that this works for $n \geq N_0$. To be precise, when implementing this algorithm, we would solve the problem by brute force if $n < N_0$ (by generating all possible n-vertex chordal graphs and then selecting one at random), and we would run Algorithm 2 as written if $n \geq N_0$.

3.3.3 Running time of Algorithm 2

The running time of Steps 1-6 is $O(n^7)$ arithmetic operations since that is the running time of COUNT_CHORDAL_LAB(n). In this section, we will show that the rest of the algorithm uses only $O(n^7)$ arithmetic operations in expectation.

If we choose $\mu = 0$ in Step 9 of Algorithm 2, then the algorithm is guaranteed to terminate in that iteration. Thus the expected number of iterations is at most B/B_0. Let T be the expected running time of Algorithm 2 after completing Step 6 (this is where we choose μ at random and the loop begins). In Lemma 45, we show that $\mathbf{E}[T]$ is at most the product of B/B_0 and the expected running time of one iteration.

Let N be the number of iterations of Algorithm 2, and let T_j be the time spent in iteration j for each $j \in [N]$. We have $T = \sum_{j=1}^{N} T_j$.

Lemma 45 *We have* $\mathbf{E}[T] \leq \frac{B}{B_0}\mathbf{E}[T_1]$.

Proof: Clearly $\mathbf{E}[T]$ is finite since the expected number of iterations is finite and the procedures COUNT_CHORDAL_LAB(n, π) and SAMPLE_CHORDAL_LAB(n, π) have worst-case running time bounds. Therefore, we can solve for $\mathbf{E}[T]$ in the following way.

The steps that we run in one iteration (Steps 9-14) do not depend on j — they are always the same, regardless of how many iterations have happened so far. Thus we have $\mathbf{E}\left[\sum_{j=2}^{N} T_j \,\middle|\, N > 1\right] = \mathbf{E}[T]$, which implies $\mathbf{E}[T \mid N > 1] = \mathbf{E}[T_1 \mid N > 1] + \mathbf{E}[T]$.

Therefore, we have

$$\begin{aligned}\mathbf{E}[T] &= \mathbf{E}[T \mid N=1]\Pr(N=1) + \mathbf{E}[T \mid N>1]\Pr(N>1) \\ &= \mathbf{E}[T_1 \mid N=1]\Pr(N=1) + \big(\mathbf{E}[T_1 \mid N>1] + \mathbf{E}[T]\big)\cdot\Pr(N>1)\end{aligned}$$

$$\begin{aligned}\implies \mathbf{E}[T](1-\Pr(N>1)) &= \mathbf{E}[T_1 \mid N=1]\Pr(N=1) + \mathbf{E}[T_1 \mid N>1]\Pr(N>1) \\ &= \mathbf{E}[T_1]\end{aligned}$$

$$\implies \mathbf{E}[T] = \frac{\mathbf{E}[T_1]}{\Pr(N=1)} \leq \frac{B}{B_0}\mathbf{E}[T_1].$$

■

For $\mu \in [n]_0$, let $T(n,\mu)$ be an upper bound on the time it takes to run one iteration, assuming we have chosen this particular value of μ in Step 9. By the running time of Count_Chordal_Lab(n,π) and Sample_Chordal_Lab(n,π), we can assume $T(n,\mu) = O(2^{7\mu}n^9)$. We have

$$\mathbf{E}[T_1] = \sum_{\mu\in[n]_0} \frac{B_\mu}{B} T(n,\mu).$$

To prove a bound on $\mathbf{E}[T_1]$, we will start by proving a bound on B_μ/B for all $\mu \in [n]_0$. Since $B_0 \leq B$, it is sufficient to prove a bound on B_μ/B_0 for all $\mu \in [n]_0$. The following lemma gives us a lower bound on B_0.

Lemma 46 *For $n \geq 2$, the number of n-vertex labeled chordal graphs is at least*

$$\frac{2^n 2^{n^2/4}}{n^2}.$$

Proof: It is enough to just consider n-vertex labeled split graphs with a split partition (C,I) such that $|C| = \lfloor \frac{n}{2} \rfloor$. Since $\binom{n}{\lfloor \frac{n}{2} \rfloor} \geq 2^n/n$ for $n \geq 2$, the number of such graphs is at least

$$\binom{n}{\lfloor \frac{n}{2} \rfloor} \frac{2^{n^2/4}}{n} \geq \frac{2^n 2^{n^2/4}}{n^2}.$$

We divide by n in the first expression since each split graph can have up to n distinct split partitions in which C is of a given size [28]. ■

Lemma 47 *Suppose $n \geq 13$. If $2 \leq \mu \leq \frac{n}{200 \log n}$, then*

$$\frac{B_\mu}{B_0} \leq \frac{3}{n^{16\mu}}.$$

Proof: Let $B_\mu^{(1)}$, $B_\mu^{(2)}$, and $B_\mu^{(3)}$ be the three terms that are added together in Equation (3.1), in order, so that $B_\mu = \left(B_\mu^{(1)} + B_\mu^{(2)} + B_\mu^{(3)}\right) n^\mu \mu!$. We have

$$\frac{B_\mu^{(1)} n^\mu \mu!}{B_0} = \left(\frac{9}{10}\right)^n n^\mu \mu!,$$

and we claim that this is at most $1/n^{16\mu}$. Since $\mu! \leq n^\mu$, it is sufficient to show $n^{18\mu} \leq (10/9)^n$, which is true when $\mu \leq \frac{n}{200 \log n}$.

For the next term, by Lemma 46 we have

$$\frac{B_\mu^{(2)} n^\mu \mu!}{B_0} \leq n^2 2^{-n^2/36} n^\mu \mu!.$$

To see that this is at most $1/n^{16\mu}$, it is sufficient to show $n^{19/200} \leq 2^{n/36}$ since $\mu \leq \frac{n}{200}$. This is indeed true for $n \geq 13$.

For the third term, by Lemma 46 we have

$$\frac{B_\mu^{(3)} n^\mu \mu!}{B_0} \leq n^3 2^{-n/2} n^\mu \mu!.$$

To see that this is at most $1/n^{16\mu}$, it is sufficient to show $n^{20\mu} \leq 2^{n/2}$, which is true when $\mu \leq \frac{n}{40 \log n}$. Adding together these three terms, we obtain $B_\mu / B_0 \leq 3/n^{16\mu}$. ■

Lemma 48 *For sufficiently large n, if $\frac{n}{200 \log n} < \mu \leq n$, then*

$$\frac{B_\mu}{B_0} \leq \frac{1}{n^{16\mu}}.$$

Proof: Lemma 46 implies $B_0 \geq 2^{n^2/4}$ for $n \geq 4$, so we have

$$\frac{B_\mu}{B_0} \leq n^{2n+1} 2^{-f(\mu)} n^\mu \mu!,$$

where $f(\mu) = \frac{\mu^2}{900} - \frac{\mu}{10}$. We claim that this expression is at most $1/n^{16\mu}$. Since $\mu! \leq n^\mu$, it is sufficient to show $n^{2n+1} n^{18\mu} \leq 2^{f(\mu)}$. When n is sufficiently large,[1] we have $\log^3 n \leq \frac{1}{200^2 \cdot 1800} \frac{n^2}{2n+1}$. Since $\mu \geq \frac{n}{200 \log n}$, this implies

$$n^{2n+1} \leq 2^{\mu^2/1800}. \tag{3.3}$$

When n is sufficiently large,[2] we also have $n \geq 18 \cdot 900 \cdot 200 \log^2 n + 90 \cdot 200 \log n$. Since $\mu \geq \frac{n}{200 \log n}$, this implies

$$n^{18\mu} \leq 2^{\mu^2/1800 - \mu/10}. \tag{3.4}$$

Multiplying Equations (3.3) and (3.4) gives us the desired bound of $n^{2n+1} n^{18\mu} \leq 2^{f(\mu)}$.

■

By Lemmas 47 and 48, the above summation for $\mathbf{E}[T_1]$ is at most $T(n, 0) + O(1)$ since $T(n, \mu)$ is certainly at most $O(n^{16\mu})$. For an iteration in which we have chosen $\mu = 0$, when counting and sampling n-vertex labeled chordal graphs with automorphism $\pi = \mathrm{id}$, we can simply run the counting and sampling algorithms of Chapter 2, rather than passing in $\pi = \mathrm{id}$ as an input. Thus the expected running time $\mathbf{E}[T_1]$ of one iteration is at most $O(n^7)$ arithmetic operations. Lemmas 47 and 48 also immediately give us a bound on B/B_0, since we have

$$\frac{B}{B_0} = \frac{B_0}{B_0} + \frac{B_2}{B_0} + \ldots + \frac{B_n}{B_0} = O(1).$$

Therefore, by Lemma 45, the overall running time is at most $O(n^7)$ arithmetic operations in expectation.

[1] This holds for $n \geq 2.6 \cdot 10^5$.
[2] This holds for $n \geq 3.3 \cdot 10^9$.

3.4 Counting Labeled Chordal Graphs with a Given Automorphism

In this section, we describe the algorithm for COUNT_CHORDAL_LAB(n, π), which counts the number of labeled chordal graphs with a given automorphism. This is the algorithm of Theorem 6. In Section 3.5 we will prove correctness, analyze the running time, and derive the corresponding sampling algorithm (Theorem 7).

Theorem 6 *There is a deterministic algorithm that given $n \in \mathbb{N}$ and $\pi \in S_n$, computes the number of labeled chordal graphs with vertex set $[n]$ for which π is an automorphism. This algorithm uses $O(2^{7\mu}n^9)$ arithmetic operations, where $\mu = |M_\pi|$.*

In Chapter 2, we gave a dynamic-programming algorithm for computing the number n-vertex labeled chordal graphs that uses $O(n^7)$ arithmetic operations, in the case when we do not require the graphs to have a particular automorphism. Our algorithm in this section is closely based on that one, but we add more arguments and more details to each of the recurrences to keep track of the behavior of the automorphism. As was done in Chapter 2, we evaluate the recurrences top-down using memoization.

3.4.1 Reducing from counting chordal graphs to counting connected chordal graphs

For $k \in \mathbb{N}$, let $a(k)$ denote the number of labeled chordal graphs with vertex set $[k]$, and let $c(k)$ denote the number of *connected* labeled chordal graphs with vertex set $[k]$. Recall that the algorithm of Chapter 2 begins by reducing from counting chordal graphs to counting connected chordal graphs via the following recurrence. (We omit the "ω-colorable" requirement since we will not need that here.)

Lemma 49 *The number of labeled chordal graphs with vertex set $[k]$ is given by*

$$a(k) = \sum_{k'=1}^{k} \binom{k-1}{k'-1} c(k')a(k-k')$$

for all $k \in \mathbb{N}$.

Here k' stands for the number of vertices in the connected component that contains the label 1. The remaining connected components have a total of $k - k'$ vertices. Since this recurrence is relatively simple, most of the difficulty in the algorithm of Chapter 2 lies in the recurrences for counting *connected* chordal graphs. However, when counting graphs with a given automorphism, the step of reducing to connected graphs is already quite a bit more involved.

Suppose we are given $n \in \mathbb{N}$ and $\pi \in S_n$ as input. From now on, whenever we refer to n or π, we mean these particular values from the input.

We will first define $a(k, p, M)$ and $c(k, p, M)$, which are variations of $a(k)$ and $c(k)$ that only count graphs for which $\pi^p|_{V(G)}$ is an automorphism (see Definition 4). Here π^p stands for π to the power p, i.e., the permutation that arises from applying π a total of p times. The reason for raising π to a power will be apparent in the recurrence for $a(k, p, M)$. In the initial, highest-level recursive call, we will have $p = 1$ and thus $\pi^p = \pi$.

In Chapter 2, when counting the number of possibilities for a subgraph of size k' (for example, a connected component), the authors essentially relabel that subgraph to have vertex set $[k']$, so that one can count the number of possibilities using, for example, $c(k')$. In our algorithm, we want to relabel the vertices of each subgraph in a similar way. However, this time, we do not relabel the vertices that are moved by π. This ensures that the automorphism in each later recursive call will still be π, or a permutation closely related to π.

As a consequence, the vertex sets of the subgraphs that we wish to count become slightly more complicated. For example, suppose we wish to count the number of possi-

bilities for a 5-vertex subgraph that originally contains the moved vertices $2, 8, 9 \in M_\pi$. Since we do not relabel the moved vertices, the resulting vertex set after relabeling is $\{1, 2, 3, 8, 9\}$ rather than $\{1, 2, 3, 4, 5\}$. More formally, suppose we have been given $k \in \mathbb{N}$ and $M \subseteq M_\pi$, where $|M| \leq k$. Let V be the set of the first $k - |M|$ natural numbers in $\mathbb{N} \setminus M_\pi$. We define $V_{k,M} \coloneqq V \cup M$. For example, if $k = 5$ and $M = M_\pi = \{2, 8, 9\}$, then $V_{k,M} = \{1, 2, 3, 8, 9\}$. Intuitively, $V_{k,m}$ is the label set of size k whose intersection with M_π is M and that otherwise contains labels that are as small as possible.

For $\hat{\pi} \in S_n$ and $M \subseteq [n]$, recall that M is *invariant* under $\hat{\pi}$ if $\hat{\pi}(i) \in M$ for all $i \in M$. For $M', M \subseteq [n]$, we write $M' \subseteq_{\hat{\pi}} M$ to indicate that $M' \subseteq M$ and M' is invariant under $\hat{\pi}$. Intuitively, M' is a subset of M that respects the cycles of $\hat{\pi}$ by taking all or nothing of each cycle.

Definition 4 *Suppose $k, p \in [n]$ and suppose $M \subseteq_{\pi^p} M_\pi$, where $|M| \leq k$. Let $a(k, p, M)$ (resp. $c(k, p, M)$) denote the number of labeled chordal graphs (resp. **connected** labeled chordal graphs) with vertex set $V_{k,M}$ for which $\pi^p|_{V(G)}$ is an automorphism.*

Our algorithm returns $a(n, 1, M_\pi)$. This is precisely the number of labeled chordal graphs with vertex set $[n]$ and automorphism π since $V_{n,M_\pi} = [n]$.

Suppose we have been given fixed values of the arguments k, p, M of $a(k, p, M)$. Let s be the smallest label in the vertex set $V_{k,M}$. For $k' \in [k]$, let $\mathcal{P}_{k'}$ be the family of sets $M' \subseteq_{\pi^p} M$ such that $|M| - k + k' \leq |M'| \leq k'$ and such that the following condition holds: if $s \in M$, then $s \in M'$, and otherwise, $|M'| \leq k' - 1$. This last condition will ensure that s belongs to the connected component of size k' in the recurrence for $a(k, p, M)$.

Suppose $C \subseteq [n]$, $\sigma \in S_n$. For an element $i \in C$, we say the *period* of i with respect to (C, σ) is the smallest positive integer j such that $\sigma^j(i) \in C$. Let $\mathcal{Q}$ be the family of sets $C \subseteq M$ such that all elements of C have the same period $j \geq 2$ with respect to (C, π^p), and such that $s \in C$. For a set $C \in \mathcal{Q}$, we write p_C to denote the period of the elements

of C (with respect to (C, π^p)), and we let $C_\sigma \coloneqq C \cup \sigma(C) \cup \cdots \cup \sigma^{p_C - 1}(C)$ denote the union of the sets that C is mapped to by powers of σ, where $\sigma = \pi^p$.

To compute $a(k, p, M)$, we use the following recurrence. The dot in front of the curly braces denotes multiplication. Note that we have $|M'| \leq k'$ and $|M \setminus M'| \leq k - k'$ by the definition of $\mathcal{P}_{k'}$.

Lemma 50 *Let $k, p \in [n]$ and suppose $M \subseteq_{\pi^p} M_\pi$, where $|M| \leq k$. We have*

$$a(k,p,M) = \sum_{\substack{1 \leq k' \leq k \\ M' \in \mathcal{P}_{k'}}} c(k', p, M') a(k - k', p, M \setminus M') \cdot \begin{cases} \binom{k-|M|}{k'-|M'|} & \text{if } s \in M \\ \binom{k-1-|M|}{k'-1-|M'|} & \text{otherwise} \end{cases}$$
$$+ \sum_{C \in \mathcal{Q}} c(|C|, p \cdot p_C, C) a(k - p_C|C|, p, M \setminus C_{\pi^p}).$$

For some intuition, suppose G is a graph counted by $a(k, p, M)$, and let C be the connected component of G that contains s. The first line of the recurrence for $a(k, p, M)$ covers the case when C is invariant under π^p. This case is analogous to Lemma 49. In the first summation, k' stands for $|C|$ and M' stands for the set of vertices in C that can be moved by π^p. If $s \in M$, then in addition to the vertices in M', we need to choose $k' - |M'|$ additional vertices for C so that $|C| = k'$. For these, we must choose non-moved vertices, so there are $k - |M|$ possible vertices to choose from. If $s \notin M$, then we subtract 1 from each of the numbers in the binomial coefficient since we already know that s is a non-moved vertex in C.

The second line covers the case when C is not invariant under π^p, which means all of C is mapped to some other connected component of G by π^p. In this case, we have p_C components of size $|C|$ that are all isomorphic to C, and the rest of the graph has $k - p_C|C|$ vertices. We do not need a binomial coefficient in this case because all of the vertices in C are moved. There are $c(|C|, p \cdot p_C, C)$ possibilities for the edges of C since $\pi^{p \cdot p_C}$ is an automorphism of C. As an example, suppose $p = 1$. If we apply $\pi^p = \pi$ to the

vertices of C a total of p_C times, then the image $\pi^{p_C}(C)$ is equal to C, although these two sets might not match up pointwise. To ensure that the image $\pi^{p_C}(C)$ matches up with the edges of C, we require that π^{p_C} is an automorphism of C. This is why we need the argument p in $a(k, p, M)$ and $c(k, p, M)$.

Note that for a graph G counted by $a(k, p, M)$, we have $M_{\pi^p} \cap V(G) \subseteq M_\pi \cap V(G) \subseteq M$ since $V(G) = V_{k,M}$. This means no vertex in $V(G) \setminus M$ is moved by π^p, so we are free to relabel these vertices in the proof of Lemma 50 without changing the automorphism. In the initial recursive call $a(n, 1, M_\pi)$, we have $p = 1$ and $M = M_\pi$, so $M_{\pi^p} \cap V(G) = M$. Later on in the algorithm, it is possible that some vertices in M are not actually moved by π^p. For example, in the previous paragraph, it could happen that $\pi^{p \cdot p_C}$ is the identity on C (and then $p \cdot p_C$ becomes the new value of p). However, we always have $M \subseteq M_\pi$ by the definition of $a(k, p, M)$, so if $|M_\pi| = \mu$, then $|M| \leq \mu$. This fact will be useful for the running time analysis in Section 3.5.3.

The proof of Lemma 50, along with the proofs of all of the following recurrences, can be found in Section 3.5.

3.4.2 Recurrences for counting connected chordal graphs

To compute $c(k, p, M)$, we will define various counter functions that are analogous to those in Definition 3 from Chapter 2. The counter functions for our algorithm will be similar, except we only want to count graphs with a particular automorphism. Therefore, we will have several more arguments in addition to the usual ones t, x, ℓ, k, z from Definition 3. As above, the argument p will indicate that π^p is the current automorphism. We also introduce the arguments M_X, M_L, M_Z, and M_K, each of which is a subset of M_π. Roughly, these sets specify which vertices — in X, $L \coloneqq L_G(X)$, $Z \coloneqq [z]$, and the rest of the graph, respectively — can be moved by the current permutation (but the sets

X, L, and Z will be modified slightly).

In Definition 3, the g-functions count graphs with vertex set $[x + k]$, and the f-functions count graphs with vertex set $[x + \ell + k]$. For our algorithm, we will make some adjustments to these vertex sets to ensure that the vertices moved by the current permutation still appear in the graph. This is similar to how we defined $V_{k,M}$ above. To do so, we define several symbols for these vertex sets and vertex subsets (V_{args}, etc.). Each of these depends on the list of arguments of the function in question, which we denote by $args$. For example, when defining g, we have $args = \begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$ (see Definition 5).

First, suppose $args$ comes from one of the g-functions in Definition 5. Let V_X be the set of the first $x - |M_X|$ natural numbers in $\mathbb{N} \setminus M_\pi$, and let V_K be the set of the first $k - |M_K|$ natural numbers in $\mathbb{N} \setminus (V_X \cup M_\pi)$. We define $X_{args} \coloneqq V_X \cup M_X$ and $V_{args} \coloneqq X_{args} \cup V_K \cup M_K$. Also, if z is included in $args$, then let V_Z be the set of the first $z - |M_Z|$ natural numbers in $\mathbb{N} \setminus M_\pi$. In this case, we define $Z_{args} \coloneqq V_Z \cup M_Z$.

For the other case, suppose $args$ comes from one of the f-functions. Let V_X be the set of the first $x - |M_X|$ natural numbers in $\mathbb{N} \setminus M_\pi$, let V_L be the set of the first $\ell - |M_L|$ natural numbers in $\mathbb{N} \setminus (V_X \cup M_\pi)$, and let V_K be the set of the first $k - |M_K|$ natural numbers in $\mathbb{N} \setminus (V_X \cup V_L \cup M_\pi)$. We define $X_{args} \coloneqq V_X \cup M_X$, $L_{args} = V_L \cup M_L$, and $V_{args} \coloneqq X_{args} \cup L_{args} \cup V_K \cup M_K$. Also, if z is included in $args$, then let V_Z be the set of the first $z - |M_Z|$ natural numbers in $\mathbb{N} \setminus M_\pi$. In this case, we again define $Z_{args} \coloneqq V_Z \cup M_Z$.

These vertex subsets still have the same sizes as they did in the original algorithm: the size of V_{args} is $x + k$ or $x + \ell + k$, $|X_{args}| = x$, $|L_{args}| = \ell$, the rest of the graph has size k, and $|Z_{args}| = z$. Just as we had $Z \subseteq X$ in the original algorithm, we now have $Z_{args} \subseteq X_{args}$. This is because we require $M_Z \subseteq M_X$ in Definition 5, and we also require $z - |M_Z| \leq x - |M_X|$, which implies $V_Z \subseteq V_X$.

For g-functions, we have $M_{\pi^p} \cap V_{args} \subseteq M_X \cup M_K$, and for f-functions, we have

$M_{\pi^p} \cap V_{args} \subseteq M_X \cup M_L \cup M_K$, by the definition of V_{args}. We are now ready to define our new counter functions.

Definition 5 *The following functions count various subclasses of chordal graphs. The arguments t, x, ℓ, k, z are nonnegative integers with the same domains as in Definition 3. We have $p \in [n]$, and the arguments M_X, M_L, M_K, M_Z are subsets of M_π. All other requirements for their domains are specified below.*

1. $g\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$, $\tilde{g}\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$, $\tilde{g}_p\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$, $\tilde{g}_1\begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix}$, *and* $\tilde{g}_{\geq 2}\begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix}$ *are the same as $g(t, x, k, z)$, $\tilde{g}(t, x, k, z)$, $\tilde{g}_p(t, x, k, z)$, $\tilde{g}_1(t, x, k)$, and $\tilde{g}_{\geq 2}(t, x, k)$, respectively, except we only count graphs for which $\pi^p|_{V(G)}$ is an automorphism, and we make the following changes to the vertices of the graph: the vertex set is V_{args} rather than $[x + k]$, the exception set is X_{args} rather than $[x]$, and $[z]$ is replaced by Z_{args}.*

   Domain: *$M_X \subseteq_{\pi^p} M_\pi$, $M_K \subseteq_{\pi^p} M_\pi \setminus M_X$, $M_Z \subseteq_{\pi^p} M_X$, $|M_X| \leq x$, $|M_K| \leq k$, $|M_Z| \leq z$, and $z - |M_Z| \leq x - |M_X|$.*

2. $f\begin{pmatrix} t & x & \ell & k \\ p & M_X & M_L & M_K \end{pmatrix}$, $\tilde{f}\begin{pmatrix} t & x & \ell & k \\ p & M_X & M_L & M_K \end{pmatrix}$, $\tilde{f}_p\begin{pmatrix} t & x & \ell & k \\ p & M_X & M_L & M_K \end{pmatrix}$, *and* $\tilde{f}_p\begin{pmatrix} t & x & \ell & k & z \\ p & M_X & M_L & M_K & M_Z \end{pmatrix}$ *are the same as $f(t, x, \ell, k)$, $\tilde{f}(t, x, \ell, k)$, $\tilde{f}_p(t, x, \ell, k)$, and $\tilde{f}_p(t, x, \ell, k, z)$, respectively, except we only count graphs for which $\pi^p|_{V(G)}$ is an automorphism, and we make the following changes to the vertices of the graph: the vertex set is V_{args} rather than $[x + \ell + k]$, the exception set is X_{args} rather than $[x]$, the last set to evaporate is L_{args} rather than $[x + 1, x + \ell]$, and $[z]$ is replaced by Z_{args}.*

   Domain: *$M_X \subseteq_{\pi^p} M_\pi$, $M_L \subseteq_{\pi^p} M_\pi \setminus M_X$, $M_K \subseteq_{\pi^p} M_\pi \setminus (M_X \cup M_L)$, $M_Z \subseteq_{\pi^p} M_X$, $|M_X| \leq x$, $|M_L| \leq \ell$, $|M_K| \leq k$, $|M_Z| \leq z$, and $z - |M_Z| \leq x - |M_X|$.*

For a graph G counted by one of these functions, $\pi^p|_{V(G)}$ is indeed a permutation of $V(G)$ since M_X and M_K (and M_L if needed) are invariant under π^p.

To compute $c(k, p, M)$, we consider all possibilities for the evaporation time. In the following recurrence, $\tilde{g}_1$ (with those particular arguments) counts the number of labeled connected chordal graphs with vertex set $V_{args} = V_{k,M}$ and automorphism $\pi^p|_{V(G)}$ that evaporate at time exactly t with empty exception set.

Lemma 51 *Let $k, p \in [n]$ and suppose $M \subseteq_{\pi^p} M_\pi$, where $|M| \leq k$. We have*

$$c(k, p, M) = \sum_{t=1}^{k} \tilde{g}_1 \begin{pmatrix} t & 0 & k \\ p & \emptyset & M \end{pmatrix}.$$

To compute $\tilde{g}_1$, we consider all possibilities for the size ℓ of L_{args}. The set M in the inner sum stands for the set of vertices in L_{args} that can be moved by π^p. Note that we have $|M| \leq \ell$ and $|M_K \setminus M| \leq k - \ell$ by the definition of $\mathcal{I}_\ell$. For $\tilde{g}_1$ and all of the following functions, we recommend reading the analogous recurrences in Chapter 2 for further insight into what is happening with the arguments t, x, ℓ, k, z in each recursive call.

Lemma 52 *For $\tilde{g}_1$, we have*

$$\tilde{g}_1 \begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix} = \sum_{\ell=1}^{k} \sum_{M \in \mathcal{I}_\ell} \binom{k - |M_K|}{\ell - |M|} f \begin{pmatrix} t & x & \ell & k - \ell \\ p & M_X & M & M_K \setminus M \end{pmatrix},$$

where $\mathcal{I}_\ell = \{M \subseteq_{\pi^p} M_K : |M_K| - k + \ell \leq |M| \leq \ell\}$.

To compute f, we consider all possibilities for the set of labels that appear in connected components of $G \setminus (X_{args} \cup L_{args})$ that evaporate at time exactly $t - 1$. These components correspond to the recursive call to $\tilde{f}$. The set M stands for the set of vertices in components that evaporate at time exactly $t - 1$ that can be moved by π^p. Note that we have $|M| \leq k'$ and $|M_K \setminus M| \leq k - k'$ by the definition of $\mathcal{I}_{k'}$. Since $|M_L| \leq \ell$, we also have $x - |M_X| \leq x + \ell - |M_X \cup M_L|$, which is required by the domain of g.

Lemma 53 *For f, we have*

$$f\begin{pmatrix} t & x & \ell & k \\ p & M_X & M_L & M_K \end{pmatrix} = \sum_{k'=1}^{k} \sum_{M \in \mathcal{I}_{k'}} \binom{k-|M_K|}{k'-|M|} \tilde{f}\begin{pmatrix} t & x & \ell & k' \\ p & M_X & M_L & M \end{pmatrix} g\begin{pmatrix} t-2 & x+\ell & k-k' & x \\ p & M_X \cup M_L & M_K \setminus M & M_X \end{pmatrix},$$

where $\mathcal{I}_{k'} = \{M \subseteq_{\pi^p} M_K : |M_K| - k + k' \leq |M| \leq k'\}$.

To compute g, we consider all possibilities for the set of labels that appear in connected components of $G \setminus X_{args}$ that evaporate at time exactly t. These components correspond to the recursive call to $\tilde{g}$. The set M stands for the set of vertices in components that evaporate at time exactly t that can be moved by π^p.

Lemma 54 *For g, we have*

$$g\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix} = \sum_{k'=0}^{k} \sum_{M \in \mathcal{I}_{k'}} \binom{k-|M_K|}{k'-|M|} \tilde{g}\begin{pmatrix} t & x & k' & z \\ p & M_X & M & M_Z \end{pmatrix} g\begin{pmatrix} t-1 & x & k-k' & z \\ p & M_X & M_K \setminus M & M_Z \end{pmatrix},$$

where $\mathcal{I}_{k'} = \{M \subseteq_{\pi^p} M_K : |M_K| - k + k' \leq |M| \leq k'\}$.

For the next recurrence, we will need a few more definitions. Suppose we have been given fixed values of the arguments of $\tilde{g}$. Let s be the smallest label in $V_{args} \setminus X_{args}$. For $k' \in [k]$, let $\mathcal{P}_{k'}$ be the family of sets $M \subseteq_{\pi^p} M_K$ such that $|M_K| - k + k' \leq |M| \leq k'$ and such that the following condition holds: if $s \in M_K$, then $s \in M$, and otherwise, $|M| \leq k' - 1$. For $x' \in [x]$, let $\mathcal{I}_{x'} = \{M' \subseteq_{\pi^p} M_X : |M'| \leq x'\}$.

Let $\mathcal{Q}$ be the family of pairs (C, M'), where $C \subseteq M_K$ and $M' \subseteq M_X$, such that all elements of C have the same period $p_C \geq 2$ with respect to (C, π^p), such that $s \in C$, and such that M' is invariant under $\pi^{p \cdot p_C}$. For a set C from such a pair, we let $C_\sigma \coloneqq C \cup \sigma(C) \cup \cdots \cup \sigma^{p_C - 1}(C)$, where $\sigma = \pi^p$.

To compute $\tilde{g}$, we consider all possibilities for the label set of the connected component C of $G \setminus X_{args}$ that contains s. In the definition of $\tilde{g}$, the components of $G \setminus X_{args}$ are similar enough (since they all evaporate at time t) that they can potentially be mapped to one another by π^p. Thus we consider two cases: either C is invariant under π^p, or C is mapped to some other component of $G \setminus X_{args}$ by π^p. We add together the summations from these two cases, in a similar way to the recurrence for $a(k, p, M)$. In the recurrence for $\tilde{g}$, k' stands for $|C|$, and x' stands for $|N(C)|$. The set M stands for the set of vertices in C that can be moved by π^p, and M' stands for the set of vertices in $N(C)$ that can be moved by π^p.

Lemma 55 *For $\tilde{g}$, we have*

$$\tilde{g}\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix} = \sum_{\substack{1\le k'\le k \\ 1\le x'\le x \\ M\in\mathcal{P}_{k'} \\ M'\in\mathcal{I}_{x'}}} \tilde{g}_1\begin{pmatrix} t & x' & k' \\ p & M' & M \end{pmatrix} \tilde{g}\begin{pmatrix} t & x & k-k' & z \\ p & M_X & M_K \setminus M & M_Z \end{pmatrix}$$
$$\cdot \begin{cases} \binom{k-|M_K|}{k'-|M|} & \text{if } s \in M_K \\ \binom{k-1-|M_K|}{k'-1-|M|} & \text{otherwise} \end{cases}$$
$$\cdot \begin{cases} \binom{x-|M_X|}{x'-|M'|} & \text{if } M' \not\subseteq M_Z \\ \binom{x-|M_X|}{x'-|M'|} - \binom{z-|M_Z|}{x'-|M'|} & \text{otherwise} \end{cases}$$

$$+ \sum_{\substack{1\le x'\le x \\ (C,M')\in\mathcal{Q}}} \tilde{g}_1\begin{pmatrix} t & x' & |C| \\ p\cdot p_C & M' & C \end{pmatrix} \tilde{g}\begin{pmatrix} t & x & k-p_C|C| & z \\ p & M_X & M_K \setminus C_{\pi^p} & M_Z \end{pmatrix}$$
$$\cdot \begin{cases} \binom{x-|M_X|}{x'-|M'|} & \text{if } M' \not\subseteq M_Z \\ \binom{x-|M_X|}{x'-|M'|} - \binom{z-|M_Z|}{x'-|M'|} & \text{otherwise.} \end{cases}$$

To compute $\tilde{f}$, we consider three cases: either no component of $G \setminus (X_{args} \cup L_{args})$ sees all of $X_{args} \cup L_{args}$, exactly one component sees all of $X_{args} \cup L_{args}$, or at least two

components see all of $X_{args} \cup L_{args}$. The recursive calls to $\tilde{g}_1$ and $\tilde{g}_{\geq 2}$ correspond to the component/components that see all of $X_{args} \cup L_{args}$. In the second and third cases, the set M stands for the set of vertices that can be moved by π^p in components that see all of $X_{args} \cup L_{args}$.

Lemma 56 *For $\tilde{f}$, we have*

$$\tilde{f}\begin{pmatrix} t & x & \ell & k \\ p & M_X & M_L & M_K \end{pmatrix} = \tilde{f}_p\begin{pmatrix} t & x & \ell & k \\ p & M_X & M_L & M_K \end{pmatrix}$$
$$+ \sum_{\substack{1 \leq k' \leq k \\ M \in \mathcal{I}_{k'}}} \binom{k - |M_K|}{k' - |M|} \tilde{g}_1\begin{pmatrix} t-1 & x+\ell & k' \\ p & M_X \cup M_L & M \end{pmatrix} \tilde{f}_p\begin{pmatrix} t & x & \ell & k-k' \\ p & M_X & M_L & M_K \setminus M \end{pmatrix}$$
$$+ \sum_{\substack{1 \leq k' \leq k \\ M \in \mathcal{I}_{k'}}} \binom{k - |M_K|}{k' - |M|} \tilde{g}_{\geq 2}\begin{pmatrix} t-1 & x+\ell & k' \\ p & M_X \cup M_L & M \end{pmatrix} \tilde{g}_p\begin{pmatrix} t-1 & x+\ell & k-k' & x \\ p & M_X \cup M_L & M_K \setminus M & M_X \end{pmatrix},$$

where $\mathcal{I}_{k'} = \{M \subseteq_{\pi^p} M_K : |M_K| - k + k' \leq |M| \leq k'\}$.

For the next recurrence, suppose we have been given fixed values of the arguments of $\tilde{g}_{\geq 2}$. Let s be the smallest label in $V_{args} \setminus X_{args}$, and let $\mathcal{P}_{k'}$ be defined as it was in the recurrence for $\tilde{g}$. Let $\mathcal{Q}$ be the family of sets $C \subseteq M$ such that all elements of C have the same period $p_C \geq 2$ with respect to (C, π^p), and such that $s \in C$. For a set $C \in \mathcal{Q}$, we let $C_\sigma \coloneqq C \cup \sigma(C) \cup \cdots \cup \sigma^{p_C - 1}(C)$, where $\sigma = \pi^p$.

To compute $\tilde{g}_{\geq 2}$, we consider all possibilities for the label set of the connected component C of $G \setminus X_{args}$ that contains s. This is somewhat similar to the recurrence for $\tilde{g}$, but this time we do not need to keep track of the neighborhood of C, since every component of $G \setminus X_{args}$ sees all of X_{args}.

Lemma 57 *For $\tilde{g}_{\geq 2}$, we have*

$$\tilde{g}_{\geq 2}\begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix} =$$
$$\sum_{\substack{1 \leq k' \leq k \\ M \in \mathcal{P}_{k'}}} \tilde{g}_1\begin{pmatrix} t & x & k' \\ p & M_X & M \end{pmatrix} \left(\tilde{g}_1\begin{pmatrix} t & x & k-k' \\ p & M_X & M_K \setminus M \end{pmatrix} + \tilde{g}_{\geq 2}\begin{pmatrix} t & x & k-k' \\ p & M_X & M_K \setminus M \end{pmatrix} \right)$$

$$\cdot \begin{cases} \binom{k-|M_K|}{k'-|M|} & \text{if } s \in M_K \\ \binom{k-1-|M_K|}{k'-1-|M|} & \text{otherwise} \end{cases}$$

$$+ \sum_{C \in \mathcal{Q}} \tilde{g}_1 \begin{pmatrix} t & x & |C| \\ p \cdot p_C & M_X & C \end{pmatrix} \left(\tilde{g}_1 \begin{pmatrix} t & x & k - p_C|C| \\ p & M_X & M_K \setminus C^\pi \end{pmatrix} + \tilde{g}_{\geq 2} \begin{pmatrix} t & x & k - p_C|C| \\ p & M_X & M_K \setminus C^\pi \end{pmatrix} \right).$$

To compute $\tilde{g}_p$, all we need to do is make a slight adjustment to the recurrence for $\tilde{g}$.

Lemma 58 *The recurrence for $\tilde{g}_p$ is exactly the same as the recurrence for $\tilde{g}$ from Lemma 55, except for the two sums over x': In both summations, rather than summing over x' such that $1 \leq x' \leq x$, we sum over x' such that $1 \leq x' \leq x - 1$.*

To compute $\tilde{f}_p$, we first observe that when $z = x$, requiring $G \setminus Z_{args}$ to be connected is the same as requiring $G \setminus X_{args}$ to be connected.

Lemma 59 *We have* $\tilde{f}_p \begin{pmatrix} t & x & \ell & k \\ p & M_X & M_L & M_K \end{pmatrix} = \tilde{f}_p \begin{pmatrix} t & x & \ell & k & x \\ p & M_X & M_L & M_K & M_X \end{pmatrix}$.

For the next $\tilde{f}_p$ recurrence, we need one last round of definitions. Suppose we have been given fixed values of the arguments of $\tilde{f}_p$, including z and M_Z. Let s be the smallest label in $V_{args} \setminus (X_{args} \cup L_{args})$. For $k' \in [k]$, let $\mathcal{P}_{k'}$ be the family of sets $M \subseteq_{\pi^p} M_K$ such that $|M_K| - k + k' \leq |M| \leq k'$ and such that the following condition holds: if $s \in M_K$, then $s \in M$, and otherwise, $|M| \leq k' - 1$. For $0 \leq x' \leq x$, let $\mathcal{I}_{x'} = \{M' \subseteq_{\pi^p} M_X : |M'| \leq x'\}$. Also, for $0 \leq \ell' \leq \ell$, let $\mathcal{J}_{\ell'} = \{M' \subseteq_{\pi^p} M_L : |M'| \leq \ell'\}$.

Let $\mathcal{Q}$ be the family of triples (C, M'_X, M'_L), where $C \subseteq M_K$, $M'_X \subseteq M_X$, and $M'_L \subseteq M_L$, such that all elements of C have the same period $p_C \geq 2$ with respect to (C, π^p), such that $s \in C$, and such that $M'_X \cup M'_L$ is invariant under $\pi^{p \cdot p_C}$. For a set C from such a triple, we let $C_\sigma \coloneqq C \cup \sigma(C) \cup \cdots \cup \sigma^{p_C - 1}(C)$, where $\sigma = \pi^p$.

To compute $\tilde{f}_p$ with the argument z, we consider all possibilities for the label set of the component C of $G \setminus (X_{args} \cup L_{args})$ that contains s. There are two possible cases: either

C is invariant under π^p, or C is mapped to some other component of $G \setminus (X_{args} \cup L_{args})$ by π^p. We add together the summations from these two cases, both of which are now rather complicated. However, the general idea is similar to the earlier recurrences. In the following recurrence, k' stands for $|C|$, x' stands for $|N(C) \cap X_{args}|$, and ℓ' stands for $|N(C) \cap L_{args}|$. For the moved vertices, M stands for the set of vertices in C that can be moved by π^p, M'_X stands for the set of vertices in $N(C) \cap X_{args}$ that can be moved by π^p, and M'_L stands for the set of vertices in $N(C) \cap L_{args}$ that can be moved by π^p.

Lemma 60 *For $\tilde{f}_p$ with the argument z, we have*

$$\tilde{f}_p\begin{pmatrix} t & x & \ell & k & z \\ p & M_X & M_L & M_K & M_Z \end{pmatrix} = \sum_{\substack{1\le k'\le k \\ 0\le x'\le x \\ 0\le \ell'\le \ell \\ 0<x'+\ell'<x+\ell}} \sum_{\substack{M\in\mathcal{P}_{k'} \\ M'_X\in\mathcal{I}_{x'} \\ M'_L\in\mathcal{J}_{\ell'}}} \tilde{g}_1\begin{pmatrix} t-1 & x'+\ell' & k' \\ p & M'_X\cup M'_L & M \end{pmatrix}$$

$$\cdot \binom{\ell - |M_L|}{\ell' - |M'_L|} \cdot \begin{cases} \binom{k-|M_K|}{k'-|M|} & \text{if } s \in M_K \\ \binom{k-1-|M_K|}{k'-1-|M|} & \text{otherwise} \end{cases}$$

$$\cdot \begin{cases} \binom{x-|M_X|}{x'-|M'_X|} & \text{if } \ell' > 0 \text{ or } M'_X \not\subseteq M_Z \\ \binom{x-|M_X|}{x'-|M'_X|} - \binom{z-|M_Z|}{x'-|M'_X|} & \text{otherwise} \end{cases}$$

$$\cdot \begin{cases} \tilde{f}_p\begin{pmatrix} t & x+\ell' & \ell-\ell' & k-k' & z \\ p & M_X\cup M'_L & M_L\setminus M'_L & M_K\setminus M & M_Z \end{pmatrix} & \text{if } \ell' < \ell \\ \tilde{g}_p\begin{pmatrix} t-1 & x+\ell & k-k' & z \\ p & M_X\cup M'_L & M_K\setminus M & M_Z \end{pmatrix} & \text{otherwise} \end{cases}$$

$$+ \sum_{\substack{0\le x'\le x \\ 0\le \ell'\le \ell \\ 0<x'+\ell'<x+\ell \\ (C,M'_X,M'_L)\in\mathcal{Q}}} \tilde{g}_1\begin{pmatrix} t-1 & x'+\ell' & |C| \\ p\cdot p_C & M'_X\cup M'_L & C \end{pmatrix} \cdot \binom{\ell - |M_L|}{\ell' - |M'_L|}$$

$$\cdot \begin{cases} \binom{x-|M_X|}{x'-|M'_X|} & \text{if } \ell' > 0 \text{ or } M'_X \not\subseteq M_Z \\ \binom{x-|M_X|}{x'-|M'_X|} - \binom{z-|M_Z|}{x'-|M'_X|} & \text{otherwise} \end{cases}$$

$$\cdot \begin{cases} \tilde{f}_p \begin{pmatrix} t & x+\ell' & \ell-\ell' & k-p_C|C| & z \\ p & M_X \cup M'_L & M_L \setminus M'_L & M_K \setminus C^\pi & M_Z \end{pmatrix} & \text{if } \ell' < \ell \\ \tilde{g}_p \begin{pmatrix} t-1 & x+\ell & k-p_C|C| & z \\ p & M_X \cup M'_L & M_K \setminus C^\pi & M_Z \end{pmatrix} & \text{otherwise.} \end{cases}$$

The base cases for all of these counter functions are the same as in Chapter 2 since they only depend on the arguments t, x, ℓ, k, z.

3.5 Proofs of Theorems 6 and 7

In this section, we prove correctness of the recurrences from Section 3.4 and complete the proofs of Theorems 6 and 7.

3.5.1 Automorphism properties

First of all, we show that automorphisms preserve evaporation time.

Lemma 61 *Suppose G is a labeled chordal graph that evaporates at time t with exception set X, where $X \subseteq V(G)$ is a clique, and let $L_1, \ldots, L_t$ be the evaporation sequence. If $\hat{\pi}$ is an automorphism of G such that X is invariant under $\hat{\pi}$, then L_i is invariant under $\hat{\pi}$ for all $i \in [t]$.*

Proof: We proceed by induction on i. Let $\widetilde{L}_1$ be the set of all simplicial vertices in G. If the neighborhood of a vertex $v \in V(G)$ is a clique, then the neighborhood of $\hat{\pi}(v)$ must also be a clique, so $\widetilde{L}_1$ is invariant under $\hat{\pi}$. Since X is also invariant under $\hat{\pi}$, this implies that $L_1 = \widetilde{L}_1 \setminus X$ is invariant under $\hat{\pi}$.

For the induction step, suppose $L_1, \ldots, L_i$ are all invariant under $\hat{\pi}$ for some $i \in [t-1]$. This means $\hat{\pi}|_{V(G)\setminus(L_1\cup\cdots\cup L_i)}$ is an automorphism of $G \setminus (L_1 \cup \cdots \cup L_i)$. By the same argument as in the base case, the set $\widetilde{L}_{i+1}$ of simplicial vertices of $G \setminus (L_1 \cup \cdots \cup L_i)$ is invariant under $\hat{\pi}|_{V(G)\setminus(L_1\cup\cdots\cup L_i)}$ and also under $\hat{\pi}$. Hence $L_{i+1} = \widetilde{L}_{i+1} \setminus X$ is invariant under $\hat{\pi}$ as well. ■

Next, we prove two lemmas that relate the automorphisms of a graph G to the automorphisms of its induced subgraphs.

Lemma 62 *Let G be a labeled chordal graph with an automorphism $\hat{\pi}$ and induced subgraphs G_1 and G_2 such that $V(G_1) \cup V(G_2) = V(G)$. If $X \coloneqq V(G_1) \cap V(G_2)$ and $A \coloneqq V(G_1) \setminus V(G_2)$ are both invariant under $\hat{\pi}$, then $\hat{\pi}|_{V(G_1)}$ is an automorphism of G_1 and $\hat{\pi}|_{V(G_2)}$ is an automorphism of G_2.*

Proof: We know $\hat{\pi}|_{V(G_1)}$ is a bijection from $V(G_1)$ to $V(G_1)$ since X and A are invariant under $\hat{\pi}$. Moreover, this map is in fact an automorphism of G_1 since $\hat{\pi}$ is an automorphism of G and G_1 is an induced subgraph. Similarly, $\hat{\pi}|_{V(G_2)}$ is a bijection from $V(G_2)$ to $V(G_2)$ since X and $V(G_2) \setminus V(G_1)$ are invariant under $\hat{\pi}$, and this map is in fact an automorphism G_2. ■

Lemma 63 *Let G be a labeled chordal graph with induced subgraphs G_1 and G_2 such that $V(G_1) \cup V(G_2) = V(G)$ and such that there are no edges from $V(G_1) \setminus V(G_2)$ to $V(G_2) \setminus V(G_1)$. If $\hat{\pi}$ is a permutation of $V(G)$ such that $\hat{\pi}|_{V(G_1)}$ is an automorphism of G_1 and $\hat{\pi}|_{V(G_2)}$ is an automorphism of G_2, then $\hat{\pi}$ is an automorphism of G.*

Proof: Let $u, v \in V(G)$. If u and v both belong to $V(G_1)$, then u and v are adjacent in G_1 if and only if $\hat{\pi}(u)$ and $\hat{\pi}(v)$ are adjacent in G_1 since $\hat{\pi}|_{V(G_1)}$ is an automorphism of G_1. Since G_1 is an induced subgraph of G, this means u and v are adjacent in G if and only if $\hat{\pi}(u)$ and $\hat{\pi}(v)$ are adjacent in G. The same statement also holds if $u, v \in V(G_2)$.

The only remaining case is when one vertex is in $V(G_1) \setminus V(G_2)$ and the other is in $V(G_2) \setminus V(G_1)$. In this case, u and v are not adjacent in G, and neither are $\hat{\pi}(u)$ and $\hat{\pi}(v)$ since $V(G_1) \cap V(G_2)$ is invariant under $\hat{\pi}$. ■

3.5.2 Proofs of recurrences

Recall that we are given n and π as input. The proofs of all of these recurrences build upon the proofs of the analogous recurrences in Chapter 2. For this reason, we highly recommend reading the proof of correctness of the counting algorithm in Chapter 2 before reading this section in detail. In the proof of almost every recurrence here, we will cite the analogous lemma from Chapter 2 to prove various properties such as the evaporation time of each subgraph.

Remark 2 *One important piece of the original proofs in Chapter 2 is that for each recurrence, the authors describe two injective functions. For example, in the proof of the recurrence for $g(t, x, k, z)$, there is an injective function that maps each graph G counted by $g(t, x, k, z)$ to a tuple (G_1, G_2, A), as well as an injective function that maps each tuple (G_1, G_2, A) to such a graph G. In this chapter, we only prove injectivity in detail in the proof of the recurrence for $\tilde{g}$, since this statement for each of the other recurrences is either similar to $\tilde{g}$ or simpler than $\tilde{g}$. This can be found in Lemmas 64 and 65, after the recurrence for $\tilde{g}$.*

We use uppercase letters to denote the set of graphs counted by each of the counter functions. For example, the set of graphs counted by $a(k, p, M)$ is denoted by $A(k, p, M)$, the set of graphs counted by $g\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$ is denoted by $G\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$, and so on. We also use similar notation for the original counter functions: $G(t, x, k, z)$ is the set of graphs counted by $g(t, x, k, z)$, $\widetilde{G}(t, x, k, z)$ is the set of graphs counted by $\tilde{g}(t, x, k, z)$, and so on. We proceed in the order in which our counter functions were

defined, so that similar functions and proofs appear consecutively. (This is somewhat different from the ordering of the lemma numbers.)

Lemma 50 *Let $k, p \in [n]$ and suppose $M \subseteq_{\pi^p} M_\pi$, where $|M| \leq k$. We have*

$$a(k,p,M) = \sum_{\substack{1 \leq k' \leq k \\ M' \in \widehat{\mathcal{P}}_{k'}}} c(k',p,M') a(k-k', p, M \setminus M') \cdot \begin{cases} \binom{k-|M|}{k'-|M'|} & \text{if } s \in M \\ \binom{k-1-|M|}{k'-1-|M'|} & \text{otherwise} \end{cases}$$
$$+ \sum_{C \in \mathcal{Q}} c(|C|, p \cdot p_C, C) a(k - p_C|C|, p, M \setminus C_{\pi^p}).$$

Proof: Let $\mathcal{G}_{inv}$ be the set of graphs $G \in A(k,p,M)$ such that C is invariant under π^p, where C is the connected component of G that contains s, and let $\mathcal{G}_{move} = A(k,p,M) \setminus \mathcal{G}_{inv}$. To prove this recurrence, we will first describe a map from $\mathcal{G}_{inv}$ to the set of tuples $(G_1, G_2, k', M', \hat{C})$ with the properties given below, and then we will describe a map from $\mathcal{G}_{move}$ to the set of tuples (G_1, G', C) given below. This will imply that $a(k,p,M)$ is at most the summation above. Next, to show that $a(k,p,M)$ is at least the summation above, we will describe a map from the set of tuples $(G_1, G_2, k', M', \hat{C})$ to $\mathcal{G}_{inv}$, as well as a map from the set of tuples (G_1, G', C) to $\mathcal{G}_{move}$. Strictly speaking, to prove these two bounds ($\leq$ and $\geq$), it remains to show that all of these maps are injective. See Remark 2.

To see that $a(k,p,M)$ is at most the summation above, suppose $G \in A(k,p,M)$. Let C be the set of vertices in the connected component of G that contains s. There are two possible cases: either C is invariant under π^p, or all of C is mapped to some other component of G by π^p. If we are in the first case, then let $G_1 = G[C]$ and $G_2 = G \setminus C$. Let $k' = |C|$, $M' = C \cap M$, $\hat{C} = C \setminus M'$, and $D = V(G_2)$. Note that $|\hat{C}| = k' - |M'|$. Since π^p maps C to itself, this means $\pi^p|_{V(G_1)}$ is an automorphism of G_1, $\pi^p|_{V(G_2)}$ is an automorphism of G_2, and we have $M' \subseteq_{\pi^p} M$. Clearly $|M'| \leq k'$. We also have $k' - |M'| \leq k - |M|$ since the number of vertices that are fixed points of π^p in G_1 is at most the number of fixed points in G. If $s \in M$, then we know $s \in M'$ by the definition

of C. Otherwise, if $s \notin M$, then we have $|M'| \leq k' - 1$ since $s \in C \setminus M'$. Hence $M' \in \mathcal{P}_{k'}$.

Now relabel G_1 by applying $\phi(\hat{C}, V_{k',M'} \setminus M')$ to the labels in $\hat{C}$, and relabel G_2 by applying $\phi(D \setminus M, V_{k-k',M\setminus M'} \setminus M)$ to the labels in $D \setminus M$. We have $M_{\pi^p} \cap V_{k,M} \subseteq M$ by the definition of $V_{k,M}$, so by only relabeling vertices outside of M, we know that we have not relabeled any vertices that are moved by π^p. Thus $\pi^p|_{V(G_1)}$ and $\pi^p|_{V(G_2)}$ continue to be automorphisms of G_1 and G_2. Clearly $D \setminus M = D \setminus (M \setminus M')$. We also have $V_{k-k',M\setminus M'} \setminus M = V_{k-k',M\setminus M'} \setminus (M \setminus M')$ since $V_{k-k',M\setminus M'} \setminus (M \setminus M')$ is disjoint from M_π and is therefore disjoint from M. Hence G_1 now has the vertex set $V_{k',M'}$ and G_2 has the vertex set $V_{k-k',M\setminus M'}$. Since G_1 is connected, we have $G_1 \in C(k', p, M')$ and $G_2 \in A(k - k', p, M \setminus M')$.

If we are in the second case, where C is mapped to some other component of G by π^p, then we have $C \in \mathcal{Q}$ since every connected component of G is mapped to a connected component of G by π^p. Let $C_1 = C$, and let $C_i = \pi^p(C_{i-1})$ for $2 \leq i \leq p_C$. Let $G_i = G[C_i]$ for $i \in [p_C]$. The graphs $G_2, \ldots, G_{p_C}$ are all isomorphic to G_1 since $\pi^p, \ldots, \pi^{p(p_C-1)}$ are automorphisms of G. (This ensures that the map that takes G to (G_1, G', C) is injective, where G' is as defined below.)

For the remainder of the graph, let $D = V(G) \setminus (C_1 \cup \cdots \cup C_{p_C})$, and let $G' = G[D]$. Leave G_1 as is, and relabel G' by applying $\phi(D \setminus M, V_{\hat{k},\hat{M}} \setminus M)$ to the labels in $D \setminus M$, where $\hat{k} = k - p_C|C|$ and $\hat{M} = M \setminus C_{\pi^p}$. We know $\pi^{p \cdot p_C}|_{V(G_1)}$ is an automorphism of G_1 since $\pi^{p \cdot p_C}|_{V(G)}$ is an automorphism of G that maps C to itself. We also know $\pi^p|_{V(G')}$ is an automorphism of G', so $G_1 \in C(|C|, p \cdot p_C, C)$ and $G' \in A(k - p_C|C|, p, M \setminus C_{\pi^p})$. Hence $a(k, p, M)$ is at most the summation above.

We now show that $a(k, p, M)$ is bounded below by this summation. For the first case, suppose we are given $1 \leq k' \leq k$, a set $M' \in \mathcal{P}_{k'}$, a graph $G_1 \in C(k', p, M')$, a graph $G_2 \in A(k - k', p, M \setminus M')$, and a subset $\hat{C} \subseteq V_{k,M} \setminus M$ of size $k' - |M'|$ that contains s if $s \notin M$. Let $C = M' \cup \hat{C}$, and let $D = V_{k,M} \setminus C$. We construct a graph G as follows:

Relabel G_1 by applying $\phi(V_{k',M'} \setminus M', \hat{C})$ to the labels in $V_{k',M'} \setminus M'$, and relabel G_2 by applying $\phi(V_{k-k',M\setminus M'} \setminus M, D \setminus M)$ to the labels in $V_{k-k',M\setminus M'} \setminus M$. Now let G be the union of G_1 and G_2 (by taking the union of the vertex sets and the edge sets). Since $\pi^p|_{V(G_1)}$ is an automorphism of G_1 and $\pi^p|_{V(G_2)}$ is an automorphism of G_2, this means $\pi^p|_{V(G)}$ is an automorphism of G, so we have $G \in A(k, p, M)$.

For the second case, suppose we are given a set $C \in \mathcal{Q}$, a graph $G_1 \in C(|C|, p \cdot p_C, C)$, and a graph $G' \in A(k - p_C|C|, p, M \setminus C_{\pi^p})$. Let $G_2, \ldots, G_{p_C}$ all be equal to G_1 initially. Leave G_1 as is, and relabel G_i by applying $\pi^{p(i-1)}$ to its label set for all $2 \leq i \leq p_C$. Let $D = V_{k,M} \setminus C_{\pi^p}$. Relabel G' by applying $\phi(V_{\hat{k},\hat{M}} \setminus M, D \setminus M)$ to the labels in $V_{\hat{k},\hat{M}} \setminus M$, where $\hat{k} = k - p_C|C|$ and $\hat{M} = M \setminus C_{\pi^p}$. Finally, let G be the union of $G_1, \ldots, G_{p_C}$, and G'. The label sets of $G_1, \ldots, G_{p_C}$ are all pairwise disjoint since $C \in \mathcal{Q}$. If we apply π^p to all of the labels in G_i for some $1 \leq i < p_C$, then the resulting graph is G_{i+1}. Furthermore, if we apply π^p to the labels in G_{p_C}, then the resulting graph is the same as the graph we obtain if we apply $\pi^{p \cdot p_C}$ to the labels in G_1. Since $\pi^{p \cdot p_C}|_{V(G_1)}$ is an automorphism of G_1, this graph is in fact equal to G_1. This means $\pi^p|_{V(\hat{G})}$ is an automorphism of $\hat{G}$, where $\hat{G}$ is the union of $G_1, \ldots, G_{p_C}$. Since we also know $\pi^p|_{V(G')}$ is an automorphism of G', this implies that $\pi^p|_{V(G)}$ is an automorphism of G. Therefore, $G \in A(k, p, M)$, so $a(k, p, M)$ is at least the summation above. ■

Lemma 51 *Let $k, p \in [n]$ and suppose $M \subseteq_{\pi^p} M_\pi$, where $|M| \leq k$. We have*

$$c(k, p, M) = \sum_{t=1}^{k} \tilde{g}_1 \begin{pmatrix} t & 0 & k \\ p & \emptyset & M \end{pmatrix}.$$

Proof: This is similar to the proof of the recurrence for $c(n)$ in Chapter 2 (see the beginning of Section 2.5.3). The only difference is that in this recurrence, both $c(k, p, M)$ and $\tilde{g}_1$ are counting graphs G with vertex set $V_{args} = V_{k,M}$ for which $\pi^p|_{V(G)}$ is an automorphism. ■

Lemma 54 *For g, we have*

$$g\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix} = \sum_{k'=0}^{k} \sum_{M \in \mathcal{I}_{k'}} \binom{k-|M_K|}{k'-|M|} \tilde{g}\begin{pmatrix} t & x & k' & z \\ p & M_X & M & M_Z \end{pmatrix} g\begin{pmatrix} t-1 & x & k-k' & z \\ p & M_X & M_K \setminus M & M_Z \end{pmatrix},$$

where $\mathcal{I}_{k'} = \{M \subseteq_{\pi^p} M_K : |M_K| - k + k' \leq |M| \leq k'\}$.

Proof: Let $\mathcal{G} = G\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$. To prove this recurrence, we will first describe a map from $\mathcal{G}$ to the set of tuples $(G_1, G_2, k', M, \hat{A})$ with the properties given below, which will imply that g is at most the summation above. Next, to show that g is at least the summation above, we will describe a map from the set of tuples $(G_1, G_2, k', M, \hat{A})$ to $\mathcal{G}$. Strictly speaking, to prove these two bounds ($\leq$ and $\geq$), it remains to show that both of these maps are injective. See Remark 2.

To see that g is at most the summation above, suppose $G \in G\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$. Let $args$, $args_1$, and $args_2$ be the arguments of g on the left, $\tilde{g}$, and g on the right, respectively. Let A be the set of vertices in components C of $G \setminus X_{args}$ such that C evaporates at time exactly t in G with exception set X_{args}, and let B be the set of vertices of all other components of $G \setminus X_{args}$. Let $G_1 = G[X_{args} \cup A]$, $G_2 = G[X_{args} \cup B]$, $k' = |A|$, $M = A \cap M_K$, and $\hat{A} = A \setminus M$. Note that $|\hat{A}| = k' - |M|$. We know X_{args} is invariant under π^p since M_X is invariant under π^p. If π^p were to map some vertex in A, which belongs to a component C of $G \setminus X_{args}$, to a vertex in B, then π^p would also map all of C to B, including the vertices of C that evaporate at time t. By Lemma 61, this is impossible, so A is invariant under π^p, and so is M. Since G has the automorphism $\pi^p|_{V(G)}$, by Lemma 62 this means G_1 has the automorphism $\pi^p|_{V(G_1)}$ and G_2 has the automorphism $\pi^p|_{V(G_2)}$. Clearly $|M| \leq k'$, and we also have $k' - |M| \leq k - |M_K|$ since the number of vertices that are fixed points of π^p in A is at most the number of fixed points in $G \setminus X_{args}$. Hence $M \in \mathcal{I}_{k'}$.

Now relabel G_1 by applying $\phi(X_{args} \setminus M_X, X_{args_1} \setminus M_X)$ to the labels in $X_{args} \setminus M_X$ and applying $\phi(\hat{A}, V_{args_1} \setminus (X_{args_1} \cup M))$ to the labels in $\hat{A}$. Relabel G_2 by applying $\phi(X_{args} \setminus M_X, X_{args_2} \setminus M_X)$ to the labels in $X_{args} \setminus M_X$ and applying $\phi(B \setminus M_K, V_{args_2} \setminus (X_{args_2} \cup M_K))$ to the labels in $B \setminus M_K$. We have only relabeled vertices that are not moved by π^p, so G_1 and G_2 still have the automorphisms $\pi^p|_{V(G_1)}$ and $\pi^p|_{V(G_2)}$. By the proof of Lemma 3 in Chapter 2, we have $G_1 \in \widetilde{G}(t, x, k', z)$ and $G_2 \in G(t-1, x, k-k', z)$.[3] Therefore, we have $G_1 \in \widetilde{G}\begin{pmatrix} t & x & k' & z \\ p & M_X & M & M_Z \end{pmatrix}$ and $G_2 \in G\begin{pmatrix} t-1 & x & k-k' & z \\ p & M_X & M_K \setminus M & M_Z \end{pmatrix}$, so g is at most the summation above.

We now show that g is bounded below by this summation. Suppose we are given $0 \leq k' \leq k$, a set $M \in \mathcal{I}_{k'}$, a graph $G_1 \in \widetilde{G}\begin{pmatrix} t & x & k' & z \\ p & M_X & M & M_Z \end{pmatrix}$, a graph $G_2 \in G\begin{pmatrix} t-1 & x & k-k' & z \\ p & M_X & M_K \setminus M & M_Z \end{pmatrix}$, and a subset $\hat{A} \subseteq V_{args} \setminus (X_{args} \cup M_K)$ of size $k' - |M|$. Let $A = M \cup \hat{A}$, and let $B = V_{args} \setminus (X_{args} \cup A)$. We construct a graph G as follows: Relabel G_1 by applying $\phi(X_{args_1} \setminus M_X, X_{args} \setminus M_X)$ to the labels in $X_{args_1} \setminus M_X$ and applying $\phi(V_{args_1} \setminus (X_{args_1} \cup M), \hat{A})$ to the labels in $G_1 \setminus (X_{args_1} \cup M)$. Relabel G_2 by applying $\phi(X_{args_2} \setminus M_X, X_{args} \setminus M_X)$ to the labels in $X_{args_2} \setminus M_X$ and applying $\phi(V_{args_2} \setminus (X_{args_2} \cup M_K), B \setminus M_K)$ to the labels in $G_2 \setminus (X_{args_2} \cup M_K)$. Since X_{args} is now a clique in both G_1 and G_2, we can glue G_1 and G_2 together at X_{args} to obtain a chordal graph G. By Lemma 63, $\pi^p|_{V(G)}$ is an automorphism of G. By the proof of Lemma 3 in Chapter 2, we have $G \in G(t, x, k, z)$.[4] Therefore, we have $G \in G\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$, so g is at least the summation above. ■

In the proof of the recurrence for $\tilde{g}$, we use the following relabeling procedures. The intuition behind these is the same as in our previous lemmas: we relabel the subgraphs

[3]Technically, this is not quite true since graphs in $\widetilde{G}(t, x, k', z)$ and $G(t-1, x, k-k', z)$ have vertex set $[x+k']$ and $[x+k-k']$, respectively. However, suppose we relabel G_1 by applying $\phi(Z_{args_1}, [z])$ to the labels in Z_{args_1}, applying $\phi(X_{args_1} \setminus Z_{args_1}, [x] \setminus [z])$ to the labels in $X_{args_1} \setminus Z_{args_1}$, and applying $\phi(V_{args_1} \setminus X_{args_1}, [x+k'] \setminus [x])$ to the labels in $V_{args_1} \setminus X_{args_1}$. After relabeling G_1 in this way, we have $G_1 \in \widetilde{G}(t, x, k', z)$. Also, if we relabel G_2 in a similar way, then we have $G_2 \in G(t-1, x, k-k', z)$.

[4]See Footnote 3.

in a similar way to Chapter 2, but we do not relabel the moved vertices.

Procedure A: Relabel G_1 by applying $\phi(\hat{X}', X_{args_1} \setminus M')$ to the labels in $\hat{X}'$ and applying $\phi(\hat{C}, V_{args_1} \setminus (X_{args_1} \cup M))$ to the labels in $\hat{C}$. Relabel G_2 by applying $\phi(X_{args} \setminus M_X, X_{args_2} \setminus M_X)$ to the labels in $X_{args} \setminus M_X$ and applying $\phi(D \setminus M_K, V_{args_2} \setminus (X_{args_2} \cup M_K))$ to the labels in $D \setminus M_K$.

Procedure B: Relabel G_1 by applying $\phi(\hat{X}', X_{args_3} \setminus M')$ to the labels in $\hat{X}'$, and leave the labels in C as they are. Relabel G' by applying $\phi(X_{args} \setminus M_X, X_{args_4} \setminus M_X)$ to the labels in $X_{args} \setminus M_X$ and applying $\phi(D \setminus M_K, V_{args_4} \setminus (X_{args_4} \cup M_K))$ to the labels in $D \setminus M_K$.

Procedure A′: In this procedure, we apply the inverse of all of the functions from *Procedure A*. For example, when relabeling G_1, we apply $\phi(X_{args_1} \setminus M', \hat{X}')$ to the labels in $X_{args_1} \setminus M'$.

Procedure B′: For all $i \in [p_C]$, relabel G_i by applying $\phi(X_{args_3} \setminus M', \hat{X}')$ to the labels in $X_{args_3} \setminus M'$. Also, for all $2 \leq i \leq p_C$, relabel G_i by applying $\pi^{p(i-1)}$ to the labels in M' and applying $\pi^{p(i-1)}$ to the labels in C. For G', we apply the inverse of both of the functions that were applied to G' in *Procedure B*.

Lemma 55 *For $\tilde{g}$, we have*

$$\tilde{g}\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix} = \sum_{\substack{1 \leq k' \leq k \\ 1 \leq x' \leq x \\ M \in \mathcal{P}_{k'} \\ M' \in \mathcal{I}_{x'}}} \tilde{g}_1\begin{pmatrix} t & x' & k' \\ p & M' & M \end{pmatrix} \tilde{g}\begin{pmatrix} t & x & k-k' & z \\ p & M_X & M_K \setminus M & M_Z \end{pmatrix}$$
$$\cdot \begin{cases} \binom{k-|M_K|}{k'-|M|} & \text{if } s \in M_K \\ \binom{k-1-|M_K|}{k'-1-|M|} & \text{otherwise} \end{cases}$$
$$\cdot \begin{cases} \binom{x-|M_X|}{x'-|M'|} & \text{if } M' \not\subseteq M_Z \\ \binom{x-|M_X|}{x'-|M'|} - \binom{z-|M_Z|}{x'-|M'|} & \text{otherwise} \end{cases}$$

$$+ \sum_{\substack{1 \le x' \le x \\ (C,M') \in \mathcal{Q}}} \tilde{g}_1 \begin{pmatrix} t & x' & |C| \\ p \cdot p_C & M' & C \end{pmatrix} \tilde{g} \begin{pmatrix} t & x & k - p_C|C| & z \\ p & M_X & M_K \setminus C_{\pi^p} & M_Z \end{pmatrix} \cdot \begin{cases} \binom{x-|M_X|}{x'-|M'|} & \text{if } M' \not\subseteq M_Z \\ \binom{x-|M_X|}{x'-|M'|} - \binom{z-|M_Z|}{x'-|M'|} & \text{otherwise.} \end{cases}$$

Proof: Let $\mathcal{G}_{inv}$ be the set of graphs $G \in \widetilde{G}\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$ such that C is invariant under π^p, where C is the connected component of $G \setminus X_{args}$ that contains s, and let $\mathcal{G}_{move} = \widetilde{G}\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix} \setminus \mathcal{G}_{inv}$. To prove this recurrence, we will first describe a map from $\mathcal{G}_{inv}$ to the set of tuples $(G_1, G_2, k', x', M, M', \hat{C}, \hat{X}')$ with the properties given below, and then we will describe a map from $\mathcal{G}_{move}$ to the set of tuples $(G_1, G', x', C, M', \hat{X}')$ given below. This will imply that $\tilde{g}$ is at most the summation above. Next, to show that $\tilde{g}$ is at least the summation above, we will describe a map from the set of tuples $(G_1, G_2, k', x', M, M', \hat{C}, \hat{X}')$ to $\mathcal{G}_{inv}$, as well as a map from the set of tuples $(G_1, G', x', C, M', \hat{X}')$ to $\mathcal{G}_{move}$. See Lemmas 64 and 65 for the proofs that all of these maps are injective.

Let $args$ be the arguments of $\tilde{g}$ (on the left side), and let $args_1$, $args_2$, $args_3$, and $args_4$ be the arguments of $\tilde{g}_1$, $\tilde{g}$, $\tilde{g}_1$ (the lower one), and $\tilde{g}$ (the lower one), in the order that they appear on the right. To see that $\tilde{g}$ is at most the summation above, suppose $G \in \widetilde{G}\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$. Let C be the set of vertices in the component of $G \setminus X_{args}$ that contains s, let $X' = N(C)$, and let $M' = X' \cap M_X$. There are two possible cases: either C is invariant under π^p, or all of C is mapped to some other component of $G \setminus X_{args}$ by π^p. If we are in the first case, then let D be the set of vertices of all other components of $G \setminus X_{args}$. Let $G_1 = G[X' \cup C]$, $G_2 = G[X_{args} \cup D]$, $k' = |C|$, $x' = |N(C)|$, $M = C \cap M_K$, $\hat{C} = C \setminus M$, and $\hat{X}' = X' \setminus M'$. Note that $|\hat{C}| = k' - |M|$ and $|\hat{X}'| = x' - |M'|$. We know $\hat{X}' \not\subseteq Z_{args}$ if $M' \subseteq M_Z$ since the definition of $\tilde{g}$ tells us that $X' \not\subseteq Z_{args}$. Since

C is invariant under π^p, this means $N(C)$ is also invariant under π^p. Therefore, by Lemma 62, $\pi^p|_{V(G_1)}$ is an automorphism of G_1 and $\pi^p|_{V(G_2)}$ is an automorphism of G_2. Since $|M'| \leq x'$, we have $M' \in \mathcal{I}_{x'}$. Clearly $|M| \leq k'$, and we also have $k'-|M| \leq k-|M_K|$ since the number of vertices that are fixed points of π^p in C is at most the number of fixed points in $G \setminus X_{args}$. If $s \in M_K$, then we know $s \in M$ by the definition of C. Otherwise, if $s \notin M_K$, then we have $|M| \leq k'-1$ since $s \in C \setminus M$. Hence $M \in \mathcal{P}_{k'}$. Now relabel G_1 and G_2 using *Procedure A* so that they have the desired vertex sets. By the proof of Lemma 4 in Chapter 2, we have $G_1 \in \widetilde{G}_1(t, x', k')$ and $G_2 \in \widetilde{G}(t, x, k-k', z)$.[5] Therefore, we have $G_1 \in \widetilde{G}_1\begin{pmatrix} t & x' & k' \\ p & M' & M \end{pmatrix}$ and $G_2 \in \widetilde{G}\begin{pmatrix} t & x & k-k' & z \\ p & M_X & M_K \setminus M & M_Z \end{pmatrix}$.

If we are in the second case, where C is mapped to some other component of $G \setminus X_{args}$ by π^p, then we claim that $(C, M') \in \mathcal{Q}$. We know every component of $G \setminus X_{args}$ is mapped to a component of $G \setminus X_{args}$ by π^p, so all vertices in C have the same period $p_C \geq 2$ with respect to (C, π^p). We have $N(\pi^{p \cdot p_C}(C)) = \pi^{p \cdot p_C}(N(C))$ since $\pi^{p \cdot p_C}|_{V(G)}$ is an automorphism of G, and we know $\pi^{p \cdot p_C}(C) = C$, so $\pi^{p \cdot p_C}(M') = M'$. Hence $(C, M') \in \mathcal{Q}$.

Let $x' = |N(C)|$ and let $\hat{X}' = X' \setminus M'$. Note that $|\hat{X}'| = x' - |M'|$. We also know $\hat{X}' \not\subseteq Z_{args}$ if $M' \subseteq M_Z$ since $X' \not\subseteq Z_{args}$. Let $C_1 = C$ and let $X'_1 = X'$. For $2 \leq i \leq p_C$, let $C_i = \pi^p(C_{i-1})$ and let $X'_i = \pi^p(X'_{i-1})$. Let $G_i = G[X'_i \cup C_i]$ for $i \in [p_C]$. For the remainder of the graph, let $D = V_{args} \setminus (X_{args} \cup C_1 \cup \cdots \cup C_{p_C})$, and let $G' = G[X_{args} \cup D]$. Now relabel G_1 and G' using *Procedure B*.

Let $\hat{G} = G[\check{X} \cup \check{C}]$, where $\check{X} = X'_1 \cup \cdots \cup X'_{p_C}$ and $\check{C} = C_1 \cup \cdots \cup C_{p_C}$. By Lemma 62, since $\check{X}$ and $\check{C}$ are both invariant under π^p, we know $\pi^p|_{V(\hat{G})}$ is an automorphism of $\hat{G}$ and $\pi^p|_{V(G')}$ is an automorphism of G'. Also, by applying Lemma 62 to $\hat{G}$, we can see that $\pi^{p \cdot p_C}|_{V(G_1)}$ is an automorphism of G_1 since $\pi^{p \cdot p_C}|_{V(\hat{G})}$ is an automorphism of $\hat{G}$ that maps C to itself and maps $N(C)$ to itself. By the proof of Lemma 4 in Chapter 2, we

[5] See Footnote 3 in Lemma 54.

have $G_1 \in \widetilde{G}_1(t, x', |C|)$ and $G' \in \widetilde{G}(t, x, k - p_C|C|, z)$.[6] Indeed, even though G' is defined slightly differently from G_2, we can similarly show that G' has the desired properties including the correct evaporation time. Therefore, we have $G_1 \in \widetilde{G}_1\begin{pmatrix} t & x' & |C| \\ p \cdot p_C & M' & C \end{pmatrix}$ and $G' \in \widetilde{G}\begin{pmatrix} t & x & k - p_C|C| & z \\ p & M_X & M_K \setminus C^\pi & M_Z \end{pmatrix}$, so $\tilde{g}$ is at most the summation above.

We now show that $\tilde{g}$ is bounded below by this summation. For the first case, suppose we are given $1 \leq k' \leq k$, $1 \leq x' \leq x$, a set $M \in \mathcal{P}_{k'}$, a set $M' \in \mathcal{I}_{x'}$, a graph $G_1 \in \widetilde{G}_1\begin{pmatrix} t & x' & k' \\ p & M' & M \end{pmatrix}$, a graph $G_2 \in \widetilde{G}\begin{pmatrix} t & x & k - k' & z \\ p & M_X & M_K \setminus M & M_Z \end{pmatrix}$, a subset $\hat{C} \subseteq V_{args} \setminus (X_{args} \cup M_K)$ of size $k' - |M|$ that contains s if $s \notin M_K$, and a subset $\hat{X}' \subseteq X_{args} \setminus M_X$ of size $x' - |M'|$ such that $M' \cup \hat{X}' \not\subseteq Z_{args}$. Let $C = M \cup \hat{C}$, $D = V_{args} \setminus (X_{args} \cup C)$, and $X' = M' \cup \hat{X}'$. We construct a graph G as follows: Relabel G_1 and G_2 using *Procedure* A', and then glue G_1 and G_2 together at X' to obtain G. By Lemma 63, $\pi^p|_{V(G)}$ is an automorphism of G. By the proof of Lemma 4 in Chapter 2, we have $G \in \widetilde{G}(t, x, k, z)$.[7] Therefore, we have $G \in \widetilde{G}\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$.

For the second case, suppose we are given $1 \leq x' \leq x$, a pair $(C, M') \in \mathcal{Q}$, a graph $G_1 \in \widetilde{G}_1\begin{pmatrix} t & x' & |C| \\ p \cdot p_C & M' & C \end{pmatrix}$, a graph $G' \in \widetilde{G}\begin{pmatrix} t & x & k - p_C|C| & z \\ p & M_X & M_K \setminus C^\pi & M_Z \end{pmatrix}$, and a subset $\hat{X}' \subseteq X_{args} \setminus M_X$ of size $x' - |M'|$ such that $M' \cup \hat{X}' \not\subseteq Z_{args}$. Let $X' = M' \cup \hat{X}'$, and let $D = V_{args} \setminus (X_{args} \cup C_{\pi^p})$. We construct a graph G as follows: Let $G_2, \ldots, G_{p_C}$ all be equal to G_1 initially. Next, relabel $G_1, \ldots, G_{p_C}$ and G' using *Procedure* B'. For all $i \in [p_C]$, glue the graph G_i to G' at $\pi^{p(i-1)}(X')$, and let G be the resulting graph. We know $X' \not\subseteq Z_{args}$, which also implies $\pi^{p(i-1)}(X') \not\subseteq Z_{args}$ for all $i \in [p_C]$ since M_Z is invariant under π^p.

An alternative way to construct the same graph G would be to do the following: Let $\check{X}$ be a clique with vertex set $X' \cup \pi^p(X') \cup \cdots \cup \pi^{p(p_C-1)}(X')$. Let $\hat{G}$ be the graph obtained by gluing G_i to $\check{X}$ at the clique $\pi^{p(i-1)}(X')$ for all $i \in [p_C]$. To obtain G, glue

[6] See Footnote 3 in Lemma 54.
[7] See Footnote 3 in Lemma 54.

$\hat{G}$ and G' together at $\check{X}$. We claim that $\pi^p|_{V(\hat{G})}$ is an automorphim of $\hat{G}$. We know $\pi^p(V(\hat{G})) = V(\hat{G})$ since $\pi^{p \cdot p_C}(C) = C$ and $\pi^{p \cdot p_C}(X') = X'$. Let $u, v \in V(\hat{G})$. If $u, v \in \check{X}$, then $\pi^p(u), \pi^p(v) \in \check{X}$, so u and v are adjacent, and so are $\pi^p(u)$ and $\pi^p(v)$. If $u \in \pi^{pi}(C)$ and $v \in \pi^{pj}(C)$ for some $i \not\equiv j \pmod{p_C}$, then $\pi^p(u) \in \pi^{p(i+1)}(C)$ and $\pi^p(v) \in \pi^{p(j+1)}(C)$, so u and v are not adjacent, and neither are $\pi^p(u)$ and $\pi^p(v)$. If $u, v \in V(G_i)$ for some $1 \leq i < p_C$, then $u, v \in V(G_{i+1})$, so u and v are adjacent if and only if $\pi^p(u)$ and $\pi^p(v)$ are adjacent by the way G_{i+1} was constructed. If $u, v \in V(G_{p_C})$, then $u, v \in V(G_1)$. This means u and v are adjacent if and only if $\pi^p(u)$ and $\pi^p(v)$ are adjacent since $\pi^{p \cdot p_C}|_{V(G_1)}$ is an automorphism of G_1. Lastly, if $u \in \pi^{pi}(C)$ for some i and $v \in \check{X} \setminus \pi^{pi}(X')$ (or vice versa), then u and v are not adjacent, and neither are $\pi^p(u)$ and $\pi^p(v)$ since $\pi^{p \cdot p_C}|_{V(G_1)}$ is an automorphism of G_1. Hence $\pi^p|_{V(\hat{G})}$ is an automorphim of $\hat{G}$. By Lemma 63, this implies that $\pi^p|_{V(G)}$ is an automorphism of G. By an argument similar to Lemma 4 in Chapter 2, we have $G \in \widetilde{G}(t, x, k, z)$.[8] Therefore, we have $G \in \widetilde{G}\begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$, so $\tilde{g}$ is at least the summation above. ■

In the following two lemmas, let $\mathcal{G}_{inv}$ and $\mathcal{G}_{move}$ be as defined in Lemma 55, and let $args = \begin{pmatrix} t & x & k & z \\ p & M_X & M_K & M_Z \end{pmatrix}$ be fixed values of the arguments of $\tilde{g}$.

Lemma 64 *Let ψ_1 be the map from $\mathcal{G}_{inv}$ to the set of tuples $(G_1, G_2, k', x', M, M', \hat{C}, \hat{X}')$ with certain properties from the first case of the "$\leq$" direction of the proof of Lemma 55, and let ψ_2 be the map from $\mathcal{G}_{move}$ to the set of tuples $(G_1, G', x', C, M', \hat{X}')$ with certain properties from the second case. The maps ψ_1 and ψ_2 are injective.*

Proof: To see that ψ_1 is injective, suppose G and $G^\star$ are distinct graphs in $\mathcal{G}_{inv}$. We want to show $\psi_1(G) \neq \psi_1(G^\star)$. Suppose that starting from G, we obtain $\psi_1(G) = (G_1, G_2, k', x', M, M', \hat{C}, \hat{X}')$, along with the sets C, D, and X' as described in Lemma 55. Suppose that starting from $G^\star$, we obtain $\psi_1(G^\star) = (G_1^\star, G_2^\star, k'^\star, x'^\star, M^\star, M'^\star, \hat{C}^\star, \hat{X}'^\star)$,

[8] See Footnote 3 in Lemma 54.

along with the sets $C^\star$, $D^\star$, and $X'^\star$. If $(k', x', M, M', \hat{C}, \hat{X}') \neq (k'^\star, x'^\star, M^\star, M'^\star, \hat{C}^\star, \hat{X}'^\star)$, then we are done, so suppose $(k', x', M, M', \hat{C}, \hat{X}') = (k'^\star, x'^\star, M^\star, M'^\star, \hat{C}^\star, \hat{X}'^\star)$. This means $C = C^\star$ since $C = M \cup \hat{C}$ and $C^\star = M^\star \cup \hat{C}^\star$, which implies $D = D^\star$. Similarly, we also have $X' = X'^\star$.

Since $G \neq G^\star$, we either have $G[X' \cup C] \neq G^\star[X' \cup C]$ or $G[X_{args} \cup D] \neq G^\star[X_{args} \cup D]$. If $G[X' \cup C] \neq G^\star[X' \cup C]$, then after relabeling, $G[X' \cup C]$ still differs from $G^\star[X' \cup C]$ since we use the same sets $M = M^\star$ and $M' = M'^\star$ in both of these relabeling procedures. Thus in this case, we have $G_1 \neq G_1^\star$. Otherwise, if $G[X_{args} \cup D] \neq G^\star[X_{args} \cup D]$, then after relabeling, $G[X_{args} \cup D]$ still differs from $G^\star[X_{args} \cup D]$ since we use the same sets M_X and M_K when relabeling, so we have $G_2 \neq G_2^\star$. Therefore, ψ_1 is injective.

To see that ψ_2 is injective, suppose G and $G^\star$ are distinct graphs in $\mathcal{G}_{move}$. We want to show $\psi_2(G) \neq \psi_2(G^\star)$. Suppose that starting from G, we obtain $\psi_2(G) = (G_1, G', x', C, M', \hat{X}')$, along with the sets $C_1, \ldots, C_{p_C}, D, X'_1, \ldots, X'_{p_C}$ and the graphs $G_2, \ldots, G_{p_C}$ as described in Lemma 55, where $C_1 = C$. Also, suppose that starting from $G^\star$, we obtain $\psi_2(G^\star) = (G_1^\star, G'^\star, x'^\star, C^\star, M'^\star, \hat{X}'^\star)$, along with the sets $C_1^\star, \ldots, C_{p_C}^\star$, $D^\star, X_1'^\star, \ldots, X_{p_C}'^\star$, and the graphs $G_2^\star, \ldots, G_{p_C}^\star$, where $C_1^\star = C^\star$. If $(x', C, M', \hat{X}') \neq (x'^\star, C^\star, M'^\star, \hat{X}'^\star)$, then we are done, so suppose $(x', C, M', \hat{X}') = (x'^\star, C^\star, M'^\star, \hat{X}'^\star)$. We have $C_i = C_i^\star$ for all $i \in [p_C]$ since $C = C^\star$, using the fact that $C_i = \pi^p(C_{i-1})$ and $C_i^\star = \pi^p(C_{i-1}^\star)$. This implies $D = D^\star$. Similarly, we also have $X'_i = X_i'^\star$ for all $i \in [p_C]$ since $X'_1 = X_1'^\star$.

Since $G \neq G^\star$, we either have $G[X'_i \cup C_i] \neq G^\star[X'_i \cup C_i]$ for some $i \in [p_C]$ or $G[X_{args} \cup D] \neq G^\star[X_{args} \cup D]$. First, suppose we have $G[X'_i \cup C_i] \neq G^\star[X'_i \cup C_i]$ for some $i \in [p_C]$. For all $2 \leq i \leq p_C$, $\pi^{p(i-1)}$ is an isomorphism from G_1 to G_i, since $\pi^p, \ldots, \pi^{p(p_C-1)}$ are automorphisms of G. Similarly, for all $2 \leq i \leq p_C$, $\pi^{p(i-1)}$ is an isomorphism from $G_1^\star$ to $G_i^\star$. Therefore, we have $G[X'_1 \cup C_1] \neq G^\star[X'_1 \cup C_1]$. After relabeling, $G[X'_1 \cup C_1]$ still differs from $G^\star[X'_1 \cup C_1]$ since we use the same set $M' = M'^\star$ in both of these

relabeling procedures. Thus in this case, we have $G_1 \neq G_1^\star$. Otherwise, if $G[X_{args} \cup D] \neq G^\star[X_{args} \cup D]$, then after relabeling, $G[X_{args} \cup D]$ still differs from $G^\star[X_{args} \cup D]$ since we use the same sets M_X and M_K when relabeling, so we have $G_2 \neq G_2^\star$. Therefore, ψ_2 is injective. ■

Lemma 65 *Let ψ_1' be the map from the set of tuples $(G_1, G_2, k', x', M, M', \hat{C}, \hat{X}')$ with certain properties to $\mathcal{G}_{inv}$ from the first case of the "$\geq$" direction of the proof of Lemma 55, and let ψ_2' be the map from the set of tuples $(G_1, G', x', C, M', \hat{X}')$ with certain properties to $\mathcal{G}_{move}$ from the second case. The maps ψ_1' and ψ_2' are injective.*

Proof: To see that ψ_1' is injective, suppose

$$(G_1, G_2, k', x', M, M', \hat{C}, \hat{X}') \text{ and } (G_1^\star, G_2^\star, k'^\star, x'^\star, M^\star, M'^\star, \hat{C}^\star, \hat{X}'^\star)$$

are distinct tuples in the domain of ψ_1' that map to G and $G^\star$, respectively. If $k' \neq k'^\star$, then either $M \neq M^\star$ or $\hat{C} \neq \hat{C}^\star$ since $k' = |M| + |\hat{C}|$ and $k'^\star = |M^\star| + |\hat{C}^\star|$. Similarly, if $x' \neq x'^\star$, then $\hat{X}' \neq \hat{X}'^\star$ or $M' \neq M'^\star$. Therefore, we know $(G_1, G_2, M, M', \hat{C}, \hat{X}') \neq (G_1^\star, G_2^\star, M^\star, M'^\star, \hat{C}^\star, \hat{X}'^\star)$. Let $X' = M' \cup \hat{X}'$ and $X'^\star = M'^\star \cup \hat{X}'^\star$.

First, suppose $(M, \hat{C}) \neq (M^\star, \hat{C}^\star)$, which means $M \cup \hat{C} \neq M^\star \cup \hat{C}^\star$. After we relabel G_1, the vertex set of $G_1 \setminus X'$ is $M \cup \hat{C}$, which contains s. Similarly, after we relabel $G_1^\star$, the vertex set of $G_1^\star \setminus X'^\star$ is $M^\star \cup \hat{C}^\star$, which also contains s. Therefore, the component of $G \setminus X_{args}$ that contains s has a different label set from the component of $G^\star \setminus X_{args}$ that contains s, so $G \neq G^\star$.

Next, suppose $(M', \hat{X}') \neq (M'^\star, \hat{X}'^\star)$, which means $X' \neq X'^\star$. The neighborhood of the component of $G \setminus X_{args}$ that contains s is X', and the neighborhood of the component of $G^\star \setminus X_{args}$ that contains s is $X'^\star$. Hence $G \neq G^\star$.

Lastly, suppose $(G_1, G_2) \neq (G_1^\star, G_2^\star)$ and $(M, M', \hat{C}, \hat{X}') = (M^\star, M'^\star, \hat{C}^\star, \hat{X}'^\star)$, since this is the only remaining possibility. If $G_1 \neq G_1^\star$, then either the component of $G \setminus X_{args}$

that contains s will differ from the component of $G^\star \setminus X_{args}$ that contains s, or the edges from those components to X_{args} will differ. Similarly, if $G_2 \neq G_2^\star$, then either the remaining components of $G \setminus X_{args}$ will differ from those of $G^\star \setminus X_{args}$, or the edges from those components to X_{args} will differ. Hence $G \neq G^\star$, so ψ_1' is injective.

To see that ψ_2' is injective, let $(G_1, G', x', C, M', \hat{X}')$ and $(G_1^\star, G'^\star, x'^\star, C^\star, M'^\star, \hat{X}'^\star)$ be distinct tuples in the domain of ψ_2' that map to G and $G^\star$, respectively. By a similar argument to the first paragraph above, we know $(G_1, G', C, M', \hat{X}') \neq (G_1^\star, G'^\star, C^\star, M'^\star, \hat{X}'^\star)$. As before, let $X' = M' \cup \hat{X}'$ and $X'^\star = M'^\star \cup \hat{X}'^\star$.

If $C \neq C^\star$, then a similar argument to the second paragraph shows that $G \neq G^\star$. If $(M', \hat{X}') \neq (M'^\star, \hat{X}'^\star)$, then a similar argument to the third paragraph shows that $G \neq G^\star$. Lastly, if $(G_1, G') \neq (G_1^\star, G'^\star)$ and $(C, M', \hat{X}') \neq (C^\star, M'^\star, \hat{X}'^\star)$, then a similar argument to the fourth paragraph shows that $G \neq G^\star$. Hence ψ_2' is injective. ■

Lemma 58 *The recurrence for $\tilde{g}_p$ is exactly the same as the recurrence for $\tilde{g}$ from Lemma 55, except for the two sums over x': In both summations, rather than summing over x' such that $1 \leq x' \leq x$, we sum over x' such that $1 \leq x' \leq x - 1$.*

Proof: The argument for $\tilde{g}_p$ is similar to the proof for $\tilde{g}$, except we do not allow $x' = x$ since no component of $G \setminus X_{args}$ sees all of X_{args}. ■

Lemma 52 *For $\tilde{g}_1$, we have*

$$\tilde{g}_1\begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix} = \sum_{\ell=1}^{k} \sum_{M \in \mathcal{I}_\ell} \binom{k - |M_K|}{\ell - |M|} f\begin{pmatrix} t & x & \ell & k-\ell \\ p & M_X & M & M_K \setminus M \end{pmatrix},$$

where $\mathcal{I}_\ell = \{M \subseteq_{\pi^p} M_K : |M_K| - k + \ell \leq |M| \leq \ell\}$.

Proof: Let $\mathcal{G} = \widetilde{G}_1\begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix}$. To prove this recurrence, we will first describe a map from $\mathcal{G}$ to the set of tuples $(G, \ell, M, \hat{L})$ with the properties given below, which will imply that $\tilde{g}_1$ is at most the summation above. Next, to show that $\tilde{g}_1$ is at least

the summation above, we will describe a map from the set of tuples $(G, \ell, M, \hat{L})$ to $\mathcal{G}$. Strictly speaking, to prove these two bounds ($\leq$ and $\geq$), it remains to show that both of these maps are injective. See Remark 2.

Let $args$ be the arguments of $\tilde{g}_1$. To see that $\tilde{g}_1$ is at most the summation above, suppose $G \in \widetilde{G}_1\begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix}$. Let $C = V_{args} \setminus X_{args}$, $L = L_G(X_{args})$, $A = C \setminus L$, $\ell = |L|$, $M = L \cap M_K$, and $\hat{L} = L \setminus M$. Note that $|\hat{L}| = \ell - |M|$. We know $\pi^p|_{V(G)}$ is an automorphism of G. By Lemma 61, L is invariant under π^p, so $M \subseteq_{\pi^p} M_K$. Clearly $|M'| \leq \ell$, and we also have $\ell - |M| \leq k - |M_K|$ since the number of vertices that are fixed points of π^p in L is at most the number of fixed points in $G \setminus X_{args}$. Hence $M \in \mathcal{I}_\ell$. Now relabel G to have the desired vertex set, in a similar way to Lemma 54. By the proof of Lemma 1 in Chapter 2, we have $G \in F(t, x, \ell, k - \ell)$.[9] Therefore, $G \in F\begin{pmatrix} t & x & \ell & k-\ell \\ p & M_X & M & M_K \setminus M \end{pmatrix}$, so $\tilde{g}_1$ is at most the summation above.

We now show that $\tilde{g}_1$ is bounded below by this summation. Suppose we are given $1 \leq \ell \leq k$, a set $M \in \mathcal{I}_\ell$, a graph $G \in F\begin{pmatrix} t & x & \ell & k-\ell \\ p & M_X & M & M_K \setminus M \end{pmatrix}$, and a subset $\hat{L} \subseteq V_{args} \setminus (X_{args} \cup M_K)$ of size $\ell - |M|$. Relabel G to have the desired vertex set (in a similar way to Lemma 54). Clearly $\pi^p|_{V(G)}$ is an automorphism of G. By the proof of Lemma 1 in Chapter 2, we have $G \in \widetilde{G}_1(t, x, k)$.[10] Therefore, $G \in \widetilde{G}_1\begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix}$, so $\tilde{g}_1$ is at least the summation above. ■

Lemma 57 *For $\tilde{g}_{\geq 2}$, we have*

$$\tilde{g}_{\geq 2}\begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix} =$$
$$\sum_{\substack{1 \leq k' \leq k \\ M \in \mathcal{P}_{k'}}} \tilde{g}_1\begin{pmatrix} t & x & k' \\ p & M_X & M \end{pmatrix} \left(\tilde{g}_1\begin{pmatrix} t & x & k-k' \\ p & M_X & M_K \setminus M \end{pmatrix} + \tilde{g}_{\geq 2}\begin{pmatrix} t & x & k-k' \\ p & M_X & M_K \setminus M \end{pmatrix} \right)$$

[9] See Footnote 3 in Lemma 54.
[10] See Footnote 3 in Lemma 54.

$$\cdot \begin{cases} \binom{k-|M_K|}{k'-|M|} & \text{if } s \in M_K \\ \binom{k-1-|M_K|}{k'-1-|M|} & \text{otherwise} \end{cases}$$

$$+ \sum_{C \in \mathcal{Q}} \tilde{g}_1 \begin{pmatrix} t & x & |C| \\ p \cdot p_C & M_X & C \end{pmatrix} \left(\tilde{g}_1 \begin{pmatrix} t & x & k - p_C|C| \\ p & M_X & M_K \setminus C^\pi \end{pmatrix} + \tilde{g}_{\geq 2} \begin{pmatrix} t & x & k - p_C|C| \\ p & M_X & M_K \setminus C^\pi \end{pmatrix} \right).$$

Proof: Let $\mathcal{G}_{inv}$ be the set of graphs $G \in \widetilde{G}_{\geq 2} \begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix}$ such that C is invariant under π^p, where C is the connected component of $G \setminus X_{args}$ that contains s, and let $\mathcal{G}_{move} = \widetilde{G}_{\geq 2} \begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix} \setminus \mathcal{G}_{inv}$. To prove this recurrence, we will first describe a map from $\mathcal{G}_{inv}$ to the set of tuples $(G_1, G_2, k', M, \hat{C})$ with the properties given below, and then we will describe a map from $\mathcal{G}_{move}$ to the set of tuples (G_1, G', C) given below. This will imply that $\tilde{g}_{\geq 2}$ is at most the summation above. Next, to show that $\tilde{g}_{\geq 2}$ is at least the summation above, we will describe a map from the set of tuples $(G_1, G_2, k', M, \hat{C})$ to $\mathcal{G}_{inv}$, as well as a map from the set of tuples (G_1, G', C) to $\mathcal{G}_{move}$. Strictly speaking, to prove these two bounds ($\leq$ and $\geq$), it remains to show that all of these maps are injective. See Remark 2.

Let $args$ be the arguments of $\tilde{g}_{\geq 2}$. To see that $\tilde{g}_{\geq 2}$ is at most the summation above, suppose $G \in \widetilde{G}_{\geq 2} \begin{pmatrix} t & x & k \\ p & M_X & M_K \end{pmatrix}$. Let C be the set of vertices in the component of $G \setminus X_{args}$ that contains s. There are two possible cases: either C is invariant under π^p, or all of C is mapped to some other component of $G \setminus X_{args}$ by π^p. If we are in the first case, then let D be the set of vertices of all other components of $G \setminus X_{args}$. Let $G_1 = G[X_{args} \cup C]$, $G_2 = G[X_{args} \cup D]$, $k' = |C|$, $M = C \cap M_K$, and $\hat{C} = C \setminus M$. Note that $|\hat{C}| = k' - |M|$. We know X_{args} is invariant under π^p, so by Lemma 62, $\pi^p|_{V(G_1)}$ is an automorphism of G_1 and $\pi^p|_{V(G_2)}$ is an automorphism of G_2. We also have $M \in \mathcal{P}_{k'}$ by an argument similar to the proof for $\tilde{g}$. Now relabel G_1 and G_2 in a similar way to Lemma 55. By the proof of Lemma 6 in Chapter 2, we have $G_2 \in \widetilde{G}_1(t, x, k - k')$ if $G \setminus X_{args}$ has exactly two connected components, and we have $G_2 \in \widetilde{G}_{\geq 2}(t, x, k - k')$ otherwise. In

either case, we have $G_1 \in \widetilde{G}_1(t, x, k')$.[11] Therefore, we have $G_1 \in \widetilde{G}_1\binom{t \quad x \quad k'}{p \quad M_X \quad M}$ and either $G_2 \in \widetilde{G}_1\binom{t \quad x \quad k-k'}{p \quad M_X \quad M_K \setminus M}$ or $G_2 \in \widetilde{G}_{\geq 2}\binom{t \quad x \quad k-k'}{p \quad M_X \quad M_K \setminus M}$.

If we are in the second case, where C is mapped to some other component of $G \setminus X_{args}$ by π^p, then we know $C \in \mathcal{Q}$ by an argument similar to the proof for $\tilde{g}$. Let $C_1 = C$, and let $C_i = \pi^p(C_{i-1})$ for $2 \leq i \leq p_C$. Let $G_i = G[X_{args} \cup C_i]$ for $i \in [p_C]$. For the remainder of the graph, let $D = V_{args} \setminus (X_{args} \cup C_1 \cup \cdots \cup C_{p_C})$, and let $G' = G[X_{args} \cup D]$. Now relabel G_1 and G' in a similar way to Lemma 55. We can see that $\pi^{p \cdot p_C}|_{V(G_1)}$ is an automorphism of G_1 and $\pi^p|_{V(G')}$ is an automorphism of G' by an argument similar to the proof for $\tilde{g}$. By an argument similar to Lemma 6 in Chapter 2, we have $G' \in \widetilde{G}_1(t, x, k - p_C|C|)$ if $G \setminus X_{args}$ has exactly two connected components, and we have $G' \in \widetilde{G}_{\geq 2}(t, x, k - p_C|C|)$ otherwise. In either case, we have $G_1 \in \widetilde{G}_1(t, x, |C|)$.[12] Therefore, we have $G_1 \in \widetilde{G}_1\binom{t \quad x \quad |C|}{p \cdot p_C \quad M_X \quad C}$ and either $G_2 \in \widetilde{G}_1\binom{t \quad x \quad k - p_C|C|}{p \quad M_X \quad M_K \setminus C^\pi}$ or $G_2 \in \widetilde{G}_{\geq 2}\binom{t \quad x \quad k - p_C|C|}{p \quad M_X \quad M_K \setminus C^\pi}$, so $\tilde{g}_{\geq 2}$ is at most the summation above.

We now show that $\tilde{g}_{\geq 2}$ is bounded below by this summation. For the first case, suppose we are given $1 \leq k' \leq k$, a set $M \in \mathcal{P}_{k'}$, a graph $G_1 \in \widetilde{G}_1\binom{t \quad x \quad k'}{p \quad M_X \quad M}$, a graph G_2 that belongs to $\widetilde{G}_1\binom{t \quad x \quad k-k'}{p \quad M_X \quad M_K \setminus M}$ or $\widetilde{G}_{\geq 2}\binom{t \quad x \quad k-k'}{p \quad M_X \quad M_K \setminus M}$, and a subset $\hat{C} \subseteq V_{args} \setminus (X_{args} \cup M_K)$ of size $k' - |M|$ that contains s if $s \notin M_K$. We construct a graph G as follows: Relabel G_1 and G_2 in a similar way to Lemma 55, and then glue G_1 and G_2 together at X_{args} to obtain G. By Lemma 63, $\pi^p|_{V(G)}$ is an automorphism of G. By the proof of Lemma 6 in Chapter 2, we have $G \in \widetilde{G}_{\geq 2}(t, x, k)$.[13] Therefore, we have $G \in \widetilde{G}_{\geq 2}\binom{t \quad x \quad k}{p \quad M_X \quad M_K}$.

For the second case, suppose we are given $C \in \mathcal{Q}$, a graph $G_1 \in \widetilde{G}_1\binom{t \quad x \quad |C|}{p \cdot p_C \quad M_X \quad C}$,

[11] See Footnote 3 in Lemma 54.
[12] See Footnote 3 in Lemma 54.
[13] See Footnote 3 in Lemma 54.

and a graph G' that belongs to $\widetilde{G}_1\binom{t \quad x \quad k-p_C|C|}{p \quad M_X \quad M_K \setminus C^\pi}$ or $\widetilde{G}_{\geq 2}\binom{t \quad x \quad k-p_C|C|}{p \quad M_X \quad M_K \setminus C^\pi}$. We construct a graph G as follows: Let $G_2, \ldots, G_{p_C}$ all be equal to G_1 initially. Next, relabel $G_1, \ldots, G_{p_C}$ and G' in a similar way to Lemma 55. For all $i \in [p_C]$, glue the graph G_i to G' at X_{args}, and let G be the resulting graph. We can see that $\pi^p|_{V(G)}$ is an automorphism of G by an argument similar to the proof for $\tilde{g}$. By an argument similar to Lemma 6 in Chapter 2, we have $G \in \widetilde{G}_{\geq 2}(t, x, k)$.[14] Therefore, we have $G \in \widetilde{G}_{\geq 2}\binom{t \quad x \quad k}{p \quad M_X \quad M_K}$, so $\tilde{g}$ is at least the summation above. ■

Lemma 53 *For f, we have*

$$f\binom{t \quad x \quad \ell \quad k}{p \quad M_X \quad M_L \quad M_K} = \sum_{k'=1}^{k} \sum_{M \in \mathcal{I}_{k'}} \binom{k-|M_K|}{k'-|M|} \tilde{f}\binom{t \quad x \quad \ell \quad k'}{p \quad M_X \quad M_L \quad M} g\binom{t-2 \quad x+\ell \quad k-k' \quad x}{p \quad M_X \cup M_L \quad M_K \setminus M \quad M_X},$$

where $\mathcal{I}_{k'} = \{M \subseteq_{\pi^p} M_K : |M_K| - k + k' \leq |M| \leq k'\}$.

Proof: Let $\mathcal{G} = F\binom{t \quad x \quad \ell \quad k}{p \quad M_X \quad M_L \quad M_K}$. To prove this recurrence, we will first describe a map from $\mathcal{G}$ to the set of tuples $(G_1, G_2, k', M, \hat{A})$ with the properties given below, which will imply that f is at most the summation above. Next, to show that f is at least the summation above, we will describe a map from the set of tuples $(G_1, G_2, k', M, \hat{A})$ to $\mathcal{G}$. Strictly speaking, to prove these two bounds ($\leq$ and $\geq$), it remains to show that both of these maps are injective. See Remark 2.

Let $args$, $args_1$, and $args_2$ be the arguments of f, $\tilde{f}$, and g, respectively. To see that f is at most the summation above, suppose $G \in F\binom{t \quad x \quad \ell \quad k}{p \quad M_X \quad M_L \quad M_K}$. Let A be the set of vertices in components C of $G \setminus (X_{args} \cup L_{args})$ such that C evaporates at time exactly $t-1$ in G with exception set X_{args}, and let B be the set of vertices of all other components of $G \setminus (X_{args} \cup L_{args})$. Let $G_1 = G[X_{args} \cup L_{args} \cup A]$, $G_2 = G[X_{args} \cup L_{args} \cup B]$, $k' = |A|$,

[14] See Footnote 3 in Lemma 54.

$M = A \cap M_K$, and $\hat{A} = A \setminus M$. Note that $|\hat{A}| = k' - |M|$. By Lemma 61, A is invariant under π^p, and so is M. Since G has the automorphism $\pi^p|_{V(G)}$, by Lemma 62 this means G_1 has the automorphism $\pi^p|_{V(G_1)}$ and G_2 has the automorphism $\pi^p|_{V(G_2)}$. We also have $M \in \mathcal{I}_{k'}$. Now relabel G_1 and G_2 so that they have the desired vertex sets, in a similar way to Lemma 54. By the proof of Lemma 2 in Chapter 2, we have $G_1 \in \widetilde{F}(t, x, \ell, k')$ and $G_2 \in G(t-2, x+\ell, k-k', x)$.[15] Therefore, we have $G_1 \in \widetilde{F}\begin{pmatrix} t & x & \ell & k' \\ p & M_X & M_L & M \end{pmatrix}$ and $G_2 \in G\begin{pmatrix} t-2 & x+\ell & k-k' & x \\ p & M_X \cup M_L & M_K \setminus M & M_X \end{pmatrix}$, so f is at most the summation above.

We now show that f is bounded below by this summation. Suppose we are given $1 \le k' \le k$, a set $M \in \mathcal{I}_{k'}$, a graph $G_1 \in \widetilde{F}\begin{pmatrix} t & x & \ell & k' \\ p & M_X & M_L & M \end{pmatrix}$, a graph $G_2 \in G\begin{pmatrix} t-2 & x+\ell & k-k' & x \\ p & M_X \cup M_L & M_K \setminus M & M_X \end{pmatrix}$, and a subset $\hat{A} \subseteq V_{args} \setminus (X_{args} \cup L_{args} \cup M_K)$ of size $k' - |M|$. We construct a graph G as follows: Relabel G_1 and G_2 so that they have the desired vertex sets, including the clique $X_{args} \cup L_{args}$ (in a similar way to Lemma 54), and then glue G_1 and G_2 together at $X_{args} \cup L_{args}$ to obtain G. By Lemma 63, $\pi^p|_{V(G)}$ is an automorphism of G. By the proof of Lemma 2 in Chapter 2, we have $G \in F(t, x, \ell, k)$.[16] Therefore, $G \in F\begin{pmatrix} t & x & \ell & k \\ p & M_X & M_L & M_K \end{pmatrix}$, so f is at least the summation above. ■

Lemma 56 *For $\tilde{f}$, we have*

$$\tilde{f}\begin{pmatrix} t & x & \ell & k \\ p & M_X & M_L & M_K \end{pmatrix} = \tilde{f}_p\begin{pmatrix} t & x & \ell & k \\ p & M_X & M_L & M_K \end{pmatrix} + \sum_{\substack{1 \le k' \le k \\ M \in \mathcal{I}_{k'}}} \binom{k - |M_K|}{k' - |M|} \tilde{g}_1\begin{pmatrix} t-1 & x+\ell & k' \\ p & M_X \cup M_L & M \end{pmatrix} \tilde{f}_p\begin{pmatrix} t & x & \ell & k-k' \\ p & M_X & M_L & M_K \setminus M \end{pmatrix}$$

$$+ \sum_{\substack{1 \le k' \le k \\ M \in \mathcal{I}_{k'}}} \binom{k - |M_K|}{k' - |M|} \tilde{g}_{\ge 2}\begin{pmatrix} t-1 & x+\ell & k' \\ p & M_X \cup M_L & M \end{pmatrix} \tilde{g}_p\begin{pmatrix} t-1 & x+\ell & k-k' & x \\ p & M_X \cup M_L & M_K \setminus M & M_X \end{pmatrix},$$

where $\mathcal{I}_{k'} = \{M \subseteq_{\pi^p} M_K : |M_K| - k + k' \le |M| \le k'\}$.

[15] See Footnote 3 in Lemma 54.
[16] See Footnote 3 in Lemma 54.

Proof: Let $\mathcal{G}$ be the set of graphs in $\widetilde{F}\binom{t \quad x \quad \ell \quad k}{p \quad M_X \quad M_L \quad M_K}$ such that at least one component of $G \setminus (X_{args} \cup L_{args})$ sees all of $X_{args} \cup L_{args}$. To prove (the second and third cases of) this recurrence, we will first describe a map from $\mathcal{G}$ to the set of tuples $(G_1, G_2, k', M, \hat{A})$ with the properties given below, which will imply that f is at most the summation above. Next, to show that f is at least the summation above, we will describe a map from the set of tuples $(G_1, G_2, k', M, \hat{A})$ to $\mathcal{G}$. Strictly speaking, to prove these two bounds ($\leq$ and $\geq$), it remains to show that both of these maps are injective. See Remark 2.

Let $args$ be the arguments of $\tilde{f}$, and let $args_1$, $args_2$, $args_3$, $args_4$, and $args_5$ be the arguments of $\tilde{f}_p$ (the first one), $\tilde{g}_1$, $\tilde{f}_p$ (the second one), $\tilde{g}_{\geq 2}$, and $\tilde{g}_p$, respectively. To see that $\tilde{f}$ is at most the summation above, suppose $G \in \widetilde{F}\binom{t \quad x \quad \ell \quad k}{p \quad M_X \quad M_L \quad M_K}$. The graph G falls into one of three cases: either zero, one, or at least two components of $G \setminus (X_{args} \cup L_{args})$ see all of $X_{args} \cup L_{args}$. In the first case, we have $G \in \widetilde{F}_p\binom{t \quad x \quad \ell \quad k}{p \quad M_X \quad M_L \quad M_K}$. If G falls into the second or third case, then let A be the set of vertices in the components that see all of $X_{args} \cup L_{args}$, and let B be the set of vertices of all other components of $G \setminus (X_{args} \cup L_{args})$. Let $G_1 = G[X_{args} \cup L_{args} \cup A]$, $G_2 = G[X_{args} \cup L_{args} \cup B]$, $k' = |A|$, $M = A \cap M_K$, and $\hat{A} = A \setminus M$. Note that $|\hat{A}| = k' - |M|$. By Lemma 61, L_{args} is invariant under π^p, which implies that A is invariant under π^p (and so is M). Since G has the automorphism $\pi^p|_{V(G)}$, by Lemma 62 this means G_1 has the automorphism $\pi^p|_{V(G_1)}$ and G_2 has the automorphism $\pi^p|_{V(G_2)}$. We also have $M \in \mathcal{I}_{k'}$. Now relabel G_1 and G_2 so that they have the desired vertex sets, in a similar way to Lemma 54. If A consists of exactly one component of $G \setminus (X_{args} \cup L_{args})$, then by the proof of Lemma 5 in Chapter 2, we have $G_1 \in \widetilde{G}_1(t-1, x+\ell, k')$ and $G_2 \in \widetilde{F}_p(t, x, \ell, k-k')$,[17] which implies $G_1 \in \widetilde{G}_1\binom{t-1 \quad x+\ell \quad k'}{p \quad M_X \cup M_L \quad M}$ and $G_2 \in \widetilde{F}_p\binom{t \quad x \quad \ell \quad k-k'}{p \quad M_X \quad M_L \quad M_K \setminus M}$. Otherwise, by the proof of Lemma 5 in Chapter 2, we have $G_1 \in \widetilde{G}_{\geq 2}(t-1, x+\ell, k')$ and

[17] See Footnote 3 in Lemma 54.

$G_2 \in \widetilde{G}_p(t-1, x+\ell, k-k', x)$,[18] which implies $G_1 \in \widetilde{G}_{\geq 2}\binom{t-1 \quad x+\ell \quad k'}{p \quad M_X \cup M_L \quad M}$ and $G_2 \in \widetilde{G}_p\binom{t-1 \quad x+\ell \quad k-k' \quad x}{p \quad M_X \cup M_L \quad M_K \setminus M \quad M_X}$. Hence $\tilde{f}$ is at most the summation above.

We now show that $\tilde{f}$ is bounded below by this summation. First of all, every graph in $\widetilde{F}_p\binom{t \quad x \quad \ell \quad k}{p \quad M_X \quad M_L \quad M_K}$ belongs to $\widetilde{F}\binom{t \quad x \quad \ell \quad k}{p \quad M_X \quad M_L \quad M_K}$. For the second and third case, suppose we are given $1 \leq k' \leq k$, a set $M \in \mathcal{I}_{k'}$, a subset $\hat{A} \subseteq V_{args} \setminus (X_{args} \cup L_{args} \cup M_K)$ of size $k' - |M|$, and either a pair of graphs $G_1 \in \widetilde{G}_1\binom{t-1 \quad x+\ell \quad k'}{p \quad M_X \cup M_L \quad M}$ and $G_2 \in \widetilde{F}_p\binom{t \quad x \quad \ell \quad k-k'}{p \quad M_X \quad M_L \quad M_K \setminus M}$ or a pair of graphs $G_1 \in \widetilde{G}_{\geq 2}\binom{t-1 \quad x+\ell \quad k'}{p \quad M_X \cup M_L \quad M}$ and $G_2 \in \widetilde{G}_p\binom{t-1 \quad x+\ell \quad k-k' \quad x}{p \quad M_X \cup M_L \quad M_K \setminus M \quad M_X}$. We construct a graph G as follows: Relabel G_1 and G_2 so that they have the desired vertex sets, including the clique $X_{args} \cup L_{args}$ (in a similar way to Lemma 54), and then glue G_1 and G_2 together at $X_{args} \cup L_{args}$ to obtain G. By Lemma 63, we know $\pi^p|_{V(G)}$ is an automorphism of G. By the proof of Lemma 5 in Chapter 2, we have $G \in \widetilde{F}(t, x, \ell, k)$.[19] Therefore, $G \in \widetilde{F}\binom{t \quad x \quad \ell \quad k}{p \quad M_X \quad M_L \quad M_K}$, so $\tilde{f}$ is at least the summation above. ■

Lemma 59 *We have* $\tilde{f}_p\binom{t \quad x \quad \ell \quad k}{p \quad M_X \quad M_L \quad M_K} = \tilde{f}_p\binom{t \quad x \quad \ell \quad k \quad x}{p \quad M_X \quad M_L \quad M_K \quad M_X}$.

Proof: This is similar to the proof that $\tilde{f}_p(t, x, \ell, k) = \tilde{f}_p(t, x, \ell, k, x)$ in Chapter 2 (see Lemma 8). The only difference is that both sides now have the automorphism $\pi^p|_{V(G)}$, and the relevant vertex subsets are V_{args}, X_{args}, and L_{args}. ■

Lemma 60 *For* $\tilde{f}_p$ *with the argument* z*, we have*

$$\tilde{f}_p\binom{t \quad x \quad \ell \quad k \quad z}{p \quad M_X \quad M_L \quad M_K \quad M_Z} = \sum_{\substack{1 \leq k' \leq k \\ 0 \leq x' \leq x \\ 0 \leq \ell' \leq \ell \\ 0 < x'+\ell' < x+\ell}} \sum_{\substack{M \in \mathcal{P}_{k'} \\ M'_X \in \mathcal{I}_{x'} \\ M'_L \in \mathcal{J}_{\ell'}}} \tilde{g}_1\binom{t-1 \quad x'+\ell' \quad k'}{p \quad M'_X \cup M'_L \quad M}$$

[18] See Footnote 3 in Lemma 54.
[19] See Footnote 3 in Lemma 54.

$$\cdot \binom{\ell - |M_L|}{\ell' - |M'_L|} \cdot \begin{cases} \binom{k-|M_K|}{k'-|M|} & \text{if } s \in M_K \\ \binom{k-1-|M_K|}{k'-1-|M|} & \text{otherwise} \end{cases}$$

$$\cdot \begin{cases} \binom{x-|M_X|}{x'-|M'_X|} & \text{if } \ell' > 0 \text{ or } M'_X \not\subseteq M_Z \\ \binom{x-|M_X|}{x'-|M'_X|} - \binom{z-|M_Z|}{x'-|M'_X|} & \text{otherwise} \end{cases}$$

$$\cdot \begin{cases} \tilde{f}_p \begin{pmatrix} t & x+\ell' & \ell-\ell' & k-k' & z \\ p & M_X \cup M'_L & M_L \setminus M'_L & M_K \setminus M & M_Z \end{pmatrix} & \text{if } \ell' < \ell \\ \tilde{g}_p \begin{pmatrix} t-1 & x+\ell & k-k' & z \\ p & M_X \cup M'_L & M_K \setminus M & M_Z \end{pmatrix} & \text{otherwise} \end{cases}$$

$$+ \sum_{\substack{0 \le x' \le x \\ 0 \le \ell' \le \ell \\ 0 < x'+\ell' < x+\ell \\ (C, M'_X, M'_L) \in \mathcal{Q}}} \tilde{g}_1 \begin{pmatrix} t-1 & x'+\ell' & |C| \\ p \cdot p_C & M'_X \cup M'_L & C \end{pmatrix} \cdot \binom{\ell - |M_L|}{\ell' - |M'_L|}$$

$$\cdot \begin{cases} \binom{x-|M_X|}{x'-|M'_X|} & \text{if } \ell' > 0 \text{ or } M'_X \not\subseteq M_Z \\ \binom{x-|M_X|}{x'-|M'_X|} - \binom{z-|M_Z|}{x'-|M'_X|} & \text{otherwise} \end{cases}$$

$$\cdot \begin{cases} \tilde{f}_p \begin{pmatrix} t & x+\ell' & \ell-\ell' & k-p_C|C| & z \\ p & M_X \cup M'_L & M_L \setminus M'_L & M_K \setminus C^\pi & M_Z \end{pmatrix} & \text{if } \ell' < \ell \\ \tilde{g}_p \begin{pmatrix} t-1 & x+\ell & k-p_C|C| & z \\ p & M_X \cup M'_L & M_K \setminus C^\pi & M_Z \end{pmatrix} & \text{otherwise.} \end{cases}$$

Proof: Let $\mathcal{G}_{inv}$ be the set of graphs $G \in \widetilde{F}_p \begin{pmatrix} t & x & \ell & k & z \\ p & M_X & M_L & M_K & M_Z \end{pmatrix}$ such that C is invariant under π^p, where C is the connected component of $G \backslash (X_{args} \cup L_{args})$ that contains s, and let $\mathcal{G}_{move} = \widetilde{F}_p \begin{pmatrix} t & x & \ell & k & z \\ p & M_X & M_L & M_K & M_Z \end{pmatrix} \setminus \mathcal{G}_{inv}$. To prove this recurrence, we will first describe a map from $\mathcal{G}_{inv}$ to the set of tuples $(G_1, G_2, k', x', \ell', M, M'_X, M'_L, \hat{C}, \hat{X}', \hat{L}')$ with the properties given below, and then we will describe a map from $\mathcal{G}_{move}$ to the set of tuples $(G_1, G', x', \ell', C, M'_X, M'_L, \hat{X}', \hat{L}')$ given below. This will imply that $\tilde{f}_p$ is at most the summation above. Next, to show that $\tilde{f}_p$ is at least the summation above, we will

describe a map from the set of tuples $(G_1, G_2, k', x', \ell', M, M'_X, M'_L, \hat{C}, \hat{X}', \hat{L}')$ to $\mathcal{G}_{inv}$, as well as a map from the set of tuples $(G_1, G', x', \ell', C, M'_X, M'_L, \hat{X}', \hat{L}')$ to $\mathcal{G}_{move}$. Strictly speaking, to prove these two bounds ($\leq$ and $\geq$), it remains to show that all of these maps are injective. See Remark 2.

Let $args$ be the arguments of $\tilde{f}_p$. To see that $\tilde{f}_p$ is at most the summation above, suppose $G \in \widetilde{F}_p\begin{pmatrix} t & x & \ell & k & z \\ p & M_X & M_L & M_K & M_Z \end{pmatrix}$. Let C be the set of vertices in the component of $G \setminus (X_{args} \cup L_{args})$ that contains s. Let $X' = N(C) \cap X_{args}$, $L' = N(C) \cap L_{args}$, $M'_X = X' \cap M_X$, and $M'_L = L' \cap M_L$. There are two possible cases: either C is invariant under π^p, or all of C is mapped to some other component of $G \setminus (X_{args} \cup L_{args})$ by π^p. If we are in the first case, then let D be the set of vertices of all other components of $G \setminus (X_{args} \cup L_{args})$. Let $G_1 = G[X' \cup L' \cup C]$, $G_2 = G[X_{args} \cup L_{args} \cup D]$, $k' = |C|$, $x' = |X'|$, $\ell' = |L'|$, $M = C \cap M_K$, $\hat{C} = C \setminus M$, $\hat{X}' = X' \setminus M'_X$, and $\hat{L}' = L' \setminus M'_L$. Note that $|\hat{C}| = k' - |M|$, $|\hat{X}'| = x' - |M'_X|$, and $|\hat{L}'| = \ell' - |M'_L|$. If $\ell' = 0$ and $M'_X \subseteq M_Z$, then we know $\hat{X}' \not\subseteq Z_{args}$ since $G \setminus Z_{args}$ is connected. Since C is invariant under π^p, this means $N(C)$ is also invariant under π^p. Therefore, by Lemma 62, $\pi^p|_{V(G_1)}$ is an automorphism of G_1 and $\pi^p|_{V(G_2)}$ is an automorphism of G_2. Furthermore, L' is invariant under π^p since L_{args} is invariant under π^p, so $M_X \cup M'_L$ and $M_L \setminus M'_L$ are both invariant under π^p. Since $|M'_X| \leq x'$ and $|M'_L| \leq \ell'$, we have $M'_X \in \mathcal{I}_{x'}$ and $M'_L \in \mathcal{J}_{\ell'}$. We also have $M \in \mathcal{P}_{k'}$ by an argument similar to the proof for $\tilde{g}$. Now relabel G_1 and G_2 in a similar way to Lemma 55. By the proof of Lemma 9 in Chapter 2, we have $G_2 \in \widetilde{F}_p(t, x + \ell', \ell - \ell', k - k', z)$ if $\ell' < \ell$, and we have $G_2 \in \widetilde{G}_p(t-1, x+\ell, k-k', z)$ otherwise. In either case, we have $G_1 \in \widetilde{G}_1(t-1, x'+\ell', k')$.[20] Therefore, we either have $G_2 \in \widetilde{F}_p\begin{pmatrix} t & x+\ell' & \ell-\ell' & k-k' & z \\ p & M_X \cup M'_L & M_L \setminus M'_L & M_K \setminus M & M_Z \end{pmatrix}$ or $G_2 \in \widetilde{G}_p\begin{pmatrix} t-1 & x+\ell & k-k' & z \\ p & M_X \cup M'_L & M_K \setminus M & M_Z \end{pmatrix}$, and we have $G_1 \in \widetilde{G}_1\begin{pmatrix} t-1 & x'+\ell' & k' \\ p & M'_X \cup M'_L & M \end{pmatrix}$.

If we are in the second case, where C is mapped to some other component of

[20] See Footnote 3 in Lemma 54.

$G \setminus (X_{args} \cup L_{args})$ by π^p, then we know $(C, M'_X, M'_L) \in \mathcal{Q}$ by an argument similar to the proof for $\tilde{g}$. Let $x' = |X'|$, $\ell' = |L'|$, $\hat{X}' = X' \setminus M'_X$, and $\hat{L}' = L' \setminus M'_L$. As before, if $\ell' = 0$ and $M'_X \subseteq M_Z$, then we know $\hat{X}' \not\subseteq Z_{args}$ since $G \setminus Z_{args}$ is connected. Let $C_1 = C$, $X'_1 = X'$, and $L'_1 = L'$. For $2 \leq i \leq p_C$, let $C_i = \pi^p(C_{i-1})$, $X'_i = \pi^p(X'_{i-1})$, and $L'_i = \pi^p(L'_{i-1})$. Let $G_i = G[X'_i \cup L'_i \cup C_i]$ for $i \in [p_C]$. For the remainder of the graph, let $D = V_{args} \setminus (X_{args} \cup L_{args} \cup C_1 \cup \cdots \cup C_{p_C})$, and let $G' = G[X_{args} \cup L_{args} \cup D]$. Now relabel G_1 and G' in a similar way to Lemma 55. We can see that $\pi^{p \cdot p_C}|_{V(G_1)}$ is an automorphism of G_1 and $\pi^p|_{V(G')}$ is an automorphism of G' by an argument similar to the proof for $\tilde{g}$. By an argument similar to Lemma 9 in Chapter 2, we have $G' \in \widetilde{F}_p(t, x + \ell', \ell - \ell', k - p_C|C|, z)$ if $\ell' < \ell$, and we have $G' \in \widetilde{G}_p(t-1, x+\ell, k-p_C|C|, z)$ otherwise. In either case, we have $G_1 \in \widetilde{G}_1(t-1, x'+\ell', |C|)$.[21] Therefore, we have either $G' \in \widetilde{F}_p\begin{pmatrix} t & x+\ell' & \ell-\ell' & k-p_C|C| & z \\ p & M_X \cup M'_L & M_L \setminus M'_L & M_K \setminus C^\pi & M_Z \end{pmatrix}$ or $G' \in \widetilde{G}_p\begin{pmatrix} t-1 & x+\ell & k-p_C|C| & z \\ p & M_X \cup M'_L & M_K \setminus C^\pi & M_Z \end{pmatrix}$, and we have $G_1 \in \widetilde{G}_1\begin{pmatrix} t-1 & x'+\ell' & |C| \\ p \cdot p_C & M'_X \cup M'_L & C \end{pmatrix}$, so $\tilde{f}_p$ is at most the summation above.

We now show that $\tilde{f}_p$ is bounded below by this summation. For the first case, suppose we are given $1 \leq k' \leq k$, $0 \leq x' \leq x$, $0 \leq \ell' \leq \ell$ such that $0 < x' + \ell' < x + \ell$, a set $M \in \mathcal{P}_{k'}$, a set $M'_X \in \mathcal{I}_{x'}$, a set $M'_L \in \mathcal{J}_{\ell'}$, a graph $G_1 \in \widetilde{G}_1\begin{pmatrix} t-1 & x'+\ell' & k' \\ p & M'_X \cup M'_L & M \end{pmatrix}$, a graph G_2 such that $G_2 \in \widetilde{F}_p\begin{pmatrix} t & x+\ell' & \ell-\ell' & k-k' & z \\ p & M_X \cup M'_L & M_L \setminus M'_L & M_K \setminus M & M_Z \end{pmatrix}$ if $\ell' < \ell$ and $G_2 \in \widetilde{G}_p\begin{pmatrix} t-1 & x+\ell & k-k' & z \\ p & M_X \cup M'_L & M_K \setminus M & M_Z \end{pmatrix}$ otherwise, a subset $\hat{C} \subseteq V_{args} \setminus (X_{args} \cup L_{args} \cup M_K)$ of size $k' - |M|$ that contains s if $s \notin M_K$, a subset $\hat{X}' \subseteq X_{args} \setminus M_X$ of size $x' - |M'_X|$ such that $M' \cup \hat{X}' \not\subseteq Z_{args}$, and a subset $\hat{L}' \subseteq L_{args} \setminus M_L$ of size $\ell' - |M'_L|$. We construct a graph G as follows: Relabel G_1 and G_2 in a similar way to Lemma 55, and then glue G_1 and G_2 together at $X' \cup L'$ to obtain G. By Lemma 63, $\pi^p|_{V(G)}$ is an automorphism of G. By the proof of Lemma 9 in Chapter 2, we have $G \in \widetilde{F}_p(t, x, \ell, k, z)$.[22]

[21] See Footnote 3 in Lemma 54.

[22] See Footnote 3 in Lemma 54.

Therefore, we have $G \in \widetilde{F}_p\begin{pmatrix} t & x & \ell & k & z \\ p & M_X & M_L & M_K & M_Z \end{pmatrix}$.

For the second case, suppose we are given $0 \leq x' \leq x$, $0 \leq \ell' \leq \ell$ such that $0 < x' + \ell' < x + \ell$, a triple $(C, M'_X, M'_L) \in \mathcal{Q}$, a graph $G_1 \in \widetilde{G}_1\begin{pmatrix} t-1 & x'+\ell' & |C| \\ p \cdot p_C & M'_X \cup M'_L & C \end{pmatrix}$, a graph G' such that $G' \in \widetilde{F}_p\begin{pmatrix} t & x+\ell' & \ell-\ell' & k-p_C|C| & z \\ p & M_X \cup M'_L & M_L \setminus M'_L & M_K \setminus C^\pi & M_Z \end{pmatrix}$ if $\ell' < \ell$ and $G' \in \widetilde{G}_p\begin{pmatrix} t-1 & x+\ell & k-p_C|C| & z \\ p & M_X \cup M'_L & M_K \setminus C^\pi & M_Z \end{pmatrix}$ otherwise, a subset $\hat{X}' \subseteq X_{args} \setminus M_X$ of size $x' - |M'_X|$ such that $M' \cup \hat{X}' \not\subseteq Z_{args}$, and a subset $\hat{L}' \subseteq L_{args} \setminus M_L$ of size $\ell' - |M'_L|$. We construct a graph G as follows: Let $G_2, \ldots, G_{p_C}$ all be equal to G_1 initially. Next, relabel $G_1, \ldots, G_{p_C}$ and G' in a similar way to Lemma 55. For all $i \in [p_C]$, glue the graph G_i to G' at $\pi^{p(i-1)}(X' \cup L')$, and let G be the resulting graph. We can see that $\pi^p|_{V(G)}$ is an automorphism of G by an argument similar to the proof for $\tilde{g}$. By an argument similar to Lemma 9 in Chapter 2, we have $G \in \widetilde{F}_p(t, x, \ell, k, z)$.[23] Therefore, we have $G \in \widetilde{F}_p\begin{pmatrix} t & x & \ell & k & z \\ p & M_X & M_L & M_K & M_Z \end{pmatrix}$, so $\tilde{f}_p$ is at least the summation above. ■

3.5.3 Running time

The running time is dominated by the arithmetic operations needed to compute $\tilde{f}_p$. In the first part of the recurrence for $\tilde{f}_p$ (corresponding to when C is invariant under π^p), there are three summations that sum over at most n possible values (the sums over k', x', and ℓ'), and there are three summations that sum over at most 2^μ possible values (the sums over M, M'_X, and M'_L). As for the arguments, there are six arguments whose values are at most n (t, x, ℓ, k, z, p), and there are four arguments that are sets of moved points with at most 2^μ possible values (M_X, M_L, M_K). The analysis for the second part of the recurrence (when C is not invariant) is similar. Therefore, the overall running time is $O(2^{7\mu} n^9)$.

[23] See Footnote 3 in Lemma 54.

3.5.4 Sampling labeled chordal graphs with a given automorphism

The algorithm for SAMPLE_CHORDAL_LAB(n, π) is given by the following theorem.

Theorem 7 *There is a randomized algorithm that given $n \in \mathbb{N}$ and $\pi \in S_n$, generates a graph uniformly at random from the set of all labeled chordal graphs with vertex set $[n]$ for which π is an automorphism. This algorithm uses $O(2^{7\mu}n^9)$ arithmetic operations, where $\mu = |M_\pi|$.*

In Chapter 2, the authors used the standard sampling-to-counting reduction of [1] to derive an algorithm for sampling labeled chordal graphs from the corresponding counting algorithm. For labeled chordal graphs with a given automorphism, Theorem 7 follows from Theorem 6 in exactly the same way.

3.6 Bound on the Number of Labeled Chordal Graphs with Automorphism π

In this section, we prove a bound on the number of n-vertex labeled chordal graphs that have a given automorphism π, in terms of n and the number of moved points of π (Theorem 8). To do this, we begin by bounding the number of graphs of this form that have a given perfect elimination ordering (PEO) and a given maximum clique size. In fact, we only need to consider PEOs that delete some maximum clique of G as late as possible. The following observation shows that every chordal graph has such a PEO.

Observation 7 *For every n-vertex chordal graph G with maximum clique size ω, there exists a PEO $v_1, \ldots, v_n$ of G such that $G \setminus \{v_1, \ldots, v_{n-\omega}\}$ is a clique of size ω.*

Proof: Let C be a maximum clique in G. For each $i \in [n-\omega]$, by Lemma 40 we can always find a vertex v_i that is simplicial in $G \setminus \{v_1, \ldots, v_{i-1}\}$ and does not belong to C. After that, the remaining ω vertices of the graph can be added to the PEO in any order, since all of these are simplicial in $G \setminus \{v_1, \ldots, v_{n-\omega}\}$. ■

Suppose G is an n-vertex chordal graph with automorphism π and maximum clique size ω. Imagine that we delete the vertices of G in order according to a given PEO that has the property from Observation 7. For our purposes, it makes sense to truncate this PEO once the graph that remains becomes a clique of size ω because at that point, there is only one possibility for the edges of the remaining graph. A *truncated PEO* of an n-vertex chordal graph G with maximum clique size ω is an ordering $v_1, \ldots, v_{n-\omega}$ of a subset $S \subseteq V(G)$ such that $V(G) \setminus S$ is a clique of size ω and such that $v_1, \ldots, v_{n-\omega}$ can be extended to a perfect elimination ordering $v_1, \ldots, v_n$ of G. By Observation 7, every chordal graph has a truncated PEO.

For most of the lemmas in this section, we will count graphs that have the truncated PEO $1, 2, \ldots, n-\omega$. For $n \in \mathbb{N}$, $1 \leq \omega \leq n$, and $\pi \in S_n$, let $\alpha_\pi(n, \omega)$ denote the number of labeled chordal graphs G with vertex set $[n]$ and maximum clique size exactly ω such that π is an automorphism of G and such that $1, 2, \ldots, n-\omega$ is a truncated PEO of G. Let $\mathcal{A}_\pi(n, \omega)$ denote the set of such graphs.

We only need to bound the number of graphs with this particular truncated PEO because the number of graphs is the same regardless of which truncated PEO we choose. More formally, suppose $v_1, v_2, \ldots, v_{n-\omega}$ is an ordering of a subset of $[n]$ of size $n-\omega$. For n, ω, and π as above, let $\beta_\pi(n, \omega)$ denote the number of labeled chordal graphs G with vertex set $[n]$ and maximum clique size exactly ω such that π is an automorphism of G and such that $v_1, v_2, \ldots, v_{n-\omega}$ is a truncated PEO of G. Note that $\alpha_\pi(n, \omega)$ and $\beta_\pi(n, \omega)$ have different truncated PEOs. The following observation shows that this makes no difference.

Observation 8 *Let $n \in \mathbb{N}$, $1 \leq \omega \leq n$, $\pi \in S_n$. We have $\alpha_\pi(n, \omega) = \beta_\pi(n, \omega)$.*

Proof: Let $v_1, \ldots, v_n$ be an ordering of $[n]$ that begins with $v_1, \ldots, v_{n-\omega}$, and let $\psi : [n] \to [n]$ be the bijection defined by $\psi(i) = v_i$ for all $i \in [n]$. We have the following bijection between $\alpha_\pi(n, \omega)$ and $\beta_\pi(n, \omega)$: given a graph $G \in \alpha_\pi(n, \omega)$, relabel the vertices of G by applying ψ to the label of each vertex. ■

Our goal is now to prove a bound on $\alpha_\pi(n, \omega)$. It will be useful to consider two different cases having to do with which points are moved by π.

Two cases for the moved points. Suppose we are given $n \in \mathbb{N}$, $1 \leq \omega \leq n$, and $\pi \in S_n$. Let $\hat{C} = \{n - \omega + 1, n - \omega + 2, \ldots, n\}$. This is the clique of size ω that would remain after deleting the truncated PEO $1, 2, \ldots, n - \omega$ from a graph $G \in \mathcal{A}_\pi(n, \omega)$. Let $\mu = |M_\pi|$. The permutation π falls into one of the following cases:

1. $|M_\pi \cap \hat{C}| \leq \frac{9\mu}{10}$, i.e., at most $\frac{9}{10}$ of the moved points of π belong to $\hat{C}$
2. $|M_\pi \cap \hat{C}| > \frac{9\mu}{10}$.

We prove bounds for these two cases in Lemmas 71 and 72. However, Lemma 71 assumes $\omega \geq \frac{9n}{20}$ and $\mu \leq \frac{4n}{5}$, and Lemma 72 assumes $\omega \leq n - \frac{\mu}{5}$. We begin by dealing with the corresponding edge cases (when ω is small or close to n, or μ is close to n) in Lemmas 67 to 70 since these cases are relatively simple and use ideas that will also be useful later on.

Lemma 66 *Let $n \in \mathbb{N}$, $1 \leq \omega \leq n$. The number of labeled chordal graphs G with vertex set $[n]$ and maximum clique size exactly ω such that $1, 2, \ldots, n - \omega$ is a truncated PEO of G is at most*

$$n^n 2^{\omega(n-\omega)}.$$

Proof: For $0 \leq i \leq n-\omega$, let $\mathcal{A}_i$ denote the set of labeled chordal graphs G with vertex set $\{n-\omega-i+1,\ldots,n\}$ and maximum clique size exactly ω such that $n-\omega-i+1,\ldots,n-\omega$ is a truncated PEO of G. We prove $|\mathcal{A}_i| \leq n^i 2^{\omega i}$ by induction on i, for all $i \in \{0,1,\ldots,n-\omega\}$. When $i=0$, there is just one graph in $\mathcal{A}_0$, namely a clique of size ω. Now let $i \in [n-\omega]$ and suppose the statement is true for $i-1$. If we delete the vertex with label $n-\omega-i+1$ from a graph $G \in \mathcal{A}_i$, then the resulting graph (say G') belongs to $\mathcal{A}_{i-1}$. We claim that there are at most $n2^\omega$ possibilities for the neighborhood of this vertex in G. The neighborhood must be a clique since this vertex is simplicial in G, and thus it is contained in some maximal clique of G'. By Lemma 41, G' has at most n maximal cliques. To choose this neighborhood, we can first choose a maximal clique of G' (which has size at most ω) and then choose a subset of it. Therefore, there are at most $n2^\omega$ possibilities. Since $G' \in \mathcal{A}_{i-1}$, the number of possible graphs in $\mathcal{A}_i$ is at most $n2^\omega|\mathcal{A}_{i-1}| \leq n2^\omega n^{i-1}2^{\omega(i-1)} \leq n^i 2^{\omega i}$. This gives us the desired bound when $i = n-\omega$. ■

We can use Lemma 66 to bound the number of chordal graphs with small or large values of ω.

Lemma 67 *Let $n \in \mathbb{N}$, $1 \leq \omega < \frac{9n}{20}$. The number of labeled chordal graphs G with vertex set $[n]$ and maximum clique size exactly ω such that $1,2,\ldots,n-\omega$ is a truncated PEO of G is at most*

$$n^n 2^{n^2/4 - n^2/400}.$$

Proof: By Lemma 66, the number of such graphs is at most

$$n^n 2^{\omega(n-\omega)} = n^n 2^{(n/2+\Delta)(n/2-\Delta)} = n^n 2^{(n/2)^2-\Delta^2},$$

where $\Delta = \frac{n}{2} - \omega$. We know $\Delta^2 > \frac{n^2}{400}$ since $\omega < \frac{9n}{20}$, so this proves the bound. ■

Lemma 68 *Let $n \in \mathbb{N}$, $\frac{3n}{5} < \omega \leq n$. The number of labeled chordal graphs G with vertex set $[n]$ and maximum clique size exactly ω such that $1,2,\ldots,n-\omega$ is a truncated PEO*

of G is at most

$$n^n 2^{n^2/4 - n^2/100}.$$

Proof: Similarly to Lemma 67, the number of such graphs is at most

$$n^n 2^{\omega(n-\omega)} = n^n 2^{(n/2)^2 - \Delta^2},$$

where $\Delta = \omega - \frac{n}{2}$ this time. We know $\Delta^2 > \frac{n^2}{100}$ since $\omega > \frac{3n}{5}$, so this proves the bound. ■

Lemma 69 *Let $n \in \mathbb{N}$, $\pi \in S_n$, and suppose $n - \frac{\mu}{5} < \omega \leq n$, where $\mu = |M_\pi|$. The number of labeled chordal graphs G with vertex set $[n]$ and maximum clique size exactly ω such that $1, 2, \ldots, n - \omega$ is a truncated PEO of G is at most*

$$n^n 2^{n^2/4 - n^2/100}.$$

Proof: This follows immediately from Lemma 68 since $\mu \leq n$, and thus $\omega > \frac{4n}{5}$. ■

Next, we will deal with the case when $\mu > \frac{4n}{5}$ (Lemma 70). When a graph $\hat{G}$ has an automorphism π, it is necessary that every subgraph G of $\hat{G}$ "respects" π, in the sense that the edges of G obey whichever properties are required by π. More precisely, this requirement is defined as follows. In Definition 6, one can think of G as being a subgraph of a graph with vertex set $[n]$ for which π is an automorphism.

Definition 6 *Let $n \in \mathbb{N}$, $\pi \in S_n$, and let G be a labeled graph such that $V(G) \subseteq [n]$. We say G **respects** π if whenever a pair of vertices $a, b \in V(G)$ and a number $0 \leq p \leq n-1$ satisfy $\pi^p(a), \pi^p(b) \in V(G)$, we have the following: a is adjacent to b if and only if $\pi^p(a)$ is adjacent to $\pi^p(b)$.*

For a graph G with vertex set $[n]$, G respects π if and only if π is an automorphism of G. Imagine that we build up a graph $G \in \mathcal{A}_\pi(n, \omega)$ by starting with an empty vertex

set and then adding each of the vertices $n, n-1, \ldots, 1$ to the graph, in that order. It is clear that the current graph must respect π at every step of this process.

Next, the following definition will be useful for bounding the number of possible neighborhoods of a vertex.

Definition 7 *Let $n \in \mathbb{N}$, $1 \leq \omega \leq n$, $\pi \in S_n$. Let $1 \leq a \leq n-\omega$, let $V' = \{a+1, \ldots, n\}$, and let $b \in [n]$ be the largest number in the cycle of π that contains a. For $c \in \mathbb{N}$, we say the element a is $\frac{1}{c}$-**retrospective** with respect to ω and π if $a \neq b$ and the following holds: for every $M \subseteq V'$ such that $\omega - \frac{\omega}{c} \leq |M| \leq \omega$, we have*

$$|\pi^p(M) \cap V'| \geq \frac{\omega}{c},$$

where p is the number such that $\pi^p(a) = b$.

To see the intuition behind Definition 7, again imagine that we build up a graph $G \in \mathcal{A}_\pi(n, \omega)$ by adding the vertices $n, n-1, \ldots, 1$ to the graph, in that order. Each time we add a vertex a to the current graph (which has vertex set V'), we want to bound the number of possible neighborhoods for this vertex. Suppose vertex a is $\frac{1}{20}$-retrospective. We know vertex b has already been added to the graph since $b > a$. The neighborhood of vertex a is contained in some maximal clique M of $G[V']$. If $|M|$ is much smaller than ω, then there clearly are few possibilities for this neighborhood. On the other hand, if $|M|$ is close to ω (at least $\omega - \frac{\omega}{20}$), then the definition of $\frac{1}{20}$-retrospective says that a significant portion of the edges/non-edges from a to vertices in M *can be copied from edges/non-edges incident to b that have already been decided in the past* since π is an automorphism. In particular, the relationship between b and every vertex of $\pi^p(M) \cap V'$ has already been decided. Therefore, there are strictly fewer than $n2^\omega$ possibilities for the neighborhood of vertex a.

Using this notion, we can prove a bound that is tighter than Lemma 66 for the case when $\mu > \frac{4n}{5}$. By the above lemmas, we can assume ω is neither too large nor too small.

Suppose we have been given n, ω, π, and $0 \leq i \leq n-\omega$. From now on, let $V_i = \{n-\omega-i+1, \ldots, n\}$, and let $\mathcal{A}_i$ denote the set of labeled chordal graphs G with vertex set V_i and maximum clique size exactly ω such that $n-\omega-i+1, \ldots, n-\omega$ is a truncated PEO of G and such that G respects π.

Lemma 70 *Let $n \in \mathbb{N}$, $\frac{9n}{20} \leq \omega \leq \frac{3n}{5}$, $\pi \in S_n$. Let $\mu = |M_\pi|$, and suppose $\mu > \frac{4n}{5}$. We have*

$$\alpha_\pi(n, \omega) \leq n^n 2^{\omega(n-\omega)} 2^{-\omega n/300 + \omega/30}.$$

Proof: There are at least $\frac{2n}{5}$ vertices in the truncated PEO $1 \ldots, n-\omega$ since $\omega \leq \frac{3n}{5}$. Let B be the set of vertices j from this truncated PEO such that $1 \leq j \leq \frac{2n}{5}$, $j \in M_\pi$, and j is not the largest number in the cycle of π that contains j. We claim $|B| \geq \frac{n}{10} - 1$. Indeed, since there are only $\lceil \frac{3n}{5} \rceil$ vertices outside of the set $\{1, 2, \ldots, \lfloor 2n/5 \rfloor\}$ and we have $\mu > \frac{4n}{5}$, that means at least $\frac{4n}{5} - \lceil \frac{3n}{5} \rceil \geq \frac{n}{5} - 1$ of the vertices in the set $\{1, 2, \ldots, \lfloor 2n/5 \rfloor\}$ are moved by π. At least half of those that are moved are not the largest number in their cycle of π, so this implies $|B| \geq \frac{n}{10} - 1$.

We proceed by a similar argument to the proof of Lemma 66, except we now just want to count graphs for which π is an automorphism. We prove the following by induction on i, for all $i \in \{0, 1, \ldots, n-\omega\}$: $|\mathcal{A}_i| \leq n^i 2^{\omega i} 2^{-\omega k/30}$, where $k = |B \cap \{n-\omega-i+1, \ldots, n\}|$, i.e., k is the number of vertices in B that belong to the vertex set so far. Since $|B| \geq \frac{n}{10} - 1$, this statement with $i = n - \omega$ will give us the desired bound.

When $i = 0$, there is just one graph in $\mathcal{A}_0$, namely a clique of size ω. Now let $i \in [n-\omega]$ and suppose the statement is true for $i-1$. Let $a = n-\omega-i+1$. If we delete the vertex with label a from a graph $G \in \mathcal{A}_i$, then the resulting graph (say G') belongs to $\mathcal{A}_{i-1}$. If $a \notin B$, then there are at most $n2^\omega$ possibilities for the neighborhood of this vertex, as we saw in the proof of Lemma 66. Now suppose $a \in B$. In this case, we can prove a tighter bound by showing that this vertex is $\frac{1}{30}$-retrospective. Let M be a subset of V_{i-1} such

that $\frac{29\omega}{30} \leq |M| \leq \omega$. Let p be the number such that $\pi^p(a) = b$, where b is the largest number in the cycle of π that contains a. We know $|\pi^p(M)| = |M| \geq \frac{29\omega}{30} \geq \frac{87n}{200}$, and there are at most $\lfloor \frac{2n}{5} \rfloor$ vertices in $[n] \setminus V_{i-1}$, so there must be at least $\frac{87n}{200} - \lfloor \frac{2n}{5} \rfloor > \frac{n}{30} \geq \frac{\omega}{30}$ vertices of $\pi^p(M)$ in V_{i-1}. Hence vertex a is $\frac{1}{30}$-retrospective.

The neighborhood of vertex a is contained in some maximal clique M of G'. If $|M| < \frac{29\omega}{30}$, then there are at most $2^\omega 2^{-\omega/30}$ possible subsets of M. If $|M| \geq \frac{29\omega}{30}$, then we know the following, since G respects π: for every $c \in \pi^p(M) \cap V(G')$, a is adjacent to $\pi^{-p}(c) \in M$ if and only if b is adjacent to c in G'. The rest of the neighborhood of a could be an arbitrary subset of $\{d \in M : \pi^p(d) \notin V(G')\}$, but this set has size at most $\frac{29\omega}{30}$ since a is $\frac{1}{30}$-retrospective, so there are at most $2^\omega 2^{-\omega/30}$ subsets of M that could be the neighborhood of a. Overall, since G' has at most n maximal cliques, there are at most $n2^\omega 2^{-\omega/30}$ possibilities for the neighborhood of a.

Therefore, we have $|\mathcal{A}_i| \leq n2^\omega |\mathcal{A}_{i-1}|$ if $n - \omega - i + 1 \notin B$ (in which case k remains the same when going from i to $i+1$), and we have $|\mathcal{A}_i| \leq n2^\omega 2^{-\omega/30} |\mathcal{A}_{i-1}|$ otherwise (in which case k increases by 1). This proves the desired bound. ■

We are finally ready to prove a bound for case (1), when $|M_\pi \cap \hat{C}| \leq \frac{9\mu}{10}$.

Lemma 71 *Let $n \in \mathbb{N}$, $\frac{9n}{20} \leq \omega \leq n$, $\pi \in S_n$. Let $\mu = |M_\pi|$, and suppose $\mu \leq \frac{4n}{5}$. Also, suppose $|M_\pi \cap \hat{C}| \leq \frac{9\mu}{10}$, where $\hat{C} = \{n - \omega + 1, n - \omega + 2, \ldots, n\}$. In this case, we have*

$$\alpha_\pi(n, \omega) \leq n^n 2^{\omega(n-\omega)} 2^{-\omega\mu/400}.$$

Proof: Let B be the set of vertices $j \in [n - \omega]$ such that $j \in M_\pi$ and j is not the largest number in the cycle of π that contains j. At least $\frac{\mu}{10}$ of the vertices in $[n - \omega]$ are moved by π since $|M_\pi \cap \hat{C}| \leq \frac{9\mu}{10}$, and among those that are moved, at least half of them are not the largest in their cycle of π. Therefore, $|B| \geq \frac{\mu}{20}$.

We prove the following by induction on i, for all $i \in \{0, 1, \ldots, n - \omega\}$: $|\mathcal{A}_i| \leq n^i 2^{\omega i} 2^{-\omega k/20}$, where $k = |B \cap \{n - \omega - i + 1, \ldots, n\}|$. Since $|B| \geq \frac{\mu}{20}$, this statement

with $i = n - \omega$ will give us the desired bound.

When $i = 0$, we have $|\mathcal{A}_0| = 1$. Now let $i \in [n - \omega]$ and suppose the statement is true for $i - 1$. Let $a = n - \omega - i + 1$. If we delete the vertex with label a from a graph $G \in \mathcal{A}_i$, then the resulting graph (say G') belongs to $\mathcal{A}_{i-1}$. If $a \notin B$, then there are at most $n2^\omega$ possibilities for the neighborhood of this vertex. Now suppose $a \in B$. In this case, we claim that vertex a is $\frac{1}{20}$-retrospective. Let M be a subset of V_{i-1} such that $\frac{19\omega}{20} \leq |M| \leq \omega$. Suppose for a contradiction that $|\pi^p(M) \cap V_{i-1}| < \frac{\omega}{20}$, where p is as above. We have $|\pi^p(M) \setminus V_{i-1}| > \frac{18\omega}{20}$ since $|\pi^p(M)| \geq \frac{19\omega}{20}$, so π^p has at least $2 \cdot \frac{18\omega}{20}$ moved points. This means π has at least $\frac{9\omega}{5} \geq \frac{81n}{100} > \frac{4n}{5}$ moved points, since $M_{\pi^p} \subseteq M_\pi$. This is a contradiction since we are assuming $\mu \leq \frac{4n}{5}$. Hence vertex a is $\frac{1}{20}$-retrospective. By an argument similar to the proof of Lemma 70, this implies that there are at most $n2^\omega 2^{-\omega/20}$ possibilities for the neighborhood of a.

Therefore, we have $|\mathcal{A}_i| \leq n2^\omega |\mathcal{A}_{i-1}|$ if $n - \omega - i + 1 \notin B$ (in which case k remains the same), and we have $|\mathcal{A}_i| \leq n2^\omega 2^{-\omega/20} |\mathcal{A}_{i-1}|$ otherwise (in which case k increases by 1). This proves the desired bound. ■

Lastly, we prove a bound for case (2), when $|M_\pi \cap \hat{C}| > \frac{9\mu}{10}$.

Lemma 72 *Let $n \in \mathbb{N}$, $\pi \in S_n$. Suppose $1 \leq \omega \leq n - \frac{\mu}{5}$, where $\mu = |M_\pi|$. Also, suppose $|M_\pi \cap \hat{C}| > \frac{9\mu}{10}$, where $\hat{C} = \{n - \omega + 1, n - \omega + 2, \ldots, n\}$. In this case, we have*

$$\alpha_\pi(n, \omega) \leq n^n 2^{\omega(n-\omega)} 2^{-\mu^2/100 + \mu/10}.$$

Proof: Let $\hat{D} \coloneqq \{v_1, \ldots, v_{n-\omega}\}$ be the set of vertices outside of $\hat{C}$. There are at least $\frac{\mu}{5}$ vertices in $\hat{D}$ since $n \geq \omega + \frac{\mu}{5}$, and there are at most $\frac{\mu}{10}$ moved vertices in $\hat{D}$ since $|M_\pi \cap \hat{C}| > \frac{9\mu}{10}$. Therefore, there are at least $\frac{\mu}{10}$ vertices in $\hat{D}$ that are fixed points of π. Out of all of those vertices in $\hat{D} \setminus M_\pi$, let B be the set of the $\lfloor \frac{\mu}{10} \rfloor$ of these with the largest labels. Note that if G' is a graph with vertex set $\{\ell, \ell + 1, \ldots, n\}$, where ℓ is the vertex in B with the smallest label, then $|V(G')| \leq \omega + \frac{\mu}{5}$.

We prove the following by induction on i, for all $i \in \{0, 1, \ldots, n-\omega\}$: $|\mathcal{A}_i| \leq n^i 2^{\omega i} 2^{-\mu k/10}$, where $k = |B \cap \{n-\omega-i+1, \ldots, n\}|$. Since $|B| \geq \frac{\mu}{10} - 1$, this statement with $i = n - \omega$ will give us the desired bound.

When $i = 0$, we have $|\mathcal{A}_0| = 1$. Now let $i \in [n-\omega]$ and suppose the statement is true for $i-1$. Let $a = n-\omega-i+1$. If we delete the vertex with label a from a graph $G \in \mathcal{A}_i$, then the resulting graph (say G') belongs to $\mathcal{A}_{i-1}$. If $a \notin B$, then there are at most $n2^{\omega}$ possibilities for the neighborhood of this vertex. Now suppose $a \in B$. The neighborhood of vertex a is contained in some maximal clique M of G'. If $|M| < \omega - \frac{\mu}{10}$, then there are at most $2^{\omega - \mu/10}$ possible subsets of M.

On the other hand, if $|M| \geq \omega - \frac{\mu}{10}$, then we have $|V(G')| - |M| \leq \frac{3\mu}{10}$ since $|V(G')| \leq \omega + \frac{\mu}{5}$. We say a vertex $v \in M$ is *singly moved* if v is moved by π and belongs to a cycle C of π such that v is the only vertex in $C \cap M$. There are at most $\frac{3\mu}{10}$ moved points of π in $V(G') \setminus M$ and at most $\frac{\mu}{10}$ in $[n] \setminus V(G')$, so we have at most $\frac{2\mu}{5}$ moved points in $[n] \setminus M$. This means at most $\frac{2\mu}{5}$ of the vertices in M are singly moved, since each of those must belong to a cycle that lies partly outside of M. There are at least $\mu - \frac{2\mu}{5} = \frac{3\mu}{5}$ moved vertices in M in total, so at least $\frac{3\mu}{5} - \frac{2\mu}{5} = \frac{\mu}{5}$ of these are not singly moved. For each of these $\frac{\mu}{5}$ vertices, there is at least one other vertex from the same cycle of π that can also be found in M. Hence it is sufficient to just choose the relationship to a (i.e., edge or non-edge) for at most half of these vertices, since vertices in M that belong to the same cycle of π must have the same relationship to a. Once those choices have been made, then all of the other edges/non-edges between a and M have also been determined since G respects π. Overall, since G' has at most n maximal cliques, there are at most $n2^{\omega}2^{-\mu/10}$ possibilities for the neighborhood of a.

Therefore, we have $|\mathcal{A}_i| \leq n2^{\omega}|\mathcal{A}_{i-1}|$ if $n-\omega-i+1 \notin B$ (in which case k remains the same), and we have $|\mathcal{A}_i| \leq n2^{\omega}2^{-\mu/10}|\mathcal{A}_{i-1}|$ otherwise (in which case k increases by 1). This proves the desired bound. ■

Theorem 8 *Let $n \in \mathbb{N}$, $\pi \in S_n$, and let $\mu = |M_\pi|$. The number of labeled chordal graphs with vertex set $[n]$ for which π is an automorphism is at most*

$$n^{2n+1}2^{n^2/4-f(\mu)},$$

where $f(\mu) = \frac{\mu^2}{900} - \frac{\mu}{10}$.

Proof: Fix a maximum clique size $1 \leq \omega \leq n$. If $\omega < \frac{9n}{20}$, $\omega > \frac{3n}{5}$, or $\omega > n - \frac{\mu}{5}$, then by Lemmas 67 to 69, we have $\alpha_\pi(n,\omega) \leq n^n 2^{n^2/4-n^2/400}$. So from now on, we can assume $\frac{9n}{20} \leq \omega \leq \frac{3n}{5}$ and $\omega \leq n - \frac{\mu}{5}$.

If $\mu > \frac{4n}{5}$, then by Lemma 70 we have $\alpha_\pi(n,\omega) \leq n^n 2^{\omega(n-\omega)} 2^{-\omega n/300+\omega/30}$. We know $\omega(n-\omega) \leq \frac{n^2}{4}$ for all values of ω. Therefore, the bound in this case is at most $n^n 2^{n^2/4-3n^2/2000+n/30}$ since $\omega \geq \frac{9n}{20}$. On the other hand, if $\mu \leq \frac{4n}{5}$, then by Lemmas 71 and 72 we either have $\alpha_\pi(n,\omega) \leq n^n 2^{\omega(n-\omega)} 2^{-\omega\mu/400}$ or $\alpha_\pi(n,\omega) \leq n^n 2^{\omega(n-\omega)} 2^{-\mu^2/100+\mu/10}$, depending on which vertices are moved by π. The first of these two is bounded above by $n^n 2^{n^2/4-9\mu n/8000}$ since $\omega \geq \frac{9n}{20}$.

Combining all of these bounds yields $\alpha_\pi(n,\omega) \leq n^n 2^{n^2/4-f(n,\mu)}$, where

$$f(n,\mu) = \min\left\{\frac{n^2}{400}, \frac{3n^2}{2000} - \frac{n}{30}, \frac{9\mu n}{8000}, \frac{\mu^2}{100} - \frac{\mu}{10}\right\} \geq \frac{\mu^2}{900} - \frac{\mu}{10}$$

since $\frac{3n^2}{2000} - \frac{n}{30} \geq \frac{n^2}{900} - \frac{n}{10}$. There are n possible values of ω, and there are at most n^n possible choices for the truncated PEO of an n-vertex graph. Therefore, by Observation 8, the number of labeled chordal graphs with vertex set $[n]$ and automorphism π is at most $n^{2n+1}2^{n^2/4-f(\mu)}$, where $f(\mu) = \frac{\mu^2}{900} - \frac{\mu}{10}$. ■

Chapter 4

Counting Yes-Instances of Vertex Deletion Problems in FPT Time

4.1 Introduction

For a graph class $\mathcal{G}$, Vertex Deletion to $\mathcal{G}$ is the decision problem where the input is a graph G and a nonnegative integer k and the goal is to decide whether there exists a vertex subset $S \subseteq V(G)$ of size at most k such that $G \setminus S \in \mathcal{G}$. In this chapter, we work with three particular classes of undirected graphs. Let $\mathcal{G}_I$ be the class of edgeless graphs (i.e., the independent set graphs), let $\mathcal{G}_C$ be the class of cluster graphs, and let $\mathcal{G}_S$ be the class of split graphs. When the graph class in question is $\mathcal{G}_I$, $\mathcal{G}_C$, or $\mathcal{G}_S$, the problem of Vertex Deletion to $\mathcal{G}$ is known as Vertex Cover, Cluster Vertex Deletion, or Split Vertex Deletion, respectively. In this chapter, we present algorithms to solve the corresponding graph counting problems. In particular, for each of the above graph classes $\mathcal{G}$, we design an FPT algorithm to compute the number of n-vertex graphs that are yes-instances of Vertex Deletion to $\mathcal{G}$. In other words, given n and k as input, we compute the number of n-vertex graphs G such that $G \setminus S \in \mathcal{G}$ for some

$S \subseteq V(G)$ of size at most k.

We also consider one class of directed graphs. A *tournament* is a directed graph G such that for every pair of vertices $u, v \in V(G)$, exactly one of (u, v) or (v, u) is a directed edge of G. Let $\mathcal{G}_A$ be the class of acyclic tournaments. The problem of Vertex Deletion to $\mathcal{G}_A$ is the decision problem where the input is a tournament G and a nonnegative integer k and the goal is to decide whether there exists a vertex subset $S \subseteq V(G)$ of size at most k such that $G \setminus S \in \mathcal{G}_A$. This problem is also known as Feedback Vertex Set in Tournaments. In this chapter, we design an FPT algorithm to compute the number of n-vertex tournaments that are yes-instances of Vertex Deletion to $\mathcal{G}_A$.

To achieve this, we design a general framework that reduces each of these four counting problems to a simpler version of the problem with given deletion sets. For each of the problems in question, we then design an FPT algorithm for the version with given deletion sets. Overall, the algorithms for counting yes-instances of Vertex Cover, Cluster Vertex Deletion, and Feedback Vertex Set in Tournaments have running time at most $2^{3^{2k+o(k)}} \cdot n^{O(1)}$, and the running time of the algorithm for counting yes-instances of Split Vertex Deletion is a triple-exponential function of k. We also design a faster algorithm for counting yes-instances of Vertex Cover that runs in time $k^{O(k^2)}n^3$, using ideas inspired by the Buss kernel for that problem.

4.2 Preliminaries

In this chapter, we implicitly assume *all graphs that we consider are labeled graphs.* A directed graph G is acyclic if and only if there is a total order $\leq$ on $V(G)$ such that $u \leq v$ for every directed edge (u, v) [36]. Such an ordering is called a *topological ordering.* Furthermore, it is easy to see that every acyclic tournament has a unique topological ordering. In an undirected graph G, distinct vertices u and v are called *true twins* if

$N(u) = N(v)$ and $(u, v) \in E(G)$. The vertices u and v are *false twins* if $N(u) = N(v)$ and $(u, v) \notin E(G)$. An *ordered* partition is a partition that comes with a total order on the set of its parts. For a set A, we write 2^A to denote the power set of A.

4.3 Framework for Several FPT Graph Counting Algorithms

Suppose $\mathcal{G}$ is one of our three classes of undirected graphs ($\mathcal{G}_I$, $\mathcal{G}_C$, or $\mathcal{G}_S$). We are given n and k as input, and our goal is to compute the number of n-vertex graphs G such that $G \setminus S \in \mathcal{G}$ for some $S \subseteq V(G)$ of size at most k. The main idea of our algorithm is to repeatedly use the inclusion-exclusion principle to reduce this problem to the following: given sets $S_1, \ldots, S_p$ and n as input, count the number of n-vertex graphs G such that $G \setminus S_i \in \mathcal{G}$ for all $1 \leq i \leq p$. This reduction is essentially the same no matter which graph class we are dealing with. For the problem that we reduce to, however, each of the above graph classes needs its own algorithm that is tailored to that particular graph class.

The algorithm for counting yes-instances of Feedback Vertex Set in Tournaments follows a similar approach with directed graphs, but for now, we first describe the algorithm for a class $\mathcal{G}$ of undirected graphs.

4.3.1 Reducing to the problem with given sets

To begin with, we describe a general framework for counting yes-instances of Vertex Deletion to $\mathcal{G}$, assuming we have an algorithm for the above problem in which the sets $S_1, \ldots, S_p$ are given in the input. The identifier γ will specify which graph class we are dealing with. For example, $\mathcal{G}_\gamma = \mathcal{G}_I$ if $\gamma = \mathtt{I}$.

Definition 8 *Let $\gamma \in \{\mathcal{I}, \mathcal{C}, \mathcal{S}\}$. For a graph G, we say $S \subseteq V(G)$ is a $\mathcal{G}_\gamma$-**deletion set** of G if $G \setminus S \in \mathcal{G}_\gamma$. We say a $\mathcal{G}_\gamma$-deletion set S of G is **minimal** if no proper subset of S is a $\mathcal{G}_\gamma$-deletion set of G.*

For every graph class $\mathcal{G}_\gamma$ that we consider, there is a well-known bound on the number of minimal $\mathcal{G}_\gamma$-deletion sets of a given graph. Every graph has at most 2^k minimal vertex covers of size at most k since the bounded search tree in the standard branching algorithm for Vertex Cover has at most 2^k leaves. For Cluster Vertex Deletion, the standard bounded search tree for this problem has at most 3^k leaves since a graph is a cluster graph if and only if it has no induced P_3. Hence every graph has at most 3^k minimal $\mathcal{G}_C$-deletion sets of size at most k. For the class of split graphs, a graph is split if and only if it does not contain a cycle on four or five vertices or a $2K_2$ (two disjoint edges) as as induced subgraph. Thus the same bounded search tree argument shows that every graph has at most 5^k minimal $\mathcal{G}_S$-deletion sets of size at most k.[1]

We define the function $g_\gamma(k)$ to be this bound for each graph class $\mathcal{G}_\gamma$. In other words, $g_I(k) = 2^k$, $g_C(k) = 3^k$, and $g_S(k) = 5^k$ for all $k \geq 0$.

Theorem 9 *Let $\gamma \in \{\mathcal{I}, \mathcal{C}, \mathcal{S}\}$, and let $g(k) = g_\gamma(k)$. Let $n \in \mathbb{N}$, $0 \leq k \leq n$, and let $S_1, \ldots, S_p$ be subsets of $[n]$ of size at most k, where $1 \leq p \leq (k+1)g(k)$. Suppose we have an algorithm that, given n and the sets $S_1, \ldots, S_p$ as input, computes the number of graphs G with vertex set $[n]$ such that $G \setminus S_i \in \mathcal{G}_\gamma$ for all $i \in [p]$. Suppose the running time of this algorithm is $f(k)q(n)$. Then there is an algorithm that given $n \in \mathbb{N}$ and $0 \leq k \leq n$, computes the number of graphs G with vertex set $[n]$ such that $G \setminus S \in \mathcal{G}_\gamma$ for some $S \subseteq V(G)$ of size at most k. The running time of this algorithm is $g(k)^{O(k \cdot g(k))} f(k)q(n)$.*

[1]The FPT algorithm in [82] implies that there are in fact at most $2^{k+O(\log^2 k)}$ minimal $\mathcal{G}_S$-deletion sets of size at most k. However, this is not explicitly stated in that paper, so for completeness, we use the bound of 5^k in our results.

To prove Theorem 9, suppose we have been given n and k as input. Let $S^{(1)}, \ldots, S^{(r)}$ be all of the subsets of $[n]$ of size at most k. Note that $r = O(kn^k)$. For $i \in [r]$, let $\mathcal{G}_i$ be the set of graphs with vertex set $[n]$ for which $S^{(i)}$ is a minimal $\mathcal{G}_\gamma$-deletion set. It is clear that G has a $\mathcal{G}_\gamma$-deletion set of size at most k if and only if G has a *minimal* $\mathcal{G}_\gamma$-deletion set of size at most k, so our goal is to compute $|\bigcup_{i=1}^{r} \mathcal{G}_i|$. By the inclusion-exculsion principle, we have

$$\Big|\bigcup_{i=1}^{r} \mathcal{G}_i\Big| = \sum_{\emptyset \neq J \subseteq [r]} (-1)^{|J|+1} \Big|\bigcap_{i \in J} \mathcal{G}_i\Big|.$$

In fact, we only need to consider sets J of size at most $g(k)$ since no graph can have more than $g(k)$ minimal $\mathcal{G}_\gamma$-deletion sets:

$$\Big|\bigcup_{i=1}^{r} \mathcal{G}_i\Big| = \sum_{\substack{\emptyset \neq J \subseteq [r] \\ |J| \leq g(k)}} (-1)^{|J|+1} \Big|\bigcap_{i \in J} \mathcal{G}_i\Big|.$$

As written, the number of terms in this sum is not FPT. However, we can compute this sum more efficiently by observing that we only need to consider certain collections of sets $S^{(i)}$ that contain relatively small vertex labels. Let J be a nonempty subset of $[r]$. We say J is *compacted* if $\bigcup_{i \in J} S^{(i)} = [\ell]$, where $\ell = |\bigcup_{i \in J} S^{(i)}|$. For example, if $S^{(1)} = \{1, 3, 5\}$, $S^{(2)} = \{4, 5\}$, and $S^{(3)} = \{2, 3, 4\}$, then $J_1 = \{1, 3\}$ is compacted, but $J_2 = \{1, 2\}$ is not. We will show that it is sufficient to just compute $\big|\bigcap_{i \in J} \mathcal{G}_i\big|$ for the sets J that are compacted.

Suppose J' is a nonempty subset of $[r]$. Let $\ell = |\bigcup_{i \in J'} S^{(i)}|$, and let ϕ stand for the function $\phi(\bigcup_{i \in J'} S^{(i)}, [\ell])$. For each $i \in J'$, let $\hat{S}^{(i)} = \{\phi(v) : v \in S^{(i)}\}$. Let J be the set of indices such that $\{\hat{S}^{(i)} : i \in J'\} = \{S^{(i)} : i \in J\}$. The set J is called the *intersection pattern* of J'. The purpose of this is to relabel the elements in the collection of sets $\{S^{(i)} : i \in J'\}$ so that the labels are as small as possible. Note that every intersection pattern J is compacted.

Lemma 73 *Let J' be a nonempty subset of $[r]$, and let J be the intersection pattern of J'. We have*

$$\Big|\bigcap_{i\in J'}\mathcal{G}_i\Big| = \Big|\bigcap_{i\in J}\mathcal{G}_i\Big|.$$

Proof: Let $\mathcal{G}' = \bigcap_{i\in J'}\mathcal{G}_i$, $\mathcal{G} = \bigcap_{i\in J}\mathcal{G}_i$, and $\ell = |\bigcup_{i\in J'} S^{(i)}|$. Let ϕ_1 be the function $\phi(\bigcup_{i\in J'} S^{(i)}, [\ell])$, and let ϕ_2 be the function $\phi([n]\setminus\bigcup_{i\in J'} S^{(i)}, [n]\setminus[\ell])$. Let $\psi\colon \mathcal{G}' \to \mathcal{G}$ be the bijection defined as follows: for each $G\in\mathcal{G}$, $\psi(G)$ is the graph obtained by applying ϕ_1 to the labels in $\bigcup_{i\in J'} S^{(i)}$ and applying ϕ_2 to the remaining labels. The map ψ is indeed a bijection since ϕ_1 and ϕ_2 are both bijections, so $|\mathcal{G}'| = |\mathcal{G}|$. ■

Lemma 74 *Suppose $\emptyset \neq J \subseteq [r]$ is compacted, and let $\ell = |\bigcup_{i\in J} S^{(i)}|$. The number of subsets $J'\subseteq[r]$ with intersection pattern J is equal to $\binom{n}{\ell}$.*

Proof: Let $\mathcal{J}$ be the family of subsets $J'\subseteq[r]$ that have intersection pattern J, and let $\mathcal{S}$ be the family of all subsets of $[n]$ of size exactly ℓ. Let $\psi : \mathcal{J}\to\mathcal{S}$ be defined by $\psi(J') = \bigcup_{i\in J'} S^{(i)}$ for all $J'\in\mathcal{J}$. The inverse of ψ can be defined as follows. Let $A\in\mathcal{S}$, and let ϕ' be the function $\phi([\ell], A)$. For each $i\in J$, let $\hat{S}^{(i)} = \{\phi'(v) : v\in S^{(i)}\}$. Let J' be the set of indices such that $\{\hat{S}^{(i)} : i\in J\} = \{S^{(i)} : i\in J'\}$. We define $\psi^{-1}(A) = J'$. This function is indeed the inverse of ψ since J is the intersection pattern of J'. Therefore, ψ is a bijection, so $|\mathcal{J}| = |\mathcal{S}| = \binom{n}{\ell}$. ■

Let $\mathcal{C}$ be the family of nonempty subsets $J\subseteq[r]$ such that J is compacted and $|J|\leq g(k)$. If J is the intersection pattern of J', then $|J| = |J'|$. Therefore, by Lemmas 73 and 74, we have

$$\Big|\bigcup_{i=1}^{r}\mathcal{G}_i\Big| = \sum_{J\in\mathcal{C}}\binom{n}{\ell_J}(-1)^{|J|+1}\Big|\bigcap_{i\in J}\mathcal{G}_i\Big|, \tag{4.1}$$

where $\ell_J = |\bigcup_{i\in J} S^{(i)}|$. The number of terms that we need to compute in the sum over J is now roughly bounded above by $(k\cdot g(k))^{k\cdot g(k)}$ since there are at most $\binom{|J|}{k}$ possibilities

for a set $S^{(i)}$ of size k, and since $|J| \leq g(k)$. (See Section 4.3.2 for a more precise bound on $|\mathcal{C}|$.)

To compute each term $|\bigcap_{i\in J} \mathcal{G}_i|$, we just need to design an algorithm for the following problem: Given subsets $S_1, \ldots, S_p$ of $[n]$, compute the number of graphs G with vertex set $[n]$ such that S_i is a minimal $\mathcal{G}_\gamma$-deletion set of G for all $i \in [p]$. Requiring that S_i is a minimal $\mathcal{G}_\gamma$-deletion set of G is equivalent to requiring that S_i is a $\mathcal{G}_\gamma$-deletion set of G and none of the sets $X_1, \ldots, X_q$ are a $\mathcal{G}_\gamma$-deletion set of G, where $X_1, \ldots, X_q$ are all of the sets that can be formed by removing one element from S_i. Therefore, it is sufficient to have an algorithm for the following problem: Given subsets $S_1, \ldots, S_p$ and $X_1, \ldots, X_q$ of $[n]$, compute the number of graphs G with vertex set $[n]$ such that S_i is a $\mathcal{G}_\gamma$-deletion set of G for all $i \in [p]$, and X_i is *not* a $\mathcal{G}_\gamma$-deletion set of G for all $i \in [q]$.

Suppose we have been given subsets $S_1, \ldots, S_p$ and $X_1, \ldots, X_q$ of $[n]$. For $i \in [q]$, let $\mathcal{H}_i$ be the set of graphs G with vertex set $[n]$ such that S_j is a $\mathcal{G}_\gamma$-deletion set of G for all $j \in [p]$, and X_i is a $\mathcal{G}_\gamma$-deletion set of G. Also for $i \in [q]$, let $\overline{\mathcal{H}_i}$ be the set of graphs G with vertex set $[n]$ such that S_j is a $\mathcal{G}_\gamma$-deletion set of G for all $j \in [p]$, and X_i is *not* a $\mathcal{G}_\gamma$-deletion set of G. By the intersection version of the inclusion-exclusion principle, we have

$$\Big|\bigcap_{i=1}^{q} \overline{\mathcal{H}_i}\Big| = \sum_{J\subseteq[q]} (-1)^{|J|} \Big|\bigcap_{i\in J} \mathcal{H}_i\Big|. \tag{4.2}$$

Here $\bigcap_{i\in\emptyset} \mathcal{H}_i$ denotes the set of graphs with vertex set $[n]$ such that S_j is a $\mathcal{G}_\gamma$-deletion set of G for all $j \in [p]$. Therefore, it is sufficient to have an algorithm for the following problem, without the sets X_i: Given subsets $S_1, \ldots, S_p$ of $[n]$ of size at most k, compute the number of graphs G with vertex set $[n]$ such that S_i is a $\mathcal{G}_\gamma$-deletion set of G for all $i \in [p]$. This is precisely the algorithm that is given in the hypothesis of Theorem 9. Indeed, for each set S_i in one of the original collections of minimal $\mathcal{G}_\gamma$-deletion sets (containing at most $g(k)$ sets), there are at most k sets that can be formed by removing

one element from S_i. Hence we have $p \leq (k+1)g(k)$. This completes the algorithm for Theorem 9. See Section 4.3.2 for the running time analysis.

We can also prove a similar theorem for counting yes-instances of Feedback Vertex Set in Tournaments.

Definition 9 *For a tournament G, we say $S \subseteq V(G)$ is a $\mathcal{G}_A$-**deletion set** of G if $G \setminus S \in \mathcal{G}_A$. We say a $\mathcal{G}_A$-deletion set S of G is **minimal** if no proper subset of S is a $\mathcal{G}_\gamma$-deletion set of G.*

Every tournament has at most 3^k minimal $\mathcal{G}_A$-deletion sets of size at most k, since a tournament contains a directed cycle if and only if it contains a triangle (i.e., a directed cycle of length 3). Let $g(k)$ be this bound, i.e., $g(k) = 3^k$ for all $k \geq 0$.

Theorem 10 *Let $n \in \mathbb{N}$, $0 \leq k \leq n$, and let $S_1, \ldots, S_p$ be subsets of $[n]$ of size at most k, where $1 \leq p \leq (k+1)g(k)$. Suppose we have an algorithm that, given n and the sets $S_1, \ldots, S_p$ as input, computes the number of tournaments G with vertex set $[n]$ such that $G \setminus S_i \in \mathcal{G}_A$ for all $i \in [p]$. Suppose the running time of this algorithm is $f(k)q(n)$. Then there is an algorithm that given $n \in \mathbb{N}$ and $0 \leq k \leq n$, computes the number of tournaments G with vertex set $[n]$ such that $G \setminus S \in \mathcal{G}_A$ for some $S \subseteq V(G)$ of size at most k. The running time of this algorithm is $g(k)^{O(k \cdot g(k))} f(k) q(n)$.*

The algorithm for Theorem 10 is exactly the same as the algorithm for Theorem 9, except every time we had a "graph," we now have a "tournament," and every time we had a "$\mathcal{G}_\gamma$-deletion set," we now have a "$\mathcal{G}_A$-deletion set." All that remains is to analyze the running time.

4.3.2 Running time

At the start of the algorithm for Theorem 9, we first compute all of the necessary binomial coefficients that appear in the sum from Equation (4.1). We can compute all

values of $\binom{a}{b}$ such that $0 \leq a \leq n$ and $0 \leq b \leq a$ in $O(n^2)$ time.

The algorithm begins with Equation (4.1), where we iterate over all $J \in \mathcal{C}$. This amounts to iterating over all collections $\{S_1, \ldots, S_p\}$ of subsets of $[n]$ of size at most k such that $1 \leq p \leq g(k)$ and $\bigcup_{i=1}^{p} S_i = [\ell]$, where $\ell = |\bigcup_{i=1}^{p} S_i|$. The largest possible value of ℓ is $\ell_{\max} \coloneqq k \cdot g(k)$. Therefore, to achieve this, we can first iterate over all *ordered* collections $S_1, \ldots, S_p$ of subsets of $[n]$ of size at most k such that $1 \leq p \leq g(k)$ and $S_i \subseteq [\ell_{\max}]$ for all $i \in [p]$. The number of such ordered collections is at most $g(k)\left((k+1)\ell_{\max}^{k}\right)^{g(k)} \leq g(k)^{O(k \cdot g(k))}$ since $k \leq g(k)$. For each ordered collection $S_1, \ldots, S_p$, we check whether the sets are listed in lexicographic order, and we check whether $\bigcup_{i=1}^{p} S_i = [\ell]$, where $\ell = |\bigcup_{i=1}^{p} S_i|$. If so, we compute the corresponding term of Equation (4.1), and if not, we discard this collection of sets. The reason we require the sets to be in lexicographic order is to ensure that the same collection is not counted twice, since we actually want to iterate over all *unordered* collections of sets. Therefore, we can compute the right side of Equation (4.1) in time $g(k)^{O(k \cdot g(k))} \cdot T(n,k)$, where $T(n,k)$ is an upper bound on the time it takes to compute each term $|\bigcap_{i \in J} \mathcal{G}_i|$.

Now $T(n,k)$ is the time it takes to solve the following problem: Given subsets $S_1, \ldots, S_p$ and $X_1, \ldots, X_q$ of $[n]$, compute the number of graphs G with vertex set $[n]$ such that S_i is a $\mathcal{G}_\gamma$-deletion set of G for all $i \in [p]$, and X_i is *not* a $\mathcal{G}_\gamma$-deletion set of G for all $i \in [q]$. Here we can assume $1 \leq p \leq g(k)$ and $q \leq k \cdot g(k)$. We solve this problem using Equation (4.2), which has at most $2^{k \cdot g(k)}$ terms since each J is a subset of $[q]$. To compute each term $|\bigcap_{i \in J} \mathcal{H}_i|$, we use the algorithm from the hypothesis of Theorem 9, whose running time is $f(k) \cdot n^2 \log^2 n$. Hence we have $T(n,k) = 2^{k \cdot g(k)} f(k) \cdot n^2 \log^2 n$. Therefore, the overall running time is $g(k)^{O(k \cdot g(k))} f(k) \cdot n^2 \log^2 n$. The proof of the running time in Theorem 10 is exactly the same.

4.3.3 The algorithm for $\mathcal{G}_I$

To complete the algorithm for the graph class $\mathcal{G}_I$, we need to design an algorithm for the problem in which the sets $S_1, \ldots, S_p$ are given in the input. The following lemma gives us a convenient way of characterizing graphs that have a given collection of vertex covers.

Lemma 75 *Let $n \in \mathbb{N}$, and let $S_1, \ldots, S_p$ be subsets of $[n]$, where $p \geq 1$. Let $A = \bigcup_{i=1}^{p} S_i$, $B = [n] \setminus A$, and $T = \bigcap_{i=1}^{p} S_i$. Suppose G is a graph with vertex set $[n]$. Then S_i is a vertex cover of G for all $i \in [p]$ if and only if G has the following properties:*

1. *S_i is a vertex cover of $G[A]$ for all $i \in [p]$*
2. *B is an independent set*
3. *G has no edges with one endpoint in $A \setminus T$ and the other in B.*

Proof: First, suppose S_i is a vertex cover of G for all $i \in [p]$. Property (1) is clearly true, and we know B is an independent set since S_1 is a vertex cover of G. If $u \in A \setminus T$ and $v \in B$, then u and v are not adjacent since there is some set S_i that does not contain u. Thus property (3) holds as well.

For the other direction, suppose properties (1)-(3) hold for G. Let $i \in [p]$. Every edge in $G[A]$ is incident to some vertex in S_i by property (1). The only edges of G that do not belong to $G[A]$ are the edges from T to B, and all of these are incident to a vertex in S_i since $T \subseteq S_i$. Hence S_i is a vertex cover of G. ■

To count how many graphs have all of the given vertex covers, we simply need to count how many graphs satisfy properties (1)-(3). Let A, B, T be as defined in Lemma 75.

Lemma 76 *Let $n \in \mathbb{N}$, $0 \leq k \leq n$, and let $S_1, \ldots, S_p$ be subsets of $[n]$ of size at most k, where $1 \leq p \leq (k+1)2^k$. There is an algorithm that computes the number of graphs G*

with vertex set $[n]$ such that S_i is a vertex cover of G for all $i \in [p]$. The running time of this algorithm is $2^{O(k^4 2^{2k})} \cdot n^2 \log^2 n$.

Proof: **Algorithm.** Let $\mathcal{A}$ be the set of graphs G_A with vertex set A such that S_i is a vertex cover of G_A for all $i \in [p]$. To compute $|\mathcal{A}|$, we iterate over all graphs with vertex set A, and for each such graph, we check whether every set S_i is a vertex cover of the graph. Let $t = |T|$ and $b = |B|$. Return $2^{tb}|\mathcal{A}|$.

Correctness. This algorithm computes the number of graphs with vertex set $[n]$ that satisfy properties (1)-(3) from Lemma 75. The set $\mathcal{A}$ is the set of possibilities for $G[A]$, and for each fixed choice of $G[A]$, there are 2^{tb} possible ways of choosing the edges from T to B. Thus the number of graphs that satisfy properties (1)-(3) is $2^{tb}|\mathcal{A}|$.

Running time. We have $|A| \leq (k^2 + k)2^k$, so the number of graphs with vertex set A is at most $2^{h(k)} \leq 2^{O(k^4 2^{2k})}$, where $h(k) = \binom{(k^2+k)2^k}{2}$. It is easy to check whether each S_i is a vertex cover of each graph with vertex set A in $O(n^2)$ time. Computing t and b takes $O(np)$ time, where $p \leq (k+1)2^k$. Using repeated squaring, we can compute 2^{tb} in $O(\log n)$ multiplications, each of which takes $n^2 \log n$ time since each number has at most $O(n^2)$ bits [91]. Hence the running time of this algorithm is $2^{O(k^4 2^{2k})} \cdot n^2 \log^2 n$. ■

By Theorem 9, this gives us an algorithm that computes the number of n-vertex graphs with a vertex cover of size at most k. The overall running time is $g(k)^{O(k \cdot g(k))} f(k) \cdot n^2 \log^2 n$, where $g(k) = 2^k$ and $f(k) = 2^{O(k^4 2^{2k})}$. This simplifies to a running time of $2^{O(k^4 2^{2k})} \cdot n^2 \log^2 n$.

4.3.4 The algorithm for $\mathcal{G}_C$

Next, we follow similar steps to design an algorithm for $\mathcal{G}_C$, the class of cluster graphs. Suppose G is a graph with vertex set $V(G) \subseteq [n]$ for some $n \in \mathbb{N}$, and suppose we have been given a collection of subsets $S_1, \ldots, S_p$ of $[n]$. We say an edge $(u, v) \in E(G)$ is

perennial if $\{u, v\} \cap S_i = \emptyset$ for some $i \in [p]$. The intuition is that a perennial edge is sometimes alive (not deleted) when we consider the graphs $G \setminus S_1, \ldots, G \setminus S_p$. A *non-edge* of G is a pair of distinct vertices $u, v \in V(G)$ such that $(u, v) \notin E(G)$. We similarly say a non-edge (u, v) of G is *perennial* if $\{u, v\} \cap S_i = \emptyset$ for some $i \in [p]$.

Lemma 77 *Let $n \in \mathbb{N}$, and let $S_1, \ldots, S_p$ be subsets of $[n]$, where $p \geq 1$. Let $A = \bigcup_{i=1}^{p} S_i$, $B = [n] \setminus A$, and $T = \bigcap_{i=1}^{p} S_i$. Suppose G is a graph with vertex set $[n]$. Then $G \setminus S_i$ is a cluster graph for all $i \in [p]$ if and only if G has the following properties:*

1. *$G[A] \setminus S_i$ is a cluster graph for all $i \in [p]$*
2. *$G[B]$ is a cluster graph*
3. *For every $v \in A \setminus T$, $N(v) \cap B$ is either the empty set or is a connected component of $G[B]$*
4. *For every $u, v \in A \setminus T$, if (u, v) is a perennial edge, then $N(u) \cap B = N(v) \cap B$*
5. *For every $u, v \in A \setminus T$, if (u, v) is a perennial non-edge, then $N(u) \cap B \neq N(v) \cap B$ unless $N(u) \cap B = N(v) \cap B = \emptyset$.*

Proof: First, suppose $G \setminus S_i$ is a cluster graph for all $i \in [p]$. Property (1) is clearly true, and we know $G[B]$ is a cluster graph since $G \setminus S_1$ is a cluster graph. For property (3), let $v \in A \setminus T$. The vertex v belongs to some cluster graph $G \setminus S_j$ since there is some set S_j that does not contain v. If v were to have a neighbor w_1 in one connected component of $G[B]$, along with a neighbor w_2 in a distinct connected component of $G[B]$, then w_1, v, w_2 would be an induced P_3 in the graph $G \setminus S_j$, which is impossible. This means $N(v) \cap B$ is contained in some component C of $G[B]$. Furthermore, if v were to have a neighbor $w_1 \in C$ and an non-neighbor $w_2 \in C$, then v, w_1, w_2 would be an induced

P_3 in the graph $G \setminus S_j$, which is impossible. Therefore, we either have $N(v) \cap B = \emptyset$ or $N(v) \cap B = C$.

For property (4), let $u, v \in A \setminus T$, and suppose (u, v) is a perennial edge. This means u and v both belong to some cluster graph $G \setminus S_j$. If $N(u) \cap B = N(v) \cap B = \emptyset$, then we are done, so suppose without loss of generality that $N(u) \cap B$ is nonempty. Let $w \in N(u) \cap B$, and suppose for a contradiction that $N(u) \cap B \neq N(v) \cap B$. By property (3), the path w, u, v is an induced P_3 in the graph $G \setminus S_j$, which is a contradiction. Hence property (4) holds.

For property (5), let $u, v \in A \setminus T$, and suppose (u, v) is a perennial non-edge. As before, we know u and v both belong to some cluster graph $G \setminus S_j$. Suppose for a contradiction that $N(u) \cap B = N(v) \cap B$ and $N(u) \cap B \neq \emptyset$. This means there is some vertex $w \in N(u) \cap B$. The path u, w, v is an induced P_3 in the graph $G \setminus S_j$, which is a contradiction. Therefore, property (5) holds.

For the other direction, suppose properties (1)-(5) hold for G. Let $i \in [p]$, and let $u, v, w \in V(G) \setminus S_i$. If $u, v, w \in A$, then these vertices (in any order) cannot form an induced P_3 since $G[A] \setminus S_i$ is a cluster graph. If $u, v, w \in B$, then these vertices cannot form an induced P_3 by property (2). Now suppose one of these vertices, say w, is in A and the other two are in B. If u and v are adjacent, then they belong to the same connected component of $G[B]$. In this case, by property (3), w is adjacent to either both of the vertices u and v or neither, so these do not form an induced P_3. If u and v are not adjacent, then they belong to distinct connected components of $G[B]$, so by property (3), these vertices cannot form an induced P_3.

The last remaining possibility is that one of the three vertices, say w, is in B and the other two are in A. Since $T \subseteq S_i$, we know $u, v \in A \setminus T$. If u and v are adjacent, then (u, v) is a perennial edge, so by property (4), these vertices cannot form an induced P_3. If u and v are not adjacent, then (u, v) is a perennial non-edge, so properties (3) and (5)

imply that these vertices cannot form an induced P_3. Hence $G \setminus S_i$ is a cluster graph. ■

Let A, B, T be as defined in Lemma 77.

Lemma 78 *Let $n \in \mathbb{N}$, $0 \leq k \leq n$, and let $S_1, \ldots, S_p$ be subsets of $[n]$ of size at most k, where $1 \leq p \leq (k+1)3^k$. There is an algorithm that computes the number of graphs G with vertex set $[n]$ such that $G \setminus S_i$ is a cluster graph for all $i \in [p]$. The running time of this algorithm is $2^{O(k^4 3^{2k})} n^2 \log n$.*

Proof: **Algorithm.** Let $\mathcal{A}$ be the set of graphs G_A with vertex set A such that $G_A \setminus S_i$ is a cluster graph for all $i \in [p]$. If $B = \emptyset$, then our algorithm returns $|\mathcal{A}|$. Otherwise, we proceed as follows.

Suppose $G_A \in \mathcal{A}$. Let G_P be the subgraph of G_A with vertex set $A \setminus T$ that contains all of the perennial edges of G_A, and no other edges. Let $\mathcal{C}$ be the set of all connected components of G_P. Let $\Gamma_1(G_A)$ be the family of all subsets $E \subseteq \mathcal{C}$ with the following property: for every connected component C of G_P, if there exists a perennial non-edge of G_A with both endpoints in C, then $C \in E$.

Suppose $E \in \Gamma_1(G_A)$, and suppose $\mathcal{P}$ is a true partition of $\mathcal{C} \setminus E$. We say $\mathcal{P}$ *satisfies property (5)* if the following holds: for every perennial non-edge (u, v) in G_A, if u and v belong to different components of G_P, say C_u and C_v, then either C_u and C_v belong to different parts in $\mathcal{P}$, or at least one of C_u, C_v belongs to E. Let $\Gamma_2(G_A, E)$ be the family of all true partitions of $\mathcal{C} \setminus E$ that satisfy property (5). Let $b = |B|$ and $t = |T|$. Our algorithm returns

$$\sum_{G_A \in \mathcal{A}} \sum_{b' \in [b]} \sum_{E \in \Gamma_1(G_A)} \sum_{\mathcal{P} \in \Gamma_2(G_A, E)} (b')_{|\mathcal{P}|} S(b, b') 2^{tb}.$$

Here $|\mathcal{P}|$ is the number of parts in the partition $\mathcal{P}$, $(b')_{|\mathcal{P}|} \coloneqq b'(b'-1)\cdots(b'-|\mathcal{P}|+1)$ denotes the falling factorial, and $S(b, b')$ stands for the number of ways of partitioning a set of size b into b' nonempty parts (a Stirling number of the second kind).

Correctness. We show that this algorithm computes the number of graphs with vertex set $[n]$ that satisfy properties (1)-(5) from Lemma 77. The graph G_A corresponds to $G[A]$, and b' corresponds to the number of connected components in $G[B]$. By property (4) of Lemma 77, vertices in the same component of G_P must have the same neighborhood in B. The set E corresponds to the set of components of G_P whose neighborhood in B is the empty set, and the partition $\mathcal{P}$ groups together components of G_P that have the same (nonempty) neighborhood in B. It is correct to only consider subsets $E \subseteq \mathcal{C}$ that belong to $\Gamma_1(G_A)$ because of properties (4) and (5) of Lemma 77. Similarly, we only consider partitions $\mathcal{P}$ that belong to $\Gamma_2(G_A, E)$ because of property (5) of that lemma.

The term $(b')_{|\mathcal{P}|}$ counts the number of injective functions from the set of parts in $\mathcal{P}$ to the set of connected components of $G[B]$, which corresponds to choosing the neighborhood of the components of G_P that make up each part. The Stirling number $S(b, b')$ is equal to the number of ways of partitioning B into b' nonempty cliques. Finally, there are 2^{tb} possibilities for the edges between $A \setminus T$ and B.

Running time. At the start of this algorithm, we first compute all of the necessary Stirling numbers, which can be done in $O(n^2)$ time using the standard recurrences. We have $|A| \leq (k^2 + k)3^k$, so the number of graphs with vertex set A is at most $2^{h(k)} \leq 2^{O(k^4 3^{2k})}$, where $h(k) = \binom{(k^2+k)3^k}{2}$. It is easy to check whether $G_A \setminus S_i$ is a cluster graph for each graph G_A with vertex set A in $O(n^2)$ time. For the second summation, there are $b \leq n$ possible values of b'. We also know $|\mathcal{C}| \leq |A|$, so there are at most $2^{(k^2+k)3^k}$ possible values of E, and at most $3^{O(k^3 3^k)}$ true partitions of $\mathcal{C} \setminus E$. Each multiplication in the summation over $G_A, b', E, \mathcal{P}$ takes $O(n^2 \log n)$ time since each number has at most $O(n^2)$ bits [91]. Computing t and b takes $O(np)$ time, where $p \leq (k+1)3^k$. Using repeated squaring, we can compute 2^{tb} in $O(\log n)$ multiplications, each of which takes $O(n^2 \log n)$ time. Hence the running time of this algorithm is $2^{O(k^4 3^{2k})} \cdot n^3 \log n$. ■

By Theorem 9, this gives us an algorithm that computes the number of yes-instances of Cluster Vertex Deletion. The overall running time is $2^{O(k^4 3^{2k})} \cdot n^3 \log n$.

4.3.5 The algorithm for $\mathcal{G}_A$

Next, we design an algorithm for $\mathcal{G}_A$, the class of acyclic tournaments. For directed graphs, perennial edges are defined in the same way as for undirected graphs. Suppose G is a directed graph with vertex set $V(G) \subseteq [n]$ for some $n \in \mathbb{N}$, and suppose we have been given a collection of subsets $S_1, \ldots, S_p$ of $[n]$. We say an edge $(u, v) \in E(G)$ is *perennial* if $\{u, v\} \cap S_i = \emptyset$ for some $i \in [p]$.

Every acyclic tournament has a unique topological ordering (see Section 4.2). If we wish to add an additional vertex to an acyclic tournament G while still maintaining the acyclic property, this vertex can either go between two consecutive vertices in the topological ordering, or it can go before or after all other vertices of G. Let $\leq$ denote the topological ordering of G, and suppose the vertices are ordered as follows: $v_1 \leq v_2 \leq \ldots \leq v_n$, where $n = |V(G)|$. A *gap* λ in the topological ordering of G is either v_1, v_n, or a pair of consecutive vertices (v_i, v_{i+1}), where $1 \leq i < n$. If $\lambda = v_1$, we say the vertices $v_1, \ldots, v_n$ *come after* λ, and if $\lambda = v_n$, we say the vertices $v_1, \ldots, v_n$ *come before* λ. If $\lambda = (v_i, v_{i+1})$, we say the vertices $v_1, \ldots, v_i$ *come before* λ and the vertices $v_{i+1}, \ldots, v_n$ *come after* λ. When we say a *gap* in G, this is short for a gap in the topological ordering of G.

Suppose G is a tournament with vertex set $[n]$ for some $n \in \mathbb{N}$, and suppose we have been given a collection of subsets $S_1, \ldots, S_p$ of $[n]$, where $p \geq 1$. Let $A = \bigcup_{i=1}^{p} S_i$, $B = [n] \setminus A$, and $T = \bigcap_{i=1}^{p} S_i$. For a vertex $v \in A \setminus T$, we say its neighborhood is *organized* if there exists a gap λ in $G[B]$ such that $(u, v) \in E(G)$ for every $u \in B$ that comes before λ and $(v, u) \in E(G)$ for every $u \in B$ that comes after λ. Note that if such

a gap exists, it is unique. We refer to λ as the gap *corresponding to* v.

Let $\leq_B$ denote the topological ordering of $G[B]$, and suppose the vertices of B are ordered as follows: $u_1 \leq_B u_2 \leq_B \ldots \leq_B u_b$, where $b = |B|$. (In fact, these vertices are distinct, so we also could have written $u_1 <_B u_2$, etc., but for simplicity, we just define the relation $\leq_B$). Let $\mathcal{B}$ be the set of size $b+1$ of all gaps in the topological ordering of $G[B]$. The ordering of $G[B]$ naturally leads to a total order on $\mathcal{B}$, denoted by $\preceq$. We have $u_1 \preceq (u_1, u_2) \preceq (u_2, u_3) \preceq \cdots \preceq (u_{b-1}, u_b) \preceq u_b$, where each of these elements is a gap in $G[B]$.

Now suppose the neighborhood of v is organized for every $v \in A \setminus T$. For such a graph G, let $h : A \setminus T \to \mathcal{B}$ be the function defined by $h(v) = \lambda_v$ for all $v \in A \setminus T$, where λ_v is the gap corresponding to v. We call h the *gap function* of G. We say this function is *orderly* if $h(v) \preceq h(w)$ for every perennial edge (v, w) in $G[A]$.

Lemma 79 *Let $n \in \mathbb{N}$, and let $S_1, \ldots, S_p$ be subsets of $[n]$, where $p \geq 1$. Let $A = \bigcup_{i=1}^{p} S_i$, $B = [n] \setminus A$, and $T = \bigcap_{i=1}^{p} S_i$. Suppose G is a tournament with vertex set $[n]$. Then $G \setminus S_i$ is acyclic for all $i \in [p]$ if and only if G has the following properties:*

1. *$G[A] \setminus S_i$ is acyclic for all $i \in [p]$*
2. *$G[B]$ is acyclic*
3. *For every $v \in A \setminus T$, the neighborhood of v is organized*
4. *The gap function h of G is orderly.*

Proof: First suppose $G \setminus S_i$ is acyclic for all $i \in [p]$. Property (1) is clearly true, and we know $G[B]$ is acyclic since $G \setminus S_1$ is acyclic. For property (3), let $v \in A \setminus T$, which means there is some $j \in [r]$ such that $v \notin S_j$. Let $\leq_B$ denote the topological ordering of $G[B]$, and suppose the elements of B are ordered as follows: $u_1 \leq_B u_2 \leq_B \ldots \leq_B u_b$,

where $b = |B|$. If we have $(v, u) \in E(G)$ for every $u \in B$, then let $\lambda = u_1$. Otherwise, let u' be the last vertex in the ordering $u_1, \ldots, u_b$ such that $(u', v) \in E(G)$. We claim that $(u, v) \in E(G)$ for all $u \in B$ such that $u \leq_B u'$. Indeed, if there were a vertex $u \neq u'$ in B such that $u \leq_B u'$ and $(v, u) \in E(G)$, then v, u, u' would be a cycle in $G \setminus S_j$. Therefore, we have $(u, v) \in E(G)$ for all $u \in B$ such that $u \leq_B u'$, and we have $(v, u) \in E(G)$ for all $u \neq u'$ in B such that $u' \leq_B u$. If $u' = u_b$, then let $\lambda = u'$, and otherwise, let $\lambda = (u', w)$, where w is the vertex immediately after u' in the ordering $u_1, \ldots, u_b$. This shows that the neighborhood of v is organized, since λ is the gap corresponding to v.

For property (4), suppose for a contradiction that h is not orderly. This means there is some perennial edge (v, w) in $G[A]$ such that $h(v) \not\preceq h(w)$, so we have $h(w) \preceq h(v)$ and $h(v) \neq h(w)$. Hence there is some $u \in B$ that comes after $h(w)$ and comes before $h(v)$. We know v and w belong to some acyclic graph $G \setminus S_j$. Therefore, u, v, w is a cycle in $G \setminus S_j$, which is a contradiction.

For the other direction, suppose properties (1)-(4) hold for G. Let $i \in [p]$. We show that $G \setminus S_i$ has a topological ordering. The graph $G[A] \setminus S_i$ has a topological ordering by property (1) (denoted by $\leq_A$), and the graph $G[B]$ has a topological ordering by property (2) (denoted by $\leq_B$). Based on these, we can define the relation $\leq$ on $V(G) \setminus S_i$ in the following way. Let $v, w \in V(G) \setminus S_i$. If $v, w \in A$, then $v \leq w$ if $v \leq_A w$, and $w \leq v$ if $w \leq_A v$. If $v, w \in B$, then $v \leq w$ if $v \leq_B w$, and $w \leq v$ if $w \leq_B v$. If $v \in A$ and $w \in B$, then $v \leq w$ if w comes after $h(v)$, and $w \leq v$ if w comes before $h(v)$.

To see that the relation $\leq$ is a topological ordering, we first check that it is a total order. Clearly, this relation is reflexive. We cannot have both $v \leq w$ and $w \leq v$ in the case when $v \in A$ and $w \in B$, so it is also antisymmetric. To see that it is transitive, suppose $u \leq v$ and $v \leq w$ for some $u, v, w \in V(G) \setminus S_i$. If $u, v, w \in A$ or $u, v, w \in B$, then clearly $u \leq w$. If $u \in A$ and $v, w \in B$, then we have $u \leq w$ since w comes after $h(u)$. If $v \in A$ and $u, w \in B$, or if $w \in A$ and $u, v \in B$, then we can similarly see that $u \leq w$. If

$u \in B$ and $v, w \in A$, then we have $u \leq w$ since $h(v) \preceq h(w)$ by property (4). If $v \in B$ and $u, w \in A$, or if $w \in B$ and $u, v \in A$, then we can similarly see that $u \leq w$. Thus the relation $\leq$ is transitive.

Now we just need to check that $v \leq w$ for every edge (v, w) in the graph $G \setminus S_i$. Suppose (v, w) is an edge in $G \setminus S_i$. If $v, w \in A$, then clearly $v \leq w$ since $v \leq_A w$, and similarly, we are done if $v, w \in B$. If $v \in A$ and $w \in B$, then w comes after $h(v)$ by property (3), so we have $v \leq w$. Lastly, if $v \in B$ and $w \in A$, then v comes before $h(w)$ by property (3), so we again have $v \leq w$. Therefore, the relation $\leq$ is a topological ordering, and thus $G \setminus S_i$ is acyclic. ■

Let A, B, T be as defined in Lemma 79.

Lemma 80 *Let $n \in \mathbb{N}$, $0 \leq k \leq n$, and let $S_1, \ldots, S_p$ be subsets of $[n]$ of size at most k, where $1 \leq p \leq (k+1)3^k$. There is an algorithm that computes the number of tournaments G with vertex set $[n]$ such that $G \setminus S_i$ is acyclic for all $i \in [p]$. The running time of this algorithm is $2^{O(k^4 3^{2k})} n^2 \log n$.*

Proof: **Algorithm.** Let $\mathcal{A}$ be the set of tournaments G_A with vertex set A such that $G_A \setminus S_i$ is acyclic for all $i \in [p]$. Suppose $G_A \in \mathcal{A}$. For a true partition $\mathcal{P}$ of $A \setminus T$, we say $\mathcal{P}$ is an *ordered partition* if we have a total order on $\mathcal{P}$ (i.e., an ordering of the parts, denoted by $\leq$). For each $v \in A \setminus T$, let P_v denote the part in $\mathcal{P}$ that contains v. We say an ordered partition $\mathcal{P}$ of $A \setminus T$ is *orderly* if $P_v \leq P_w$ for every perennial edge (v, w) in G_A. Let $\Gamma(G_A)$ be the family of all ordered partitions $\mathcal{P}$ of $A \setminus T$ such that $\mathcal{P}$ is orderly. Let $b = |B|$ and $t = |T|$. Our algorithm returns

$$b! \cdot \sum_{G_A \in \mathcal{A}} \sum_{\mathcal{P} \in \Gamma(G_A)} \binom{b+1}{|\mathcal{P}|} 2^{tb},$$

where $|\mathcal{P}|$ is the number of parts in $\mathcal{P}$.

Correctness. We show that this algorithm computes the number of tournaments with vertex set $[n]$ that satisfy properties (1)-(4) from Lemma 79. There are $b!$ possibilities for $G[B]$ since $G[B]$ is acyclic. Therefore, we simply need to prove the following: the number of tournaments G with vertex set $[n]$ such that $G[B] = G_B$ and G satisfies properties (1)-(4) is equal to

$$\sum_{G_A \in \mathcal{A}} \sum_{\mathcal{P} \in \Gamma(G_A)} \binom{b+1}{|\mathcal{P}|} 2^{tb}$$

for each fixed acyclic tournament G_B with vertex set B. Let $G_A \in \mathcal{A}$, and let G_B be an acyclic tournament with vertex set B. Let $\mathcal{G}$ be the set of tournaments G with vertex set $[n]$ such that $G[A] = G_A$, $G[B] = G_B$, and G satisfies properties (1)-(4). Since $\mathcal{A}$ is the set of possibilities for $G[A]$, it is sufficient to show that $|\mathcal{G}| = \sum_{\mathcal{P} \in \Gamma(G_A)} \binom{b+1}{|\mathcal{P}|} 2^{tb}$.

From the gap function h of a tournament $G \in \mathcal{G}$, we can obtain an ordered partition of $A \setminus T$. Let $\mathcal{P}$ be the (true) partition of $A \setminus T$ such that $v, w \in A \setminus T$ belong to the same part if and only if $h(v) = h(w)$. Now let $\mathcal{P}$ be ordered as follows: for $v, w \in A \setminus T$, we have $P_v \leq P_w$ if and only if $h(v) \preceq h(w)$. When $\mathcal{P}$ is defined in this way, we say $\mathcal{P}$ is the ordered partition *specified by* h.

We only consider ordered partitions $\mathcal{P}$ that belong to $\Gamma(G_A)$ to ensure that the graphs we are counting satisfy property (4). Let $\mathcal{P} \in \Gamma(G_A)$, and let $\mathcal{B}$ be the set of all gaps in the topological ordering of $G[B]$. The number of functions $h : A \setminus T \to \mathcal{B}$ such that $\mathcal{P}$ is the ordered partition specified by h is equal to $\binom{b+1}{|\mathcal{P}|}$, since $|\mathcal{B}| = b + 1$. In other words, this is number of possible gap functions h for a graph $G \in \mathcal{G}$ such that $\mathcal{P}$ is the ordered partition specified by h. Given such a function h, the number of graphs $G \in \mathcal{G}$ such that h is the gap function of G is equal to 2^{tb}, since h specifies the directions of all of the edges between $A \setminus T$ and B, and there are 2^{tb} possibilities for the edges between T and B. Therefore, the number of graphs $G \in \mathcal{G}$ such that $\mathcal{P}$ is the ordered partition specified by the gap function of G is equal to $\binom{b+1}{|\mathcal{P}|} 2^{tb}$. This implies $|\mathcal{G}| = \sum_{\mathcal{P} \in \Gamma(G_A)} \binom{b+1}{|\mathcal{P}|} 2^{tb}$, as

desired.

Running time. At the start of this algorithm, we first compute all of the necessary binomial coefficients, which can be done in $O(n^2)$ time. We have $|A| \leq (k^2+k)3^k$, so the number of tournaments with vertex set A is at most $2^{h(k)} \leq 2^{O(k^4 3^{2k})}$, where $h(k) = \binom{(k^2+k)3^k}{2}$. It is easy to check whether $G_A \backslash S_i$ is acyclic for each tournament G_A with vertex set A in $O(n^2)$ time. For the second summation, there are at most $|A|^{|A|}|A|! = 3^{O(k^3 3^k)}$ ordered true partitions of $A \setminus T$. Each multiplication in the summation over $G_A, \mathcal{P}$ takes $O(n^2 \log n)$ time since each number has at most $O(n^2)$ bits [91]. Computing t and b takes $O(np)$ time, where $p \leq (k+1)3^k$. Using repeated squaring, we can compute 2^{tb} in $O(\log n)$ multiplications, each of which takes $O(n^2 \log n)$ time. Hence the running time of this algorithm is $2^{O(k^4 3^{2k})} \cdot n^2 \log^2 n$. ■

By Theorem 10, this gives us an algorithm that computes the number of yes-instances of Feedback Vertex Set in Tournaments. The overall running time is $2^{O(k^4 3^{2k})} \cdot n^2 \log^2 n$.

4.3.6 The algorithm for $\mathcal{G}_S$

Split graph properties

If C is the always-clique set, I is the always-independent set, and Q is the questioning set of a split graph G, then (C, I, Q) is called the *split decomposition* of G. More generally, a *decomposition* of a graph G is a triple (A_1, A_2, A_3) such that $\{A_1, A_2, A_3\}$ is a partition of $V(G)$. The following lemma is a more compact version of the results proved in Chapter 2 (see Observation 6 and Lemmas 25 and 26).

Lemma 81 *The split decomposition (C, I, Q) of a split graph G has the following properties:*

1. *C is a clique and I is an independent set*

2. *Q is either a clique or an independent set*

3. *Every vertex in Q is adjacent to every vertex in C and is non-adjacent to every vertex in I*

4. *If Q is a clique, then every vertex in C has a neighbor in I*

5. *If Q is an independent set, then every vertex in I has a non-neighbor in C.*

The converse is also true, as we can see in the following lemma, which is a compact version of Lemmas 27 and 30 from Chapter 2.

Lemma 82 *Suppose G is a graph, and suppose (C, I, Q) is a decomposition of G that satisfies properties (1)-(5) from Lemma 81. Then G is a split graph, and furthermore, (C, I, Q) is the split decomposition of G.*

We will also need two more properties of split decompositions.

Lemma 83 *Let G be a split graph. If $u, v \in V(G)$ are true or false twins, then they belong to the same part in the split decomposition (C, I, Q) of G. In other words, we have $u, v \in C$, $u, v \in I$, or $u, v \in Q$.*

Proof: Suppose $u \in C'$ for some split partition (C', I') of G. We claim that in this case, there must also be a split partition $(C^\star, I^\star)$ of G such that $v \in C^\star$. If $v \in C'$, then we are done. Otherwise, we have $u \in C'$ and $v \in I'$, which means $(C' \backslash \{u\} \cup \{v\}, I' \backslash \{v\} \cup \{u\})$ is a split partition of G that places v in the clique. Similarly, we can see that if u belongs to the independent set of some split partition of G, then the same is true for v. Therefore, we have $u, v \in C$, $u, v \in I$, or $u, v \in Q$. ■

Lemma 84 *Suppose G is a split graph with an induced subgraph $\hat{G}$. Let (C, I, Q) be the split decomposition of G, and let $(\hat{C}, \hat{I}, \hat{Q})$ be the split decomposition of $\hat{G}$. We have $\hat{C} \subseteq C$ and $\hat{I} \subseteq I$.*

Proof: To see that $\hat{C} \subseteq C$, suppose $v \in V(G) \setminus C$. This means there is some split partition (C', I') of G such that $v \in I'$. Let $C^\star = C' \cap V(\hat{G})$ and $I^\star = I' \cap V(\hat{G})$. If $v \in V(\hat{G})$, then the pair $(C^\star, I^\star)$ is a split partition of $\hat{G}$ with the property that $v \in I^\star$, so $v \notin \hat{C}$. Otherwise, if $v \notin V(\hat{G})$, then clearly $v \notin \hat{C}$. Hence $\hat{C} \subseteq C$. A similar argument (reversing the roles of the always-clique set and the always-independent set) shows that $\hat{I} \subseteq I$. ■

Guess lists

The goal of this section (Section 4.3.6) is to prove the following lemma.

Lemma 85 *Let $n \in \mathbb{N}$, $0 \leq k \leq n$, and let $S_1, \ldots, S_p$ be subsets of $[n]$ of size at most k, where $1 \leq p \leq (k+1)5^k$. There is an algorithm that computes the number of graphs G with vertex set $[n]$ such that $G \setminus S_i$ is a split graph for all $i \in [p]$. The running time of this algorithm is $4^{2^{O(k^2 5^k)}} \cdot n^7 \log n$.*

In this algorithm, we will guess certain information about these graphs in order to group them into smaller families of graphs that are easier to count. Suppose we have been given $n \in \mathbb{N}$, as well as a collection of subsets $S_1, \ldots, S_p$ of $[n]$, where $p \geq 1$. Let $A = \bigcup_{i=1}^{p} S_i$, and let $B = [n] \setminus A$.

Suppose G is a graph with vertex set $[n]$ such that $G \setminus S_i$ is a split graph for all $i \in [p]$. Let (C_B, I_B, Q_B) be the split decomposition of $G[B]$, and let (C_i, I_i, Q_i) be the split decomposition of $G \setminus S_i$ for each $i \in [p]$. For such a graph G, we define the sets $C_B^{\text{yes}}, C_B^{\text{no}}, I_B^{\text{yes}}, I_B^{\text{no}}$ in the following way. Let C_B^{yes} be the set of vertices in C_B that have

a neighbor in I_B, and let C_B^{no} be the set of vertices in C_B that do not have a neighbor in I_B. Let I_B^{yes} be the set of vertices in I_B that have a non-neighbor in C_B, and let I_B^{no} be the set of vertices in I_B that do not have a non-neighbor in C_B. In Definition 10, the identifiers `C`, `I`, and `Q` stand for the parts in the split decomposition of $G \setminus S_i$, `D` stands for "deleted," and `E` stands for "empty set."

Definition 10 *A **guess list** is a tuple*

$$(G_A, c_{yes}, c_{no}, i_{yes}, i_{no}, q, z, f_C, f_I, f_Q, g_A^{(1)}, \ldots, g_A^{(p)}, g_Q^{(1)}, \ldots, g_Q^{(p)}),$$

where G_A is a graph with vertex set A, the numbers $c_{yes}, c_{no}, i_{yes}, i_{no}, q$ are nonnegative integers such that $c_{yes}+c_{no}+i_{yes}+i_{no}+q = |B|$, z is a boolean, and the remaining entries are functions of the following form:

- $f_C, f_I, f_Q \colon 2^A \to \{0, 1, 2\}$
- $g_A^{(i)} \colon A \to \{\texttt{D}, \texttt{C}, \texttt{I}, \texttt{Q}\}$ *for all* $i \in [p]$
- $g_Q^{(i)} \colon 2^A \to \{\texttt{E}, \texttt{C}, \texttt{I}, \texttt{Q}\}$ *for all* $i \in [p]$.

The graph G_A specifies the edges of $G[A]$, the numbers $c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q$ are the sizes of C_B^{yes}, C_B^{no}, I_B^{yes}, I_B^{no}, Q_B, respectively, and z indicates whether or not Q_B is a clique. For $A' \subseteq A$, the values $f_C(A')$, $f_I(A')$, and $f_Q(A')$ specify the number of vertices in certain sets (namely C_B^{no}, I_B^{no}, and Q_B) whose neighborhood in A is equal to A'. Each of these numbers is either 0, 1, or at least 2. When we say $f_C(A')$ *matches with* a number ℓ, this means $\ell = 0$ if $f_C(A') = 0$, $\ell = 1$ if $f_C(A') = 1$, and $\ell \geq 2$ if $f_C(A') = 2$. We also define *matches with* in the same way for $f_I(A')$ and $f_Q(A')$.

For each $i \in [p]$, the functions $g_A^{(i)}$ and $g_Q^{(i)}$ specify the roles certain vertices play in the split decomposition of $G \setminus S_i$. Let $v \in A$, $i \in [p]$. When we say $g_A^{(i)}(v)$ *matches with* the part in the split decomposition of $G \setminus S_i$ that contains v, this means the following:

if $g_A^{(i)}(v) = \texttt{D}$, then $v \in S_i$; if $g_A^{(i)}(v) = \texttt{C}$, then $v \in C_i$; if $g_A^{(i)}(v) = \texttt{I}$, then $v \in I_i$; and if $g_A^{(i)}(v) = \texttt{Q}$, then $v \in Q_i$.

For $A' \subseteq A$, the *neighborhood class* of A' is the set $\mathcal{N}(A') \coloneqq \{v \in Q_B : N(v) \cap A = A'\}$. Let $A' \subseteq A$, $i \in [p]$. When we say $g_Q^{(i)}(A')$ *matches with* the part in the split decomposition of $G \setminus S_i$ that contains $\mathcal{N}(A')$, this means the following: if $g_Q^{(i)}(A') = \texttt{E}$, then $\mathcal{N}(A') = \emptyset$; if $g_Q^{(i)}(A') = \texttt{C}$, then $\mathcal{N}(A') \subseteq C_i$; if $g_Q^{(i)}(A') = \texttt{I}$, then $\mathcal{N}(A') \subseteq I_i$; and if $g_Q^{(i)}(A') = \texttt{Q}$, then $\mathcal{N}(A') \subseteq Q_i$.

Definition 11 *Suppose G is a graph with vertex set $[n]$. For a guess list*

$$L = (G_A, c_{yes}, c_{no}, i_{yes}, i_{no}, q, z, f_C, f_I, f_Q, q, g_A^{(1)}, \ldots, g_A^{(p)}, g_Q^{(1)}, \ldots, g_Q^{(p)}),$$

we say G ***conforms to*** *L if $G \setminus S_i$ is a split graph for all $i \in [p]$ and the following holds:*

- $G[A] = G_A$
- $|C_B^{yes}| = c_{yes}$, $|C_B^{no}| = c_{no}$, $|I_B^{yes}| = i_{yes}$, $|I_B^{no}| = i_{no}$, $|Q_B| = q$
- *Q_B is a clique if and only if $z = 1$*
- *For all $A' \subseteq A$, $f_C(A')$ matches with $|\{v \in C_B^{no} : N(v) \cap A = A'\}|$*
- *For all $A' \subseteq A$, $f_I(A')$ matches with $|\{v \in I_B^{no} : N(v) \cap A = A'\}|$*
- *For all $A' \subseteq A$, $f_Q(A')$ matches with $|\mathcal{N}(A')|$*
- *For all $i \in [p]$, $v \in A$, $g_A^{(i)}(v)$ matches with the part in the split decomposition of $G \setminus S_i$ that contains v*
- *For all $i \in [p]$, $A' \subseteq A$, $g_Q^{(i)}(A')$ matches with the part in the split decomposition of $G \setminus S_i$ that contains $\mathcal{N}(A')$.*

For each guess list L, our algorithm will count the number of graphs that conform to L. Suppose $L = (G_A, c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q, z, f_C, f_I, f_Q, g_A^{(1)}, \ldots, g_A^{(p)}, g_Q^{(1)}, \ldots, g_Q^{(p)})$ is a guess list. For this guess list to make sense, we need its entries to be consistent with one another. In particular, q and z are both related to whether Q_B is a clique, the functions f_Q and $g_Q^{(i)}$ both have the ability to specify whether each neighborhood class $\mathcal{N}(A')$ is empty or not, and the functions $g_Q^{(i)}$ need to agree with which vertices are actually deleted. We say L is *consistent* if the following conditions hold:

- If $q \leq 1$, then $z = 1$
- For all $A' \subseteq A$, $f_Q(A') = 0$ if and only if we have $g_Q^{(i)}(A') = \texttt{E}$ for all $i \in [p]$
- For all $v \in A$, $i \in [p]$, we have $g_A^{(i)}(v) = \texttt{D}$ if and only if $v \in S_i$.

For every graph G counted by our algorithm, the split graph $G \setminus S_i$ needs to satisfy properties (1)-(5) of Lemma 81 for each $i \in [p]$. Recall that the split decomposition of $G \setminus S_i$ is denoted by (C_i, I_i, Q_i) for each $i \in [p]$. To begin with, we need a way of checking whether Q_i is a clique or independent set (or both). Let $i \in [p]$. We say L is *of clique type for* i if the following holds:

- $\{v \in A : g_A^{(i)}(v) = \texttt{Q}\}$ is a clique in G_A
- If $\sum_{A' : g_Q^{(i)}(A') = \texttt{Q}} f_Q(A') \geq 2$, then $z = 1$
- For all $A' \subseteq A$, if $g_Q^{(i)}(A') = \texttt{Q}$, then $\{v \in A : g_A^{(i)}(v) = \texttt{Q}\} \subseteq A'$.

We say L is *of independent type for* i if the following holds:

- $\{v \in A : g_A^{(i)}(v) = \texttt{Q}\}$ is an independent set in G_A
- If $\sum_{A' : g_Q^{(i)}(A') = \texttt{Q}} f_Q(A') \geq 2$, then $z = 0$

- For all $A' \subseteq A$, if $g_Q^{(i)}(A') = \mathtt{Q}$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{Q}\} \subseteq A \setminus A'$.

The intuition is that L is of clique type for i if Q_i is a clique, and L is of independent type for i if Q_i is an independent set. In the penultimate bullet point, checking whether $z = 0$ is equivalent to checking whether Q_B is an independent set, since here we have $|Q_B| \geq 2$.

In the following table, we assume $i \in [p]$ is fixed. We list some necessary conditions for $G \setminus S_i$ to be a split graph, grouped according to the five properties from Lemma 81. The phrases in italics are intuition, and the bullet points are requirements for L. We say the guess list L is *locally valid* if it satisfies all of the bullet points in the following table for every $i \in [p]$.

1. C_i is a clique and I_i is an independent set
$C_i \cap A$ is a clique and $I_i \cap A$ is an independent set • $\{v \in A : g_A^{(i)}(v) = \mathtt{C}\}$ is a clique in G_A • $\{v \in A : g_A^{(i)}(v) = \mathtt{I}\}$ is an independent set in G_A
$C_i \cap Q_B$ is a clique is a clique and $I_i \cap Q_B$ is an independent set • If $\sum_{A' : g_Q^{(i)}(A') = \mathtt{C}} f_Q(A') \geq 2$, then $z = 1$ • If $\sum_{A' : g_Q^{(i)}(A') = \mathtt{I}} f_Q(A') \geq 2$, then $z = 0$
All edges between $C_i \cap A$ and C_B^{no} are present, and we only have non-edges between $I_i \cap A$ and I_B^{no} • For all $A' \subseteq A$, if $f_C(A') > 0$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{C}\} \subseteq A'$ • For all $A' \subseteq A$, if $f_I(A') > 0$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{I}\} \subseteq A \setminus A'$

All edges between $C_i \cap A$ and $C_i \cap Q_B$ are present, and we only have non-edges between $I_i \cap A$ and $I_i \cap Q_B$

- For all $A' \subseteq A$, if $g_Q^{(i)}(A') = \mathtt{C}$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{C}\} \subseteq A'$
- For all $A' \subseteq A$, if $g_Q^{(i)}(A') = \mathtt{I}$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{I}\} \subseteq A \setminus A'$

2. Q_i is either a clique or an independent set

- L is of clique type or independent type for i

3. Every vertex in Q_i is adjacent to every vertex in C_i and is non-adjacent to every vertex in I_i

Every vertex in $Q_i \cap A$ is adjacent to every vertex in $C_i \cap A$ and is non-adjacent to every vertex in $I_i \cap A$

- In G_A, every vertex in $\{v \in A : g_A^{(i)}(v) = \mathtt{Q}\}$ is adjacent to every vertex in $\{v \in A : g_A^{(i)}(v) = \mathtt{C}\}$ and is non-adjacent to every vertex in $\{v \in A : g_A^{(i)}(v) = \mathtt{I}\}$

The same property holds for edges between $Q_i \cap A$ and $C_B^{no} \cup I_B^{no}$

- For all $A' \subseteq A$, if $f_C(A') > 0$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{Q}\} \subseteq A'$
- For all $A' \subseteq A$, if $f_I(A') > 0$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{Q}\} \subseteq A \setminus A'$

The same property holds for edges between A and Q_B

- For all $A' \subseteq A$, if $g_Q^{(i)}(A') = \mathtt{Q}$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{C}\} \subseteq A'$
- For all $A' \subseteq A$, if $g_Q^{(i)}(A') = \mathtt{C}$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{Q}\} \subseteq A'$
- For all $A' \subseteq A$, if $g_Q^{(i)}(A') = \mathtt{Q}$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{I}\} \subseteq A \setminus A'$
- For all $A' \subseteq A$, if $g_Q^{(i)}(A') = \mathtt{I}$, then $\{v \in A : g_A^{(i)}(v) = \mathtt{Q}\} \subseteq A \setminus A'$

If $Q_i \cap Q_B$ and $C_i \cap Q_B$ (resp. $I_i \cap Q_B$) are nonempty, then Q_B is a clique (resp. an independent set)

- If $g_Q^{(i)}(A') = \mathtt{Q}$ and $g_Q^{(i)}(\widetilde{A}) = \mathtt{C}$ for some $A', \widetilde{A} \subseteq A$, then $z = 1$
- If $g_Q^{(i)}(A') = \mathtt{Q}$ and $g_Q^{(i)}(\widetilde{A}) = \mathtt{I}$ for some $A', \widetilde{A} \subseteq A$, then $z = 0$

4. If Q_i is a clique, then every vertex in C_i has a neighbor in I_i

If Q_i is a clique, then every vertex in C_B^{no} has a neighbor in I_i, which can come from A or Q_B

- If L is of clique type for i, then at least one of the following holds:
  - $A' \cap \{v \in A : g_A^{(i)}(v) = \mathtt{I}\} \neq \emptyset$ for every $A' \subseteq A$ such that $f_C(A') > 0$
  - $g_Q^{(i)}(A') = \mathtt{I}$ for some $A' \subseteq A$

If Q_i is a clique, then every vertex in $C_i \cap Q_B$ has a neighbor in I_i, which can come from A or Q_B

- If L is of clique type for i, then for every $A' \subseteq A$ such that $g_Q^{(i)}(A') = \mathtt{C}$, at least one of the following holds:
  - $g_A^{(i)}(v) = \mathtt{I}$ for some $v \in A'$
  - $z = 1$ and $g_Q^{(i)}(\widetilde{A}) = \mathtt{I}$ for some $\widetilde{A} \subseteq A$

5. If Q_i is an independent set, then every vertex in I_i has a non-neighbor in C_i.

If Q_i is an independent set, then every vertex in I_B^{no} has a non-neighbor in C_i, which can come from A or Q_B

- If L is of independent type for i, then at least one of the following holds:
  - $(A \setminus A') \cap \{v \in A : g_A^{(i)}(v) = \mathtt{C}\} \neq \emptyset$ for every $A' \subseteq A$ such that $f_I(A') > 0$
  - $g_Q^{(i)}(A') = \mathtt{C}$ for some $A' \subseteq A$

If Q_i is an independent set, then every vertex in $I_i \cap Q_B$ has a non-neighbor in C_i, which can come from A or Q_B

- If L is of independent type for i, then for every $A' \subseteq A$ such that $g_Q^{(i)}(A') = \mathtt{I}$, at least one of the following holds:
  - $g_A^{(i)}(v) = \mathtt{C}$ for some $v \in A \setminus A'$
  - $z = 0$ and $g_Q^{(i)}(\widetilde{A}) = \mathtt{C}$ for some $\widetilde{A} \subseteq A$

If Q_i is a clique, then we also need to make sure every vertex in $C_i \cap A$ has a neighbor in I_i. Similarly, if Q_i is an independent set, then we need to make sure every vertex in $I_i \cap A$ has a non-neighbor in C_i. The edges between $C_i \cap A$ and I_i (resp. $I_i \cap A$ and C_i) are not completely specified by L since we do not guess information about the neighborhoods of vertices in I_B^{yes} (resp. C_B^{yes}). However, we can obtain two more necessary conditions for L.

Fix $i \in [p]$. We say a vertex $v \in A$ such that $g_A^{(i)}(v) = \mathtt{C}$ is *satisfied for* i if at least one of the following holds:

- In G_A, v has a neighbor in $\{u \in A : g_A^{(i)}(u) = \mathtt{I}\}$
- $v \in A'$ for some $A' \subseteq A$ such that $f_I(A') > 0$ or $g_Q^{(i)}(A') = \mathtt{I}$.

In other words, v is satisfied if it has a neighbor in I_i that comes from A, I_B^{no}, or Q_B (in the graphs that conform to L). Similarly, we say a vertex $v \in A$ such that $g_A^{(i)}(v) = \mathtt{I}$ is *satisfied for* i if at least one of the following holds:

- In G_A, v has a non-neighbor in $\{u \in A : g_A^{(i)}(u) = \mathtt{C}\}$
- $v \notin A'$ for some $A' \subseteq A$ such that $f_C(A') > 0$ or $g_Q^{(i)}(A') = \mathtt{I}$.

Let $T = \bigcap_{i=1}^{p} S_i$. A vertex $v \in A \setminus T$ is a *universally clique* vertex if we have $g_A^{(i)}(v) = \mathtt{C}$ or $v \in S_i$ for all $i \in [p]$. A vertex $v \in A \setminus T$ is a *universally independent* vertex if we have $g_A^{(i)}(v) = \mathtt{I}$ or $v \in S_i$ for all $i \in [p]$. For a vertex $v \in A \setminus T$, if there is some $i \in [p]$

such that L is of clique type for i, $g_A^{(i)}(v) = \texttt{C}$, and v is not satisfied for i, then we say v is *neighbor needy*. Similarly, if there is some $i \in [p]$ such that L is of independent type for i, $g_A^{(i)}(v) = \texttt{I}$, and v is not satisfied for i, then we say v is *non-neighbor needy*. We say L is *globally valid* if the following holds:

- For all $v \in A \setminus T$, if v is neighbor needy, then v is a universally clique vertex
- For all $v \in A \setminus T$, if v is non-neighbor needy, then v is a universally independent vertex.

Putting this all together, we say the guess list L is *valid* if it is consistent, locally valid, and globally valid.

The algorithm

As part of our algorithm, we will need to compute the number of ordered partitions of C_B^{no}, I_B^{no}, and Q_B that are consistent with our guesses for the sizes of the parts. For a nonnegative integer j and an ordered list $A_1, \ldots, A_r$ of subsets of $\{0, 1, \ldots, n\}$, let $p_B(j, \mathcal{F})$ denote the number of ordered partitions of $[j]$ into r parts $P_1, \ldots, P_r$ such that $|P_i| \in A_i$ for all $i \in [r]$.

We will also need to compute the number of possibilities for $G[B]$. For a split graph G with split decomposition (C, I, Q), let C_{yes} be the set of vertices in C that have a neighbor in I, and let C_{no} be the set of vertices in C that do not have a neighbor in I. Let I_{yes} be the set of vertices in I that have a non-neighbor in C, and let I_{no} be the set of vertices in I that do not have a non-neighbor in C. For nonnegative integers $c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q$ and

a boolean z, let $s_B(c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q, z)$ denote the number of split graphs G such that

$$\begin{aligned} C_{\text{yes}} &= [c_{\text{yes}}], \\ C_{\text{no}} &= [c_{\text{yes}} + 1, c_{\text{yes}} + c_{\text{no}}], \\ I_{\text{yes}} &= [c_{\text{yes}} + c_{\text{no}} + 1, c_{\text{yes}} + c_{\text{no}} + i_{\text{yes}}], \\ I_{\text{no}} &= [c_{\text{yes}} + c_{\text{no}} + i_{\text{yes}} + 1, c_{\text{yes}} + c_{\text{no}} + i_{\text{yes}} + i_{\text{no}}], \\ Q &= [c_{\text{yes}} + c_{\text{no}} + i_{\text{yes}} + i_{\text{no}} + 1, c_{\text{yes}} + c_{\text{no}} + i_{\text{yes}} + i_{\text{no}} + q], \end{aligned}$$

and such that Q is a clique if and only if $z = 1$.

For now, we assume we have algorithms that compute $s_B(c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q, z)$ and $p_B(j, \mathcal{F})$. See the following two sections for the details of these algorithms. Suppose we have been given a fixed valid guess list

$$L = (G_A, c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q, z, f_C, f_I, f_Q, g_A^{(1)}, \ldots, g_A^{(p)}, g_Q^{(1)}, \ldots, g_Q^{(p)}).$$

Let $\hat{c}$ be the number of vertices in $A \setminus T$ that are neighbor needy, and let $\check{c}$ be the number of universally clique vertices in $A \setminus T$ that are *not* neighbor needy. Let $\hat{i}$ be the number of vertices in $A \setminus T$ that are non-neighbor needy, and let $\check{i}$ be the number of universally independent vertices in $A \setminus T$ that are *not* non-neighbor needy.

We define the set families $\mathcal{F}_C$, $\mathcal{F}_I$, and $\mathcal{F}_Q$ in the following way. Let $A'_1, \ldots, A'_r$ be all of the subsets of A, where $r = 2^{|A|}$. For $i \in [r]$, let $A_i^C = \{0\}$ if $f_C(A'_i) = 0$, let $A_i^C = \{1\}$ if $f_C(A'_i) = 1$, and let $A_i^C = \{2, 3, \ldots, c_{\text{no}}\}$ if $f_C(A'_i) = 2$. Similarly, let $A_i^I = \{0\}$ if $f_I(A'_i) = 0$, let $A_i^I = \{1\}$ if $f_I(A'_i) = 1$, and let $A_i^I = \{2, 3, \ldots, i_{\text{no}}\}$ if $f_I(A'_i) = 2$. Let $A_i^Q = \{0\}$ if $f_Q(A'_i) = 0$, let $A_i^Q = \{1\}$ if $f_Q(A'_i) = 1$, and let $A_i^Q = \{2, 3, \ldots, q\}$ if $f_Q(A'_i) = 2$. Now let $\mathcal{F}_C = \{A_1^C, \ldots, A_r^C\}$, $\mathcal{F}_I = \{A_1^I, \ldots, A_r^I\}$, and $\mathcal{F}_Q = \{A_1^Q, \ldots, A_r^Q\}$.

Let $\mathcal{L}$ be the set of all valid guess lists. Let $b = |B|$ and $t = |T|$. Our algorithm returns

$$\sum_{L \in \mathcal{L}} \binom{b}{c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q} (2^{c_{\text{yes}}} - 1)^{\hat{i}} (2^{i_{\text{yes}}} - 1)^{\hat{c}}\, 2^{c_{\text{yes}}\check{i} + i_{\text{yes}}\check{c} + tb} h(L),$$

where

$$h(L) = s_B(c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q, z)\, p_B(c_{\text{no}}, \mathcal{F}_C)\, p_B(i_{\text{no}}, \mathcal{F}_I)\, p_B(q, \mathcal{F}_Q),$$
$$\binom{b}{c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q} = \frac{b!}{c_{\text{yes}}!\, c_{\text{no}}!\, i_{\text{yes}}!\, i_{\text{no}}!\, q!},$$

and $c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q, z$ are entries of L.

Counting ordered partitions

For a nonnegative integer j and an ordered list $A_1, \ldots, A_r$ of subsets of $\{0, 1, \ldots, n\}$, we defined $p_B(j, \mathcal{F})$ as the number of ordered partitions of $[j]$ into r parts $P_1, \ldots, P_r$ such that $|P_i| \in A_i$ for all $i \in [r]$. We now describe how to compute this function.

Suppose we have been given j and $A_1, \ldots, A_r$. For $0 \leq s \leq j$, $1 \leq t \leq r$, let $p(s, t)$ be the number of ordered partitions of $[s]$ into t parts $P_1, \ldots, P_t$ such that $|P_i| \in A_i$ for all $i \in [t]$. It is easy to verify that this satisfies the recurrence

$$p(s, t) = \sum_{\substack{a \in A_t \\ a \leq s}} \binom{s}{a} p(s - a, t - 1).$$

For the base case, we have $p(s, 0) = 1$ for all $0 \leq s \leq j$. In the end, we compute $p_B(j, \mathcal{F}) = p(j, r)$ using dynamic programming.

Counting split graphs

Next, we describe how to compute $s_B(c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q, z)$, which corresponds to the number of possibilities for $G[B]$. Suppose we have been given c_{yes}, c_{no}, i_{yes}, i_{no}, q, and z. If $z = 1$, then we return 0 if $c_{\text{no}} > 0$. If $q \leq 1$ or $z = 0$ (i.e., if Q_B is an independent set), then we return 0 if $i_{\text{no}} > 0$. Otherwise, we proceed as follows.

We just need to compute the number of possibilities for the edges between C_{yes} and I_{yes}, since there is only one possibility for all other edges. For $v \in [c_{\text{yes}}]$, let $\mathcal{G}_v$ be the set of graphs with vertex set $[c_{\text{yes}}+i_{\text{yes}}]$ such that $\widetilde{C} \coloneqq [c_{\text{yes}}]$ is a clique, $\widetilde{I} \coloneqq [c_{\text{yes}}+1, c_{\text{yes}}+i_{\text{yes}}]$

is an independent set, the vertex $v \in \widetilde{C}$ has a neighbor in $\widetilde{I}$, and every vertex in $\widetilde{I}$ has a non-neighbor in $\widetilde{C}$. Our goal is to compute $s_B(c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q, z) = |\bigcap_{i=1}^{c_{\text{yes}}} \mathcal{G}_v|$. By the intersection version of the inclusion-exclusion principle, we have

$$\Big|\bigcap_{v=1}^{c_{\text{yes}}} \mathcal{G}_v\Big| = \sum_{j=0}^{c_{\text{yes}}} \binom{c_{\text{yes}}}{j} (-1)^j \Big|\bigcap_{v=1}^{j} \overline{\mathcal{G}_v}\Big|.$$

If $j > 0$, then

$$\Big|\bigcap_{v=1}^{j} \overline{\mathcal{G}_v}\Big| = 2^{(c_{\text{yes}}-j)i_{\text{yes}}}$$

since there are $c_{\text{yes}} - j$ possible neighbors for each vertex in $\widetilde{I}$. If $j = 0$, then

$$\Big|\bigcap_{v=1}^{j} \overline{\mathcal{G}_v}\Big| = (2^{c_{\text{yes}}} - 1)^{i_{\text{yes}}}$$

since every vertex in $\widetilde{I}$ must have a non-neighbor in $\widetilde{C}$.

Proof of correctness

First, we show that every graph we wish to count conforms to a unique guess list.

Lemma 86 *Suppose G is a graph with vertex set $[n]$ such that $G \setminus S_i$ is a split graph for all $i \in [p]$. Then there is a unique guess list L such that G conforms to L.*

Proof: Let $G_A = G[A]$, $c_{\text{yes}} = |C_B^{\text{yes}}|$, $c_{\text{no}} = |C_B^{\text{no}}|$, $i_{\text{yes}} = |I_B^{\text{yes}}|$, $i_{\text{no}} = |I_B^{\text{no}}|$ and $q = |Q_B|$. Let $z = 1$ if Q_B is a clique, and let $z = 0$ otherwise. For each $A' \subseteq A$, we define $f_C(A')$, $f_I(A')$, and $f_Q(A')$ in the natural way. Let $\hat{f}_C(A')$ (resp. $\hat{f}_I(A')$) be the number of vertices $v \in C_B^{\text{no}}$ (resp. $v \in I_B^{\text{no}}$) such that $N(v) \cap A = A'$. If $\hat{f}_C(A') \leq 1$, then let $f_C(A') = \hat{f}_C(A')$, and otherwise, let $f_C(A') = 2$. Similarly, if $\hat{f}_I(A') \leq 1$, then let $f_I(A') = \hat{f}_I(A')$, and otherwise, let $f_I(A') = 2$. If $|\mathcal{N}(A')| \leq 1$, then let $f_Q(A') = |\mathcal{N}(A')|$, and otherwise, let $f_Q(A') = 2$.

Next, for each $i \in [p]$, we define the functions $g_A^{(i)}$ and $g_Q^{(i)}$ in the following way. For each $v \in A$, let $g_A^{(i)}(v) = \texttt{D}$ if $v \in S_i$, let $g_A^{(i)}(v) = \texttt{C}$ if $v \in C_i$, let $g_A^{(i)}(v) = \texttt{I}$ if $v \in I_i$,

and let $g_A^{(i)}(v) = \mathtt{Q}$ otherwise. To define $g_Q^{(i)}$, suppose $A' \subseteq A$. If $\mathcal{N}(A') = \emptyset$, then let $g_Q^{(i)}(A') = \mathtt{E}$. Now suppose $\mathcal{N}(A')$ is nonempty. By Lemma 81, the vertices of $\mathcal{N}(A')$ are either all true twins or all false twins of one another in G. By Lemma 83, this implies that the vertices of $\mathcal{N}(A')$ all belong to the same part in the split decomposition of $G \setminus S_i$. Let $g_Q^{(i)}(A')$ be defined accordingly: let $g_Q^{(i)}(A') = \mathtt{C}$ if $\mathcal{N}(A') \subseteq C_i$, let $g_Q^{(i)}(A') = \mathtt{I}$ if $\mathcal{N}(A') \subseteq I_i$, and let $g_Q^{(i)}(A') = \mathtt{Q}$ otherwise. Now let

$$L = (G_A, c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q, z, f_C, f_I, f_Q, g_A^{(1)}, \ldots, g_A^{(p)}, g_Q^{(1)}, \ldots, g_Q^{(p)}).$$

The values of each entry in L have been defined so that G conforms to L. Furthermore, L is unique since the split decompositions of $G[B]$ and each graph $G \setminus S_i$ are unique. ■

In fact, every such graph conforms to a unique *valid* guess list.

Lemma 87 *Suppose G is a graph with vertex set $[n]$ such that $G \setminus S_i$ is a split graph for all $i \in [p]$. If L is the guess list from Lemma 86 such that G conforms to L, then L is valid.*

Proof: Clearly L is consistent. By the definition of clique type, we know L is of clique type for i if and only if Q_i is a clique, for each $i \in [p]$. Since $G \setminus S_i$ is a split graph, this means $G \setminus S_i$ satisfies all five properties of Lemma 81 for all $i \in [p]$, so L is locally valid. To see that L is globally valid, suppose there is a vertex $v \in A \setminus T$ that is neighbor needy, i.e., there is some $i \in [p]$ for which property (4) of Lemma 81 implies that v has a neighbor in I_B^{yes}. If $v \in I_i \cup Q_i$ for some $i \in [p]$, then Lemma 81 implies that v is non-adjacent to every vertex in I_B^{yes}, since $I_B^{\text{yes}} \subseteq I_i$ by Lemma 84. This a contradiction, so v must be a universally clique vertex. Similarly, if there is a vertex $v \in A \setminus T$ that is non-neighbor needy, then v must be a universally independent vertex. Hence L is globally valid, which means L is valid. ■

Finally, we show that we compute the correct number of graphs for each valid guess list.

Lemma 88 *Suppose L is a valid guess list. The number of graphs G with vertex set $[n]$ that conform to L is*

$$\binom{b}{c_{yes}, c_{no}, i_{yes}, i_{no}, q} (2^{c_{yes}} - 1)^{\hat{i}} (2^{i_{yes}} - 1)^{\hat{c}}\, 2^{c_{yes}\check{i} + i_{yes}\check{c} + tb} h(L),$$

where

$$h(L) = s_B(c_{yes}, c_{no}, i_{yes}, i_{no}, q, z)\, p_B(c_{no}, \mathcal{F}_C)\, p_B(i_{no}, \mathcal{F}_I)\, p_B(q, \mathcal{F}_Q).$$

Proof: The multinomial coefficient $\binom{b}{c_{\text{yes}}, c_{\text{no}}, i_{\text{yes}}, i_{\text{no}}, q}$ counts the number of ways of assigning the labels in B to the sets C_B^{yes}, C_B^{no}, I_B^{yes}, I_B^{no}, and Q. Suppose C_B^{yes}, C_B^{no}, I_B^{yes}, I_B^{no}, and Q are fixed label sets with sizes c_{yes}, c_{no}, i_{yes}, i_{no}, and q, respectively. Also, suppose $\mathcal{P}_1$, $\mathcal{P}_2$, and $\mathcal{P}_3$ are fixed ordered partitions chosen from the ones counted by $p_B(c_{\text{no}}, \mathcal{F}_C)$, $p_B(i_{\text{no}}, \mathcal{F}_I)$, and $p_B(q, \mathcal{F}_Q)$ respectively. Suppose G is a graph that conforms to L, has those particular label sets, and is such that the set of vertices in the ith neighborhood class of C_B^{no} (resp. I_B^{no}, Q) is equal to the ith part in the partition $\mathcal{P}_1$ (resp. $\mathcal{P}_2$, $\mathcal{P}_3$) for all i. All of the edges of $G[A]$ are specified by G_A, and all of the edges from A to $C_B^{\text{no}} \cup C_B^{\text{no}} \cup Q_B$ are specified by L. We know C_B is a clique, I_B is an independent set, Q_B is a clique or independent set (depending on the value of z in L), and every vertex in Q_B is adjacent to every vertex in C_B and is non-adjacent to every vertex in I_B.

The only edges that have not already been determined by L are the edges between A and $C_B^{\text{yes}} \cup I_B^{\text{yes}}$ and the edges between C_B and I_B. The function s_B computes the number of possibilities for the second of these, so now we just need to show that the number of possibilities for the edges between A and $C_B^{\text{yes}} \cup I_B^{\text{yes}}$ is $(2^{c_{\text{yes}}} - 1)^{\hat{i}} (2^{i_{\text{yes}}} - 1)^{\hat{c}}\, 2^{c_{\text{yes}}\check{i} + i_{\text{yes}}\check{c} + tb}$. There are 2^{tb} possibilities for the edges between T and B. For the edges between $A \setminus T$ and B, the only ones that have not already been determined are the edges from I_B^{yes} to a universally clique vertex in $A \setminus T$ and the edges from C_B^{yes} to a universally independent vertex in $A \setminus T$. Each universally clique vertex $v \in A \setminus T$ has $2^{i_{\text{yes}}} - 1$ possible neighborhoods in I_B^{yes} if v is neighbor needy and $2^{i_{\text{yes}}}$ possible neighborhoods otherwise. Similarly, each

universally independent vertex $v \in A \setminus T$ has $2^{c_{\text{yes}}} - 1$ possible neighborhoods in C_B^{yes} if v is non-neighbor needy and $2^{c_{\text{yes}}}$ possible neighborhoods otherwise. This yield a total of $(2^{c_{\text{yes}}} - 1)^{\hat{i}}(2^{i_{\text{yes}}} - 1)^{\hat{c}}\, 2^{c_{\text{yes}}\check{i} + i_{\text{yes}}\check{c} + tb}$ possibilities.

■

Running time

Each of the necessary multinomial coefficients can be computed in $O(n)$ time. When we sum over $L \in \mathcal{L}$, the number of possible functions in the entries of L is dominated by the number of possibilities for $g_Q^{(i)}$, which is

$$4^{2^{|A|}} = 4^{2^{O(k^2 5^k)}}.$$

The number of possibilities for G_A is at most $2^{|A|^2}$, which is much smaller than $4^{2^{|A|}}$. There are $O(n^4)$ possibilities for the values of c_{yes}, c_{no}, i_{yes}, i_{no}, and q since c_{no} and i_{no} cannot both be nonzero at once. Each multiplication in the summation over L takes $O(n^2 \log n)$ time since each number has at most $O(n^2)$ bits [91]. Computing t and b takes $O(np)$ time, where $p \leq (k+1)5^k$. Using repeated squaring, we can compute 2^{tb} in $O(\log n)$ multiplications, each of which takes $O(n^2 \log n)$ time. Hence the running time of this algorithm is $4^{2^{O(k^2 5^k)}} \cdot n^7 \log n$.

By Theorem 9, we obtain an algorithm that computes the number of yes-instances of Split Vertex Deletion. The overall running time is $4^{2^{O(k^2 5^k)}} \cdot n^7 \log n$.

4.4 Faster Algorithm for Counting Graphs with a Small Vertex Cover

In the previous section, we designed algorithms for counting yes-instances of four vertex deletion problems, including Vertex Cover. However, the running time of

each of these algorithms was a double-exponential or triple-exponential function of k. In particular, the running time for counting graphs with a small vertex cover was $2^{O(k^4 2^{2k})} \cdot n^2 \log^2 n$. In this section, we show that using ideas inspired by the Buss kernel for this problem, we can design a much faster algorithm for counting yes-instances of Vertex Cover.

Theorem 11 *There is an algorithm that given $n \in \mathbb{N}$ and $0 \leq k \leq n$, computes the number of labeled graphs with vertex set $[n]$ that have a vertex cover of size at most k. This algorithm uses $k^{O(k^2)} n^3$ arithmetic operations.*

To design this algorithm, we first observe that the vertices of each graph can be partitioned into three parts, H, I, and R, defined as follows.

Definition 12 *The **high-degree set** of a graph G is the set $H \coloneqq \{v \in V(G) : \deg(v) > k\}$. The **isolated set** of G is the set $I \coloneqq \{v \in V(G) \setminus H : N(v) \subseteq H\}$. The **remaining set** of G is the set $R \coloneqq V(G) \setminus (H \cup I)$.*

If G has a vertex cover of size at most k, then the sizes of H and R can be bounded above by a function of k.

Lemma 89 *If G is a graph with a vertex cover of size at most k, then the high-degree set satisfies $|H| \leq k$, the remaining set satisfies $|R| \leq k^2 + k$, and $G[R]$ has at most k^2 edges.*

Proof: Let $v \in H$, and suppose S is a vertex cover of G of size at most k. We know $v \in S$ because otherwise all neighbors of v would belong to S, which is impossible since $|S| \leq k$. Therefore, we have $|H| \leq |S| \leq k$.

The bound on the size of R follows from the standard Buss kernelization algorithm for Vertex Cover. Consider the following two reduction rules: (1) If G contains an

isolated vertex v, then delete v from G; (2) If G contains a vertex v of degree more than k, then delete v from G and decrease k by 1. If we apply these rules exhaustively to a graph with a vertex cover of size at most k, then the resulting graph has at most $k^2 + k$ vertices and at most k^2 edges [94].

Let G be a graph with a vertex cover of size at most k, and let H, I, and R be the high-degree set, the isolated set, and the remaining set of G, respectively. Suppose we exhaustively apply the above reduction rules to G. Let H' be the set of vertices deleted by reduction rule (2), let I' be the set of vertices deleted by reduction rule (1), and let $R' = V(G) \setminus (H' \cup I')$.

We know $G[R']$ has at most $k^2 + k$ vertices and at most k^2 edges, and we claim that $R \subseteq R'$. We have $H' \subseteq H$ since the degree of a vertex can only decrease when we delete other vertices. To see that $I' \subseteq I$, let $v \in I'$. We first check that $v \notin H$. If $v \in H$, then for v to become isolated, all of its neighbors would need to be deleted by reduction rule (2). However, that would mean G has more than k vertices of degree more than k, which is impossible since G has a vertex cover of size at most k. To see that $N(v) \subseteq H$, we first observe that $N(v) \subseteq H' \cup I'$ since v is isolated at the moment when it is deleted. No two vertices in I' can be adjacent because if they were, then neither one would ever be isolated. This means $N(v)$ is in fact contained in $H' \subseteq H$, so $v \in I$. Thus we have $I' \subseteq I$, so $R' \supseteq R$ as desired. ■

Algorithm overview. We consider all possibilities for the sizes of H, I, and R that could occur in an n-vertex graph G with a vertex cover of size at most k. Suppose we have guessed that $|H| = h$, $|I| = i$, and $|R| = r$, where $h + i + r = n$. We will describe how to count the number of graphs with $H = [h]$, $I = [h+1, h+i]$, and $R = [h+i+1, n]$. To consider all possibilities for the label set of these three subgraphs, we then multiply this result by $\binom{n}{h}\binom{n-h}{i}$.

We loop over all $2^{\binom{h}{2}}$ possibilities for the edges of $G[H]$, and for each of these, we

proceed as follows. For each vertex $v \in H$, we consider all possibilities for its degree into I and R, i.e., we guess the values of $|N(v) \cap I|$ and $|N(v) \cap R|$. Degree values that are greater than k (and some others) can all be bucketed into a single category. For each of these guesses, we compute the number of possibilities for $G[H \cup I]$ and $G[H \cup R]$ and then multiply those two results.

The subgraph $G[H \cup I]$: There are no edges in $G[I]$ since I is an independent set, so we just need to count the possibilities for the edges between H and I. If we have guessed that the vertex $v \in H$ has $|N(v) \cap I| = d_v$, then there are $\binom{i}{d_v}$ possibilities for $N(v) \cap I$. Even if we just have a lower bound, e.g., we have guessed that $|N(v) \cap I| > k$, it is easy to compute the number of possibilities for $N(v) \cap I$ in a similar way.

The subgraph $G[H \cup R]$: We loop over all possibilities for the edges of $G[R]$, knowing that this subgraph has at most $k^2 + k$ vertices and at most k^2 edges. In this loop, we only consider subgraphs $G[R]$ such that all vertices have degree at least 1 and such that $G[R]$ has a vertex cover of size at most $k - h$. Suppose we have a particular guess for $G[R]$ in mind, as well as a guess for $G[H]$ from above. What remains now is to count the possibilities for the edges between H and R. For each vertex $v \in H$, we have already guessed the degree of v into R. For each vertex $v \in R$, the degree of v into H must be at most $k - d_v$, where d_v is the degree of v in $G[R]$, since $v \notin H$. Using this information about the degrees, we run a divide and conquer algorithm to compute the number of possibilities for $G[H \cup R]$.

4.4.1 Algorithm for Theorem 11

The following procedures describe the algorithm for Theorem 11 in detail. To run the algorithm, one would call Count_VC_Graphs(n, k).

We refer to h-dimensional vectors as h-tuples, where h is a nonnegative integer. For

h-tuples $d = (d_1, \ldots, d_h)$ and $\hat{d} = (\hat{d}_1, \ldots, \hat{d}_h)$, we write $d - \hat{d}$ to denote the h-tuple $(d_1 - \hat{d}_1 \ldots, d_h - \hat{d}_h)$.

As mentioned above, each guess for $|N(v) \cap R|$ is either an exact number or a lower bound (e.g., $\geq k+1$). To distinguish between these two types of guesses, we use an h-tuple of booleans $e_H = (e_1, \ldots, e_h)$: the guess for $v \in [h]$ is an exact number if and only if $e_v = \text{TRUE}$. This is will be used in the function $\text{COUNT\_HR}(k, i, r, d_H)$.

COUNT_VC_GRAPHS(n, k):

Return

$$\sum_{(h,i,r)\in\mathcal{I}} \binom{n}{h}\binom{n-h}{i} \text{COUNT\_HIR}(k, h, i, r),$$

where $\mathcal{I} = \{(h, i, r) : 0 \leq h \leq k,\ 0 \leq i \leq n-h,\ r = n-h-i,\ r \leq k^2 + k\}$.

COUNT_HIR(k, h, i, r):

Let $\mathcal{H}$ be the set of all graphs with vertex set $[h]$. For $G_H \in \mathcal{H}$, let $\mathcal{D}_{G_H}$ be the set of pairs (d_H^I, d_H^R) such that $d_H^I = (d_1^I, \ldots, d_h^I)$ and $d_H^R = (d_1^R, \ldots, d_h^R)$ are h-tuples with the following property: for each $v \in [h]$, we have $0 \leq d_v^I \leq k + 1 - \deg_{G_H}(v)$ and $d_v^R = k + 1 - \deg_{G_H}(v) - d_v^I$. Return

$$\sum_{\substack{G_H\in\mathcal{H}\\(d_H^I,d_H^R)\in\mathcal{D}_{G_H}}} \text{COUNT\_HI}(k, i, G_H, d_H^I) \cdot \text{COUNT\_HR}(k, i, r, d_H^R).$$

COUNT_HI(k, i, G_H, d_H):

Let $s = 1$, and write $d_H = (d_1, \ldots, d_h)$. For each $v \in V(G_H)$, we do the following. If $d_v \leq k - \deg_{G_H}(v)$, then we multiply s by $\binom{i}{d_v}$. Otherwise, we multiply s by $\sum_{j=d_v}^{i}\binom{i}{j}$. Return s.

COUNT_HR(k, i, r, d_H):

Let h be the length of the tuple d_H. Let $\mathcal{R}$ be the set of graphs G_R with vertex set $\{h+i+1, \ldots, h+i+r\}$ and at most k^2 edges such that $1 \leq \deg_{G_R}(v) \leq k$ for all

$v \in V(G_R)$, and such that G_R has a vertex cover of size at most $k-h$. For $G_R \in \mathcal{R}$, let $d_{G_R} = (k-d_1, k-d_2, \ldots, k-d_r)$, where $d_v = \deg_{G_R}(h+i+v)$ for each $v \in [r]$. Let e_H be an h-tuple in which every entry is FALSE. Return $\sum_{G_R \in \mathcal{R}} \textsc{Div\_Conq}(d_H, d_{G_R}, e_H, 1, r)$.

$\textsc{Div\_Conq}(d_H, d_R, e_H, s, t)$:

Write $d_H = (d_1, \ldots, d_h)$, $d_R = (d'_1, \ldots, d'_r)$, and $e_H = (e_1, \ldots, e_h)$. For the base case, if $t-s \leq 0$, we do the following. If $d_v > 1$ for some $v \in [h]$, we return 0. Now suppose $d_v \leq 1$ for all $v \in [h]$. If $s = t+1$, then let $d_{\text{available}} = 0$, and otherwise, let $d_{\text{available}} = d'_s$. Let d_{needed} be the number of vertices $v \in [h]$ such that $d_v = 1$, and let $d_{\text{diff}} = d_{\text{available}} - d_{\text{needed}}$. Lastly, let d_{optional} be the number of vertices $v \in [h]$ such that $d_v = 0$ and $e_v = \text{FALSE}$. If $d_{\text{optional}} \leq d_{\text{diff}}$, return $2^{d_{\text{optional}}}$. Otherwise, return $\sum_{i=0}^{d_{\text{diff}}} \binom{d_{\text{optional}}}{i}$.

If $t - s > 0$, meaning we are not in the base case, then we proceed as follows. Let $m = \lceil \frac{s+t}{2} \rceil$. Let $\mathcal{D}$ be the set of pairs $(\hat{d}_H, \hat{e}_H)$ such that $\hat{d}_H = (\hat{d}_1, \ldots, \hat{d}_h)$ and $\hat{e}_H = (\hat{e}_1, \ldots, \hat{e}_h)$ are h-tuples that satisfy the following for all $v \in [h]$:

- If $e_v = \text{TRUE}$, then $0 \leq \hat{d}_v \leq d_v$ and $\hat{e}_v = \text{TRUE}$.
- If $e_v = \text{FALSE}$, then we either have $\hat{e}_v = \text{TRUE}$ and $0 \leq \hat{d}_v < d_v$, or we have $\hat{e}_v = \text{FALSE}$ and $\hat{d}_v = d_v$.

Return

$$\sum_{(\hat{d}_H, \hat{e}_H) \in \mathcal{D}} \textsc{Div\_Conq}(\hat{d}_H, d_R, \hat{e}_H, s, m-1) \cdot \textsc{Div\_Conq}(d_H - \hat{d}_H, d_R, e_H, m, t).$$

4.4.2 Proof of correctness

Suppose we have been given n and k as input. To prove correctness of the algorithm above, we need to show that $\textsc{Count\_VC\_Graphs}(n, k)$ returns the number of graphs with vertex set $[n]$ that have a vertex cover of size at most k. We work our way up through each of the procedures, starting with Div_Conq.

We say the arguments (d_H, d_R, e_H, s, t) are *in the domain of* DIV_CONQ if they have the following properties for some $(h, i, r) \in \mathcal{I}$: The h-tuples $d_H = (d_1, \ldots, d_h)$ and $d_R = (d'_1, \ldots, d'_r)$ are such that $0 \leq d_v \leq k+1$ for all $v \in [h]$ and $0 \leq d'_v < k$ for all $v \in [r]$, and e_H is an h-tuple of booleans. We also require $s, t \in [r]$ and $t \geq s - 1$.

We say the arguments (d_H, d_R, e_H, s, t) *come from* (h, i, r) if the above properties hold for this particular triple $(h, i, r) \in \mathcal{I}$. For some intuition, this is true when the calculations that we are currently doing come from calling COUNT_HIR(k, h, i, r) in COUNT_VC_GRAPHS.

Let $(h, i, r) \in \mathcal{I}$ be fixed. Suppose $d_H = (d_1, \ldots, d_h)$ and $d_R = (d'_1, \ldots, d'_r)$ are such that $0 \leq d_v \leq k+1$ for all $v \in [h]$ and $0 \leq d'_v < k$ for all $v \in [r]$. Also, suppose e_H is an h-tuple of booleans. Let $H = [h]$, and let $R = [h+i+1, h+i+r]$. For a bipartite graph G_B with bipartition (H, R'), where $R' \subseteq R$, we say G_B has *degrees given by* (d_H, d_R, e_H) if G_B has the following properties:

- for every $v \in H$ such that $e_v = \text{TRUE}$, we have $\deg(v) = d_v$
- for every $v \in H$ such that $e_v = \text{FALSE}$, we have $\deg(v) \geq d_v$
- for every $h + i + v \in R'$, we have $\deg(h + i + v) \leq d'_v$.

Lemma 90 *Suppose the arguments (d_H, d_R, e_H, s, t) are in the domain of* DIV_CONQ*, and suppose they come from $(h, i, r) \in \mathcal{I}$. Let $H = [h]$, and let $\widetilde{R} = [h+i+s, h+i+t]$. The procedure* DIV_CONQ(d_H, d_R, e_H, s, t) *computes the number of bipartite graphs with bipartition $(H, \widetilde{R})$ that have degrees given by (d_H, d_R, e_H).*

Proof: Write $d_H = (d_1, \ldots, d_h)$, $d_R = (d'_1, \ldots, d'_r)$, and $e_H = (e_1, \ldots, e_h)$. We proceed by induction. For the base case, suppose $t - s \leq 0$. This means we either have $s = t$ or $s = t + 1$, so $|\widetilde{R}|$ is either 1 or 0. If $d_v > 1$ for some $v \in [h]$, then there are no such graphs

since there is at most one vertex (the one in $\widetilde{R}$, if it exists) that v can be adjacent to. Now suppose $d_v \leq 1$ for all $v \in [h]$.

First, consider the case when $s = t$, which means $\widetilde{R} = \{h+i+s\}$. We have $d_{\text{available}} = d'_s$, which is the upper bound on the degree of the vertex $h + i + s \in \widetilde{R}$. Each vertex in H that is counted in the definition of d_{needed} must have degree 1 in every bipartite graph that has the properties described above. Also, d_{optional} is equal to the number of vertices $v \in H$ that are allowed to have degree 0 or 1. We can choose any subset W of these d_{optional} vertices to have degree 1, as long as the degree of the vertex $h + i + s \in \widetilde{R}$ does not exceed $d_{\text{available}}$. Recall that $d_{\text{diff}} = d_{\text{available}} - d_{\text{needed}}$. If $d_{\text{optional}} \leq d_{\text{diff}}$, then the degree of $h + i + s \in \widetilde{R}$ will never exceed $d_{\text{available}}$, so the number of possible graphs is $2^{d_{\text{optional}}}$. Otherwise, $h + i + s \in \widetilde{R}$ has degree at most $d_{\text{available}}$ if and only if $|W| \leq d_{\text{diff}}$, so the number of possible graphs is $\sum_{i=0}^{d_{\text{diff}}} \binom{d_{\text{optional}}}{i}$.

The case when $s = t+1$ is a bit simpler. In this case, we have $\widetilde{R} = \emptyset$ and $d_{\text{available}} = 0$, so $d_{\text{diff}} \leq 0$. If $d_{\text{diff}} < 0$, then there clearly are no possible graphs, and indeed we return 0. Now suppose $d_{\text{diff}} = 0$. Regardless of whether or not we have $d_{\text{optional}} \leq d_{\text{diff}}$, we know there is only one possible graph, namely the graph with no edges. Hence it is correct to return $2^{d_{\text{optional}}}$ or $\sum_{i=0}^{d_{\text{diff}}} \binom{d_{\text{optional}}}{i}$, both of which are equal to 1 in their respective cases.

Next, suppose we are not in the base case. In other words, suppose the arguments (d_H, d_R, e_H, s, t) satisfy $t - s > 0$. For the induction hypothesis, suppose Div_Conq returns the correct number of graphs, as described in the lemma statement, whenever the last two arguments s' and t' satisfy $t'-s' < t-s$. We claim that Div_Conq(d_H, d_R, e_H, s, t) correctly computes the desired number of graphs as well.

Let $\widetilde{R}_1 \coloneqq [h+i+s, h+i+m-1]$, and let $\widetilde{R}_2 = [h+i+m, h+i+t]$, where $m = \lceil \frac{s+t}{2} \rceil$. Using these sets, the edges of a bipartite graph with bipartition $(H, \widetilde{R})$ can be partitioned into two parts: the edges incident to vertices in $\widetilde{R}_1$, and the edges incident to vertices in $\widetilde{R}_2$. More precisely, let $(\hat{d}_H, \hat{e}_H) \in \mathcal{D}$. Let $\mathcal{G}_B$ be the set of bipartite graphs G_B with

bipartition $(H, \widetilde{R})$ such that G_B has degrees given by (d_H, d_R, e_H) and $G_B[H \cup \widetilde{R}_1]$ has degrees given by $(\hat{d}_H, d_R, \hat{e}_H)$.

Claim 1. The set $\mathcal{G}_B$ is in one-to-one correspondence with the set of pairs (G_1, G_2) with the following properties:

- G_1 is a bipartite graph with bipartition $(H, \widetilde{R}_1)$, and G_2 is a bipartite graph with bipartition $(H, \widetilde{R}_2)$
- G_1 has degrees given by $(\hat{d}_H, d_R, \hat{e}_H)$
- G_2 has degrees given by $(d_H - \hat{d}_H, d_R, e_H)$.

Proof of Claim 1. Suppose $G_B \in \mathcal{G}_B$. Let $G_1 = G_B[H \cup \widetilde{R}_1]$, and let $G_2 = G_B[H \cup \widetilde{R}_2]$. Suppose $v \in H$. If we are in the case where $e_v = \text{TRUE}$, then $\hat{e}_v = \text{TRUE}$ by the definition of $\mathcal{D}$. In this case, we have $\deg_G(v) = d_v$ and $\deg_{G_1}(v) = \hat{d}_v$ since G_1 has degrees given by $(\hat{d}_H, d_R, \hat{e}_H)$, so $\deg_{G_2}(v) = d_v - \hat{d}_v$. If we are in the case where $e_v = \text{FALSE}$ and $\hat{e}_v = \text{TRUE}$, then we have $\deg_G(v) \geq d_v$ and $\deg_{G_1}(v) = \hat{d}_v$, which implies $\deg_{G_2}(v) \geq d_v - \hat{d}_v$. Lastly, if we are in the case where $e_v = \text{FALSE}$ and $\hat{e}_v = \text{FALSE}$, then $\hat{d}_v = d_v$ by the definition of $\mathcal{D}$, so $\deg_{G_2}(v) \geq 0 = d_v - \hat{d}_v$. The graph G_2 also satisfies the requirements for the degrees of the vertices in $\widetilde{R}_2$ since the same is true for G_B. Therefore, G_2 has degrees given by $(d_H - \hat{d}_H, d_R, e_H)$. This shows that we have a map that takes each graph $G_B \in \mathcal{G}_B$ to a pair (G_1, G_2) with the desired properties, and it is easy to see that this map is a bijection. This proves the claim.

By Claim 1, for each choice of $(\hat{d}_H, \hat{e}_H) \in \mathcal{D}$, the number of graphs in $\mathcal{G}_B$ is equal to $\text{DIV\_CONQ}(\hat{d}_H, d_R, \hat{e}_H, s, m-1) \cdot \text{DIV\_CONQ}(d_H - \hat{d}_H, d_R, e_H, m, t)$. If $\mathcal{G}_B$ is defined as above using the pair $(\hat{d}_H, \hat{e}_H) \in \mathcal{D}$, and if $\mathcal{G}'_B$ is defined as above using a distinct pair $(\hat{d}'_H, \hat{e}'_H) \in \mathcal{D}$, then we know $\mathcal{G}_B \cap \mathcal{G}'_B = \emptyset$ by the definition of $\mathcal{D}$. Furthermore, $\mathcal{D}$ was

defined so that it covers all possibilities for the degrees of the vertices in $G_B[H \cup \widetilde{R}_1]$. Therefore, DIV_CONQ correctly computes the desired number of graphs. ■

Let $(h, i, r) \in \mathcal{I}$, $H = [h]$, and $R = [h + i + 1, n]$. In the next lemma, we show that COUNT_HR computes the number of possibilities for $G[H \cup R]$.

Lemma 91 *Let G_H be a graph with vertex set H, and suppose $d_H = (d_1, \ldots, d_h)$ is such that $0 \leq d_v \leq k + 1$ for all $v \in H$. The procedure* COUNT_HR(k, i, r, d_H) *computes the number of graphs G with vertex set $H \cup R$ such that $G[H] = G_H$, $|N(v) \cap R| \geq d_v$ for all $v \in H$, $G[R]$ has at most k^2 edges, $\deg_G(v) \leq k$ and $\deg_{G[R]}(v) \geq 1$ for all $v \in R$, and $G[R]$ has a vertex cover of size at most $k - h$.*

Proof: The set $\mathcal{R}$ is precisely the set of possibilities for $G[R]$. For a vertex $v \in R$ that has degree d_v in the induced subgraph $G[R]$, requiring that $|N(v) \cap H| \leq k - d_v$ is equivalent to requiring that $\deg_G(v) \leq k$. For $G_R \in \mathcal{R}$, let $d_{G_R} = (d'_1, \ldots, d'_r)$ and e_H be defined as in COUNT_HR. To see that the arguments $(d_H, d_{G_R}, e_H, 1, r)$ are in the domain of DIV_CONQ, we just need to check that $0 \leq d'_v < k$ for all $v \in [r]$. This is true since $1 \leq \deg_R(v) \leq k$ for all $v \in V(G_R)$ by the definition of $\mathcal{R}$. Therefore, by Lemma 90, is it correct to return $\sum_{G_R \in \mathcal{R}}$ DIV_CONQ$(d_H, d_{G_R}, e_H, 1, r)$. ■

Let $H = [h]$, and let $I = [h + 1, h + i]$. In the next lemma, we show that COUNT_HI computes the number of possibilities for $G[H \cup I]$.

Lemma 92 *Let G_H be a graph with vertex set H, and suppose $d_H = (d_1, \ldots, d_h)$ is such that $0 \leq d_v \leq k + 1 - \deg_{G_H}(v)$ for all $v \in H$. The procedure* COUNT_HI(k, i, r, d_H) *computes the number of graphs G with vertex set $H \cup I$ such that $G[H] = G_H$, such that I is an independent set, and such that the following holds for all $v \in H$: $|N(v) \cap I| = d_v$ if $d_v \leq k - \deg_{G_H}(v)$, and $|N(v) \cap I| \geq d_v$ if $d_v = k + 1 - \deg_{G_H}(v)$.*

Proof: Since $G[H]$ and $G[I]$ are fixed, we just need to compute the number of

possibilities for $N(v) \cap I$ for each vertex $v \in H$. If $d_v \leq k - \deg_{G_H}(v)$, then the number of possibilities is $\binom{i}{d_v}$, and otherwise, the number of possibilities is $\sum_{j=d_v}^{i} \binom{i}{j}$. ■

For COUNT_HIR, we first need a way to characterize the high-degree set, isolated set, and remaining set of the graph.

Lemma 93 *Let G be a graph with vertex set $[n]$. Suppose H, I, and R are disjoint subsets of $[n]$ such that $H \cup I \cup R = [n]$ and $|H| \leq k$. Then G has a vertex cover of size at most k and H is the high-degree set, I is the isolated set, and R is the remaining set of G if and only if these sets have the following properties:*

1. $\deg(v) > k$ *for all* $v \in H$
2. I *is an independent set*
3. $\deg_G(v) \leq k$ *for all* $v \in R$
4. $\deg_{G[R]}(v) \geq 1$ *for all* $v \in R$
5. $G[R]$ *has a vertex cover of size at most* $k - |H|$
6. G *has no edges with one endpoint in* I *and the other in* R
7. $G[R]$ *has at most* k^2 *edges.*

Proof: For the forward direction, suppose G has a vertex cover of size at most k and H is the high-degree set, I is the isolated set, and R is the remaining set of G. Properties (1), (3), and (6) follow immediately from the definitions of the high-degree set, the isolated set, and the remaining set. Property (2) holds because if there were a vertex $v \in I$ with a neighbor in I, then we would not have $N(v) \subseteq H$. Property (4) holds because if there were a vertex $v \in R$ with $\deg_{G[R]}(v) = 0$, then v would actually belong to I. For property (5), suppose S is a vertex cover of G of size at most k. We

know $H \subseteq S$, so $S \setminus (H \cup I)$ is a vertex cover of $G[R]$ of size at most $k - |H|$. Property (7) follows from Lemma 89.

For the other direction, suppose properties (1)-(7) hold. Let $\widetilde{H}$, $\widetilde{I}$, and $\widetilde{R}$ be the high-degree set, isolated set, and remaining set of G, respectively. Clearly $H \subseteq \widetilde{H}$. We have $N(v) \subseteq H$ for all $v \in I$ by properties (2) and (6), and no vertex in I can have degree more than k since $|H| \leq k$. Hence $I \subseteq \widetilde{I}$. We know $R \cap \widetilde{H} = \emptyset$ by property (3), and property (4) says that $\deg_{G[R]}(v) \geq 1$ for all $v \in R$, so every vertex in R has a neighbor that does not belong to $\widetilde{H}$. This implies $R \subseteq \widetilde{R}$. Therefore, $H = \widetilde{H}$, $I = \widetilde{I}$, and $R = \widetilde{R}$. To see that G has a vertex cover of size at most k, let S be vertex cover of $G[R]$ of size at most $k - |H|$. The set $S \cup H$ is a vertex cover of G since I is an independent set, and we have $|S \cup H| \leq k$. ■

Lemma 94 *Let $(h, i, r) \in \mathcal{I}$, $H = [h]$, $I = [h+1, h+i]$, and $R = [h+i+1, n]$. The procedure* Count_HIR(k, h, i, r) *computes the number of graphs G with vertex set $[n]$ that have a vertex cover of size at most k and have high-degree set H, isolated set I, and remaining set R.*

Proof: In Count_HIR, we sum over $G_H \in \mathcal{H}$ to consider all possiblities for $G[H]$. For each $v \in H$, the value d_v^I is a guess for $|N(v) \cap I|$. If $d_v^I \leq k - \deg_{G_H}(v)$, then our guess is that $|N(v) \cap I| = d_v^I$; if $d_v^I = k + 1 - \deg_{G_H}(v)$, then our guess is that $|N(v) \cap I| \geq d_v^I$. Similarly, each value d_v^R is a guess for $|N(v) \cap R|$. Every one of these is a lower bound, i.e. our guess is that $|N(v) \cap R| \geq d_v^R$. The set $\mathcal{D}_{G_H}$ covers all possibilities for d_v^I and d_v^R in a graph such that $\deg(v) > k$ for all $v \in H$. Therefore, by Lemmas 91 and 92, the summation returned by Count_HIR computes the number of graphs with vertex set $[n]$ such that H, I, and R satisfy properties (1)-(7) from Lemma 93. By that lemma, this is equal to the number of graphs with vertex set $[n]$ that have a vertex cover of size at most k and have high-degree set H, isolated set I, and remaining set R. ■

For Count_VC_Graphs, we need to show that the number of graphs does not depend on the particular choices of the labels in each of the sets H, I, and R. Suppose $A = \{a_1, \ldots, a_r\}$ and $B = \{b_1, \ldots, b_r\}$ are finite subsets of $\mathbb{N}$ such that $|A| = |B|$, where the elements a_i and b_i are listed in increasing order. We define $\phi(A, B)\colon A \to B$ as the bijection that maps a_i to b_i for all $i \in [r]$.

Lemma 95 *Let $(h, i, r) \in \mathcal{I}$. Let $H = [h]$, $I = [h+1, h+i]$, and $R = [h+i+1, n]$. Suppose H', I', and R' are disjoint subsets of $[n]$ such that $H' \cup I' \cup R' = [n]$, $|H'| = h$, $|I'| = i$, and $|R'| = r$. Let $\mathcal{G}_{n,k}$ be the set of graphs with vertex set $[n]$ that have a vertex cover of size at most k. Let $\mathcal{G}$ be the set of graphs in $\mathcal{G}_{n,k}$ that have high-degree set H, isolated set I, and remaining set R. Also, let $\mathcal{G}'$ be the set of graphs $\mathcal{G}_{n,k}$ that have high-degree set H', isolated set I', and remaining set R'. Then $|\mathcal{G}| = |\mathcal{G}'|$.*

Proof: Let $f\colon \mathcal{G} \to \mathcal{G}'$ be the bijection defined as follows: for each $G \in \mathcal{G}$, $f(G)$ is the graph obtained by applying $\phi(H, H')$ to the labels in H, applying $\phi(I, I')$ to the labels in I, and applying $\phi(R, R')$ to the labels in R. The map f is a bijection since each of the relabeling functions is a bijection, so $|\mathcal{G}| = |\mathcal{G}'|$. ■

Lemma 96 *The procedure* Count_VC_Graphs(n, k) *computes the number of graphs G with vertex set $[n]$ that have a vertex cover of size at most k.*

Proof: Let G be such a graph, and let H, I and R be the high-degree set, isolated set, and remaining set, respectively. We have $|H| \leq k$ and $|R| \leq k^2 + k$ by Lemma 89, so $\mathcal{I}$ indeed covers all possible values of $h = |H|$, $i = |I|$, $r = |R|$.

Given $(h, i, r) \in \mathcal{I}$, there are $\binom{n}{h}\binom{n-h}{i}$ possible ways of choosing disjoint subsets H, I, and R of $[n]$ such that $H \cup I \cup R = [n]$, $|H| = h$, $|I| = i$, and $|R| = r$. Fix any such choice of H, I, and R. By Lemmas 94 and 95, the number of graphs with vertex set

$[n]$ and a vertex cover of size at most k that have high-degree set H, isolated set I, and remaining set R is equal to $\textsc{Count\_HIR}(k, h, i, r)$. Therefore, it is correct to return

$$\sum_{(h,i,r)\in\mathcal{I}} \binom{n}{h}\binom{n-h}{i}\textsc{Count\_HIR}(k, h, i, r).$$

■

4.4.3 Running time

All of the running times that we discuss here are measured in terms of the number of arithmetic operations. At the start of this algorithm (Theorem 11), we first compute all of the necessary binomial coefficients. In particular, we compute all values of $\binom{a}{b}$ such that $0 \leq a \leq n$ and $0 \leq b \leq a$, which can be done in $O(n^2)$ time.

For $\ell \geq 0$, let $T(\ell)$ be the maximum running time of $\textsc{Div\_Conq}(d_H, d_R, e_H, s, t)$ across all values of (d_H, d_R, e_H, s, t) in the domain such that $\ell = t - s + 1$. The base case of $\textsc{Div\_Conq}$ takes $O(k)$ time. We have $|\mathcal{D}| \leq (k+2)^k \cdot 2^k$ since $h \leq k$, which gives us the recurrence

$$T(\ell) = (k+2)^k \cdot 2^k \cdot \Big(T(\lceil \ell/2 \rceil) + T(\lfloor \ell/2 \rfloor)\Big) + O(1).$$

By standard arguments such as the master theorem, we can see that $T(\ell) = \ell^{O(k \log(k))}$.

For $\textsc{Count\_HR}$, suppose the arguments are (k, i, r, d_H). We have

$$|\mathcal{R}| \leq (k^2+1)\binom{\binom{k^2+k}{2}}{k^2} = k^{O(k^2)}$$

since each graph in $\mathcal{R}$ has $r \leq k^2 + k$ vertices and at most k^2 edges. To construct $\mathcal{R}$, we first build the set of all graphs with vertex set $\{h+i+1, \ldots, h+i+r\}$ and at most k^2 edges (and the correct degrees), and then for each such graph, we check if it has a vertex cover of size at most $k-h$, which can be done in time $O(2^k n^2)$ using the standard

branching algorithm. Calling DIV_CONQ with $s = 1$ and $t = r$ takes $r^{O(k \log k)} = k^{O(k \log k)}$ time, so the running time of COUNT_HR(k, i, r, d_H) is $k^{O(k^2)}n^2$.

It is easy to see that the running time of COUNT_HI(k, i, G_H, d_H) is $O(hn) = O(kn)$. For COUNT_HIR, suppose the arguments are (k, h, i, r). We have $|\mathcal{H}| = 2^{\binom{h}{2}} \leq 2^{k^2}$ and $|\mathcal{D}_{G_H}| \leq (k+2)^k$, so the running time of COUNT_HIR(k, h, i, r) is dominated by the running time of COUNT_HR, which is $k^{O(k^2)}n^2$. For COUNT_VC_GRAPHS, we have $|\mathcal{I}| = O(kn)$, so the overall running time is $k^{O(k^2)}n^3$ arithmetic operations.

Chapter 5

Concluding Remarks and Open Problems

Interval graphs and permutation graphs are two graph classes with a relatively simple structure for which there might be efficient counting and sampling algorithms. These problems are semi-open, in the sense that they might be open, but it might also be the case that polynomial-time algorithms follow from other results about these graph classes that are phrased in different language (e.g., generating function results). The next step for these two graph classes would be to analyze what is known (see Section 1.2) and determine whether polynomial-time algorithms follow from these results.

There are many graph classes for which the problems of efficient counting and sampling are fully open including graphs of bounded degeneracy, graphs of bounded treewidth, chordal bipartite graphs, distance-hereditary graphs, perfect graphs, strongly chordal graphs, and weakly chordal graphs. These problems are also open for many other graph classes characterized by a finite set of forbidden minors, subgraphs, or induced subgraphs. It is worth noting that for some well-known graph classes of this form, such as planar graphs, polynomial-time algorithms are known [23, 24]. In the next few sections, we

discuss open problems that follow naturally from our results.

5.1 Labeled Chordal Graphs

In Chapter 2, we designed algorithms for exact counting and uniform sampling of labeled chordal graphs that use $O(n^7)$ arithmetic operations (i.e., in $O(n^9 \log n)$ time in the RAM model). A natural open problem is to design a substantially faster algorithm for exact counting or uniform sampling of chordal graphs. We also designed approximate counting and approximate sampling algorithms for labeled chordal graphs that run in $O(n^3 \log n \log^7(1/\varepsilon))$ deterministic time and $O(n^3 \log n \log^7(1/\varepsilon))$ expected time, respectively. Both of these algorithms are quite fast. However, one could argue that our approximate sampling algorithm is unsatisfactory in the sense that it can never output a non-split chordal graph. Therefore, another interesting open problem would be to ask for an approximate sampling algorithm with a stronger guarantee: namely, a sampling algorithm such that for every chordal graph G, we have $(1-\varepsilon)p \leq \Pr(G) \leq (1+\varepsilon)p$, where p is the probability given by the uniform distribution over all chordal graphs and $\Pr(G)$ is the probability that the approximate sampling algorithm outputs the graph G. Two interesting approaches to consider for this are Markov chain Monte Carlo algorithms and the Boltzmann sampling scheme used in [24] for planar graphs.

5.2 Unlabeled Chordal Graphs

In Chapter 3, we built upon the algorithm of Wormald for generating random unlabeled graphs to design an algorithm that, given n, generates a random unlabeled chordal graph on n vertices in expected polynomial time. This serves as a proof of concept that one can obtain a sampling algorithm for unlabeled graphs from a GI-complete graph class

$\mathcal{G}$ using the following two ingredients: (1) an FPT algorithm for counting labeled graphs in $\mathcal{G}$ with a given automorphism π parameterized by the number of moved points of π and (2) a bound on the probability that a labeled graph in $\mathcal{G}$ has a given automorphism. A few potential candidates for this are bipartite graphs, strongly chordal graphs, and chordal bipartite graphs, all of which are GI-complete. An additional open problem would be to design a uniform, or approximately uniform, sampling algorithm — either for unlabeled chordal graphs or general unlabeled graphs — that runs in expected polynomial time even when we condition on the output graph.

5.3 Yes-Instances of Parameterized Vertex Deletion Problems

In Chapter 4, we designed algorithms for counting labeled yes-instances of several vertex deletion problems: Vertex Cover, Cluster Vertex Deletion, Feedback Vertex Set in Tournaments, and Split Vertex Deletion. Our best algorithm for counting yes-instances of Vertex Cover uses $k^{O(k^2)}n^3$ arithmetic operations, and the other three counting algorithms have running times that are double exponential in k, the number of vertices deleted. From the first of these, we can also derive a sampling algorithm that generates a random yes-instance of Vertex Cover using $k^{O(k^2)}n^3$ arithmetic operations. The topic of counting yes-instances of parameterized problems has been largely unexplored until now, so there are many open problems. Here is a list of some open problems that come to mind. In this list, all graphs are labeled unless specified otherwise.

- Is there an algorithm for counting yes-instances of Vertex Cover that runs in time $2^{O(k^2)}n^{O(1)}$? How about in time $2^{o(n^2)}$?

- Is there an FPT algorithm for counting yes-instances of Cluster Vertex Deletion, Feedback Vertex Set in Tournaments, or Split Vertex Deletion whose running time is single exponential in k?

- Is there an FPT sampling algorithm that generates random yes-instances of Cluster Vertex Deletion, Feedback Vertex Set in Tournaments, or Split Vertex Deletion?

- Are there FPT algorithms for counting yes-instances of other vertex deletion problems, such as Cograph Vertex Deletion or Feedback Vertex Set in general graphs?

- So far, all known algorithms for counting yes-instances of a vertex deletion problem are inspired by the FPT algorithm or kernel for the corresponding deletion problem. Is there an FPT (or perhaps even polynomial-time) algorithm for counting yes-instances of a W[1]-hard vertex deletion problem (such as deletion to bounded degeneracy graphs, perfect graphs, or weakly chordal graphs)? Or can this be ruled out under plausible assumptions?

To design the faster algorithm (with single-exponential running time) for counting yes-instances of Vertex Cover in Chapter 4, we used ideas inspired by the Buss kernel for Vertex Cover. It is likely that one could also design a faster algorithm for counting yes-instances of Cluster Vertex Deletion by using ideas from the kernel for that problem.

For graph classes where we do not yet have an FPT counting algorithm, if the graph class can be characterized by a finite set of forbidden induced subgraphs, then one can use the inclusion-exclusion approach from Chapter 4 to reduce to a variation of the problem with given deletion sets. As long as we can design an FPT algorithm for the problem with

given deletion sets, then we will also have an FPT algorithm for the original counting problem.

Bibliography

[1] M. R. Jerrum, L. G. Valiant, and V. V. Vazirani, *Random generation of combinatorial structures from a uniform distribution*, *Theoretical Computer Science* **43** (1986) 169–188.

[2] M. Jerrum, *Counting, sampling and integrating: Algorithms and complexity*. Springer Science & Business Media, 2003.

[3] G. Tinhofer, *Generating graphs uniformly at random*, *Computational Graph Theory* (1990) 235–255.

[4] C. Greenhill, *The complexity of counting colourings and independent sets in sparse graphs and hypergraphs*, *Computational Complexity* **9** (2000) 52–72.

[5] L. G. Valiant, *The complexity of enumeration and reliability problems*, *SIAM Journal on Computing* **8** (1979) 410–421.

[6] L. G. Valiant, *The complexity of computing the permanent*, *Theoretical Computer Science* **8** (1979) 189–201.

[7] N. Linial, *Hard enumeration problems in geometry and combinatorics*, *SIAM Journal on Algebraic Discrete Methods* **7** (1986) 331–335.

[8] M. Jerrum, A. Sinclair, and E. Vigoda, *A polynomial-time approximation algorithm for the permanent of a matrix with nonnegative entries*, *Journal of the ACM (JACM)* **51** (2004) 671–697.

[9] M. Jerrum, *A very simple algorithm for estimating the number of k-colorings of a low-degree graph*, *Random Structures & Algorithms* **7** (1995) 157–165.

[10] J. Liu, A. Sinclair, and P. Srivastava, *A deterministic algorithm for counting colorings with 2Δ colors*, in *2019 IEEE 60th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 1380–1404, IEEE, 2019.

[11] Z. Chen, K. Liu, and E. Vigoda, *Optimal mixing of glauber dynamics: Entropy factorization via high-dimensional expansion*, in *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, pp. 1537–1550, 2021.

[12] A. Sly, *Computational transition at the uniqueness threshold*, in *2010 IEEE 51st Annual Symposium on Foundations of Computer Science*, pp. 287–296, IEEE, 2010.

[13] J. H. Van Lint and R. M. Wilson, *A course in combinatorics*. Cambridge University Press, 2001.

[14] P. W. Kasteleyn, *Dimer statistics and phase transitions*, *Journal of Mathematical Physics* **4** (1963) 287–293.

[15] W. Sun and I. Bezáková, *Sampling random chordal graphs by MCMC (student abstract)*, in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 13929–13930, 2020.

[16] N. C. Wormald, *Generating random unlabelled graphs*, *SIAM Journal on Computing* **16** (1987) 717–727.

[17] H. S. Wilf, *generatingfunctionology*. CRC press, 2005.

[18] T. Saitoh, Y. Otachi, K. Yamanaka, and R. Uehara, *Random generation and enumeration of bipartite permutation graphs*, *Journal of Discrete Algorithms* **10** (2012) 84–97.

[19] D. G. Corneil, H. Lerchs, and L. S. Burlingham, *Complement reducible graphs*, *Discrete Applied Mathematics* **3** (1981) 163–174.

[20] P. Hanlon, *Counting interval graphs*, *Transactions of the American Mathematical Society* **272** (1982) 383–426.

[21] M. Bodirsky and M. Kang, *Generating outerplanar graphs uniformly at random*, *Combinatorics, Probability and Computing* **15** (2006) 333–343.

[22] A. Brandstädt, V. B. Le, and J. P. Spinrad, *Graph classes: A survey*. SIAM, 1999.

[23] M. Bodirsky, C. Gröpl, and M. Kang, *Generating labeled planar graphs uniformly at random*, *Theoretical Computer Science* **379** (2007) 377–386.

[24] É. Fusy, *Uniform random sampling of planar graphs in linear time*, *Random Structures & Algorithms* **35** (2009) 464–522.

[25] M. Bodirsky, C. Gröpl, and M. Kang, *Sampling unlabeled biconnected planar graphs*, in *Algorithms and Computation: 16th International Symposium, ISAAC 2005, Sanya, Hainan, China, December 19-21, 2005. Proceedings 16*, pp. 593–603, Springer, 2005.

[26] M. Bodirsky, C. Gröpl, and M. Kang, *Generating unlabeled connected cubic planar graphs uniformly at random*, *Random Structures & Algorithms* **32** (2008) 157–180.

[27] T. Saitoh, K. Yamanaka, M. Kiyomi, and R. Uehara, *Random generation and enumeration of proper interval graphs*, *IEICE TRANSACTIONS on Information and Systems* **93** (2010) 1816–1823.

[28] V. Bína and J. Přibil, *Note on enumeration of labeled split graphs*, *Commentationes Mathematicae Universitatis Carolinae* **56** (2015) 133–137.

[29] L. Comtet, *Permutations by number of rises; Eulerian numbers*, *Advanced Combinatorics: The Art of Finite and Infinite Expansions* (1974) 51.

[30] S. Spiro, *Counting labeled threshold graphs with Eulerian numbers*, *Australasian Journal of Combinatorics* **77** (2020) 249–255.

[31] J. S. Beissinger and U. N. Peled, *Enumeration of labelled threshold graphs and a theorem of Frobenius involving Eulerian polynomials*, *Graphs and Combinatorics* **3** (1987) 213–219.

[32] H. Prüfer, *Neuer beweis eines satzes über permutationen*, *Arch. Math. Phys* **27** (1918) 742–744.

[33] OEIS Foundation Inc., *Entry A058862 in the on-line encyclopedia of integer sequences*, 2023. Published electronically at http://oeis.org.

[34] H. S. Wilf, *The uniform selection of free trees*, *Journal of Algorithms* **2** (1981) 204–207.

[35] D. R. Lick and A. T. White, *k-degenerate graphs*, *Canadian Journal of Mathematics* **22** (1970) 1082–1096.

[36] M. Cygan, F. V. Fomin, Ł. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk, M. Pilipczuk, and S. Saurabh, *Parameterized algorithms*, vol. 5. Springer, 2015.

[37] A. Hajnal and J. Surányi, *Über die auflösung von graphen in vollständige teilgraphen*, *Ann. Univ. Sci. Budapest, Eötvös Sect. Math* **1** (1958) 113–121.

[38] P. Buneman *et. al.*, *A characterisation of rigid circuit graphs.*, *Discrete Math* **9** (1974) 205–212.

[39] J. R. Walter, *Representations of rigid cycle graphs*. Wayne State University, 1972.

[40] F. Gavril, *Algorithms for minimum coloring, maximum clique, minimum covering by cliques, and maximum independent set of a chordal graph*, *SIAM Journal on Computing* **1** (1972) 180–187.

[41] F. Gavril, *The intersection graphs of subtrees in trees are exactly the chordal graphs*, *Journal of Combinatorial Theory, Series B* **16** (1974) 47–56.

[42] N. Korte and R. H. Möhring, *An incremental linear-time algorithm for recognizing interval graphs*, *SIAM Journal on Computing* **18** (1989) 68–81.

[43] D. J. Rose, R. E. Tarjan, and G. S. Lueker, *Algorithmic aspects of vertex elimination on graphs*, *SIAM Journal on Computing* **5** (1976) 266–283.

[44] M. Farber, *Independent domination in chordal graphs*, *Operations Research Letters* **1** (1982) 134–138.

[45] Y. Okamoto, T. Uno, and R. Uehara, *Counting the number of independent sets in chordal graphs*, *J. Discrete Algorithms* **6** (2008) 229–242.

[46] I. Bezáková and W. Sun, *Mixing of Markov chains for independent sets on chordal graphs with bounded separators*, in *International Computing and Combinatorics Conference (COCOON)*, pp. 664–676, 2020.

[47] H. N. de Ridder *et. al.*, *Information system on graph classes and their inclusions (ISGCI)*, *www.graphclasses.org* (2016).

[48] D. Gusfield, *The multi-state perfect phylogeny problem with missing and removable data: Solutions via integer-programming and chordal graph theory*, in *Research in Computational Molecular Biology (RECOMB)*, pp. 236–252, 2009.

[49] T. Przytycka, G. Davis, N. Song, and D. Durand, *Graph theoretical insights into evolution of multidomain proteins*, *Journal of Computational Biology* **13** (2006) 351–363.

[50] M. Wienöbst, M. Bannach, and M. Liśkiewicz, *Polynomial-time algorithms for counting and sampling Markov equivalent DAGs*, in *The Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI)*, pp. 12198–12206, 2021.

[51] D. J. Rose, *A graph-theoretic study of the numerical solution of sparse positive definite systems of linear equations*, in *Graph Theory and Computing*, pp. 183–217. Elsevier, 1972.

[52] M. Yannakakis, *Computing the minimum fill-in is NP-complete*, *SIAM Journal on Algebraic Discrete Methods* **2** (1981) 77–79.

[53] E. Bender, L. Richmond, and N. Wormald, *Almost all chordal graphs split*, *Journal of the Australian Mathematical Society* **38** (1985) 214–221.

[54] S. Kratsch and M. Wahlström, *Compression via matroids: A randomized polynomial kernel for odd cycle transversal*, *ACM Transactions on Algorithms (TALG)* **10** (2014) 1–15.

[55] D. Lokshtanov, N. Narayanaswamy, V. Raman, M. Ramanujan, and S. Saurabh, *Faster parameterized algorithms using linear programming*, *ACM Transactions on Algorithms (TALG)* **11** (2014) 1–31.

[56] J. Derbisz, L. Kanesh, J. Madathil, A. Sahu, S. Saurabh, and S. Verma, *A polynomial kernel for bipartite permutation vertex deletion*, *Algorithmica* **84** (2022) 3246–3275.

[57] Ł. Bożyk, J. Derbisz, T. Krawczyk, J. Novotná, and K. Okrasa, *Vertex deletion into bipartite permutation graphs*, *Algorithmica* **84** (2022) 2271–2291.

[58] M. Xiao, *On a generalization of Nemhauser and Trotter's local optimization theorem*, *Journal of Computer and System Sciences* **84** (2017) 97–106.

[59] M. Xiao, *A parameterized algorithm for bounded-degree vertex deletion*, in *International Computing and Combinatorics Conference*, pp. 79–91, Springer, 2016.

[60] L. Mathieson, *The parameterized complexity of editing graphs for bounded degeneracy*, *Theoretical Computer Science* **411** (2010) 3181–3187.

[61] F. V. Fomin, D. Lokshtanov, N. Misra, and S. Saurabh, *Planar F-deletion: Approximation, kernelization and optimal FPT algorithms*, in *2012 IEEE 53rd Annual Symposium on Foundations of Computer Science*, pp. 470–479, IEEE, 2012.

[62] A. Agrawal, D. Lokshtanov, P. Misra, S. Saurabh, and M. Zehavi, *Feedback vertex set inspired kernel for chordal vertex deletion*, *ACM Transactions on Algorithms (TALG)* **15** (2018) 1–28.

[63] Y. Cao and D. Marx, *Chordal editing is fixed-parameter tractable*, *Algorithmica* **75** (2016) 118–137.

[64] F. V. Fomin, T. Le, D. Lokshtanov, S. Saurabh, S. Thomassé, and M. Zehavi, *Subquadratic kernels for implicit 3-Hitting Set and 3-Set Packing problems*, *ACM Transactions on Algorithms (TALG)* **15** (2019) 1–44.

[65] D. Tsur, *Faster parameterized algorithm for cluster vertex deletion*, *Theory of Computing Systems* **65** (2021) 323–343.

[66] F. V. Fomin, S. Gaspers, D. Kratsch, M. Liedloff, and S. Saurabh, *Iterative compression and exact algorithms*, *Theoretical Computer Science* **411** (2010) 1045–1053.

[67] E. J. Kim and O. Kwon, *A polynomial kernel for distance-hereditary vertex deletion*, *Algorithmica* **83** (2021) 2096–2141.

[68] E. Eiben, R. Ganian, and O. Kwon, *A single-exponential fixed-parameter algorithm for distance-hereditary vertex deletion*, *Journal of Computer and System Sciences* **97** (2018) 121–146.

[69] S. Thomassé, *A $4k^2$ kernel for feedback vertex set*, *ACM Transactions on Algorithms (TALG)* **6** (2010) 1–8.

[70] Y. Cao, J. Chen, and Y. Liu, *On feedback vertex set: New measure and new structures*, *Algorithmica* **73** (2015) 63–86.

[71] J. Chen, I. A. Kanj, and W. Jia, *Vertex cover: Further observations and further improvements*, *Journal of Algorithms* **41** (2001) 280–301.

[72] D. G. Harris and N. S. Narayanaswamy, *A faster algorithm for vertex cover parameterized by solution size*, in *41st International Symposium on Theoretical Aspects of Computer Science (STACS 2024)*, vol. 289 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pp. 40:1–40:18, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024.

[73] A. Agrawal, D. Lokshtanov, P. Misra, S. Saurabh, and M. Zehavi, *Polynomial kernel for interval vertex deletion*, *ACM Transactions on Algorithms (TALG)* **19** (2023) 1–68.

[74] Y. Cao, *Linear recognition of almost interval graphs*, in *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 1096–1115, SIAM, 2016.

[75] H. Donkers, B. M. Jansen, and M. Włodarczyk, *Preprocessing for outerplanar vertex deletion: An elementary kernel of quartic size*, *Algorithmica* **84** (2022) 3407–3458.

[76] P. Heggernes, P. Van't Hof, B. M. Jansen, S. Kratsch, and Y. Villanger, *Parameterized complexity of vertex deletion into perfect graph classes*, *Theoretical Computer Science* **511** (2013) 172–180.

[77] B. M. Jansen and M. Włodarczyk, *Lossy planarization: A constant-factor approximate kernelization for planar vertex deletion*, in *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, pp. 900–913, 2022.

[78] B. M. Jansen, D. Lokshtanov, and S. Saurabh, *A near-optimal planarization algorithm*, in *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 1802–1811, SIAM, 2014.

[79] F. V. Fomin, S. Saurabh, and Y. Villanger, *A polynomial kernel for proper interval vertex deletion*, *SIAM Journal on Discrete Mathematics* **27** (2013) 1964–1976.

[80] Y. Cao, *Unit interval editing is fixed-parameter tractable*, *Information and Computation* **253** (2017) 109–126.

[81] E. Ghosh, S. Kolay, M. Kumar, P. Misra, F. Panolan, A. Rai, and M. Ramanujan, *Faster parameterized algorithms for deletion to split graphs*, *Algorithmica* **71** (2015) 989–1006.

[82] M. Cygan and M. Pilipczuk, *Split vertex deletion meets vertex cover: New fixed-parameter and exact exponential-time algorithms*, *Information Processing Letters* **113** (2013) 179–182.

[83] Ú. Hébert-Johnson, D. Lokshtanov, and E. Vigoda, *Counting and sampling labeled chordal graphs in polynomial time*, in *31st Annual European Symposium on Algorithms (ESA 2023)*, vol. 274 of *Leibniz International Proceedings in Informatics (LIPIcs)*, (Dagstuhl, Germany), pp. 58:1–58:17, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023.

[84] Ú. Hébert-Johnson and D. Lokshtanov, *Sampling unlabeled chordal graphs in expected polynomial time*, in *42nd International Symposium on Theoretical Aspects of Computer Science (STACS 2025)*, vol. 327 of *Leibniz International Proceedings in Informatics (LIPIcs)*, (Dagstuhl, Germany), pp. 46:1–46:20, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025.

[85] N. C. Wormald, *Counting labelled chordal graphs*, *Graphs and Combinatorics* **1** (1985) 193–200.

[86] L. Markenzon, O. Vernet, and L. H. Araujo, *Two methods for the generation of chordal graphs*, *Annals of Operations Research* **157** (2008) 47–60.

[87] O. Şeker, P. Heggernes, T. Ekim, and Z. C. Taşkın, *Linear-time generation of random chordal graphs*, in *Proceedings of the 10th International Conference on Algorithms and Complexity (CIAC)*, pp. 442–453, 2017.

[88] T. Ekim, M. Shalom, and O. Şeker, *The complexity of subtree intersection representation of chordal graphs and linear time chordal graph generation*, *Journal of Combinatorial Optimization* **41** (2021) 710–735.

[89] A. Mukhopadhyay and M. Z. Rahman, *Algorithms for generating strongly chordal graphs*, in *Transactions on Computational Science XXXVIII*, pp. 54–75, 2021.

[90] J. R. Blair and B. Peyton, *An introduction to chordal graphs and clique trees*, in *Graph Theory and Sparse Matrix Computation*, pp. 1–29. Springer, 1993.

[91] D. Harvey and J. Van Der Hoeven, *Integer multiplication in time $O(n \log n)$*, *Annals of Mathematics* **193** (2021) 563–617.

[92] J. D. Dixon and H. S. Wilf, *The random selection of unlabeled graphs*, *Journal of Algorithms* **4** (1983) 205–213.

[93] W. Oberschelp, *Kombinatorische anzahlbestimmungen in relationen*, *Mathematische Annalen* **174** (1967) 53–78.

[94] F. V. Fomin, D. Lokshtanov, S. Saurabh, and M. Zehavi, *Kernelization: Theory of parameterized preprocessing*. Cambridge University Press, 2019.